

NEAR REAL-TIME DETECTION OF ANDROID SPYWARE
LEVERAGING DIRECTIONAL NETWORK TRAFFIC
FEATURES AND RULE-BASED LABELLING

MOUSUMI AHMED MIMI

DOCTOR OF PHILOSOPHY

MULTIMEDIA UNIVERSITY

AUGUST 2025

NEAR REAL-TIME DETECTION OF ANDROID SPYWARE LEVERAGING DIRECTIONAL NETWORK TRAFFIC FEATURES AND RULE-BASED LABELLING

BY

MOUSUMI AHMED MIMI

BCSE, IUBAT-International University of Business Agriculture and Technology,
Bangladesh

M.Sc.in CSE, UIU-United International University, Bangladesh

THESIS SUBMITTED IN FULFILMENT OF THE
REQUIREMENT FOR THE DEGREE OF
DOCTOR OF PHILOSOPHY

(By Research)

in the

Faculty of Computing & Informatics

MULTIMEDIA UNIVERSITY

MALAYSIA

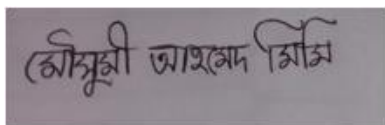
August 2025

© 2025 Universiti Telekom Sdn. Bhd. ALL RIGHTS RESERVED.

Copyright of this thesis belongs to Universiti Telekom Sdn. Bhd. as qualified by Regulation 7.2 (c) of the Multimedia University Intellectual Property and Commercialisation Policy. No part of this publication may be reproduced, stored in or introduced into a retrieval system, or transmitted in any form or by any means (electronic, mechanical, photocopying, recording, or otherwise), or for any purpose, without the express written permission of Universiti Telekom Sdn. Bhd. Due acknowledgement shall always be made of the use of any material contained in, or derived from, this thesis.

DECLARATION

I hereby declare that the work has been done by myself and no portion of the work contained in this Thesis has been submitted in support of any application for any other degree or qualification of this or any other university or institution of learning.



Mousumi Ahmed Mimi

ID- 1221404218

Faculty of Computing & Informatics
Multimedia University

Date: 27/10/2025

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude to my supervisor, Assist. Prof. Dr. Ng Hu, and my co-supervisor, Assist. Prof. Dr. Timothy Yap Tzen Vun. This thesis would not have been possible without their invaluable guidance and suggestions. I feel much fortunate to have their supervision as their continuous motivation, words of encouragement and valuable advice have been instrumental in the completion of this thesis.

My heartfelt thanks also go to my family for their unwavering love, constant encouragement, and endless patience. Their support has been a source of strength and motivation, helping me persevere through the challenges of this endeavour.

Lastly, I am deeply grateful to my friends for their unconditional support and encouragement. Their companionship and understanding have been invaluable, providing inspiration and reassurance every step of the ways.

ABSTRACT

Android spyware creates a significant threat to individual privacy and organizational security by secretly monitoring user activities. Spyware is a subclass of malware, but its unique behaviour makes traditional malware datasets and signature-based detection techniques less effective for accurate spyware classification. This study addresses the critical challenge of detecting Android spyware, emphasizing cybersecurity risks and the limitations of existing approaches. Previous works often rely on signature-based methods or static network traffic analysis, which are effective for known spyware but may fail to capture evolving spyware behaviours. This research addresses these gaps by developing Android spyware datasets based on directional features and rule-based labelling, evaluating different classification models, and enabling near real-time detection.

To address these limitations, this research proposes an approach to Android spyware classification by developing two datasets derived from real-time network traffic: MAS-2024-A (4.2 million samples, single-directional) and MAS-2024-B (2.0 million samples, bi-directional). A total of 420.3 million network traffic packets are collected using PCAPDroid, with each data collection session containing only a single type of spyware. To enhance spyware classification, directional feature engineering is applied by converting raw PCAP network traffic into structured datasets. Static features such as IP addresses, ports, and protocols are extracted and transformed into dynamic features like bandwidth and packet rate, ensuring traffic analysis. Rule-based labelling is implemented to distinguish spyware from Normal traffic, ensuring accurate classification by analysing unique destination IP counts, the ratio of uplink to downlink traffic. These rules capture spyware activity that is different from traditional network communication patterns, enabling precise classification.

A classification pipeline is applied, starting with batch processing using various learning models. Each dataset reserves 20% as a test set to evaluate final model performance. A total of 288 experiments are conducted, covering balanced approaches (Synthetic Minority Over-sampling Technique, Random-Under-Sampling, fixed sample sizes of 150,000 for MAS-2024-A and 70,000 for MAS-2024-B) and the

Original datasets scenarios. Among the tested models, Long Short-Term Memory with Extreme Gradient Boosting (LSTM-XGB) achieves the highest accuracy of 99.03% for MAS-2024-A and 99.04% for MAS-2024-B. The best-performing models are saved and applied for near real-time detection. They are identified from the Original datasets experiments using an 80-20 split ratio across all learning models.

Micro-batch processing is introduced to extend classification to near real-time detection, focusing on packet sizes ranging from 50 to 250. The optimal micro-batch size of 150 packets yields average classification accuracies of 76.29%–84.95% (MAS-2024-A) and 74.18%–83.23% (MAS-2024-B) for the spyware classes. Among all the saved models, XGB provides the best results for near real-time detection, achieving 80.01% accuracy using single-directional features. Execution time analysis highlights single-directional features as more efficient for near real-time detection, requiring 36.86 seconds compared to bidirectional features 39.89 seconds.

TABLE OF CONTENTS

DECLARATION	iii
ACKNOWLEDGEMENTS	iv
ABSTRACT	v
TABLE OF CONTENTS	vii
LIST OF TABLES	xi
LIST OF FIGURES	xiii
LIST OF ALGORITHMS	xv
LIST OF ABBREVIATIONS	xvi
LIST OF APPENDICES	xvii
CHAPTER 1 INTRODUCTION	1
1.1 Problem Statement	3
1.2 Research Questions	4
1.3 Research Objectives	4
1.4 Research Contributions	5
1.5 Thesis Outline	6
CHAPTER 2 LITERATURE REVIEW	8
2.1 Dataset Collection	9
2.1.1 Spyware Dataset Collection	9
2.1.2 Malware Dataset Collection	10
2.1.3 Dataset Collection for Anomaly Detection	12
2.1.4 Imbalanced Dataset	12
2.2 Behaviour and Feature Analysis	14
2.3 Spyware or Malware Classification	14
2.4 Feature Extraction and Engineering	16
2.4.1 Network Traffic Based Feature Engineering	17
2.5 Real-Time or Near Real-Time Network Traffic Classification	19
2.6 Network Traffic Classification using ML	21
2.6.1 Network Traffic Based Malware Classification	22
2.7 Real-Time or Near Real-Time Anomaly Detection	25
2.8 Rule-Based Malware Classification	28

2.8.1	Rule-Based Feature Extraction	33
2.9	Batch Micro-Batch and Stream Processing	34
2.10	ML, DL, and Ensemble Models	35
2.11	Summarization of Literature Review	37
CHAPTER 3 METHODOLOGY		46
3.1	Introduction	46
3.2	First Part: Data Collection	46
3.3	Second Part: Feature Engineering and Classification (Batch Processing)	46
3.4	Third Part: Near Real-Time Detection	48
3.5	Conclusions	48
CHAPTER 4 DATASET COLLECTION		50
4.1	Introduction	50
4.2	Dataset Information	50
4.3	Data Collection Method	51
4.3.1	Root the Device	51
4.3.2	How the Attacker Monitors the Victim's Device	52
4.3.3	Important Information During Spyware Network Traffic Capture	53
4.3.4	PCAP File Description	54
4.4	Ethical Considerations and Privacy Assurance	55
4.5	Methodology for Data Collection	56
4.5.1	Day-Wise Data Collection Approach	56
4.5.2	Spyware Data Collection Method	57
4.5.3	Normal Traffic Data Collection Method	58
4.5.4	Variability in Packet Numbers and File Sizes Across Spyware Classes and Normal Traffic	60
4.6	Conclusions	61
CHAPTER 5 FEATURE ENGINEERING AND DATASET DEVELOPMENT		63
5.1	Introduction	63
5.2	Data Preprocessing	63

5.3	Feature Engineering	64
5.4	Dataset Development and Labelling	65
5.4.1	Development of MAS-2024-A	65
5.4.2	Development of MAS-2024-B	66
5.4.3	Comparisons of Datasets	67
5.4.4	Sample Labelling	69
5.4.4.1	Rule-1-Flow Status	70
5.4.4.2	Rule-2-Number of Unique DIP	70
5.4.4.3	Rule-3-Uplink and Downlink Traffic Ratio	71
5.5	Exploratory Data Analysis	73
5.6	Directional Properties Impact on Dataset Development	77
5.7	Comparative Summary with Existing Spyware Datasets	79
5.8	Conclusions	80
CHAPTER 6 BATCH PROCESSING		82
6.1	Introduction	82
6.2	Metrics for Evaluation	83
6.3	Results	84
6.4	Detailed Analysis of the Results	90
6.5	Discussions	93
6.5.1	Learning Models	93
6.5.2	Network Traffic Analysis for Feature Engineering	95
6.6	Conclusions	96
CHAPTER 7 MICRO-BATCH PROCESSING		99
7.1	Introduction	99
7.2	Labelling Ground Truth During Micro-Batch and Near Real-Time Detection	100
7.3	Results	102
7.4	Detailed Analysis of the Results	105
7.5	Conclusions	110
CHAPTER 8 NEAR REAL-TIME DETECTION		112
8.1	Introduction	112
8.2	Prototype	113

8.3	Results	116
8.4	Detailed Analysis of the Results	118
8.5	Discussions	120
8.5.1	Near Real-Time Detection	120
8.5.2	Advancements in ML/DL Methods for Near Real-Time Analysis	122
8.5.3	Performance Validation on Alternative Dataset	123
8.5.4	Limitations of Real-Time Detection: Computational and Practical Constraints	126
8.5.5	Comparative Analysis with Existing Studies	127
8.6	Conclusions	128
	CHAPTER 9 CONCLUSIONS	130
9.1	Conclusions	130
9.2	Recommendations for Future Work	131
	REFERENCES	132
	APPENDICES	151
	LIST OF PUBLICATIONS	206

LIST OF TABLES

Table 2.1	Systematic literature analysis illustrating the insides and considerations.	42
Table 4.1	Dataset information.	50
Table 4.2	Activity list during data collection.	57
Table 4.3	Sniffing description for each activity.	57
Table 5.1	Dynamic features.	65
Table 5.2	Differences between MAS-2024-A and MAS-2024-B.	68
Table 5.3	Sample rows for some features for MAS-2024-A.	68
Table 5.4	Sample rows for some features for MAS-2024-B.	68
Table 5.5	Packet status.	70
Table 5.6	Comparison Between the Proposed Dataset and Existing Spyware Datasets	79
Table 6.1	Evaluation metrics results (accuracy, precision, recall, and f1-score) for 80-20 split for both datasets.	86
Table 6.2	Comparison accuracy for 80-20 split for different balancing scenario with the Original scenario.	86
Table 6.3	Run time for every model in different scenario.	87
Table 7.1	Micro-batch processing-average accuracy for all the saved models.	103
Table 7.2	Micro-batch processing-average precision for all the saved models.	104
Table 7.3	Micro-batch processing-average recall for all the saved models.	104
Table 7.4	Micro-batch processing-average f1-score for all the saved models.	105
Table 8.1	Evaluation matrix results for near real-time analysis.	117
Table 8.2	Execution time for near real-time analysis.	118
Table 8.3	Batch processing results for 70-30 train test validation results.	124

Table 8.4	Micro-batch processing accuracy of RF across different classes in the downloaded dataset.	125
Table 8.5	Micro-batch processing accuracy of XGBoost across different classes in the downloaded dataset.	125
Table 8.6	Table 8.6 Micro-batch processing accuracy of SVM across different classes in the downloaded dataset.	125
Table 8.7	Near real-time detection results with the downloaded dataset.	126
Table 8.8	Performance comparison between developed and downloaded datasets.	126

LIST OF FIGURES

Figure 3.1	Methodology.	47
Figure 3.2	General steps in the data collection process.	47
Figure 4.1	Root the device	52
Figure 4.2	Attacker monitors victim's device.	53
Figure 4.3	PSP	55
Figure 4.4	Spyware data collection workflow.	58
Figure 4.5	Normal Traffic data collection workflow.	59
Figure 4.6	Packet distribution for each class.	59
Figure 4.7	File size for each class.	60
Figure 5.1	Bi-directional packet flow.	67
Figure 5.2	Class distribution for MAS-2024-A.	69
Figure 5.3	Class distribution for MAS-2024-B.	69
Figure 5.4	Workflow for applying rule-based labelling	72
Figure 5.5	Heatmap of correlation matrix for MAS-2024-A.	74
Figure 5.6	Heatmap of correlation matrix for MAS-2024-B.	75
Figure 5.7	Comparison of the number of packets with MAS-2024-A (total sample number), and MAS-2024-B (total sample number) among the classes.	77
Figure 5.8	Comparison of the unique directional properties for each class.	78
Figure 6.1	Steps for batch processing.	83
Figure 6.2	Models accuracy comparison across different split ratios for MAS-2024-A.	88
Figure 6.3	Models accuracy comparison across different split ratios for MAS-2024-B.	88
Figure 6.4	Comparison of accuracy for balancing (SMOTE, RUS, 150K) and original dataset (MAS-2024-A) for 80-20 split ratio.	89
Figure 6.5	Comparison of accuracy for balancing (SMOTE, RUS, 70K) and original dataset (MAS-2024-B) for 80-20 split ratio.	89
Figure 7.1	Average accuracy for 15 classes (MAS-2024-A).	106
Figure 7.2	Average accuracy for 15 classes (MAS-2024-B).	106

Figure 8.1	Flow chart-shows the steps for the set-up for near real-time spyware detection.	113
Figure 8.2	User interface for model and flow type selection.	115
Figure 8.3	Interface showing packet capture after model selection.	115
Figure 8.4	Classification result output of the selected model.	116

LIST OF ALGORITHMS

Algorithm 5.1	Conversion of PCAP to CSV file.	63
Algorithm 5.2	Development of MAS-2024-A.	66
Algorithm 5.3	Development of MAS-2024-B.	66
Algorithm 5.4	Applying pre-defined rules for labelling.	71
Algorithm 7.1	Micro-batch processing workflow.	100

LIST OF ABBREVIATIONS

Abbreviation/Symbol	Description
AT-B-D-M	All Tracker Business Device Monitoring
AT-C-M	All Tracker Communication Monitoring
AT-P-C	All Tracker Parent Control
Bwd	Backward
CatBoost	Categorical Boosting
CNN	Convolutional Neural Network
DIP	Destination IP
DL	Deep Learning
DP	Destination Port
FD	Flow Duration
FFNN	Feedforward Neural Network
Fwd	Forward
FGP	FamiGuardPro
KGP	KidsGuardPro
Len	Length
LGB	Light Gradient Boosting Machine
LSTM	Long Short-Term Memory
MAS-2024	MMU Android Spyware-2024
MMP	MoniMasterPro
ML	Machine Learning
PL	Packet Length
RF	Random Forest
RUS	Random Under Sampling
SIP	Source IP
SMOTE	Synthetic Minority Over-sampling Technique
SP	Source Port
TabNet	Tabular Network
XGB	Extreme Gradient Boosting

LIST OF APPENDICES

Appendix A Experimental Results for Batch Processing	151
Appendix B Experimental Results for Micro-Batch Processing	165

CHAPTER 1

INTRODUCTION

Cybercrimes have been on the rise due to the increasing prevalence of technology and online business (Al-Hashedi, Soon, Goh, Lim, & Siew, 2023; Munusamy & Khodadi, 2023). Among these, spyware has emerged as a significant threat. Spyware is malicious software designed to secretly collect information and monitor user activities on a mobile device without user consent (Amro, 2018). It frequently operates hidden within apparently harmless programs or websites and transmits the collected data to external parties for various purposes (Sipior, Ward, & Roselli, 2005). It represents a critical security threat to organizations and individuals who rely on the internet, as it can lead to loss of privacy, financial fraud, and other malicious activities (Ali, Elgharabawy, Duchaussoy, Mannan, & Youssef, 2020).

Smartphones, particularly 3.3 billion devices those running the Android operating system, has become an essential component of daily life (Kumar, 2024 & Howarth, 2024). The widespread adoption of the Android platform, associated with over 33.3 million cyberattacks involving malware, adware, or unwanted software hindered in 2024, has increased its vulnerability to viruses and malicious software intrusions (Kumar, 2024 & Zou, Zhang, Wan, & Pan, 2014). The ability to install applications from untrusted websites in recent Android versions poses a significant security vulnerability, which hackers can exploit (Shezan, Afroze, & Iqbal, 2017). This situation has led to an increase in third-party platforms distributing Android applications, ranging from legitimate software clones to various forms of malware, including spyware (Rawat, Kumar, Singh, & Arora, 2024).

One of the major issues in cybersecurity research is the frequent confusion between spyware and malware. Although spyware is a subclass of malware, it operates with different specifics (Heinrich, Bittner, & Hollick, 2022). However, much of the existing research has often focused on malware in general, but studies specifically addressing the different challenges have associated with spyware remain relatively

limited (Qabalin, Naser, & Alkasassbeh, 2022). Relying on malware datasets for spyware classification is ineffective, as spyware can disguise itself as legitimate applications, making it difficult to distinguish from normal applications (Stafford & Urbaczewski, 2004).

Malware is a broad category that includes various types such as viruses, trojans, ransomware, and adware, all designed to disrupt, damage, or gain unauthorized access to devices or networks (Buxton, 2025). In contrast, spyware operates covertly, focusing on silently monitoring and transmitting user information without consent, often with minimal disruption to the system's normal behaviour (This vs. that, 2025). This stealthy nature allows spyware to avoid many traditional malware detection techniques (Guardian Digital Spyware, 2024). Consequently, detection strategies that are effective against traditional malware may fail to identify spyware due to its unique behaviour. (Guardian Digital Spyware, 2024).

Existing spyware datasets suffer from limitations, such as a lack of diversity in spyware sample types and insufficient sample sizes (Qabalin et al., 2022). These gaps hinder the development of robust spyware classification models, limiting the generalization of research findings (Botacin, Ceschin, Sun, Oliveira, & Grégio, 2021). To address these limitations, this study makes several contributions. First, spyware network traffic is collected. It collects 14 different types of spyware with Normal Traffic. Feature engineering is then performed using directional properties to develop two datasets and rule-based labelling is applied: MMU Android Spyware-2024-A (MAS-2024-A) based on single direction and MMU Android Spyware-2024-B (MAS-2024-B) based on bi-direction. Following the dataset development, batch processing is applied using various machine learning (ML), deep learning (DL), and ensemble models. The best performing models are saved using Pickle for micro-batch and near real-time detection. Micro-batch is applied for finding the optimal batch size from 50 packets to 250 packets. Finally, near real-time detection is applied using optimal micro-batch size and applying the saved models for spyware detection.

Current spyware classification models often rely on signature information (YARA rule), which may fail to effectively analyse application behaviour (Abri, Siami-Namini, Khanghah, Soltani, Namin, 2019). This poses a significant challenge in developing robust classification systems that can reliably differentiate between legitimate applications and various types of malwares, such as spyware, rootkits, adware, and ransomware (Loi, Borile, Ucci, 2021). Until now, spyware classification has been done using static analysis (Shahzad, 2024), while some earlier works (Malik, Kaushal, Credroid, 2016 & Qabalin et al., 2022) have explored dynamic approaches such as network behaviour-based classification. However, these earlier studies either focused on general malware (Malik et al., 2022) or did not specifically address near real-time classification based on directional traffic features (Qabalin et al., 2022) as proposed in this work. Near real-time classification is challenging due to the need for dynamic monitoring and behavioural analysis, which require advanced tools to capture evolving threat patterns, significant computational resources to process large-scale data, and robust algorithms to differentiate between benign and malicious behaviours (Aboaoja, Zainal, Ghaleb, Al-Rimy, Eisa, Elnour, 2022).

To address these gaps, this work contributes to dataset development based on directional features and rule-based labelling, evaluates directional properties and classification robustness using several learning models, and enabled near real-time detection for execution feasibility. It advances the state-of-the-art by providing efficient Android spyware detection. Unlike previous works, this study applies batch, micro-batch, and near real-time detection across different learning models, offering a detailed approach to Android spyware detection.

1.1 Problem Statement

The problem statements for this thesis are:

- i. Existing datasets often focus on traditional malware rather than spyware samples, may make spyware classification less effective.
- ii. Current models rely on batch processing, often lack adaptation for near real-time detection, may limit their practicality in dynamic threat monitoring.

- iii. Existing spyware classification often focus on static analysis, may not address real-time behaviour.

1.2 Research Questions

The research questions for this thesis are:

- i. How can spyware dataset effectively be collected based on network traffic to enhance spyware classification?
- ii. How do directional properties enhance spyware classification, and how do different ML, DL, and ensemble models perform in batch processing?
- iii. How can micro-batch processing be applied to extend spyware classification to near real-time scenarios?

1.3 Research Objectives

The overall objective of this research is to collect Android spyware dataset based on network traffic for spyware classification. Proper labelling of the samples for spyware and Normal Traffic is crucial, as it ensures the effectiveness of batch processing for each model. During training, the models learn from the features along with their true labels (IBM, 2024; ML Journey, 2024).

Spyware classification often faces challenges due to insufficient datasets, affecting the accuracy of classification models (Qabalin et al., 2022). Dataset limitations contribute to bias in classification, influencing prediction reliability (Loi et al., 2021). Spyware traffic mimic legitimate software, complicating efforts to achieve unbiased classification when sample diversity is constrained (Hu & Dinev, 2005). ML models require adequate data representation to ensure accurate predictions and robust evaluation (Petrini, 2023). The specific objectives of this research are:

- i. To collect and develop Android spyware datasets from different spyware types with Normal Traffic using directional properties and rule-based labelling.
- ii. To evaluate the impact of directional properties on spyware classification, assess the performance of various ML, DL, and ensemble models using batch

processing to identify the best-processing models and saved them for applying during micro-batch and near real-time detection.

- iii. To apply the saved models for micro-batch processing, assess the efficiency and reliability of the models for spyware prediction in near real-time scenarios, and determine the feasibility of near real-time spyware detection based on execution time.

1.4 Research Contributions

The main contributions of this research are as follows:

- i. **Dataset Collection** –Android spyware network traffic is systematically collected, comprising approximately 420.3 million packets from 14 different types of spyware alongside Normal Traffic. This addresses the limitations of existing spyware datasets by collecting packets that enhance the accuracy of spyware classification.
- ii. **Feature Engineering and Classification (Batch Processing)** – Directional properties are applied as a feature engineering approach to improve spyware classification and apply rule-based labelling to distinguish between spyware and Normal Traffic samples by developing two datasets: MAS-2024-A (single directional with 4.2 million samples) and MAS-2024-B (bidirectional with 2.0 million samples). Various ML, DL, and ensemble models are evaluated through batch processing to identify the best-performing models and save those best models for near real-time detection.
- iii. **Near Real-Time Detection** – The saved models are applied for micro-batch processing, evaluating packet sizes ranging from 50 to 250 to determine the optimal batch size for near real-time detection. The research provides a practical solution for near real-time spyware detection, demonstrating the feasibility of applying saved models to live network traffic.

1.5 Thesis Outline

Chapter 2 provides a detailed literature review of previous research related to spyware dataset collection, malware dataset collection, network traffic classification using learning models, and feature engineering. It discusses real-time detection techniques, batch and micro-batch processing, rule-based labelling, and existing anomaly classification methods. Additionally, it highlights research gaps in spyware dataset collection and classification methodologies.

Chapter 3 outlines the research methodology, illustrating the workflow of dataset collection, feature extraction and classification. It introduces batch processing, micro-batch processing, and near real-time detection as key components of the study. The chapter provides a flowchart for summarizing the methodology.

Chapter 4 details the collection of spyware and normal network traffic datasets. It describes the types of spyware that are collected, the data collection methodology, and the challenges associated with spyware data collection.

Chapter 5 introduces feature extraction techniques that are applied to the collected PCAP files. It details the development of two datasets, MAS-2024-A and MAS-2024-B, based on single and bi-directional packet analysis. Additionally, it discusses the rule-based labelling approach that is applied to classify spyware and Normal Traffic samples.

Chapter 6 focuses on batch processing for spyware classification using ML, DL, and ensemble models. It provides evaluation metrics for each model and compares results between different training-validation-test splits. The chapter also includes a discussion on class balancing techniques, such as SMOTE and RUS, and their impact on classification accuracy.

Chapter 7 applies micro-batch processing for near real-time detection. It evaluates different batch sizes (from 50 to 250 packets) to determine optimal performance. The results are analysed for multiple classification models, and comparisons are made between different spyware types.

Chapter 8 examines the application of saved models for near real-time spyware detection. It applies the optimal micro-batch size in near real-time environments and analyses execution times for various models. The chapter also discusses performance for accuracy and execution time.

Chapter 9 summarizes the key findings of the research, highlighting contributions in spyware dataset collection, feature engineering, batch processing, and near real-time detection. It also proposes future research directions to enhance spyware classification.

CHAPTER 2

LITERATURE REVIEW

Mobile technology, especially Android devices, has led to growing cybersecurity risks, with spyware becoming a major concern (Amro, 2018). Spyware is a type of malware designed to secretly collect user data, which can threaten personal privacy and security in organizations (Ali, Elgharabawy, Duchaussoy, Mannan, & Youssef, 2020). Understanding how spyware spreads and operates is important for developing better protection against it (Qabalin et al., 2022).

As highlighted in this Chapter, existing spyware datasets often lacked diversity and sufficient sample sizes, may rely on static features such as permissions and API calls (Nawaz, 2021). Many studies often combined spyware with malware, may overlook behavioural variations that set them apart (Zahan, Burckhardt, Lysenko, Aboukhadijeh, & Williams, 2024). Network traffic analysis has shown potential for malware classification, but its application to spyware classification often remain underexplored (Noetzold, Rossetto, & Leithardt, 2023). Furthermore, previous researchers often focused on offline analysis, may rarely address the real-time or near real-time classification capabilities essential for dynamic threat environments (Xu, Han, Li, & Han, 2023).

The significance of near real-time and real-time traffic detection, as discussed in this Chapter, emerged as critical for identifying spyware's short burst communications with remote servers, as packet flow patterns are important for effective classification (Mohammed & Nor 2009; Gandh & Ribeiro, 2022). However, near real-time frameworks depend on micro-batch packet streams or utilized directional network features, despite their proven importance in capturing spyware communication behaviours (Deepthi, Rani, Krishna, & Saritha, 2024; Najm, Saeed, Ahmed, Ahmed, Sekhar, Shah, & Veena, 2024).

2.1 Dataset Collection

Data collection is a foundational component in cybersecurity research, enabling details studies and classifying malicious applications, including spyware and malware (Noetzold et al., 2023). Studies focused on spyware collection using various tools and methodologies, such as VirusTotal, to extract features and build datasets that supported classification tasks, achieving significant accuracy rates with different learning models (Pierazzi, Mezzour, Han, Colajanni, & Subrahmanian, 2020; Conti, Rigoni, & Toffalini, 2020). Malware dataset collection combined to categorize diverse families and types, often having large repositories such as Android Malware Dataset and Malware Open-source Threat Intelligence Family (MOTIF) for malware classification (Almomani, Almashat, & El-Shafai, 2024; Joyce, Amlani, Nicholas, & Raff, 2023). Furthermore, anomaly classification datasets, such as MaDroid, utilized innovative approaches to trace application behaviours at the operating system level (Duan, Liu, Cai, Sun, & Chen, 2024). Despite notable advancements, the limited availability of technical insights into advanced malicious systems posed challenges, emphasizing the need for continued efforts in expanding datasets to support robust security measures (Conti et al., 2020).

2.1.1 Spyware Dataset Collection

Regarding distinguishing spyware from normal applications, (Pierazzi et al., 2020) used the VirusTotal website to collect spyware and identify five types of spyware include: HeHe, UaPush, AceCard, Pincer, and USBCleaver— and extracted 25 features from each spyware and apply the Ensemble Late Fusion (ELF) method with Random Forest (RF), achieving an F1-score of 0.96 in distinguishing spyware from normal applications through features like file size, permissions, and histograms.

Concerning network traffic analysis and class imbalance, Conti et al. (2020), utilized the ASAIN (A Spy App Identification System based on Network Traffic) application to gather spyware data, Wireshark captured network packets; to differentiate spyware (tSpy, mSpy, cSms, cImg) from normal applications (Dropbox, Google Foto), employing a four-step process of data collection, pre-processing

(applying SMOTE), training, and testing, with dataset effectiveness evaluated using RF, Linear Regression (LR), and K-Nearest Neighbour (KNN) algorithms, where RF achieved the highest F1 score of 0.85. Following this, (Qabalin et al., 2022) used PCAPDroid to capture network traffic and developed a spyware dataset with five variants (Flexispy, Mobilespy, uMobix, TheWispy, mSPY), where Decision Tree (DT) achieved 79% accuracy for binary classification and 77% for multi-class classification, highlighting the utility of network traffic analysis for spyware classification.

The challenge of limited reporting and insufficient technical insights into advanced spyware systems, particularly for smartphones, restricts comprehensive studies on spyware datasets. To address this, Noetzold et al. (2023), developed an integrated computer monitoring system using spyware techniques within a microservices architecture, including Application Programming Interface (API) gateways and JSON Web Token (JWT) token-based authentication, incorporating security features like encrypted logins and passwords, a central API Gateway for communication management, a Front-End application for alert generation, image saving, user management, and authentication, with hate speech classification trained on a Portuguese-language where the Twitter dataset achieving the highest accuracy of 80% using Naïve Bayes (NB)

2.1.2 Malware Dataset Collection

Concerning malware dataset collection, Almomani et al. (2024), developed an Android Malware Dataset with 47,971 samples across 345 families and 7 categories (adware, backdoor, banking, ransomware, riskware, SMS malware, and spyware), collected from various sources, categorized, and validated using ML models, with Support Vector Machine (SVM) achieving 92.46% accuracy, addressing imbalances by adding underrepresented samples, verifying family categorizations with VirusTotal, and providing a categorization scheme and in-depth analysis of malware families' behaviour to differentiate between malware and spyware.

For enhancing malware classification tool, Zahan et al. (2017), developed the MalwareBench dataset by collecting 20,792 samples (6,659 malicious) of malicious and benign packages from npm and PyPI, with malicious samples sourced from BKC, Datadog, Maloss, and Socket's internal dataset, verified by the Socket AI Scanner and security experts, while neutral packages were manually annotated and confirmed for neutrality using the Socket AI Scanner, providing a framework for developing spyware datasets due to overlap in classification challenges.

Exploring device security, it is necessary to gain insight into the crucial permissions of Android devices as (Hindarto & Djajadi, 2023) provided insight into important permissions and features in Android APK files that distinguished various malware forms; malware and benign APK files were collected, decompiled, and validated, categorized into Benign, Ransomware, Riskware, Banking, and SMS, with key steps including reverse engineering APKs, analysing permissions in the AndroidManifest.xml file, and using Jadx for extracting app structure and source code; the dataset supported malware classification with various algorithms like DT, SVM, Artificial Neural Network (ANN), and KNN, where SVM achieved 90% F1-Score, confirming its reliability for Android malware classification.

Focusing on a valuable resource for malware classification, Joyce et al. (2023), developed the MOTIF dataset, which included 3,095 disarmed Portable Executable (PE) files from 14 cybersecurity organizations, featuring 454 malware families and expert-verified labels, with accuracy rates of 62.10% for antivirus majority voting. For introducing the Endgame Malware Benchmark for Research (EMBER) dataset, (Anderson & Roth, 2018) comprising 900,000 samples with key data types like SHA256 hashes, achieving a 92.2% detection rate for training ML models to classification malicious Windows executable files. Regarding to create a large, balanced malware dataset for training models in malware classification across diverse characteristics, like behaviour and platform, (Joyce et al., 2023) developed Malware Datasets for Infrequent Classification Tasks (MalDICT) datasets from over 40 million VirusTotal reports, addressing diverse characteristics with unique malicious files and achieving an F1-Score of 0.56 using MalConv2. For presenting Sophos Reversing Labs-20 Million (SOREL-20M) with nearly 20 million labelled files, (Harang & Rudd,

2020) used SQLite3 and LMDB databases with high baseline model performance, including an ROC of 99.70% for Feedforward Neural Network (FFNN). Exploring public dataset for advanced research, (Yang, Ciptadi, Laziuk, Ahmadzadeh, & Wang, 2021) introduced the Blue Hexagon Open Dataset for Malware Analysis (BODMAS) dataset, with 57,293 malware samples supporting ML in malware analysis, demonstrating improved classifier performance with incremental retraining, maintaining F1 scores above 97%.

2.1.3 Dataset Collection for Anomaly Detection

Concerning large-scale data collection and improving the efficiency of large-scale Android app data collection. Duan et al. (2024), introduced the MaDroid dataset for anomaly detection in mobile apps, using high-dimensional feature sequence data and VirusTotal ratings; the dataset included 50,429 APK files sourced from Androzoo (normal) and Drebin (malware), with application behaviours traced from the Operating System (OS) kernel level using Anbox and Tracee in a virtualized Docker setup, while four primary features (System call class, Timestamp, System call parameter, and return value) were extracted, achieving 84.72% accuracy applying RF.

2.1.4 Imbalanced Dataset

Addressing class imbalance, Liu et al. (2020), introduced the Difficult Set Sampling Technique (DSSTE) algorithm to address the class imbalance problem in network intrusion detection systems (NIDS); the study used the NSL-KDD dataset with 41 attributes and four attack categories (DoS, R2L, Probe, and U2R), and the CSE-CIC-IDS2018 dataset with 83 attributes and six attack scenarios (DDoS, DoS, Brute Force, Bot, Web Attack, and Infeltration), where both datasets were highly imbalanced, reflecting real-world network conditions; data preprocessing involved removing duplicates, handling outliers, transforming features through classification and OneHot encoding, and standardizing numerical values; the DSSTE algorithm segmented the training set into difficult (near other-class samples) and easy (surrounded mostly by samples of the same class) samples using the Edited Nearest Neighbour (ENN) algorithm, augmented minority class data through attribute scaling,

and combined these with easy samples to enhance classifier learning; the scaling factor (k) from the ENN balanced difficult and easy samples, optimizing the training set; model performance was evaluated using RF, SVM, LSTM, XGB, Alex Krizhevsky Network (AlexNet), and mini Visual Geometry Group Network (miniVGGNet), where results showed AlexNet achieved the better accuracy for the NSL-KDD dataset at 82.84%, while miniVGGNet achieved the better accuracy for the CSE-CIC-IDS2018 dataset at 96.99%.

Regarding addressing class imbalance using Random Under Sampling (RUS), (Rafrastara, Supriyanto, Paramita, Astuti, & Ahmed, 2023) improved malware classification by handling class imbalance using RUS; data was sourced from the UCI Machine Learning Repository, containing malware and goodware samples from VirusTotal and VxHeaven, where the originally imbalanced dataset (1:9.5 ratio) was balanced using RUS, resulting in 595 goodware and 595 malware samples; the study applied RF, KNN, NB, and Logistic Regression, evaluating them through two experiments: one with a balanced dataset using RUS and another with the Original dataset; the dataset included 3 files (Goodware, Malware from VirusTotal, Malware from VxHeaven) and contained 6,248 records before balancing and 1,190 records after balancing, with 1,084 features retained; results showed RF provided better performance than other methods, achieving 98.3% accuracy.

For improving classification performance on imbalanced datasets by using Synthetic Minority Over-sampling Technique (SMOTE), (Bouchaib & Bouhorma, 2021) balanced the imbalanced malware dataset and applied Transfer Learning with a pre-trained Visual Geometry Group 16 (VGG16) model trained on ImageNet to improve classification performance; the binary malware files were converted into grayscale images to exploit visual similarity for classification, with the approach evaluated using the Microsoft Malware dataset; the methodology involved converting malware binary files into 8-bit unsigned vectors, arranging them into 2D arrays, and visualizing them as grayscale images; the Microsoft Malware Dataset (BIG 2015) contained 21,741 samples from 9 malware families; data balancing with SMOTE generated synthetic samples for underrepresented classes, resulting in 2,942 samples per class; the study employed a CNN trained from scratch and Transfer Learning with

VGG16, using 5-fold and 3-fold cross-validation, respectively; results showed that both the CNN trained from scratch and the CNN using Transfer Learning achieved 98% accuracy.

2.2 Behaviour and Feature Analysis

Screen logger is one of the crucial features for distinguishing spyware from other malware. For developing the "Screeminals" dataset to analyse screen logger behaviour (Sbai, Happa, Goldsmith, & Meftali, 2020) by collecting 600 spyware samples from VirusShare, AVCaesar, Malshare, and VirusSign, and 94 legitimate screenshot-utilizing applications; the dataset included 118 malware samples with verified screen capture functionality; security reports and generated missing behaviours were analysed, revealing 57% of malware used JPG coding and 61% used memory representation for captures; the study enhanced understanding of distinguishing behaviours in legitimate and spyware applications that actively captured screenshots. Expanding the use of the Endgame Malware Benchmark for Research (EMBER) dataset for malware classification, (Corlatescu, Dinu, Gaman, & Sumedrea, 2024) enhanced the 2018 EMBER dataset by adding new features, including metadata fields such as Class, File, BEH (Behavior), and FAM (Family); the dataset was split into balanced training and test subsets, enriched by querying VirusTotal for detection results, and standardized detection names with AVClass v2; Binary Code Similarity (BCS) was applied using XGB, achieved an average precision value of 96.60% with a tag ranking algorithm to improve tag relevance for cybersecurity searches.

2.3 Spyware or Malware Classification

To develop classification methods that can identify various threats, (Mahesh & KA, 2020) applied a Particle Swarm Optimization (PSO) algorithm coupled with Artificial Neural Network (ANN) to investigate malware attacks; using a Kaggle dataset with 7,107 samples across eight malware classes including spyware, downloader, trojan, worms, adware, dropper, virus, and backdoor, multi-objective PSO and standard scaling for data preprocessing were applied; a multilayer perceptron and Jordan canonical form were applied to remove less significant features, achieving 99% accuracy. To identify spyware classification, Naser et al. (2023) used the Android

Spyware-2022 dataset to identify Android spyware, focusing on two out of five spyware classes: FlexiSpy and MobileSPY; during pre-processing, duplicate and null entities were removed and Source IP (SIP), Destination IP (DIP), Source Port, (SP) Destination Port (DP), Duration, and Protocol were analysed; testing involved the application of a Fine Decision Tree (FDT), resulting in a 98% accuracy rate.

Using different normalization techniques, (Adair, Varol, Liang, & Pham, 2024) trained various DL and ML models for malware classification using Catak's tabular Windows malware dataset with Portable Executable (PE) API calls, containing 7,101 samples across eight malware classes including Trojan, Backdoor, Downloader, Worms, Virus, Dropper, Spyware, and Adware; they included only samples with at least 10 API calls and conduct frequency counts to analyse the original dataset, identifying 342 unique API calls; two new datasets were created excluding low-frequency features and applying two normalization techniques to enhance data utility; for training and evaluation, several ML models (RF, XGB, SVM) and DL models (Convolutional Neural Network (CNN), Long Short-Term Memory (LSTM), Bidirectional Long Short-Term Memory Network (BiLSTM), Multilayer Perceptron (MLP)) were used, finding XGB achieved the highest accuracy of 65% on the Normalization2.csv dataset; the study offered insights into ML and DL model performance based on frequency count analysis and various normalization techniques.

Exploring comparative analysis for static and online datasets, (Brown, Gupta, & Abdelsalam, 2024) performed a relative analysis of static and online malware classification using Automated Machine Learning (AutoML); CNN was applied for online classification and FFNN for static classification using the EMBER-2018 and SOREL-20M datasets; AutoML tuning involved selecting 150 architecture configurations, with models trained for varying epochs; the models achieved 99.80% accuracy for SOREL-20M and 98.40% for EMBER-2018, while online classification using Ubuntu virtual machines achieved 98.61% accuracy; AutoML showed better performance in static classification compared to online classification.

For comparing the performance of various classifiers, including DT, CNN, SVM, KNN, NB, and RF (Akhtar & Feng, 2022) presented ML-based malware classification by involving various learning algorithms to classify malware samples,

preprocessing the dataset with PEiD software for unpacking, and extracting features (n-gram, PE, entropy etc.); ranking techniques were applied for feature selection; the dataset, obtained from the Canadian Institute for Cybersecurity, contains 17,394 files, including various types of malware and benign files, with 279 features; results demonstrated that DT outperforms other classifiers with the highest accuracy (99%). This study showed that ML-based malware classification was efficient in classifying polymorphic malware, making it valuable for cybersecurity applications.

2.4 Feature Extraction and Engineering

Regarding a details approach to incorporating DL-derived features, (Gibert, Planes, Mateu, & Le, 2022) outlined a feature-extraction process that combined hand-crafted features from hexadecimal and assembly-language source codes with deep features derived from DL architectures; hand-crafted features included metadata, byte unigrams, entropy statistics, Haralick features, and local binary patterns; assembly-language features encompassed metadata, opcode unigrams, register features, symbol frequency, pixel intensity, API function calls, data define features, section features, and miscellaneous features; deep features were extracted from raw data, such as grayscale image-based features, entropy-based features, opcode N-gram features, and byte N-gram features; these features were fused using an early fusion mechanism to create a joint representation, which trains a Gradient Boosting (GB) model for malware classification with 99.81% accuracy. To classify polymorphic malware that can transform into various forms, (Masabo, Kaawaase, Sansa-Otim, Ngubiri, & Hanyurwimfura, 2020) developed a feature-engineering method; five malware classes (API, Crypto, Locker, Zeus, and Shadow Brokers) were collected, the data was pre-processed, and feature engineering was performed to identify eleven top features, including static analysis of portable executable files, packing techniques, file access, and registry reading; the developed feature-engineering approach achieved 94% accuracy by applying GB. To classify Android malware, (Nawaz, 2021) proposed a system using the Drebin dataset (Humayun, Jhanjhi, & Alamri, 2020); Android intents and permissions were applied for static analysis, while API calls and network requests were applied for dynamic analysis; Apktool was used to decompile and decoded the APK files; feature selection using Information Gain reduced the dimensionality of

permissions, intents, API calls, and network features; extracted from the APK components, these features train ML classifiers, with RF and GB performing better on the permission features, achieving an F1-score of 0.98. From an APK file, another feature selection method was applied by (Jung, Park, Cho, Han, Park, & Cho, 2021); the APK file was utilized from the AndroZoo dataset (Allix, Bissyandé, Klein, & Le Traon, 2016) to extract information on API calls and permissions, generating a feature vector for each application; feature-selection methods were applied to choose the top 20 features from API calls and permissions (User Account API, Bluetooth API, User Account Permission, Location Permission etc.); RF was applied with a grid search to establish optimal hyperparameters, achieving an accuracy of 96.95% using Gini importance. Concerning comparing different feature-engineering methods, (Low, Yap, Soo, Ng, Goh, Chin, & Kuek, 2022) applied two feature-engineering methods, evidence counting and label encoding, for malware classification; four main steps were involved: data pre-processing, feature selection, model construction, and evaluation; five malware classes (Advanced Persistent Threats (APT), Crypto, Zeus, Locker, and Shadow Brokers) were extracted from the dataset; during pre-processing, data integration, cleaning, and transformation were applied; Boruta was used for feature selection and several ML models were constructed, with the optimal parameters identified through grid search; the dataset was balanced using SMOTE; LSTM achieved better accuracy of 94.64% for evidence counting, while RF achieved better accuracy for label encoding at 91.34%.

2.4.1 Network Traffic Based Feature Engineering

Regarding feature engineering for network traffic classification, (Najm, Saeed, Ahmad, Ahmed, Sekhar, Shah, & Veena 2024) applied ML algorithms to address limitations in existing methods by implementing feature engineering techniques and a variety of algorithms, including DT, RF, SVM, and Recurrent Neural Network (RNN), emphasizing the performance of these models in recognizing network traffic patterns, in dynamic and encrypted communication scenarios; the approach improved precision, speed, and applicability for real-world conditions and reliable network traffic classification; the methodology involved applying DT for straightforward decision boundaries, RF for robust ensemble learning, SVMs for

optimal hyperplane separation in high-dimensional spaces, and RNNs for capturing temporal dependencies in network traffic; feature engineering included data preprocessing, feature selection using correlation analysis and tree-based models, feature extraction from payload statistics, and feature transformation using PCA to reduce dimensionality while preserving essential information; the models were trained and evaluated using an 80:20 train-test split with k-fold cross-validation; the dataset used is the Computer Network Traffic Data, containing approximately 500,000 entries collected over three months, covering 10 local workstation IP addresses, half of which were associated with botnet attacks reflecting malicious activity; the dataset included attributes such as date, remote IP address, ASN, and flow count; results demonstrated that RF achieved better accuracy of 97.89% than other ML models.

Exploring secured classification focusing on VPN and non-VPN traffic, (Afuwape, Xu, Anajemba, & Srivastava 2021) designed ML network traffic analysis system that investigated various ML classifiers, including Adaptive Boosting (AdaBoost), GB, RF, Bagging DT, KNN, Logistic Regression, NB, and Multilayer Perceptron (MLP), comparing their performance on real-world Peer-to-Peer (P2P) network traffic; the proposed model followed a structured workflow: data collection, data pre-processing (including normalization and encoding), feature correlation analysis, and computation using both single and ensemble ML models; the classifiers were evaluated using metrics such as Accuracy, Precision, Recall, and F1-score, with ensemble models like RF, AdaBoost, and GB employed to reduce bias and variance in classification; the dataset, collected from the Canadian Institute for Cybersecurity, included VPN and non-VPN traffic data, with categories like browsing, chat, email, streaming, file transfer, VoIP, and Peer-to-Peer (P2P) records; the data was generated using Wireshark and TCP-dump, with normalization performed using min-max scaling; results showed the RF classifier achieved better accuracy of 93.80% for correctly identifying non-VPN and VPN traffic.

Focusing on feature extraction techniques for ML-based intrusion detection in IoT networks, (Sarhan, Layeghy, Moustafa, Gallagher, & Portmann 2024) systematically evaluated the generalizability of various Feature Extraction (FE) techniques and ML models for different Network Intrusion Detection System (NIDS)

datasets, investigating whether combinations of FE methods and ML classifiers consistently achieved better performance across multiple datasets; the study applied three feature extraction algorithms—Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA), and Autoencoder (AE)—and evaluated their impact on six ML models: Deep Feed Forward (DFF), CNN, RNN, DT, Logistic Regression, and NB; eighteen different combinations of these techniques were examined, offering insights into the effectiveness across three benchmark datasets: UNSW-NB15, ToN-IoT, and CSE-CIC-IDS2018; results showed 98.02% accuracy by applying DT with AE.

2.5 Real-Time or Near Real-Time Network Traffic Classification

For addressing challenges like low latency, fault tolerance, and scalability, (Deepthi, Rani, Krishna, & Saritha, 2024) developed Flexible Real-Time Traffic Stream Processing System (FRTSPS), a system to process real-time traffic data streams that combined batch processing, micro-batch processing, and stream processing to handle large-scale, complex datasets, enabling real-time traffic analysis and providing insights for batch, micro, and real-time network traffic processing; this system developed on the Apache Flink platform; focused on processing real-time traffic data streams, the methodologies and processing techniques provided valuable insights for addressing similar data handling challenges in other domains; in this context, the approach to integrating batch, micro-batch, and real-time processing served as a significant reference for designing efficient data analytics pipelines, such as utilized in spyware classification. To address challenges in real-time network traffic monitoring and classification, particularly for high-bandwidth applications like video streaming, online gaming, and peer-to-peer (P2P), (Mohammed et al., 2009) developed the Near Online Flow-based Internet Traffic Classification (NOFITC) system for near real-time internet traffic classification; they applied the C4.5 algorithm, using an offline module for training and testing and an online module for real-time classification; NetFlow data was applied in an online environment with a buffer for real-time processing, achieving minimal delay and optimized bandwidth for mission-critical applications, with error rates ranging from 0.3% to 6.4%.

Another investigation on packet batching for system resource consumption and detection performance for IoT botnet detection, (Gandhi & Ribeiro, 2022) collected datasets from UNSW IoT Analytics, VirusTotal, and VirusShare; batch processing with network packets of 1,000, 5,000, and 10,000 sizes was applied to train various ML models, including Logistic Regression (LR), RF, SVM, XGB, DNN; a batch size of 1,000 achieved accuracy rates ranging from 94.60% to 82.64%, whereas a batch size of 10,000 achieved accuracy rates between 74.70% and 84.88%; these results suggested that lower batch sizes performed better and reduced system resource consumption.

Regarding near real-time network traffic classification based on online applications, Ahmed et al. (2021), developed NetScraper, a user interface that enabled users to upload captured PCAP files or captured network packets in real-time, providing flexibility and ease for users to analyse network traffic from various sources; the researchers incorporated several ML models, including ANN, RF, and KNN, to classify traffic from 53 distinct online applications, with ANN achieving 99.86% accuracy.

Concerning video streaming traffic, (Dias, Pongelupe, Caminhas, & Errico, 2019) proposed ML-based classification to address limitations of traditional methods like DPI and port-based classification in real-time network scenarios; traffic was identified based on three classes: class A (subscription and higher-rate video), class B (YouTube videos), and class C (remaining traffic); network packets were captured using Wireshark in promiscuous mode (TechTarget Security, 2024), stored in .pcap files, and adapted to various network infrastructures for flexible traffic profiling and analysis; the pre-processing block handled .pcap files by performing Traffic Identification and Variable Extraction, labelling packets, and extracting statistical features from IPv4 headers; a total of 14 features were extracted, including IP address, ports, protocol, packet length, duration, mean arrival time, and variance of IP datagram length; Python scripts using the dpkt library generated files with labels and features from packet contents; the training block selected features from the extracted data, removing redundant variables with high correlation to maintain relevant features; the validation block identifies the most accurate classifier for network traffic

classification, balancing accuracy, class hit rates, and computational complexity; for experiment and analysis, a dataset was generated using Linux bash scripts to capture network traffic for YouTube streaming, Netflix streaming, and file downloads; the captured data was processed by a Python script, resulting in files with extracted features and labels for classification, facilitating future updates via a task scheduler; dataset analysis revealed uneven distributions with narrow peaks or bimodal/multimodal profiles; three redundant features were identified based on strong correlations and removed to enhance classifier performance; the model's performance was measured using metrics balancing overall performance and computational complexity, with 10-fold cross-validation ensuring the model was trained and tested on distinct subsets without data contamination; experiments showed cross-validation with NB demonstrated better performance, achieving 98.79% accuracy.

2.6 Network Traffic Classification using ML

For real-time network traffic classification based on live video streaming, (Patel, Gupta, Kumari, Singh, & Sharma, 2018) compared the accuracy of KNN, NB, DT, and SVM for classifying network traffic data extracted from live video feeds using ML models; five key steps were involved: network data extraction, featuring, sampling, algorithm implementation, and results observation; real-time network traffic was captured using the Wireshark tool, resulting in approximately 85,000 entries from one hour of Internet surfing; converted the network traffic into CSV format, and after analysis, four key features were identified—SIP, DIP, interval between packets, and packet length—to accurately classify network traffic protocols; in the data sampling step, supervised ML techniques were used to label the data, aiding in the classification of unknown network traffic; the researchers then applied SVM, KNN, DT, and NB to the data; results showed that KNN provided better mean accuracy of 92.42%, outperforming the other algorithms; this comparative analysis, conducted by running the models 10 times, demonstrates the reliability and effectiveness of KNN in network traffic classification.

For exploring different network traffic classification approaches, evaluating their strengths, weaknesses, and challenges, (Azab, Khasawneh, Alrabaaee, Choo, & Sarsour, 2024) highlighted the effectiveness of ML and DL techniques while

addressing issues such as untrained version detection and classification accuracy improvements; the study applied multiple feature selection methods like Correlation-based Feature Selection (CFS), Consistency-based subset selection (CON), Gain Ratio (GR), Information Gain (IG), and Chi-square tests; classification models were trained using supervised, unsupervised, and DL techniques, including CNN, RNN, GRU, and SAE; accuracy, precision, recall, F-measure, and ROC curve metrics were used to evaluate classification models, and cross-validation ensured robust model evaluation; utilizing both public and private datasets for training and evaluation, including Moore (2005), ISCX (2016), CTU-13 (2011), MAWI (2000–2020), and others, the research demonstrated that RF improved untrained version detection by 40% and achieved an 11% improvement in classification accuracy using an optimized feature selection technique; it also achieved 96% accuracy in early-stage classification using the first 8 packets.

Exploring network traffic classification by combining clustering algorithms with ML models, (Elngar & Burlea-Schiopoiu, 2023) applied five steps: network traffic capture, feature extraction and selection, training process data sampling, implementation of algorithms, and results and observation; the Wireshark was used to capture network traffic and the UCI Machine Learning Repository (2023, October 1) network traffic dataset, organized by IP, port, and protocol, was utilized, emphasizing the importance of data preprocessing to correct errors and improved dataset reliability; the K-Means algorithm was applied to cluster the network traffic data by iteratively assigning data points (ports, IP, and protocol) to disjoint subsets based on distance to randomly selected cluster centred, enhancing classification accuracy for unfamiliar applications; the extracted features include IP address, ports, protocol, packet length, time interval, flow bytes/sec, flow pkt/s, number of TCP flags, number of ACK and PSH flags, maximum packet size, and mean packet size; finally, network traffic analysis was evaluated using KNN, DT, and NB, with the DT algorithm achieving the highest accuracy of 86.45%.

2.6.1 Network Traffic Based Malware Classification

Regarding ransomware classification through network traffic analysis, (Wiles, Colombo, & Mascorro, 2024) introduced a Generative Adversarial Network (GAN)-

based model that provided a dynamic and adaptive detection mechanism, highlighted the effectiveness of GANs in identifying subtle network traffic anomalies associated with ransomware, enhanced cybersecurity defences with a real-time and resource-efficient detection model, employed a methodology involving data collection using simulated network environments to generate controlled ransomware and benign traffic as well as public datasets, captured raw network traffic in PCAP format, pre-processes it to remove noise, segments it into network flows, normalized it for consistency, focuses feature extraction on packet size, flow duration, packet inter-arrival time, and the number of packets per flow analysed using statistical techniques, utilized a GAN model architecture with a generator creating synthetic traffic and a discriminator differentiating real and synthetic samples, included training through adversarial competition and convolutional layers for feature extraction, and achieved 94.2% accuracy in results.

Concerning the effectiveness of using DL and visual representation for accurate and efficient malware classification, (Bendiab, Shiaeles, Alruban, & Kolokotronis, 2020) proposed an IoT malware traffic analysis combining DL and visual representation for faster detection and classification of zero-day malware at the packet level, evaluate it on a dataset of 1,000 PCAP files using the ResNet50 model, converted network traffic into 2D visual representations processed by the pre-trained ResNet50 CNN for classification, analysed images with the ResNet50 model to detect malware while continuously improving the classifier's accuracy with detected traffic, achieved visual transformation of network traffic by converting PCAP files into RGB images using the Binvis tool that maps byte values to colours based on the ASCII table and arranged them using a space-filling curve for optimal clustering, used a dataset consisting of 1,000 Binvis images generated from both normal traffic PCAP files and malware traffic PCAP files, and achieved 94.50% accuracy after training and testing.

For malware classification for network traffic analysis, (Kim, Park, & Lee, 2020) developed an AI-based Intrusion Detection System (AI-IDS) using a CNN-LSTM model for real-time network attack classification, collected and labelled real-time HTTP traffic data to train DL models for detecting attacks, balanced normal and malicious traffic by expanding and re-training the dataset with 6 GB of data daily,

followed three steps including data saving and splitting, data preprocessing and training with labelled analysis information, and predicting suspicious payloads in new web traffic, collected and stored labelled analysis and real-time HTTP traffic in Docker images for training, split data by attack types (AUT, SQL, APP) for DL models and backed up raw data older than 24 hours, pre-process raw traffic data into trained formats, searched for the best model performance, saved optimal DL models in Docker volumes, inspected real-time web traffic for suspicious payloads, store prediction results in JSON files, periodically saved these events for security analysts to review and label, optimize the CNN-LSTM model by encoding inputs, passing through CNN, LSTM, and DNN layers, and fine-tune parameters through repeated experiments, calculated malicious probability using Adam, and evaluated the model's performance using the CSIC-2010, CICIDS 2017, and real-time HTTP datasets, achieving an accuracy of 99.99%.

Concerning network traffic anomaly classification, (Wei & Wang 2021) applied a network traffic anomaly detection method by combining CNN and RNN to improve detection efficiency, converted network packets into two-dimensional images using heat coding with CNN learning spatial characteristics and RNN learning temporal relationships between packets, applied the Softmax classifier for anomaly detection validated through k-fold cross-validation, used the DARPA1998 dataset including 7 weeks of training traffic and 2 weeks of test traffic with normal traffic and attacks such as DoS, Probe, U2R, and R2L, as well as the ISCX2012 dataset containing 7 days of traffic with normal traffic and attacks like Brute force SSH, DDoS, HttpDoS, and Infiltrating, achieved 98.68% accuracy on the DARPA1998 dataset with a network flow length of 800, and achieved 99.69% accuracy on the ISCX2012 dataset with a network flow length of 600.

Focusing on integrated temporal and spatial features, (Li, Han, Yin, Liu, & Li, 2021) proposed the Improved Traffic Series Network (ITSN) for anomaly network traffic classification, integrated temporal and spatial features using a three-layer parallel network structure, combined two parallel convolutional layers for spatial feature extraction with a spatial-temporal layer incorporating LSTM networks for temporal feature extraction, enhanced the learning of spatial and temporal traffic

features with feature fusion techniques addressing data imbalance and improving classification accuracy, involved data processing steps such as comparing label files with PCAP files, converting binary data to hexadecimal, forward filling short traffic data, and padding with zeros to ensure consistent traffic flow features for neural network training, reduced noise and improved data balance for better feature extraction, applied the ISCX-IDS 2012 and CICIDS 2017 datasets providing traffic data for anomaly classification, and achieved 99.94% accuracy on the CICIDS 2017 dataset and 99.88% accuracy on the ISCX-IDS 2012 dataset.

2.7 Real-Time or Near Real-Time Anomaly Detection

For real-time anomaly detection, (Yu, Yang, Cui, Sui, & Wu, 2024) proposed a method based on Renyi entropy to measure network traffic randomness, selected key features such as SIP, DIP, Ports, Packet, and Byte Counts, calculated Renyi entropy for traffic distribution patterns, introduced an Exponentially Weighted Moving Average (EWMA) model to dynamically adjusted thresholds for classifying network traffic as normal or anomalous, achieved F1-scores of 97.68% on UNSW-NB15, 96.21% on CIC-IDS2017, and 96.09% on CICDDoS2019, used one-hot encoding for categorical features, and applied oversampling techniques for data balancing. Regarding real-time malware classification, (Al-Musawi, 2022) applied two DL models, CNN and LSTM, for malware classification, discovered that LSTM performs better than CNN, collected 7,513 malicious PE files from VirusShare covering 29 families such as AKO, Banker.LY, Delf.DU, and Autorun.NE, and 1,000 benign PE files from Cent Download, pre-processed data in the CNN model by converting PE files to grayscale, resizing, and normalizing them, pre-processed data in the LSTM model by extracting and one-hot encoding the top 50 API calls, achieved 98.23% accuracy with CNN and 99.45% accuracy with LSTM; during training it combined both models to improve classification accuracy, highlighted that real-time evaluation lacked specific evaluation metrics and only applied to samples from the same malware families.

In the approach for near real-time anomaly detection in mobile network traffic, (Huabing, Sisi, Xiaoming, & Zhida, 2021) combined a Bayesian classifier with the Hierarchical Attention Spatio-Temporal Network Anomaly Detector (HAST-NAD)

model to enhance anomaly classification, monitored user behaviour and incorporated DL techniques, specifically CNN and LSTM, to improve both detection accuracy and processing speed, processed user data through the HAST-NAD model to extract spatio-temporal features crucial for identifying anomalies, classify traffic based on a Bayesian classifier utilizing predefined rules (uplink-downlink traffic ratio, replay or request packet) for efficient classification, improved anomaly classification by ensuring both temporal and spatial features were captured and analysed, demonstrated through simulations that this integrated method achieved anomaly classification within 104 seconds, showcasing its practical application and effectiveness in real-time scenarios, and highlighted significant advancements in anomaly classification through the fusion of user behaviour monitoring and DL techniques.

For enhancing Address Resolution Protocol (ARP) spoofing detection and prevention using real-time data analysis, Kumar et al. (2024), applied an RF algorithm, achieving a 94% accuracy rate, collected a dataset including legitimate ARP requests/responses and ARP spoofing attack samples gathered under varying conditions such as different network traffic intensity levels and diverse network topologies, extracted features including the frequency of ARP requests, the ratio of unique MAC addresses to IP addresses, and the time intervals of ARP packets, used these features for training ML models to detect ARP spoofing attacks, address vulnerabilities in IPv6 networks focusing on ICMPv6 which was crucial for IPv6 communication but highly susceptible to Denial of Service (DoS) and Distributed Denial of Service (DDoS) attacks, highlighted the importance of robust feature extraction and selection in building effective security models for classifying network attacks, and ensured the models distinguish between normal and malicious network activities, thereby enhancing network security.

Through continuous data collection and feature analysis, Kumar et al. (2024), provided valuable insights into the classification of ARP spoofing and vulnerabilities in IPv6, contributed to the advancement of cybersecurity measures, and emphasized the need for continuous improvement in anomaly classification techniques to safeguard modern network infrastructures against evolving threats. Given the growing adoption of IPv6 due to its expanded address space and new functionalities, (Bdair,

Abdullah, Manickam, & Al-Ani, 2020) aimed to detect and prevent cyberattacks targeting IPv6, ensuring network security and reliability, highlighted IPv4 limitations such as its small address pool and lack of security features which IPv6 overcomes, noted that IPv6 inherited some vulnerabilities from IPv4 while introducing new ones due to its dependence on ICMPv6 which was crucial for IPv6 operations but susceptible to attacks like DoS and DDoS, examined various Intrusion Detection Systems (IDS) including signature-based, anomaly-based, and hybrid systems while discussing their effectiveness and limitations in detecting ICMPv6 attacks, review existing studies on ICMPv6-based DoS and DDoS attack classification, highlighted ICMPv6 vulnerabilities and common attack patterns, evaluated different IDS approaches, emphasized the need for more accurate and cost-effective solutions, proposed using Optimization Algorithm techniques for improving IDS detection accuracy, categorized ICMPv6-based attacks into protocol exploitation, amplification, and flooding attacks, evaluated IDS effectiveness, identified limitations in current IDS techniques particularly in distinguishing between normal and malicious traffic, concluded that signature-based IDS techniques were inadequate for detecting ICMPv6 attacks due to their reliance on predefined patterns while anomaly-based systems provided better classification rates but have high false positives, found that hybrid systems showed promise but need further optimization for improved accuracy and cost-effectiveness, and provided a detailed review of ICMPv6 vulnerabilities and different IDS approaches, identifying research gaps and suggesting future directions including Optimization Algorithms and hybrid IDS solutions for enhanced IPv6 network security.

For real-time detection and prevention of network activities, Chanesh (2017), demonstrated configuring Snort for deep packet inspection on a single machine, capturing traffic data using Wireshark, and achieving real-time traffic monitoring and intervention in Network Intrusion Detection System (NIDS)/ Network Intrusion Prevention System (NIPS) modes with operational testing, used Snort as a cost-effective Deep Packet Inspection (DPI) tool for real-time anomaly classification, designed and implemented a low-cost DPI system with Snort to make it accessible for small organizations unable to afford expensive routers, configured Snort as a NIDS and an inline NIPS across simulated Demilitarized Zone (DMZ) and Local Area

Network (LAN) environments, captured data using Wireshark and filter it to create Snort rules that classify and blocked suspicious traffic patterns including Facebook activities, terrorism-related content, and brute-force attacks, collected data focusing on Facebook applications, chats, comments, terrorism content, and brute-force attacks while analysing Domain Name System (DNS) queries and encrypted traffic to develop robust classification, demonstrated Snort's efficiency and speed in unencrypted traffic anomaly classification and prevention, highlighted its limitation with encrypted traffic and suggested additional configurations, and found that Snort was a viable, cost-effective alternative to expensive DPI systems, especially for smaller organizations seeking network security solutions.

By combining semi-supervised HAST-NAD networks and federated learning, Xu et al. (2023), proposed a real-time anomaly traffic classification system, achieved 98.4% accuracy in anomaly detection by using spatial and temporal data features from raw traffic data, ensured computational efficiency and reduced burdens on satellite networks, analysed space-air-ground integration network (SAGIN) architecture and proposed a federated learning architecture to enhance security and timeliness, developed an anomaly traffic detection method using DL and semi-supervision, collected data packets over time, organized them by shared characteristics, and divided them into sequences for analysis, pre-processed raw traffic data by segmenting it, removing redundancy, converting packets into tensors, and normalizing them for neural network input, improved the HAST-NAD model by integrating a semi-supervised autoencoder for spatial feature extraction and using LSTM to capture temporal relationships, enhanced anomaly detection accuracy, deploy the anomaly detection algorithm on GEO satellites to monitor and analysed traffic, blocked anomalies while caching them for further analysis, enable federated learning for collaborative training of anomaly detection models across devices without transferring raw data, optimize model size through quantization, and achieved an anomaly traffic classification accuracy of 98.44% after semi-supervised learning.

2.8 Rule-Based Malware Classification

Concerning rule-based malware classification, (D'Angelo, Ficco, & Palmieri, 2021) introduced an association rule-based malware detection approach using

recurring sub sequences of API calls, enhanced robustness by considering both transition probabilities and the timeline of API calls to effectively classify minor malicious behaviours common to subsets of malware, identified recurring subsets of API calls expressed as non-contiguous sequences that maintain order through association rules, included key steps such as rule extraction and transition modelling, graph construction and merging, supported and confidence calculation, classification using a softmax function, and parameter tuning and pruning, represented rules as API call sequences while considering transition probabilities and timelines normalized to handle varying sequence lengths and imbalanced datasets, used a dataset comprising 7.3K malware samples from multiple families and 1.2K benign applications with API call sequences extracted using the Cuckoo Sandbox tool, achieved an accuracy of 99%, and combined the strengths of sequence alignment and Markov chain models to offer enhanced robustness and improved classification accuracy in malware classification.

For evaluating rule-based algorithms, (Talabani & Abdulhadi, 2022) enhanced ransomware classification within Bitcoin networks by utilizing rule-based algorithms to classify ransomware transactions, specifically employing DT and Partial Decision Tree (PART) models, aimed to improve classification accuracy and compare the performance of these models on a large dataset, filled a gap in applying rule-based methods to Bitcoin ransomware classification, utilized DT to organize rules in an IF-THEN format and evaluated feature subsets using best-first search and cross-validation, utilized PART to construct partial decision trees with a divide-and-conquer approach and generated rules from the best-performing branches, implemented both models using Weka software and evaluated them with 10-fold cross-validation to ensure robustness, derived the dataset from Bitcoin transactions between January 2009 and December 2018 including 61,004 addresses from 29 ransomware families and focusing on ransomware-related transactions, demonstrated through experimental results that the PART algorithm performs better than the DT model, and achieved 96.01% accuracy.

To evaluate the effectiveness of another rule-based classification, (Lockett, 2021) applied Yet Another Recursive Acronym (YARA) rules for signature-based malware classification, compared them with cryptographic hashing (SHA-256) and

fuzzy hashing (SSDEEP), demonstrated YARA's effectiveness in classifying obfuscated and new malware strains, highlighted the limitations of traditional cryptographic hashing in identifying modified or unknown malware, and suggested improvements for YARA rules by integrating fuzzy hashing and enhancing rule descriptions, utilize 15 malware samples from GitHub and 15 recent malware samples from VirusTotal along with over 410,000 SSDEEP malware signatures from MalwareBazaar, applied YARA rules from a public GitHub repository, SHA-256 for cryptographic hashing, and SSDEEP for fuzzy hashing with a similarity threshold of 50% or more for classification, used an evaluation method involving these techniques applied to malware samples to assess classification accuracy, and showed results indicating that for GitHub samples, SHA-256 achieved 100% classification, SSDEEP achieved 33% classification, and YARA achieved 80% classification, while for VirusTotal samples, SHA-256 failed to detect or classify any malware, SSDEEP achieved 33% classification, and YARA achieved 33% classification.

Focusing on signature-based ransomware classification, (Sangher, Singh, & Pandey, 2024) presented an approach using RF with SMOTE for ML and CNN with Adam optimizer for DL, introduced hyperparameter optimization to enhance classification accuracy and address data imbalance issues, demonstrated improved performance in predicting ransomware variants without the need for feature engineering or binary code analysis, compared the effectiveness of RF with SMOTE against other ML models while showcasing CNN's superior performance with Adam optimizer for large datasets, employed RF with SMOTE to handle class imbalance and optimized performance in small datasets, use Lazy Predict AutoML for model selection and parameter tuning with evaluation metrics such as accuracy, F1 score, ROC AUC, and Matthews Correlation Coefficient (MCC), applied Repeated Stratified K-fold Cross Validation to maintain class distribution in each fold, utilized CNN with Adam optimizer for better learning rate adjustment and performance in large datasets while ANN with RMSprop and Adam optimizers discovered hidden patterns inaccessible through ML, apply SMOTE-Tomek Links to overcome overfitting, use Local Interpretable Model-Agnostic Explanations (LIME) to provide local explanations and feature importance analysis, analysed a dataset including 96,724 malware samples and 41,323 legitimate files focusing on Windows executables from

the VirusShare repository with 57 attributes like file name, code size, version, and operational information, and achieved results showing RF with SMOTE achieved 99.3% accuracy while CNN with Adam optimizer achieved 99% accuracy.

To evaluate feature-based malware classification using rule-based methods, (Tirumala, Valluri, & Nanadigam, 2020) explored the effectiveness of feature-based training using autoencoders for malware classification, created a synthetic malware dataset based on both signatures and features to enhance the performance of AI and traditional malware detection systems, demonstrated that feature-based training was more effective than signature-based training particularly when using stacked autoencoders, achieved superior classification accuracy across multiple datasets including synthetic ones, prepared two types of datasets: signature-based and feature-based, extracted features using a stacked autoencoder network while extracting signatures using an ANN with three layers trained via backpropagation, created a synthetic dataset of 3,500 samples comprising 3,000 samples from signatures and features and 500 randomly selected samples from the original dataset, applied two deep autoencoder networks (DAE): DAEF trained with the feature dataset and DAES trained with the signature dataset, conducted experiments using two architectures (3-layered and 5-layered DAE) with the Stochastic Gradient Descent (SGD) algorithm for training and various parameters for optimization, performed experiments using Matrix Laboratory 2019 (MATLAB 2019) and normalize data with International Business Machines Corporation (IBM), Statistical Package for the Social Sciences (SPSS) and MATLAB statistical packages, and achieved classification accuracy showing that DAEF achieved 95.96% accuracy on the synthetic dataset.

Integrating visual data representation, (Aslan & Yilmaz, 2021) highlighted the combination of rule-based and DL techniques for malware classification, introduced a DL-based hybrid architecture for classifying malware variants by visualizing executable binary files as grayscale images, converted grayscale images into binary code as 8-bit unsigned integers reshaped into 2D matrices and interpreted as images, designed a model consisting of four phases: data collection, architecture design, training, and evaluation, employed pre-trained networks as feature extractors and utilized transfer learning to enhance classification performance across various

malware datasets, applied the method to the Maling-Microsoft BIG 2015 dataset with 9,339 samples and 25 classes and the Malevis dataset with 9,100 samples and 25 classes, combined ResNet-50 and AlexNet pre-trained networks to create a hybrid model for feature extraction, fed the extracted features into fully connected layers and a softmax classifier, and achieved 97.78% accuracy with the Maling dataset and 94.88% accuracy for the Microsoft BIG 2015 dataset.

In terms of network traffic classification using signature-based methods, (Khandait, Hubballi, & Mazumdar, 2020) proposed a signature-based approach for network traffic classification that reduced computational complexity, utilized a single scan of flow payloads for application signature matching, generated application signatures by processing the first 100 bytes of network flow payloads involving bidirectional flows, cluster similar flows based on word similarity, extracted high-frequency words, and selected meaningful keywords to create application-specific signatures, represented signatures by generating a common Finite State Machine (FSM) from the first keywords of each application and combining it with application-specific portions, employed the Aho-Corasick algorithm for efficient pattern matching to enable quick identification of potential signatures by scanning only the initial bytes of the payload and significantly reducing search overhead, created application-specific signatures using individual FSMs with the remaining keywords and their offset values for each application, enable efficient searches by leveraging keyword ordering and reducing unnecessary string comparisons, classified network flows during the testing phase using both common and application-specific signatures by processing the first K bytes of the payload through the classifier FSMs, performed bidirectional flow reconstruction and examine initial bytes using the common FSM to identify potential applications based on the maximum offset value of the first keywords, conducted subsequent string matching with relevant application-specific keywords when a matching keyword was found to maintain sub-linear search complexity, used a dataset comprising raw tcpdump-readable traffic from various sources including in-house deployments and external repositories with data from 10 application protocols (FTP, HTTP, POP, SIP, SMB, SMTP, SSH, etc.), verified the ground truth using Wireshark, implement their two-part signature representation and matching approach using the optimized Aho-Corasick algorithm in a Java application, and achieved a classification

performance with a minimum recall rate of 93.72% across 10 application protocols with no observed misclassifications.

2.8.1 Rule-Based Feature Extraction

In terms of signature-based feature extraction, (Miha, Kawasaki, Makuda, Takana, & Barakata, 2024) introduced an Adaptive Signature Extraction Framework for ransomware detection, utilized dynamic signature generation combined with ML featuring an adaptive mechanism that updated classification signatures in real-time, designed an architecture including data collection modules (network traffic, file system activities, process behaviours), a feature extraction engine identifying key ransomware characteristics, and an adaptive signature mechanism for dynamic detection signatures, applied ensemble learning and anomaly detection using Principal Component Analysis (PCA) to reduce data dimensionality, temporal analysis to track changed over time, and higher-order statistics for feature selection, used SVM and DT along with ensemble classification and adaptive thresholding for anomaly detection, collected multiple datasets including 1,200 ransomware samples from cybersecurity repositories, 800 benign user activity samples from internal logs, 1,000 system processed logs, and 600 network traffic logs, pre-processed data through cleaning, labelling, feature normalization, and anomaly detection, and achieved a classification accuracy of 95.70%.

Adaptive Signature Analysis (ASA) framework for ransomware classification, (Clarry, Andersen, Papadopoulos, Grigoriev, & Hofmann, 2024) combined ASA for dynamic pattern recognition with adaptive learning to detect both known and previously unseen ransomware attacks, designed it for real-time classification with limited computational resources, demonstrate its resilience against evasion techniques like obfuscation and polymorphism, included a methodology involving data collection from multiple datasets covering 130 ransomware families and 5,323 samples, applied data preprocessing with PCA for dimensionality reduction and a feature extraction module isolating ransomware indicators, used a Dynamic Pattern Recognition Engine for anomaly detection and an Adaptive Learning Unit for continuous parameter updates, implemented the framework utilizing Python, Pandas, Scikit-learn, and neural networks, and achieved a classification accuracy of 98.70%.

2.9 Batch Micro-Batch and Stream Processing

To optimize data stream processing in real-time network scenarios and improve performance through dynamic micro-batching and data stream frequency management, (Garcia, Griebler, Schepke, & Fernandes, 2022) extended a benchmarking framework to support dynamic micro-batching, focused on data stream processing by examining latency and performance under different conditions, explored how varying data stream frequency affected latency and overall performance of Threading Building Blocks (TBB) and FastFlow, conducted experiments on a machine with 32 GB RAM and two Intel Xeon processors running Ubuntu Server 18.04 with GCC 7.5.0, benchmark four applications using FastFlow and Intel TBB libraries with static micro-batch sizes under different parallelism degrees, applied four data frequency strategies (increasing, decreasing, wave, and binary) with item delays from 0 to 300 ms, used the Farm parallelism pattern with a three-stage pipeline (source, multiple worker stages, final sink) to evaluate micro-batching and data stream frequency, test micro-batches with 12 and 24 replicas while varying micro-batch size from 1 to 5 items, observed results showing that Lane Detection and Bzip2 applications achieved higher throughput gained compared to latency increased while Person Recognition experiences higher latency increases, found that FastFlow achieved better throughput with 24 replicas but suffers from latency spikes at higher frequencies whereas TBB maintained lower latency during high frequency periods but took longer at the end of execution, and revealed that testing micro-batching configurations under different levels of parallelism demonstrates throughput gained up to 33% with TBB and 26.5% with FastFlow.

Concerning robust handling of dynamic data and real-time analysis capabilities, (Rettig, Khayati, Cudré-Mauroux, & Piórkowski, 2019) presented an online anomaly classification pipeline addressing challenges in classifying anomalies in streaming applications, making it relevant for micro-batch and stream processing, used relative entropy and Pearson correlation metrics to dynamically identified anomalies in large-scale telecommunication networks, ensured generality and scalability with Kafka and Spark Streaming, collected datasets using probes between

the Radio Access Network (RAN) and the Core Network (CN) to capture signalling traffic, processed these binary events through the Firehose, serialized them using Avro, pipeline them into Kafka queues for further processing, and stored them in the Hadoop Distributed File System for batch and real-time analysis, utilized Apache Spark on YARN to process network monitoring events through micro-batches, connected streaming receivers to Kafka for parallel event consumption enabling load balancing, highlighted the pipeline's ability to manage data frequency variations and maintained efficiency in micro-batch and stream processing scenarios, calculated deviations in event types using relative entropy while Pearson correlation evaluated user ID consistency across interfaces, and conducted experiments using real-world Big Data streams from Swisscom's network, demonstrating efficiency by focusing on events from human behaviour and IT infrastructure failures, effectively identifying significant changes in user behaviour patterns and detecting anomalies to provide insights for network monitoring and management.

2.10 ML, DL, and Ensemble Models

In the context of ensemble models, (Chowdhury, Barua, & Uddin, 2024) combined TabNet, XGB, and MLP for improved accuracy, preprocess data by removing duplicate records, handling missing values with mean or median imputation, and applying standardization, applied several ML models (KNN, NB, SVM, DT, RF, Logistic Regression) and DL models (TabNet and MLP), combined TabNet, XGB, and MLP in an ensemble model for enhanced prediction accuracy, used the Pima Indian Diabetes Dataset (768 records) and a larger diabetes dataset (2000 records) with 9 attributes each, achieved 90% accuracy on the Pima Indian Diabetes Dataset and 96% accuracy on the larger dataset, while combining LSTM and XGB for Human Activity Recognition (HAR), (Mekruksavanich, Jantawong, & Jitpattanakul, 2022) designed the LSTM-XGB architecture with LSTM layers for time-series feature extraction and an XGB classifier for classification, preprocess sensor data, used 5-fold cross-validation for model evaluation, pass extracted features from LSTM to the XGB classifier, compared LSTM-XGB with CNN and LSTM models, applied the UniMiB-SHAR dataset containing 11,771 instances of human activities and falls from 30 participants aged 18-60 years, collected data with a Samsung-I9250 smartphone using

a Bosh-BMA220 accelerometer at 50 Hz frequency covering 17 activity types (9 ADLs and 8 falls), and achieved 92.59% accuracy with LSTM-XGB.

Whereas (Khade, Vidhate, & Vidhate, 2023) developed an ensemble model combining FFNN and XGB for early prediction of Chronic Kidney Disease (CKD), implemented SMOTE for class balancing, used Recursive Feature Elimination (RFE) to select 13 out of 21 features, collected real-time data from DY Patil Hospitals, Navi Mumbai (350 records: 150 non-CKD and 200 CKD patients), used the UCI-CKD dataset with 24 attributes and 400 instances, pre-processed data through imputing missing values, encoding categorical data as binary, applying Z-score normalization, and using SMOTE for class balancing, designed the hybrid FFNN-XGB model with an input layer of 13 neurons, two hidden layers, and an output layer with two neurons, used FFNN for feature extraction followed by XGB for classification, optimized using the Adam optimizer and ReLU activation function, and achieved an accuracy of 94.60%.

In terms of handling tabular data, (Arik & Pfister 2021) introduced the DL architecture TabNet, which used sequential attention for instance-wise feature selection, enabling interpretability and efficient learning by focusing on the most salient features, matched state-of-the-art models on various classification and regression datasets while providing both local and global interpretability, employed sparse feature selection through a learnable mask to enhance interpretability and learning efficiency, utilized feature transformers with shared and decision step-dependent layers to ensure robust learning, applied masked self-supervised learning to predict masked features and improved representation learning, conduct optimization and training using the Adam optimizer with Glorot uniform initialization, softmax cross-entropy for classification, and mean squared error for regression, tested the model on six synthetic datasets and several real-world datasets including Forest Cover Type, Poker Hand, Sarcos, Higgs Boson, and Rossmann Store Sales, and achieved an accuracy of 96.99%.

For enhancing performance in tabular datasets, (Zhong, Li, Song, Qu, Wang, Wang, & Duan, 2022) applied FFNN using the Surface Ocean CO₂ Atlas (SOCAT) dataset version 2020, which incorporated 33 predictors including sea surface

temperature (SST), sea surface salinity (SSS), chlorophyll concentration, and atmospheric CO₂ mixing ratio. The Self-Organizing Map (SOM) method was used to divide the global ocean into 11 biogeochemical provinces to account for regional variability. The stepwise FFNN algorithm consisted of input, hidden (25 neurons with a tan-sigmoid function), summation, and output layers, with predictors added or removed one by one based on Mean Absolute Error (MAE) to select the optimal combination. A K-fold cross-validation method estimated performance, and neuron numbers were optimized to minimize MAE in each province. Model training was conducted using fixed random number streams in MATLAB to maintain consistent error space. The algorithm was executed twice per province: once with all predictors and once without chlorophyll-related predictors when such data was unavailable. Overall, the FFNN algorithm achieved a Mean Absolute Error (MAE) of 11.32 μatm and a Root Mean Square Error (RMSE) of 17.99 μatm .

2.11 Summarization of Literature Review

This chapter has provided an in-depth review of the literature related to spyware and malware dataset collection, spyware classification, and network traffic analysis using ML, DL, and ensemble models. The discussion includes different aspects of data collection methodologies, the challenges associated with imbalanced datasets, and feature extraction techniques for effective spyware classification. This section summarizes key findings and highlights their implications for the development of robust spyware classification.

One of the notable contributions of this review is the detailed examination of spyware dataset collection methodologies. Previous studies rely on static datasets from repositories such as VirusTotal or using PCAPDroid to capture spyware traffic. However, those approaches often lack real-time adaptability and fail to account for dynamic network traffic variations. This research extends the existing body of work by incorporating both batch and micro-batch processing, ensuring that models are trained on network behaviours. The application of learning techniques enhances the classification accuracy by utilizing feature engineering for network traffic analysis.

For collecting spyware samples based on network traffic analysis, it is essential to consider previous research in the field, addressing a gap in studies which primarily focused on APK-based methods. Unlike previous works that utilize APK files for malware and spyware collection, such as those by (Almomant et al., 2024; Dual et al., 2024; Hindarto et al., 2023; & Pieazzi et al., 2020) that study worked on the potential of network traffic. Notably, prior research highlighted the diverse approaches: Qabalin et al. (2022), examined Android spyware like Flexispy and Mobilespy, achieving moderate accuracy rates using DT, while Conti et al. (2020), focused on classification types like cImg and mSpy with RF, showing moderate performance. Meanwhile, Noetzold et al. (2023), explored workplace monitoring spyware, achieving significant accuracy by applying NB. On the malware, studies like Almomant et al. (2024), examined a range of Android malware, achieving high accuracy using SVM, and Brown et al. (2024), applied different datasets such as SOREL-20M and EMBER-2018 for malware classification, achieving accuracy 98%. Hindarto et al., (2023) similarly demonstrated strong performance with SVM in analyzing APK files. This research prioritizing network traffic analysis as the foundation for spyware collection, aiming to enhance classification methodologies.

Furthermore, this review shows the critical issue of class imbalance in spyware dataset collection. Several studies have demonstrated that traditional classification models struggle with imbalanced distributions, where the number of normal traffic samples significantly exceeds the spyware samples. To address this challenge, various balancing techniques such as SMOTE, RUS and rule-based feature extraction have been studied. The analysis shows that while SMOTE enhances classification performance, which introduces synthetic noise, thereby affecting generalization. Thus, a hybrid approach that combines both under-sampling and domain-specific feature engineering arises as a feasible solution.

Another crucial topic discusses in this chapter is feature engineering based on network traffic properties. Traditional feature extraction techniques, which focus on static attributes like API calls and permissions, often fail to capture real-time spyware behaviour. In contrast, network traffic-based feature engineering utilizes dynamic attributes such as flow duration, packet rate, and bandwidth consumption, providing a

more detail representation of spyware activity. This study has reviewed the effectiveness of network traffic properties; wherein packet flow analysis contribute to enhanced classification accuracy. This differentiation is particularly useful in identifying spyware that demonstrates stealthy data transmission patterns.

The chapter also highlights the application of ML and DL models for network traffic classification. Ensemble methods, particularly combinations of learning models such as CNN with LSTM have demonstrated better performance in distinguishing malware from normal traffic. The literature suggests that while ML models like RF performs well on structured datasets, they often lack adaptability to real-time classification tasks. On the other hand, DL models like CNN and LSTM shows better capability in capturing temporal dependencies in network traffic. The combination of these approaches within ensemble frameworks results in improved classification accuracy and robustness.

This research summarizes the results for several existing approaches for network traffic-based anomaly classification. It highlights the performance outcomes of different learning models as evaluated by various authors. Wei et al. (2021) introduced the HAST-NAD model, which has integrated CNN with LSTM. Their method has tested on the DARPA1998 and ISCX2012 datasets, achieving accuracy rates of 98.68% and 99.69%, respectively. Meanwhile, Dias et al. (2019), utilized a model built on the independence assumption for attribute analysis. Their approach has analysed a dataset comprising 551,095 packets sourced from YouTube, Netflix, and file downloads, yielding accuracy of 98.88%. These results underscore the efficacy of diverse methodologies in handling and analysing complex datasets.

A critical insight has gained from the review is the potential of real-time spyware classification through batch and micro-batch processing. Traditional batch processing methods, which has involved training models on pre-collected data, has been limited in their ability to classify spyware threats. Micro-batch processing, which has involved analysing smaller chunks of network packets in real-time, has offered a more responsive approach. The literature has indicated that an optimal micro-batch size enhanced the efficiency of real-time classification while maintaining high

accuracy. This finding has particularly relevant for scenarios where immediate threat classification has required to prevent data exfiltration by spyware.

The summarization of previous research highlights various approaches for real-time anomaly classification, emphasizing their methodologies, datasets, results, and considerations. Xu et al. (2023) implemented traffic segmentation applied with an LSTM-based anomaly classification mechanism on the ISCXIDS2012 dataset, achieving an accuracy of 98.40%. Their model was enabling faster anomaly classification while avoiding complex traffic analysis. Niu et al. (2022) introduced the IARF model, which dynamically updated with streaming data, tested on a mixed dataset including sources like malware-traffic-analysis.net, Lastline Inc, MCFP, and CTU-13, the method achieved 99.69% F1-score but in continuously training on real-time traffic data, which showed computational overhead.

Li et al. (2021) developed the ITSN model for anomaly network traffic classification, integrating CNN and LSTM with feature fusion, tested on ISCX-IDS 2012 and CICIDS 2017 datasets, achieving 99.94% accuracy and 99.88%, respectively. However, the model didn't design for unknown attack classification. Bendiab et al. (2020) explored image-based classification using ResNet50 combined with Binvis for PCAP-to-image conversion, evaluating on a custom dataset of 1,000 PCAP files, the approach reached an accuracy of 94.50%, but showed computational overhead due to its conversion requirements. Finally, Kim et al. (2020), applied an optimized CNN-LSTM model for real-time HTTP traffic, tested on datasets such as CSIC-2010 and CICIDS2017, achieving accuracy ranging from 91% to 93%., while the model supported continuous training on real-time traffic data, providing computational overhead and more execution time for real-time classification

These studies show applications in real-time and near real-time anomaly detection, while also revealing potential limitations, such as computational overhead and restrictions in handling unknown attack classifications.

The review further has discussed the rule-based classification approaches, which has discussed several learning methods by applying domain expertise. Rule-based feature selection, which has involved defining thresholds for parameters such as

packet rate and uplink-downlink traffic ratio, has enhanced transparency and reduced false positive rates. Several studies have successfully applied rule-based methods with ML models, leading to more better decision-making processes. This hybrid approach is particularly useful in real-world deployment scenarios which is a crucial requirement.

For analysing DNS queries and HTTP traffic patterns to detect malware via network logs, (Malik & Kaushal, 2016) introduced a method named CREDROID which was a semi-automated approach that worked on various factors like the remote server where the application was connecting, data being sent and the protocol being used for communication for identifying the trustworthiness of the application. However, their focus was on general malware, and they operated in offline mode. Qabalin et al. (2022) proposed a dataset for dynamic analysis using ML for spyware detection, focusing on five types of spyware. While their work provides a strong foundation, it does not integrate micro-batch and real-time evaluation across different ML models as presented in this study. A systematic literature analysis is executed to demonstrate the identified research gap for this research. Table 2.1 below summarizes the key studies in related fields, including their focus, methodology, insights and considerations. The analysis highlights the lack of focused research on real-time spyware classification, which this study aims to address through feature engineering based on directional properties, rule-based labelling, and the development of datasets for several learning models to enable real-time network traffic analysis. Unlike previous studies focused solely on offline spyware classification, this methodology represents a significant advancement by enhancing the classification capabilities of spyware in dynamic environments.

Table 2.1 Systematic literature analysis illustrating the insights and considerations.

Author	Focus	Methodology	Execution Platform	Strength	Insights and Considerations	Rule-Based Analysis	Static Analysis	Real-Time Analysis
Ahmed et al. (2021)	Online application traffic classification (NetScrapper)	Used ANN, RF, KNN on real-time user input	Network Interface Card and CICFlow Meter	Developed user interface	Limited to selected applications	No	No	Yes
Al-Musawi (2022)	Malware detection	Used grayscale PE files and API sequences	Not mentioned	Diversity in malware detection	Inefficient handling of null API call sequences in dynamic analysis	No	Yes	No
Alsumai dae et al. (2025)	Malware detection	Hybrid Deep Learning (CNN-LSTM for static and CNN-1D LSTM for dynamic)	Experiments run on Jupyter Notebook	Combine static and dynamic analysis	Computational overhead	No	Yes	Yes
Arjunan ,et al. (2024)	Anomaly detection	Applied CNN-LSTM on traffic data	Zeek (formerly Bro) network security monitor	Applied different datasets (NSL-KDD, CTU-13, ISCX-IDS)	Does not address classification challenges	No	Yes	No
Bdair,et al. (2019)	IPv6 network security	ICMPv6 DoS/DDoS detection	Snort	Offer improvements over IPv4	Signature-based IDSs like Snort cannot detect new or modified IPv6 attacks	No	Yes	No
Chanes h (2017)	Snort-based real-time monitoring	Manual PCAP inspection	Snort	Filtering network traffic	Snort is not applied for encrypted traffic	Yes	No	Yes

Table 2.1 Systematic literature analysis illustrating the insights and considerations, continued.

Author	Focus	Methodology	Execution Platform	Strength	Insights and Considerations	Rule-Based Analysis	Static Analysis	Real-Time Analysis
Conti et al. (2020)	Spyware classification	RF with SMOTE	Simulated corporate network and Wireshark for traffic analysis	Effective use of AS SAINT	Limited scope to only four spyware types	No	Yes	No
D'Angelo et al. (2021)	Rule-based malware classification	Association rules on API call sequence	Not mentioned	Robust against obfuscation technique	Limited to subsequent calls of API calls	Yes	Yes	No
Deepthi et al. (2024)	Real-time traffic processing (FRTSPS)	Combined batch and stream processing	Apache Flink	Low latency	Limited evaluation on diverse traffic types	No	No	Yes
Gandhi et al. (2022)	IoT botnet detection via batching	Used ML (LR, RF, SVM, XGB, DNN) on varying batch sizes	Edge devices and IoT network devices	Reducing system resource	Relay on batch size optimization	No	Yes	No
Huabin g et al. (2022)	Mobile anomaly detection (HAST-NAD)	Combined Bayesian classifier and spatio-temporal DL	Data capture under Linux operating system	Fast detection time	Limited to binary classification	No	Yes	Yes
Mohammed et al. (2009)	Flow-based traffic classification (NOFITC)	C4.5 algorithm with online preprocessing	NetFlow-based online traffic processing	Flow-based traffic classification and control system	Limited scalability for high-bandwidth environments	No	No	Yes
Noetzold et al. (2023)	Spyware and hate speech classification	NB-based monitoring system	Spring Boot Framework	Integrated spyware for workplace monitor	Limited in handling complex spyware behaviors	No	Yes	No

Table 2.1 Systematic literature analysis illustrating the insights and considerations, continued.

Author	Focus	Methodology	Execution Platform	Strength	Insights and Considerations	Rule-Based Analysis	Static Analysis	Real-Time Analysis
Kumar et al. (2024)	ARP spoofing detection	Used RF, CNN, LSTM, SVM and Isolation Forest on ARP traffic	Not mentioned	Applied several learning models	Limited to binary classification	No	Yes	No
Lockett (2021)	Signature-based classification	Applied binary/text pattern matching	SHA-256	YARA rules for malware-based binary patterns	Avoid traditional detection mechanisms	Yes	Yes	No
Qabalin et al. (2022)	Spyware classification	Collected traffic from 5 spyware apps; used DT	Android smartphone with PCAPdroid	Dataset availability	Does not explore various ML models	No	Yes	No
Sangher et al. (2024)	Hybrid malware detection	CNN with Adam optimizer	Windows 11	Reveal hidden patterns	May not handle emerging malware with unknown signatures effectively	No	Yes	No
Talabani et al. (2022)	Ransomware transaction detection	DT and PART on Bitcoin Heist data	Weka (Waikato Environment for Knowledge Analysis)	Effectively handling non-linear datasets	Does not generalize to broader malware types	No	Yes	No
Tirumala et al. (2020)	Feature-based malware classification	Autoencoders with feature extraction	Matlab 2019a in MacBook and Microsoft Surface Pro	Creating synthetic malware dataset based on signature	Limited scalability and relies on synthetic data.	No	Yes	No
Xu et al. (2023)	Anomaly detection	HAST-NAD + federated DL	Space-air-ground integration network	Collaborative federal learning	High communication costs	No	Yes	Yes

Table 2.1 Systematic literature analysis illustrating the insights and considerations, continued.

Author	Focus	Methodology	Execution Platform	Strength	Insights and Considerations	Rule-Based Analysis	Static Analysis	Real-Time Analysis
Yu et al. (2024)	Real-time anomaly detection using Renyi entropy	Used EWMA with selected IP and port features	Not mentioned	Within fixed time window set threshold dynamically	Focus on specific datasets; lacks generalizability across broader anomalies.	Yes	No	Yes

Finally, this chapter emphasizes the necessity for future research to address certain limitations observed in existing spyware classification methods. While some models have shown high classification accuracy, their ability to generalize across different network environments remains a challenge. Factors such as network topology, encryption protocols, and advancing spyware tactics introduce variability that may impact model performance.

In summary, this literature review has provided a detailed overview of spyware dataset collection, feature engineering, classification methodologies, and real-time classification approaches. The findings show the importance of hybrid models that combine several learning models and rule-based techniques for robust spyware classification. The exploration of batch and micro-batch processing to enhance real-time classification represents a significant advancement in the field.

CHAPTER 3

METHODOLOGY

3.1 Introduction

Chapter 3 outlines the methodology of this research work. The methodology is divided into 3 parts according to the objectives of the research. In first part it discusses about the data collection. In second part it discusses about the feature engineering, dataset development and batch processing. In third part it discusses about near real-time detection. Figure 3.1 illustrates the steps of this research.

3.2 First Part: Data Collection

Data is collected over four months (April-2024 to July-2024), during this time 14 different types of spyware network traffic, along with Normal Traffic, are gathered. The 14 types of spyware include Air-Droid, All Tracker Business Device Monitoring (AT-B-D-M), All Tracker Parent Control Monitoring (AT-P-C-M), All Tracker Communication Monitoring (AT-C-M), Eyezy, mLite, MoniMasterPro (MMP), mSpy, FamiSafe, FamiGuardPro (FGP), Qustodio, TiSpy, kids-360, and KidsGuardPro (KGP).

3.3 Second Part: Feature Engineering and Classification (Batch Processing)

After collecting network traffic, feature engineering is applied based on directional properties like IP address, Ports, and Protocols, and two datasets are developed: MAS-2024-A, which is based on single direction, and MAS-2024-B, which is based on bi-direction. Rule-based labelling is then applied to distinguish between spyware and Normal Traffic. Figure 3.2 illustrates the general steps for dataset collection

After datasets development, batch processing is applied. It splits the dataset as train-validation-test split ratios. For model performance, first it reserves 20% of the dataset as a test set to evaluate the final model's performance. From the remaining samples it splits several train-validation ratios. Batch processing is performed using

various ML, DL, and ensemble models. The models use for ML are eXtreme Gradient Boosting (XGB), Categorical Boosting (CatBoost), Light Gradient Boosting Machine (LGB), and Random Forest (RF), while those for DL are CNN, LSTM, FFNN, and Tabular Network (TabNet). Ensemble models such as CNN-LSTM, LSTM-XGB, FFNN-XGB, and TabNet-XGB various scenarios like oversampling, under-sampling, and selected sample numbers for each class. The best models are saved using Pickle.

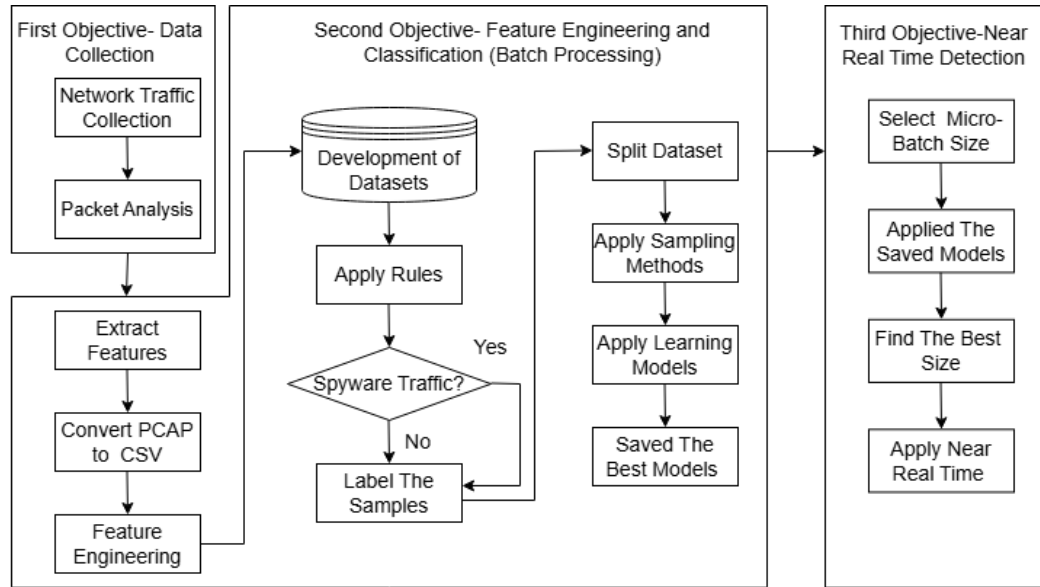


Figure 3.1 Methodology.

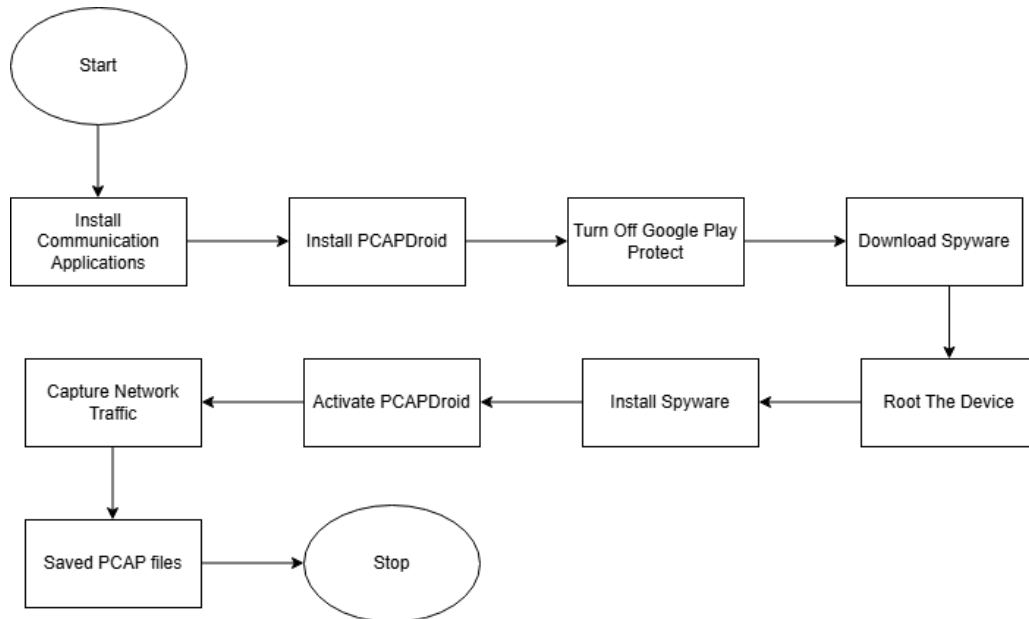


Figure 3.2 General steps in the data collection process.

3.4 Third Part: Near Real-Time Detection

Following batch processing, micro-batch processing is conducted before near real-time detection. It starts with 50 packets and increasing to 250 packets. The most appropriate batch size is determined, and near real-time detection is applied. During near real-time detection, the best micro-batch size is selected, and the saved models are applied to predict spyware classification. The models are not retrained during micro-batch and near real-time detection; instead, the saved models predict from new samples, which are sniffed directly in real-time.

Near real-time network traffic analysis is important because it allows dynamic monitoring and behavioural analysis, enabling the classification of malicious activities as they occur (Kunasaikaran, Ismail,& Ahmad, 2019). Until now, spyware classification has been done using static analysis (Shahzad, 2024), while some earlier works (Malik et al., 2016; Qabalin et al., 2022) have explored dynamic approaches such as network behaviour-based classification, often spyware detection systems still emphasize static methods. However, these earlier studies either focused on general malware (Malik et al., 2016) or did not specifically address real-time classification based on directional traffic features, Qabalin et al. (2022), as proposed in this work. Real-time classification is challenging due to the need for dynamic monitoring and behavioural analysis (Kunasaikaran et al., 2019), which require advanced tools to capture evolving threat patterns, significant computational resources to process data, and robust algorithms to differentiate between benign and malicious behaviours (Aboaoja et al., 2022). This study significantly contributes to methodology for near real-time spyware detection by integrating feature engineering based on directional properties, rule-based labelling, and the development of datasets for various ML, DL, and ensemble models.

3.5 Conclusions

This Chapter provided a structured outline for the methodology of the processes conducted to achieve the research objectives. It was segmented into three main components: dataset collection; feature engineering, dataset development, batch processing; and near real-time detection. Each section briefly introduced the respective

methodologies that would be elaborated upon in following chapters. The dataset collection phase focused on collecting network traffic data and performing feature engineering to develop two datasets, MAS-2024-A and MAS-2024-B for spyware classification. Batch processing involved the application several learning models to evaluate and enhanced model performance under various scenarios. Finally, near real-time detection addressed the dynamic classification of spyware using saved models, bridging the gap between offline and near real-time behavioural analyses.

This chapter illustrated as an overview of the methodological approach, with detailed discussions and analyses presents in Chapters 4 through 8. These Chapters expand upon each step, providing deeper insights into the methodologies, results, and their implications.

CHAPTER 4

DATASET COLLECTION

4.1 Introduction

Chapter 4 provides a details description of the data collection method for spyware classification. Understanding the spyware behaviour and sniffing mechanisms is crucial for effective data collection and analysis. This chapter discusses the primary steps required before data collection, ensuring a robust and accurate dataset structure. It discusses the configuration of the root device, and the strategic linking with attacker devices to capture spyware traffic. Additionally, the chapter discusses 14 different types of spyware with Normal Traffic along with detailed descriptions of the PCAP files for network traffic analysis.

4.2 Dataset Information

This Section will present the dataset information about data source, data class, data format, including the title, data type, targeted applications, and other relevant parameters that are necessary for this research. By using network traffic analysis 14 types of android spyware samples are collected. All models' training and evaluation are conducted on a machine running Windows 10 Pro Education with an AMD Ryzen 5 7600X 6-Core processor and 64 GB RAM. Network traffic collection is performed on devices running Android 14 using PCAPdroid. For data collection and install the spyware; the smartphone has rooted manually, each spyware application has activated with its full features.

Table 4.1 Dataset information.

Parameter	Details
Dataset Title	MMU Android Spyware-2024-A(MAS-2024-A), MMU Android Spyware-2024-B (MAS-2024-B)
Dataset Type	MAS-2024-A (Single Directional), MAS-2024-B (Bidirectional)
Data Format	PCAP files

Table 4.1 Dataset information, continued.

Parameter	Details
Data Source	Android Spyware
Spyware Types	FGP, KGP, MMP, mSpy, TiSpy, Qustodio, Air-Droid, kids-360, mLite, Eyezy, FamiSafe, AT-P-C, AT-B-D-M, AT-C-M.
Number of Files	1,793
Total Data Size	625.9 GB
Collection Strategy	Time-bounded and size-limited sequential
Data Scope	Open Systems Interconnection (OSI) layers 2-7; data link, network, transport, session, presentation and application
Deployment Approach	Hybrid, host-based and network-based
Time Constraints	Unified time interval
Number of Classes	15 (14 types of spyware and 1 Normal Traffic class)
Data Collection Tool	PCAPDroid
Data Transfer Tool	USB Drive
Data Read Tool	Wireshark
Data Analysis Tool	Wireshark, Jupyter Notebook
Data Preparation Tool	Jupyter Notebook

4.3 Data Collection Method

This Section will explain the process for data collection. It shows the details about the characteristics of 14 types of spyware, such as their permissions, structure, impact on the victim's device, and file structure with the instruction to root the device manually before install spyware. This Section will also discuss the pre-defined rules to be followed before installing spyware on a victim's device.

4.3.1 Root the Device

Before installing the spyware, it needs to be rooted the device. Without rooting, the device won't be able to install spyware. Figure 4.1 illustrates the steps to root the device. Additionally, before installing spyware, Google Play Protect must be deactivated. If Google Play Protect remains active, the system will display a security warning and block the installation of spyware. During installation, the following permissions must be activated: message tracker, location access, call tracking, social

media tracking, background data usage, live streaming, auto screen logger, key logger, auto voice recorder, and auto camera access. Without these permissions, attacker will not be able to detect the victim's device.

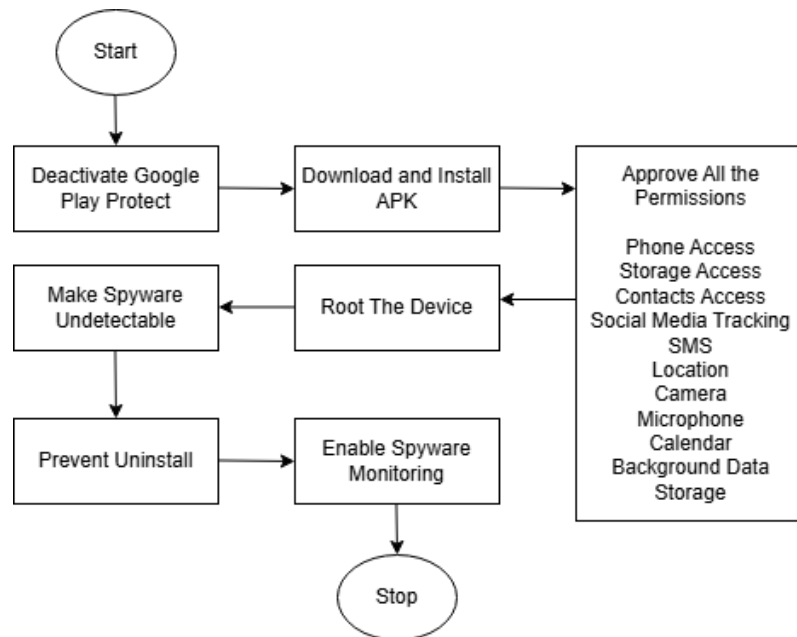


Figure 4.1 Root the device.

4.3.2 How the Attacker Monitors the Victim's Device

Once the device is successfully rooted, spyware such as mSpy, FlexiSpy, AT-B-D-M are installed to simulate real-world attacker behaviour. These applications are configured using their official web portals to remotely monitor a range of victim activities including SMS messages, call logs, GPS location, app usage, and clipboard data. Figure 4.2 illustrates the attacker's monitoring interface. The attacker can remotely access sensitive information and even issue commands without the victim's knowledge. This simulation mirrors actual spyware deployment scenarios, enabling the capture of realistic traffic patterns. The attacker's perspective is validated by cross-checking dashboard logs and PCAP files to ensure successful data exfiltration is reflected in the network traces. This process is repeated for each spyware type, with only one spyware running per session to avoid interference of traffic patterns.

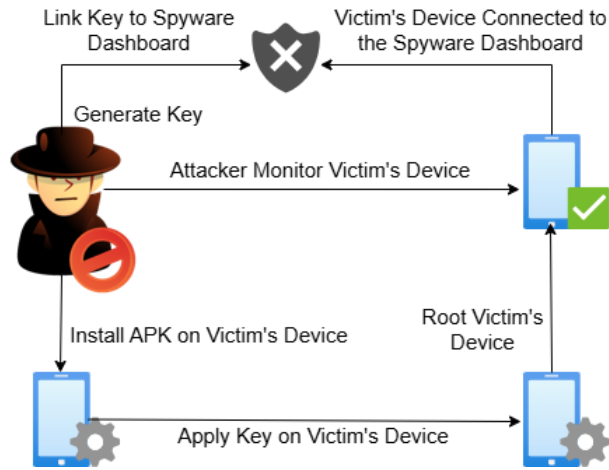


Figure 4.2 Attacker monitors victim's device.

4.3.3 Important Information During Spyware Network Traffic Capture

This research collects 14 types of spyware along with Normal Traffic over a four-month period (April 2024 to July 2024). For each data collection session, a single type of spyware is installed on the device. Before installing spyware, the device is rooted, as this step was necessary for the installation process. Specific permissions such as message tracking, location access, call tracking, and social media monitoring etc. are activated to enable the spyware to function. Without these permissions, the spyware will not be able to gather information or transmit it to the attacker's device. Similarly, installing spyware is impossible without first rooting the device.

Spyware APK files are used for installation. The attacker must turn off Google Play Protect on the victim's device, as this feature blocks the installation of APK files. Once Google Play Protect is disabled, the attacker can install the spyware APK file.

Data collection was carried out for each spyware type over a five-day period, with 15 hours of data captures daily. However, continuous packet capture (PCAP) was not performed throughout the 15 hours. Instead, PCAPDroid was used to capture traffic in 30-minute intervals. After each 30-minute session, the application was stopped, the PCAP file then save on the device, and PCAPDroid restarted to begin a new session. If the file size reached 800 MB before the 30-minute duration, the capture

was stopped, the file was saved, and the application was restarted to maintain manageable file sizes. During the network traffic capture process, the file size was limited to a maximum of 800 MB within each 30-minute interval. If the size remained below this limit, packets were collected for the entire 30-minute session. Figures 4.6 and 4.7 illustrate the packet distribution and file sizes corresponding to different spyware classes.

4.3.4 PCAP File Description

After collecting the PCAP files, analyse them through Wireshark. The Packet Summary Pane (PSP) provides information about each packet during the analysis of PCAP files. PSP includes details such as No, Time, Source, Destination, Protocol, Length, and Info. Figure 4.3 illustrates the PSP of the PCAP file.

In a PCAP file, each row represents a single packet. 420,303,217 packets were collected for this research. The 'No' column represents the serial number for each packet. The 'Time' column represents the duration for the packet's transit from source to destination. The 'Source' and 'Destination' columns represent the IP addresses. The 'Protocol' column indicates the application layer protocol. The 'Length' column shows the packet length. The 'Info' column provides a brief, human-readable description of the packet contents. This summary varies depending on the type of protocol and the traffic patterns. For example, for Transmission Control Protocol (TCP), the 'Info' column shows information like SYN, ACK, FIN, or RST, indicating the TCP connection state. For User Datagram Protocol (UDP), the 'Info' column provides the source and destination ports involved in the communication. For Domain Name System (DNS), the 'Info' column represents the DNS query along with the domain name being resolved. To access detailed information for each packet, it needs to double-click on the packet, which will display details such as Ports, IP Addresses, Transport Layer Protocol, Application Layer Protocol, Frame, Timestamps, Flags, Checksum information, and Sequence Analysis. This information varies depending on the application layer protocol.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.215.173.1	163.70.137.15	TCP	60	41094 → 41094
2	0.015826	10.215.173.1	10.215.173.2	DNS	68	Standard query
3	0.023866	10.215.173.1	10.215.173.2	DNS	60	Standard query
4	0.030460	163.70.137.15	10.215.173.1	TCP	48	443 → 41094
5	0.030617	10.215.173.2	10.215.173.1	DNS	108	Standard query
6	0.030854	10.215.173.1	163.70.137.15	TCP	40	41094 → 41094
7	0.033521	10.215.173.1	163.70.137.15	TLSv1.3	422	Client Hello
8	0.033776	163.70.137.15	10.215.173.1	TCP	40	443 → 41094
9	0.038656	10.215.173.1	163.70.137.15	TLSv1.3	1049	Change Cipher
10	0.038801	163.70.137.15	10.215.173.1	TCP	40	443 → 41094

Figure 4.3 PSP

4.4 Ethical Considerations and Privacy Assurance

This research was exclusively focused on the detection of spyware activity on Android devices through the analysis of directional network traffic features. The study did not aim to monitor or interpret the personal behaviour, habits, or communication of device users. No human behaviour or individual phone activity—such as call content, chat messages, or app usage patterns—was examined or analysed. Instead, only technical network characteristics (e.g., packet size, packet rate, bandwidth, flow duration, etc.) were processed to identify unusual patterns associated with spyware. All network traffic, including both “Normal Traffic” and “Victim” scenarios, was entirely simulated by the researcher in a controlled laboratory environment. The “victim” role was represented by the researcher using dedicated test devices configured with preinstalled applications and spyware samples. The experiment reproduced attacker-victim behaviour solely for network traffic analysis. No real end-users’ personal data or private communications were recorded. All data used in the research were anonymized and analysed solely to detect and classify spyware using various learning models, including Machine Learning, Deep Learning, and Ensemble techniques. To ensure complete anonymization, all captured data underwent additional processing to remove MAC addresses, timestamps, device IDs, and any residual application identifiers. Only statistical network features were retained for analysis. All recorded data was pseudo-anonymised to eliminate any personally identifiable information (PII). Furthermore, informed consent was obtained from four individuals whose communication metadata may have been indirectly captured during traffic logging. This study complies fully with the ethical guidelines and data protection

policies of Multimedia University (MMU), Cyberjaya, Malaysia, and ethical clearance is applied from the university's Research Management Centre.

4.5 Methodology for Data Collection

For collecting data to maintain nine key factors: quality of data labelling, consistency of data collection process, realistic network condition, data format quality, data volume quality, data rationality quality, data diversity quality, data documentation quality and data eligibility for ML quality have been adopted. A networking technique known as packet capture involves intercepting data packets across Open Systems Interconnection (OSI) layers (Qabalin et al., 2022). In this research, data is collected from the data link, network, transport, session, presentation, and application layers (Qabalin et al., 2022). The data collection methodology is divided into two main sections based on Spyware and Normal Traffic. The sniffing process continuously monitors and captures all data packets traversing a network or device. This research monitors network traffic, to build detection models by analysing sniffed data using ML. A dynamic deployment approach is adopted in the sniffing process, which collects network traffic using PCAPDroid. Figure 4.6 will illustrate the packet distribution after capturing the network traffic.

4.5.1 Day-Wise Data Collection Approach

To ensure data integrity, this research maintains consistent time intervals and engages in identical activities across all spyware types. The data has collected over a period of four months. For each type of spyware with Normal Traffic, the data collection process takes 5 days for each type. During each day, 15 hours are dedicated to the task, resulting in a total of 75 hours applies for collecting data on each class of spyware. Table 4.2 lists the activities that are carried out during the data collection process and Table 4.3 lists the detailed description for each sniffing activity for spyware.

Table 4.2 Activity list during data collection.

Day	7.00-10.00	10.00-13.00	14:00-17:00	17:00-20:00	20:00-23:00
Day-1	Facebook & Messenger	WhatsApp	Gmail, Outlook & OneDrive	YouTube	Search Engine
Day-2	Facebook & Messenger	WhatsApp	Gmail, Outlook & OneDrive	YouTube	Search Engine
Day-3	Facebook & Messenger	WhatsApp	Gmail, Outlook & OneDrive	YouTube	Search Engine
Day-4	Facebook & Messenger	WhatsApp	Gmail, Outlook & OneDrive	YouTube	Search Engine
Day-5	Facebook & Messenger	WhatsApp	Gmail, Outlook & OneDrive	YouTube	Search Engine

Table 4.3 Sniffing description for each activity.

Activity	Activity Type	Sniffing Description
System Unlock	Operating System Security	Collection PIN through key logger
WhatsApp, Messenger, Gmail, Outlook, One Drive	Exchange images, files, videos, links, audios, messages.	Simulate conversation, monitor data exchange and collect passwords through key logger during spying.
Facebook, Search Engine	Visit the pages, download PDF files	Monitor activities and auto screen capture during spying.
YouTube	Download videos, watch videos	Monitor activities and auto screen capture during spying.
Using encrypted end-to-end calls through messaging apps (WhatsApp, Messenger)	Audio and video calls.	Audio and video recording, auto screen capture during spying

4.5.2 Spyware Data Collection Method

Figure 4.4 represents the installation and data collection process for spyware operating on smartphone. Each time the device undergoes factory reset before each spyware installation. Then the process involves installing PCAPDroid and other applications (WhatsApp, Messenger, Facebook, YouTube, Gmail, Outlook, One Drive), then downloading the spyware package, installing the spyware, and linking it to the attacker's device (discussed on Section 4.3). After this, the attacker monitors the targeted device. Next, PCAPDroid is activated, and for 5 days, the listed activities (Tables 4.2 and 4.3) are carried out on the devices to collect PCAP files for network

traffic analysis. The installation mechanisms for all the spyware targets in this research are identical in terms of procedures, and these procedures are followed accurately as described by the manufacturers. The essential steps to install and activate these spywares have explained in Sections 4.3.1, 4.3.2 and 4.3.3.

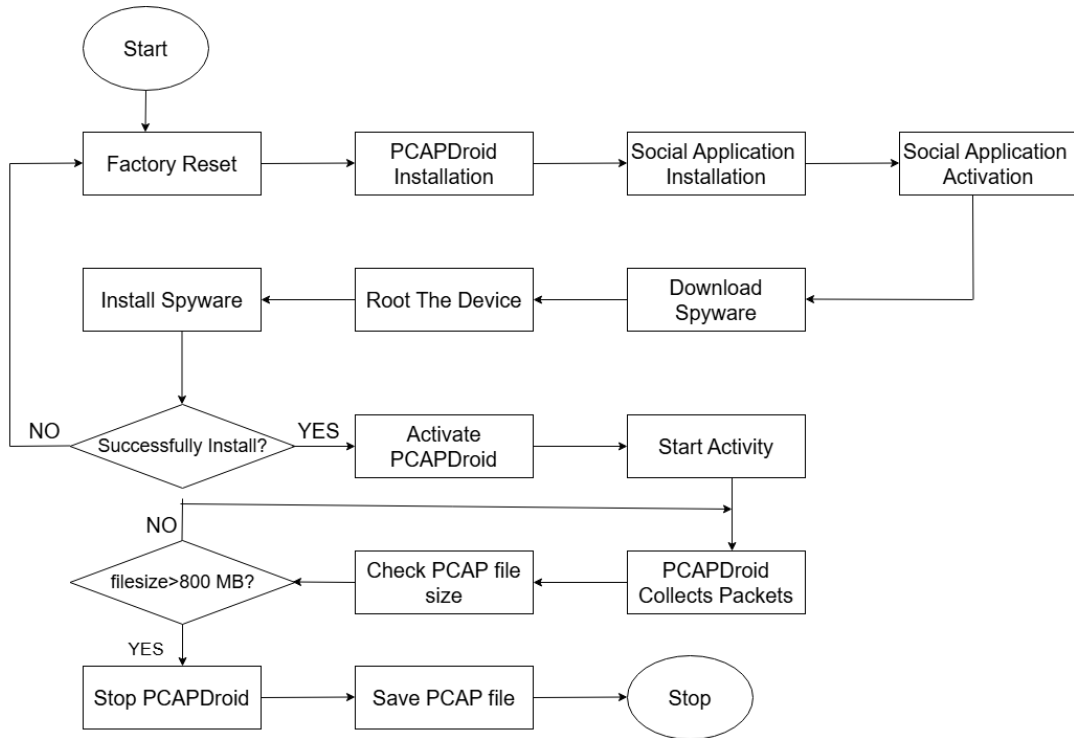


Figure 4.4 Spyware data collection workflow.

4.5.3 Normal Traffic Data Collection Method

Figure 4.5 represents the process, which begins with a format reset of the device to ensure no other applications are installed. Then PCAPDroid and other social applications listed in Table 4.3 are installed. All the social applications are then activated, and the verification process is completed. Afterward, PCAPDroid is activated, and records network traffic of social activities for 5 days, with 15 hours per day, totalling 75 hours of normal traffic collection. This research collects 41,236,958 packets, with a total file size of 33.3 GB. The purpose of this step is to collect clean network traffic without any malicious traffic.

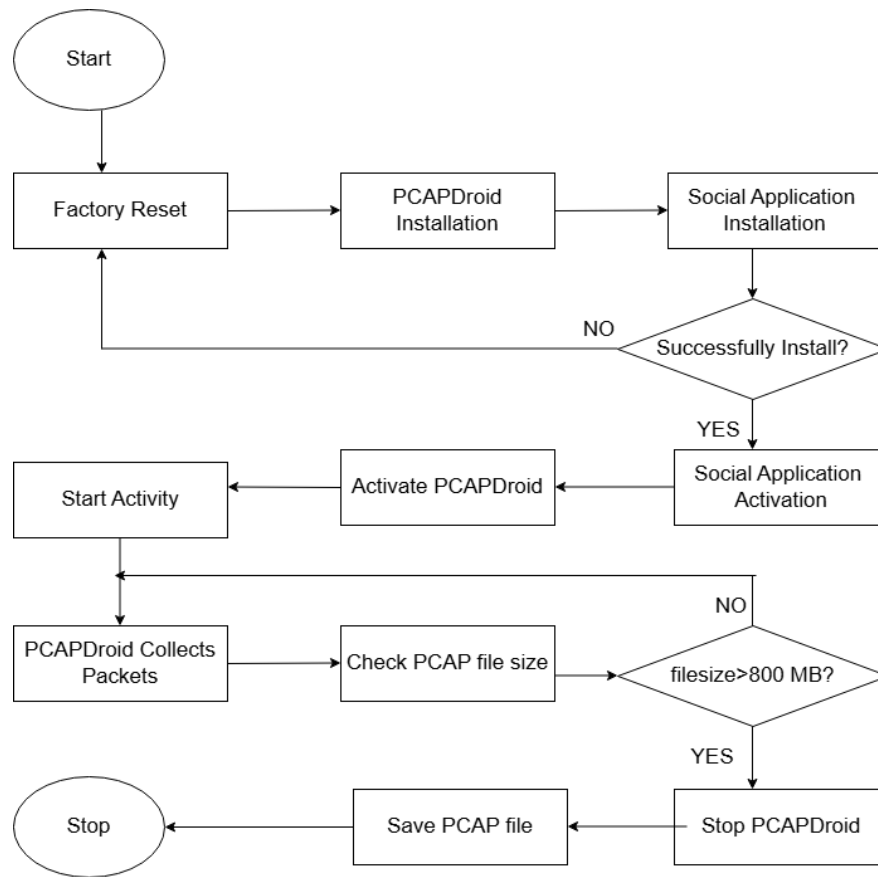


Figure 4.5 Normal Traffic data collection workflow.

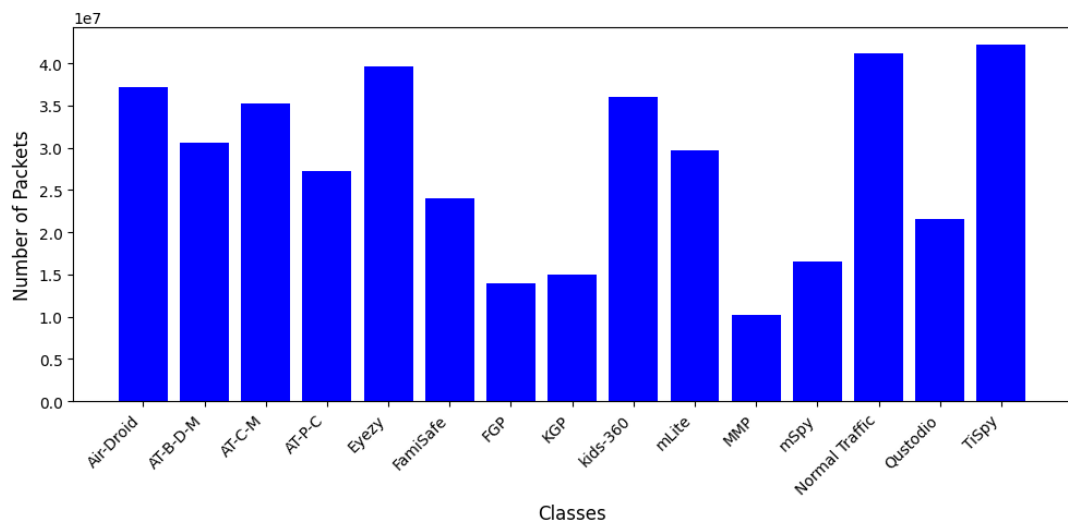


Figure 4.6 Packet distribution for each class.

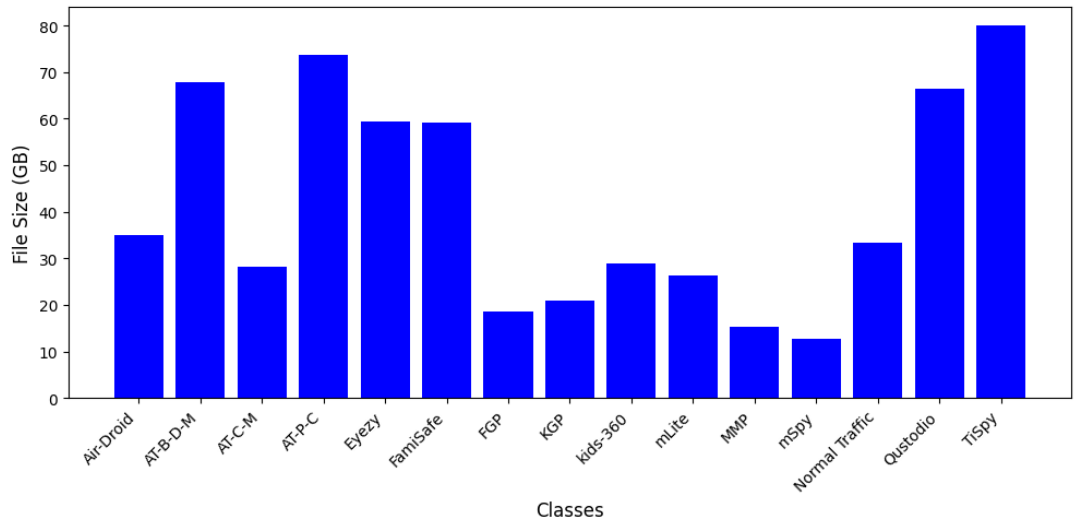


Figure 4.7 File size for each class.

4.5.4 Variability in Packet Numbers and File Sizes Across Spyware Classes and Normal Traffic

The variability in packet numbers and file sizes across the 14 spyware classes and the Normal Traffic arises due to differences in the design and operation of the spyware compared to normal network traffic (Mannan et al., 2023). Spyware like AT-B-D-M, AT-P-C, Eyezy, FamiSafe,, Qustodio, and TiSpy; which demonstrate large file sizes and a significant number of packets, likely due to aggressively capturing details data, such as multimedia files. Such spyware also collects extensive metadata and content, inflating both packet count and file size capture large amounts of metadata and actual content, contributing to both packet inflation and increased file sizes (Mannan et al., 2023). The design focus of these spyware is typically on thorough monitoring, leading to higher data capture rates (Mangeard et al., 2024; Mannam et al.,2023; & Peng et al., 2024). On the other hand, spyware such as KGP, FGP, MMP, and mSpy generate smaller file sizes and packet counts due to their more targeted data collection strategies, that prioritize certain types of data or use advanced compression techniques to minimize storage requirements (Ong, 2023; FamiGuard, 2024; MSpy Team,2024; Wikipedia contributors: KidGuard, 2024 & Wireshark, 2024).

Spyware sniffing modes significantly impact data volume. Every spyware is reactive to specific triggers. Spyware like TiSpy, Qustodio, Eyezy etc which capture

data on every possible trigger, naturally produce a higher packet count compared to spyware like mSpy, KGP, MMP, and FGP which might only capture periodically (Peng et al., 2024).

Moreover, the variability between spyware classes and Normal Traffic arises from the absence of sniffing in Normal Traffic. Normal Traffic reflects unfiltered network activity, where packets follow standard protocols without interference from spyware (IETF, 2008). This results in file sizes and packet counts that accurately represent the operation of applications like YouTube and messaging platforms. Unlike spyware-influenced data, Normal Traffic maintains natural data flow, with packets neither intercepted nor bundled by third-party tools, ensuring consistent packet sizes and counts (IETF, 2008).

Lastly, encryption overhead also explains the differences. Spyware may encrypt data during collection, thereby increasing file sizes without altering the packet count (Peng et al., 2024). This is evident in the imbalanced file size with packet counts of AT-P-C, which likely encrypts captured content extensively, compared to Normal Traffic, where encryption protocols such as TLS are lightweight and optimized for efficiency (IETF, 2008).

4.6 Conclusions

Chapter 4 presented a detailed approach to spyware dataset collection, ensuring a structured methodology for capturing network traffic data. The study emphasized the importance of understanding spyware behaviour and its sniffing mechanisms to build representative datasets. A key aspect of the methodology involved manually rooting the device before spyware installation, ensuring that all necessary permissions were activated to real-world spyware activities. The dataset included 14 types of spyware and Normal Traffic, with a total of 420 million packets with 625.9 GB of data.

The data collection strategy was designed to balance time constraints by applying a sequential time-bounded and size-limited approach. PCAPDroid was used for data collection, while Wireshark and Jupyter Notebook facilitated data analysis and preparation. The dataset structure incorporated OSI layers 2-7, capturing vital network

parameters that contributed to spyware classification. Additionally, the study ensured that data transfer was securely handled using a USB drive.

A fundamental component of the data collection process was the establishment of a secure connection between the victim's device and the attacker's monitoring device. The study showed the permissions required for spyware to function effectively, including message tracking, call tracking, location access, background data monitoring, auto-screen logging etc. These permissions were critical in simulating real-world spyware operations, ensuring that the dataset accurately captured spyware behaviour under different conditions. Furthermore, to maintain consistency and reduce external interference, Google Play Protect was disabled before installing spyware APK files.

The research applied a structured schedule for data collection, conducting 15-hour daily monitoring sessions over a five-day period for each spyware type. Network traffic was captured in 30-minute intervals, with a maximum file size limit of 800 MB per session. This approach facilitated efficient storage management while maintaining data integrity.

Overall, the dataset collection methodology in Chapter 4 established a robust foundation for subsequent feature engineering and classification tasks. The structured approach ensured high-quality data acquisition, allowing for further analysis and model training in later chapters. The collected data provided a unique and large resource for researchers working on spyware classification, addressing critical gaps in existing datasets.

CHAPTER 5

FEATURE ENGINEERING AND DATASET DEVELOPMENT

5.1 Introduction

To ensure that the collected PCAP files are suitable for spyware classification, it's crucial to convert these files into a format that can be easily validate through ML models. This section discusses data preprocessing, feature engineering, and datasets development.

5.2 Data Preprocessing

After collecting the PCAP files to make it readable and eligible for validation through ML models, it needs to be converted into a CSV file. Before converting to a CSV file, it is necessary to understand the PCAP file with each packet's information, which contains network traffic information, as discussed in sub chapter 4.3.4. After converted into CSV, where each packet's attributes are organized into rows and columns. This conversion enables the curation of key features, such as Bandwidth, Packet Rate, and Flow Bytes/s, required for further analysis. By transforming the data into a readable format, it ensures that the dataset is prepared for analysis, facilitating the development of robust models for spyware classification. To convert the PCAP file to a CSV file, it is necessary to identify the important static features that need to be collected from the PCAP file. The major static features collect from the PCAP file are Source IP (SIP), Destination IP (DIP), Source Port (SP), Destination Port (DP), Protocol (P), Packet Length (PL) and Flow Duration (FD). Algorithm 5.1 shows the steps to convert the PCAP file to a CSV file with these important static features.

Algorithm 5.1 Conversion of PCAP to CSV file.

1. pcap = read(open(pcap_file))
 2. Require: SIP, DIP, SP, DP , PL, FD
 3. for packet in pcap do
-

Algorithm 5.1 Conversion of PCAP to CSV file, continued.

```
4.  try
5.    SIP = packet.ip.src
6.    DIP = packet.ip.dst
7.    SP = packet[transport_layer].srcport
8.    DP = packet[transport_layer].dstport
9.    P = packet.transport_layer
10.   PL = packet._length
11.   FD = flow_duration
12.   Append [SIP, DIP, SP, DP, P, PL, FD] to data list
13.   except if AttributeError occurs:
14.     continue to the next packet
15.  end try
16. end for
17. function writeCsv(data, outputFile):
18.  processedData = convert_pcap_to_csv(pcapFile)
19.  write processedData to outputFile
20. writeCsv(data, output_csv_file)
```

5.3 Feature Engineering

After converting PCAP files to CSV and collecting the static features, dynamic features are then curated from those static features. Table 5.1 shows the list of these dynamic features, which are calculated based on the directional properties. From Features 1 to 17 represent the calculation for each dynamic features. Here FD represents flow duration, T_c for current time and T_e for end time within incoming or outgoing flow. FP, BP, F_L , and B_L represent total forward (Fwd) packet (Pkt), total Backward (Bwd) Pkt, total Fwd Pkt Length (Len) and total Bwd Pkt Len respectively. Features 16 and 17 represent the calculations for Bandwidth and Packet Rate, which are based on the collection duration for each PCAP file. Here, pl represents Pkt Len, t represents the duration for capturing the PCAP file, pl_i represents the Length of the i -th packet in the PCAP file, pt represents the number of packets in the PCAP file, and pt_i represents the index of the i -th packet in the PCAP file. After feature engineering next this research develops 2 datasets.

Table 5.1 Dynamic features.

No	Name	Representation
1	Flow Duration	$FD = T_e - T_c$
2	Total Fwd Pkt	$FP = \sum_{i=1}^n FP$
3	Total Bwd Pkt	$BP = \sum_{i=1}^n BP$
4	Total Fwd Len	$F_L = \sum_{i=1}^n F_L$
5	Total Bwd Len	$B_L = \sum_{i=1}^n B_L$
6	Min Fwd Len	Min_{F_L}
7	Max Fwd Len	Max_{F_L}
8	Mean Fwd Len	$\mu(F_L)$
9	Std Fwd Len	$\sigma(F_L)$
10	Min Bwd Len	Min_{B_L}
11	Max Bwd Len	Max_{B_L}
12	Mean Bwd Len	$\mu(B_L)$
13	Std Bwd Len	$\sigma(B_L)$
14	Flow Bytes/s	$FB_s = \frac{\sum_{i=1}^n (F_L + B_L)}{FD}$
15	Flow Pkt/s	$F P_s = \frac{\sum_{i=1}^n (FP + BP)}{FD}$
16	Bandwidth	$B = \frac{\sum_{i=1}^n pl_i}{t}$
17	Packet Rate	$R = \frac{\sum_{i=1}^n pt_i}{t}$

5.4 Dataset Development and Labelling

During observing directional properties, and developing dynamic features, on the base of incoming and outgoing properties, two datasets are developed. This sub-chapter will discuss about the method.

5.4.1 Development of MAS-2024-A

A single-directional packet flow is applied to develop MAS-2024-A. It examines the IP pairs, Ports and Protocol for each row. When the SIP, DIP, SP, DP and P remain same across two or more consecutive rows, and dynamic features

develop which discussed in Section 5.3. The features are already presented in Table 5.1. These consecutive rows are combined into a single group, which represents a single-directional packet flow. However, if any of the directional properties changed for consecutive rows, they cannot be considered part of the same group. They will be considered as part of a different packet flow. Algorithm 5.2. illustrates the steps for developing MAS-2024-A.

Algorithm 5.2 Development of MAS-2024-A.	
1.	csv = read (open (csv file))
2.	Require: SIP, DIP, SP, DP, P, PL, previous row (pr), next row (nr)
3.	for row number in row do
4.	if (pr.IP pairs == nr.IP pairs && pr.port == nr.port && pr.P == nr. P) then
5.	calculate feature value
6.	else
7.	move to the next row
8.	end if
9.	end for
10.	function writeCsv(data, outputFile):
11.	processedData = processCsv(csvfile)
12.	writeCsv(processedData, outputFile)

5.4.2 Development of MAS-2024-B

For MAS-2024-B, the process is like MAS-2024-A (Section 5.4.1) apart from utilizing a bi-directional packet flow instead of a single-directional packet flow. Figure 5.1 illustrates this bi-directional flow. The SIP in Row-1 and Row-2 becomes the DIP in Row-3 and Row-4. This same process occurs in Row-5 too. A similar process occurs for the DIPs as well. Algorithm 5.3. shows the steps for development of MAS-2024-B.

Algorithm 5.3 Development of MAS-2024-B.	
1.	pcap = read (open (csv file))
2.	Require: SIP, DIP, SP, DP, P, PL, pr, nr
3.	for row number in row do

```

4.   if (pr.SIP || pr.DIP == nr.SIP || nr.DIP && pr.SP || pr.DP == nr.SP || nr.DP && pr.P =
      = nr.P) then
5.       calculate feature value
6.   else
7.       move to the next row
8.   end if
9. end for
10. function writeCsv(data, outputFile):
11. processedData = processCsv(csvfile)
12. writeCsv(processedData, outputFile)

```

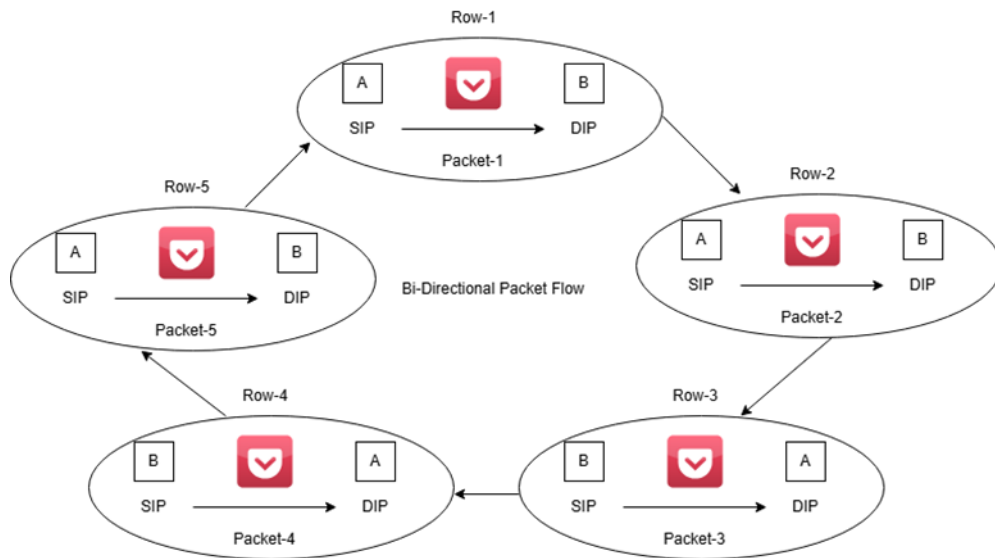


Figure 5.1 Bi-directional packet flow.

5.4.3 Comparisons of Datasets

After developing MAS-2024-A and MAS-2024-B, both datasets have same features, but all the features' values are different. Table 5.2 presents the differences between MAS-2024-A and MAS-2024-B. Tables 5.3 and 5.4 represents some features values which represents that though the features are all same, but the features values are different. Figures 5.2 and 5.3 illustrates the class distribution for both datasets.

Table 5.2 Differences between MAS-2024-A and MAS-2024-B.

Scenario	MAS-2024-A	MAS-2024-B
Method	Single Direction- Incoming or Outgoing Packet	Bi Direction- Incoming and Outgoing Packet
Samples	4,217,799	2,041,082
File Size	617.84 KB	331.44 KB

Table 5.3 Sample rows for some features for MAS-2024-A.

SIP	DIP	Protocol	Total	Total	Total	Total	Mean	Std	Mean	Std
			Fwd	Bwd	Fwd	Bwd	Fwd	Fwd	Bwd	Bwd
			Pkt	Pkt	Len	Len	Len	Len	Len	Len
10.215.173.1	163.70.137.35	TLSv1.3	2	0	389	0	194.50	111.02	0.00	0.00
192.168.0.2	10.215.173.1	DTLSv1.2	0	2	0	641	0.00	0.00	320.50	358.50
10.215.173.1	142.251.222.227	TCP	4	0	692	0	173.00	252.84	0.00	0.00
216.58.199.238	10.215.173.1	TLSv1.2	0	4	0	2078	0.00	0.00	519.50	377.86
10.215.173.1	142.251.223.74	TLSv1.3	8	0	1360	0	170.00	236.39	0.00	0.00
172.217.166.131	10.215.173.1	QUIC	0	8	0	6338	0.00	0.00	792.25	568.65

Table 5.4 Sample rows for some features for MAS-2024-B.

SIP	DIP	Protocol	Total	Total	Total	Total	Mean	Std Fwd	Mean	Std Bwd
			Fwd	Bwd	Fwd	Bwd	Fwd	Len	Bwd	Len
			Pkt	Pkt	Len	Len	Len		Len	
10.215.173.1	216.58.199.238	TLSv1.3	2	2	167	1438	83.50	43.13	719.00	417.19
10.215.173.1	75.2.2.223	TCP	2	2	100	88	50.00	14.14	44.00	5.66
10.215.173.1	157.240.236.34	TLSv1.3	4	3	1115	559	278.75	241.84	186.33	108.31
10.215.173.1	104.17.171.101	TCP	8	5	852	208	106.50	180.14	41.60	3.58
10.215.173.1	142.250.199.2	QUIC	8	8	3191	3141	398.87	544.63	392.63	451.54
10.215.173.1	163.70.137.34	TCP	4	5	180	208	45.00	10.00	41.60	3.58

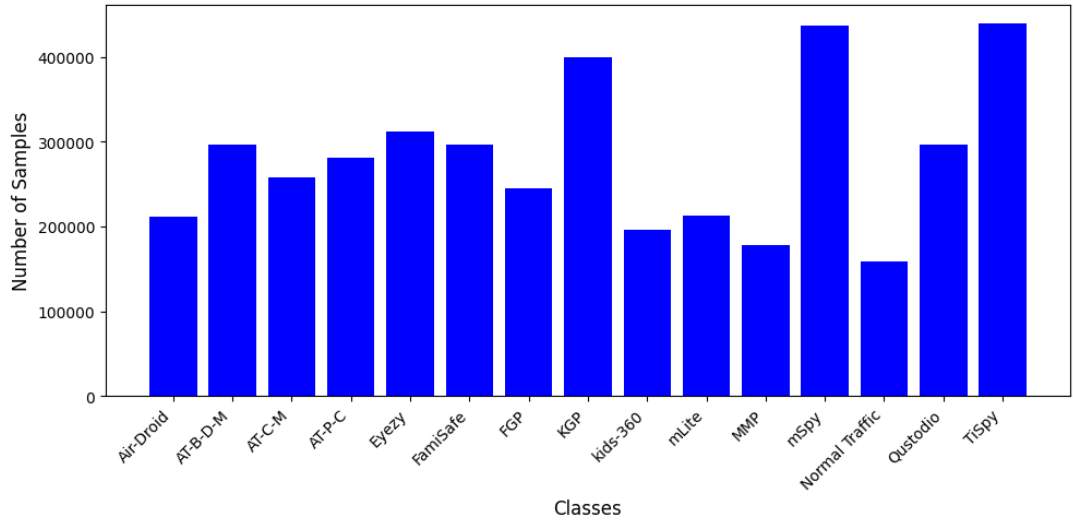


Figure 5.2 Class distribution for MAS-2024-A.

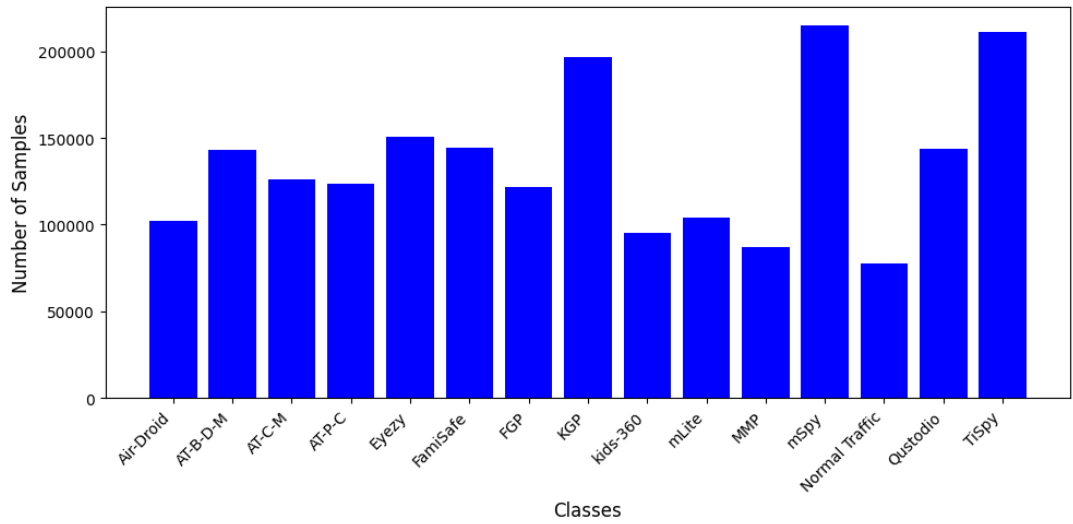


Figure 5.3 Class distribution for MAS-2024-B.

5.4.4 Sample Labelling

After feature engineering and developing both datasets, the correct label must be applied. Network traffic is collected in PCAP format, and before further experiments, it is necessary to correct the labelling for each class. This research follows three rules to label the flow as either spyware or normal traffic. If any of the rules are true, the flow is considered spyware traffic. Each specific spyware traffic is collected one at a time on a format reset device, ensuring that the specific PCAP file contained that specific spyware with Normal Traffic. Based on these rules, the flows are labelled accordingly as either the specific spyware or Normal Traffic. Traffic feature attributes

include the ratio of uplink traffic to downlink traffic respectively represented by Total Fwd Len and Total Bwd Len, the number of unique DIP, and the status of the flow for SIP and DIP. Algorithm 5.4. shows the steps for labelling using predefined rules, which is also illustrated in the workflow shown in Figure 5.4.

5.4.4.1 Rule-1-Flow Status

To accurately classify whether it is a spyware traffic or Normal Traffic, this research defined specific rules based on packet status, which outline how DIP and SIP interact in different communication scenarios (Packetlabs, 2024). By using Scapy library, it finds out whether it represents as replay or request flow. Table 5.5 summarizes these scenarios for both traffic types, alongside their respective equations. Notations (1) to (4) represent scenarios typically found in Normal Traffic, where the user initiates communication with a server and receives responses. Notations (5) and (6) describe patterns commonly associated with Spyware Traffic, where an attacker's IP (DIP) interacts with the user's device (SIP) in a specific manner.

Table 5.5 Packet status.

Traffic Set	Scenario	SIP	DIP	Status	Notation
Normal (N)	User initiates communication	First SIP	DIP	SIP sends Request Pkt (RQ)	$SIP \xrightarrow{RQ} DIP$
	Server responds	DIP	First SIP	DIP sends Replay Pkt (RP)	$SIP \xleftarrow{RP} DIP$
	Server-Initiated Communication	DIP	First SIP	DIP sends RQ	$SIP \xleftarrow{RQ} DIP$
	User Responds	First SIP	DIP	SIP sends RP	$SIP \xrightarrow{RP} DIP$
Spyware (S)	Attacker-Initiated Request	First SIP	DIP	DIP sends RQ	$SIP \xleftarrow{RQ} DIP$
	Attacker Server Response	DIP	First SIP	SIP sends RP	$SIP \xrightarrow{RP} DIP$

5.4.4.2 Rule-2-Number of Unique DIP

This research collects normal traffic over a period of five days, totalling 75 hours. Analysis shows that the maximum number of unique DIP addresses observes in any 1-second interval is fewer than 3. In certain flows, the unique destination IP addresses varied between 4 to 8. According to the highest observation number of

unique DIP is 3, the research establishes a threshold: any flow with more than 3 unique DIP within one second is considered as suspicious flow (Faizal, Zaki, Shahrin, S., Robiah, Rahayu, & Nazrulazhar, 2009).

5.4.4.3 Rule-3-Uplink and Downlink Traffic Ratio

Usually, the downlink traffic (D_T) is much larger than the uplink traffic (U_T), unless there is spyware or suspicious activity occurs, which transfer data, files, screen short, sniffing information without user concern (Huabing & Ozcan, 2021; RF Wireless World, 2024). This data transfers from the source to the 3rd party in the network from time to time, so that the uplink traffic will be abnormal, resulting in ($U_T > D_T$) and considered suspicious flow.

$$U_T = \sum_{i=1}^n FP \quad (5.1)$$

$$D_T = \sum_{i=1}^m BP \quad (5.2)$$

In equations 5.1 and 5.2, FP represents Total Forward Packet Length and BP represents Total Backward Packet Length. n represents the number of Total Forward Packet and m represents the number of Total Backward Packet in a flow (Ericsson, Fortinet, & Hewlett Packard Enterprise, 2024).

Algorithm 5.4 Applying pre-defined rules for labelling.

1. pcap = read (open (pcap file))
 2. **Require:** aggregate flow, DIP,
 3. **for** aggregate flow number in aggregate flow **do**
 4. **if** (DIP= = S_RQ)
 5. flag = True
 6. Label= Specific Spyware
 7. **else if** (SIP= = S_RP)
 8. flag = True
 9. Label = Specific Spyware
 10. **else if** (len(group[DIP].unique()) > 3)
 11. flag = True
-

Algorithm 5.4 Applying pre-defined rules for labelling, continued.

```

12.           Label = Specific Spyware
13.       else if (uplink_total.sum()>downlink_total.sum())
14.           flag=True
15.           Label = Specific Spyware
16.       else
17.           flag = False
18.           Label = Normal-Traffic
19.       end if
20.   end for
21.   processedData = processPcap(pcapFile)

```

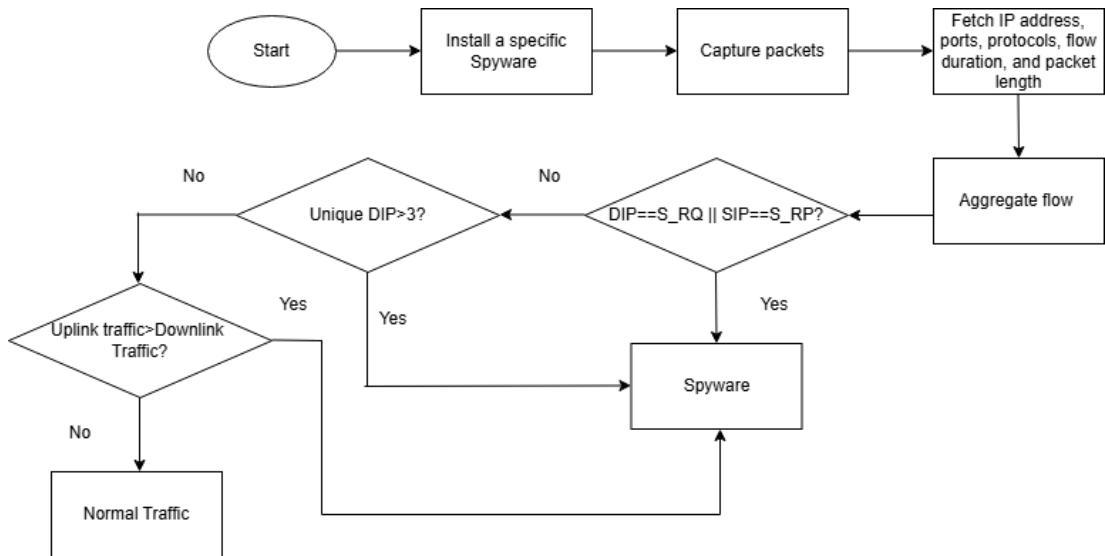


Figure 5.4 Workflow for applying rule-based labelling.

Rule-based labelling is applied for distinguishing spyware traffic from Normal Traffic due to its ability to provide a structured approach to classification (Awasthi, 2025). It ensures precise identification based on predefined characteristics (Huabing et al., 2021). Network traffic pattern analysis is crucial for detecting anomalies and potential threats, as it allows security teams to spot unauthorized access, data exfiltration, or malware attack (Awasthi, 2025). Security policy rules define traffic matching criteria, including applications, users, and source/destination addresses, which help in narrowing the scope of detection (Palo Alto Networks, 2024). Spyware

often communicates with external servers in a manner distinct from normal user activity, such as frequent outbound connections to unknown IPs or abnormal uplink-to-downlink traffic ratios (Huabing et al., 2021 & Tchesnokov, 2017).

To ensure the reliability and accuracy of the rule-based labelling, a validation phase was conducted prior to applying the rules to the full dataset. During data collection, normal traffic was captured separately using a factory-reset Android device to guarantee that no spyware was active or interfering during the capture process. The rule set—comprising conditions such as “more than three unique destination IPs within one second”, “uplink-dominant traffic ratio”, and “flow status”—were first applied to this clean normal traffic PCAP file. None of the normal traffic flows matched any of the rule conditions, confirming that the rules did not produce false positives under benign scenarios. This initial validation provided confidence that the rules were specific to spyware-induced patterns. Additionally, a subset of traffic flows from various spyware samples was inspected by those three rules to verify that the rule conditions consistently aligned with known spyware behaviours. This dual validation—against both clean normal traffic and representative spyware flows—ensured that the rule-based labelling was both precise and scalable for large-scale annotation. The methodology thus supported the integrity of the ground truth labels used for training and evaluation across millions of samples.

5.5 Exploratory Data Analysis

After development the datasets, it is essential to apply exploratory data analysis (EDA). It is an important step which involves thoroughly examining and visualizing the dataset to uncover underlying patterns and relationships among the features. It provides a deep understanding of the datasets structure and characteristics, which helps in identifying trends, addressing data quality issues, and guiding further analysis for model development (Attobrah, 2024). For this research, EDA is important because it allows for the validation of assumptions, the discovery of significant insights, and the informed selection of appropriate methodologies, ultimately ensuring the reliability and robustness of the results (Holdaway, 2009). Figures 5.5 and 5.6 represents heatmap of correlation matrix for MAS-2024-A and MAS-2024-B respectively.

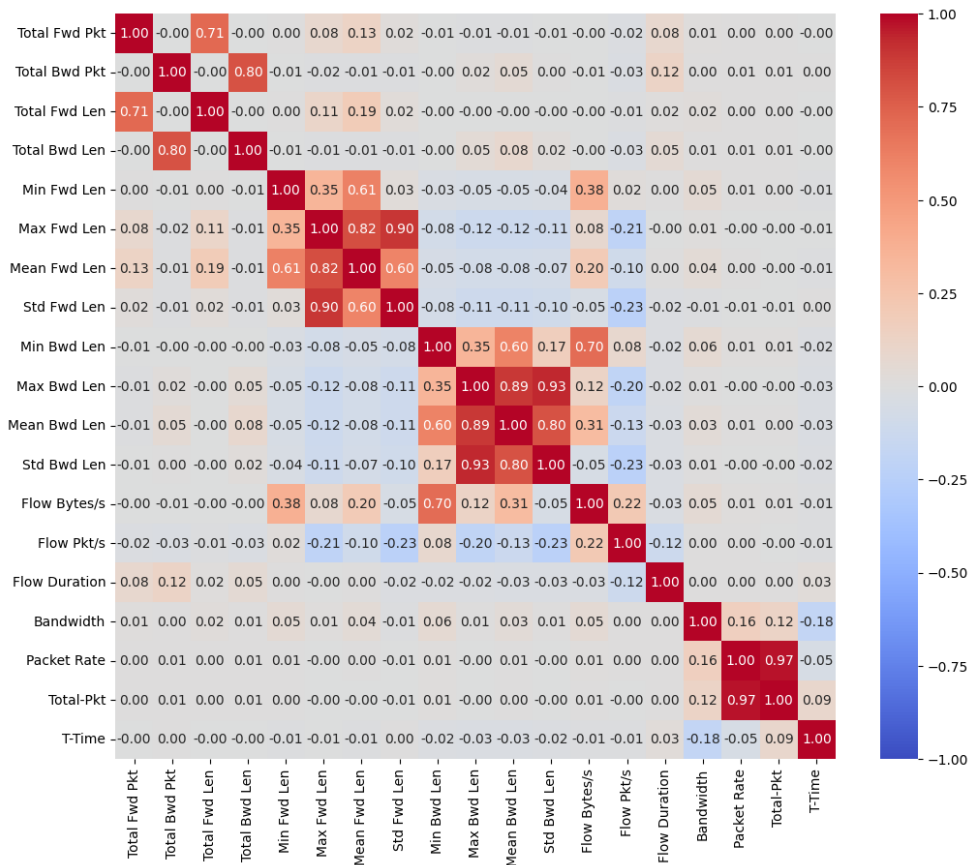


Figure 5.5 Heatmap of correlation matrix for MAS-2024-A.

The heatmap diagram for the correlation matrix effectively visualizes the relationships between the features in the dataset. For MAS-2024-A, Total Fwd Pkt is strongly correlated with Total Fwd Len, with a correlation score of 0.71, indicating a significant positive linear relationship. Similarly, Total Bwd Pkt is strongly correlated with Total Bwd Len, achieving a score of 0.80. Min Fwd Len exhibits a moderate positive correlation with Mean Fwd Len, scoring 0.61, while it is weakly positively correlated with Max Fwd Len and Flow Bytes/s, with scores of 0.35 and 0.38, respectively. These values suggest varying degrees of dependence between these features.

Max Fwd Len displays strong correlations with both Mean Fwd Len and Std Fwd Len, scoring 0.82 and 0.90, respectively, highlighting their close positive relationships. On the other hand, Min Bwd Len shows a strong correlation with Flow Bytes/s, with a score of 0.70, while it moderately positively correlates with Mean Bwd

Len, scoring 0.60. However, its correlations with Max Bwd Len and Std Bwd Len are weak, with scores of 0.35 and 0.17, indicating only minor dependencies.

Flow Bytes/s and Flow Pkt/s are weakly positively correlated, with a score of 0.22, suggesting limited interaction. Flow Duration shows weak positive correlations with Total Fwd Pkt and Total Fwd Len, with scores of 0.08 and 0.12, respectively. Lastly, Bandwidth and Packet Rate are also weakly positively correlated, with a correlation score of 0.16. These observations provide a detail understanding of feature relationships, supporting logical insights into the dataset's structure and dependencies.

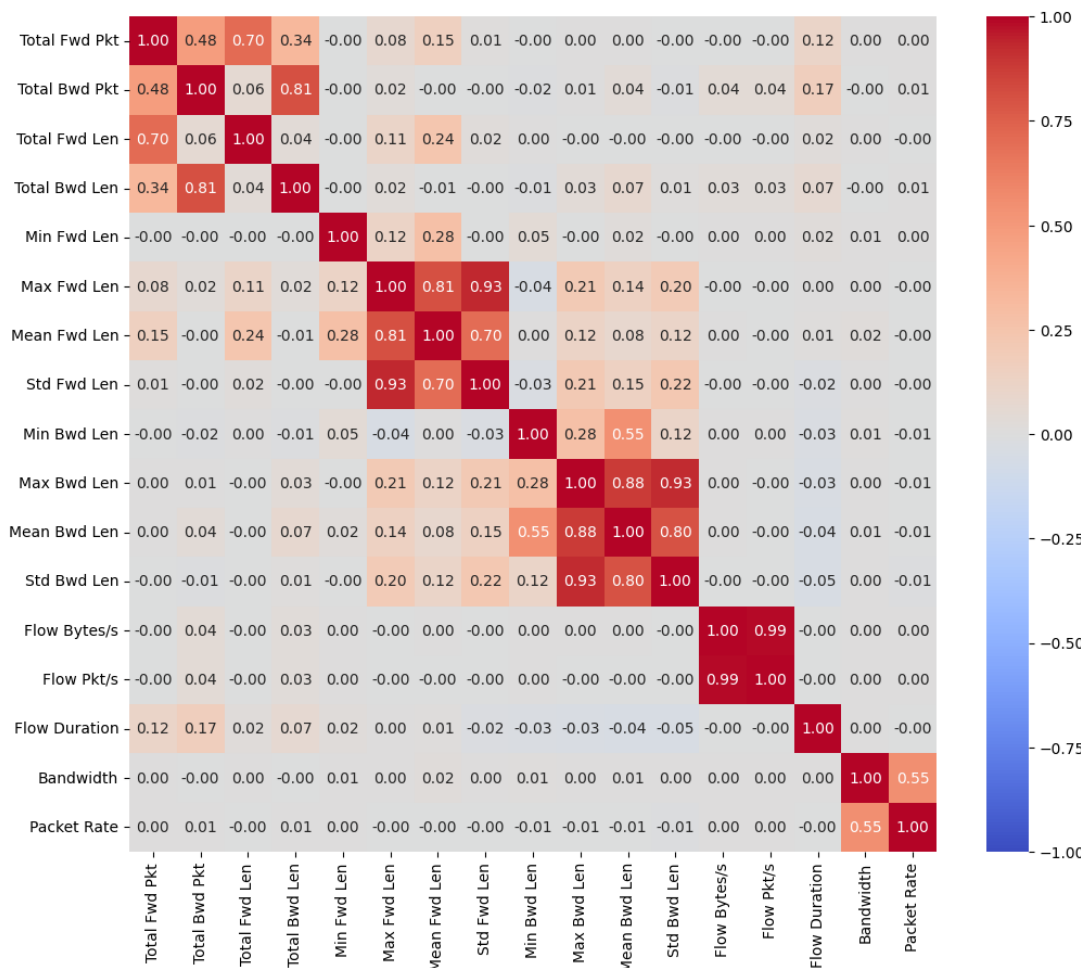


Figure 5.6 Heatmap of correlation matrix for MAS-2024-B.

The heatmap diagram for the correlation matrix clearly illustrates the relationships between the features in MAS-2024-B, providing valuable insights into their dependencies. Total Fwd Pkt is found to be positively correlated with Total Fwd

Len, with a correlation score of 0.71, indicating a strong linear relationship. However, Total Fwd Pkt demonstrates a weak positive correlation with Total Bwd Pkt and Total Bwd Len, scoring 0.48 and 0.34, respectively. Similarly, Total Bwd Pkt exhibits a strong positive correlation with Total Bwd Len, with a score of 0.81, while showing a moderate positive correlation with Total Fwd Pkt, scoring 0.48. These correlations suggest that Total Fwd Pkt and Total Bwd Pkt interact differently with Total Fwd Len, Total Bwd Len, and Total Fwd Pkt, reflecting varying strengths in their relationships.

Min Fwd Len displays weak positive correlations with Max Fwd Len and Mean Fwd Len, with scores of 0.12 and 0.28, respectively. On the other hand, Max Fwd Len shows strong positive correlations with both Mean Fwd Len and Std Fwd Len, scoring 0.81 and 0.93, respectively. Similarly, Mean Fwd Len is strongly correlated with Max Fwd Len and Std Fwd Len, scoring 0.81 and 0.70. Std Fwd Len also shows strong positive correlations with Max Fwd Len and Mean Fwd Len, scoring 0.93 and 0.70, respectively. These results show the significant relationships among these features.

For backward-related features, Min Bwd Len shows weak positive correlations with Max Bwd Len and Std Bwd Len, scoring 0.28 and 0.12, respectively, but shows a moderate positive correlation with Mean Fwd Len, with a score of 0.55. Max Bwd Len shows strong positive correlations with both Mean Bwd Len and Std Bwd Len, scoring 0.88 and 0.93, respectively. Similarly, Mean Bwd Len displays strong correlations with Std Bwd Len and Max Bwd Len, scoring 0.80 and 0.88, reflecting a high degree of dependency among these features.

Flow Bytes/s and Flow Pkt/s are strongly positively correlated, with an almost perfect score of 0.99, highlighting a direct and proportional relationship between these two metrics. In contrast, Flow Duration shows no significant correlations with most features, but it shows weak positive correlations with Total Fwd Len and Total Fwd Pkt, with scores of 0.17 and 0.12, respectively. Finally, Bandwidth and Packet Rate displays a moderate positive correlation, scoring 0.55, suggesting a moderate dependency between these two aspects of the data.

5.6 Directional Properties Impact on Dataset Development

Figure 5.7 illustrates a comparison of the number of packets and the sample numbers for MAS-2024-A and MAS-2024-B using a line chart with separate scales. The y-axis on the left represents the number of packets, while the y-axis on the right represents the sample numbers for both datasets. During the packet collection process, the total number of packets are 420 million. However, after the datasets are developed based on directional properties, the total sample numbers are reduced to 4.2 million for MAS-2024-A and 2.2 million for MAS-2024-B. Each sample corresponds to a flow, which is a combination of packets grouped based on directional properties, as discussed in Section 5.4.

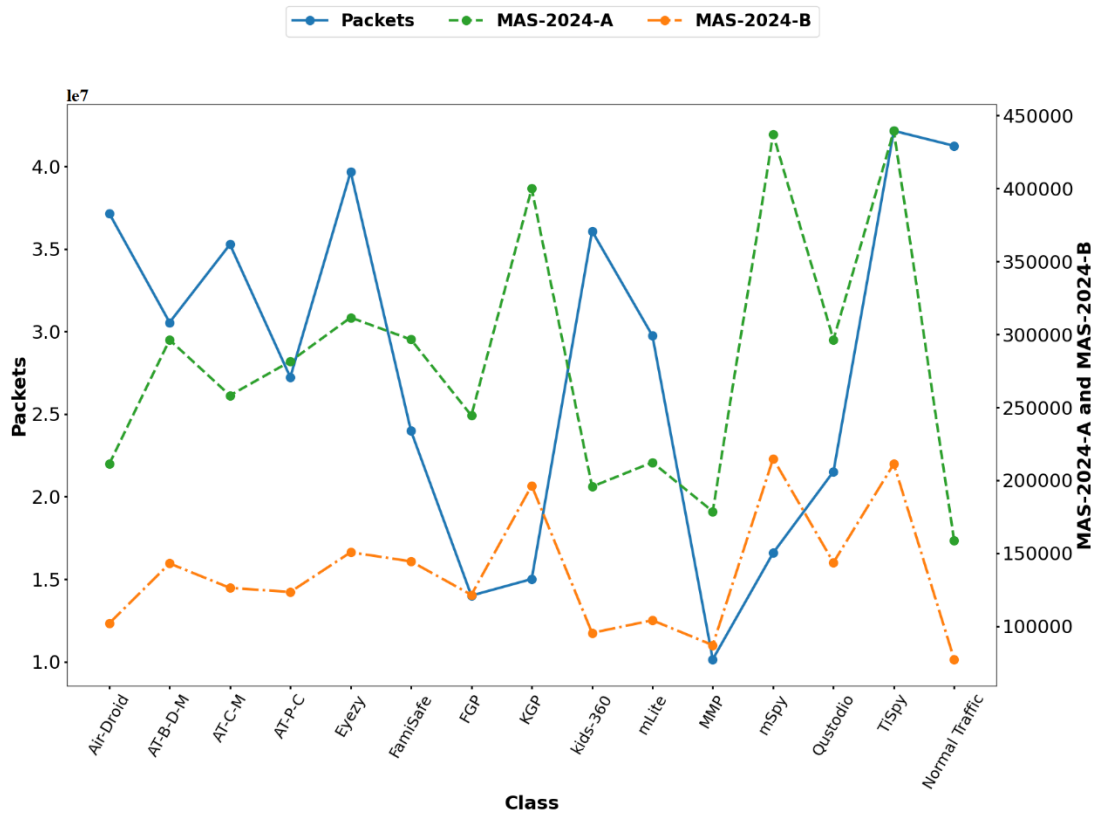


Figure 5.7 Comparison of the number of packets with MAS-2024-A (total sample number), and MAS-2024-B (total sample number) among the classes.

From Figure 5.7, it is evident that although the total number of packets are significantly higher for Normal Traffic compared to other spyware classes, the total number of samples for normal traffic is much smaller. This difference arises due to the

unique directional properties. The flow count directly depends on the number of unique directional properties, and in the case of Normal Traffic, these unique directional properties are fewer compared to spyware traffic.

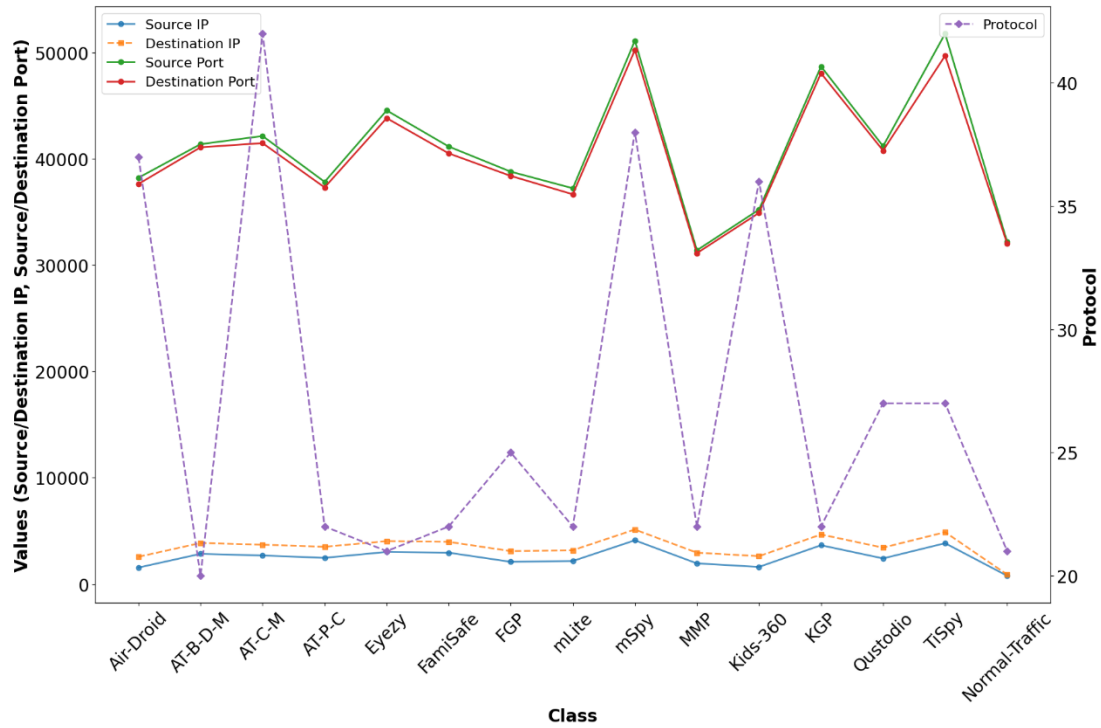


Figure 5.8 Comparison of the unique directional properties for each class.

Figure 5.8 further illustrates this by comparing the unique number of directional properties—SIP, DIP, SP, DP, and Protocol—for each class. It shows that normal traffic has the smallest number of unique directional properties, which leads to fewer flows and, consequently, a smaller number of samples compares to spyware traffic.

The reason Normal Traffic has less unique directional properties compared to spyware traffic lays in the nature of the communication patterns. Normal Traffic consists of standard, predictable interactions such as routine web browsing, file transfers, or common application usage, which typically involves a limited set of IP addresses, ports, and protocols (Fidelis Security, 2024). This regularity leads to less unique combinations of directional properties, resulting in less flows.

Additionally, spyware traffic is characterized by more dynamic and varies communication patterns, as it often engages in covert operations such as data exfiltration, command-and-control communication, or activity masking. These behaviours involve a wider range of SIP, DIP, SP, DP, and protocols, generating a larger number of unique directional properties (Yu et al., 2024). Consequently, spyware traffic produces more flows and a higher number of samples during dataset development (Ferreira, Ravazzi, Dabbene, Calafiore, & Fiore, 2023).

5.7 Comparative Summary with Existing Spyware Datasets

To highlight the novelty of the proposed dataset, this section compares it with several well-known datasets used in prior research. Table 5.6 summarizes the main differences in terms of data source, analysis level, and feature representation.

Table 5.6 Comparison Between the Proposed Dataset and Existing Spyware Datasets.

Dataset	Data Type	Source / Environment	No. of Samples	Feature Type	Remarks
Qabalin et al. (2022)	Network Traffic Based	Install Spyware APK files	386,963 packets	Network Traffic Based	Offline analysis and only DT applied for classification
Conti et al. (2020)	Network traffic (offline)	Captured from controlled emulators	25,000 packets	Packet headers	Offline analysis; no real-time flow segmentation.
CICIDS2017	Network traffic (multi-type attacks)	Mixed intrusion dataset	80,000+	Generic network stats	General attack dataset, not specific to Android spyware, offline analysis.
Proposed Dataset (This Study)	Network traffic (Directional)	Real Android devices (lab environment)	4.2 million flows for MAS-2024-A and 2.0 million flows for MAS-2024-B	Directional network traffic features (packet rate, flow duration, bandwidth, etc.)	Designed for real-time Android spyware detection using rule-based labelling and directional analysis.

Unlike existing works such as Conti et al. (2022), which emphasized controlled emulators, the proposed dataset focused on dynamic, network-based spyware behaviours derived from controlled attacker–victim simulations. Additionally, the rule-based labelling process ensured automatic, accurate, and scalable annotation of millions of network flows, which was absent in prior datasets.

The inclusion of directional flow statistics (bandwidth, packet rate, etc.) provided a unique perspective that supported near real-time classification tasks.

5.8 Conclusions

The feature engineering and dataset development process discussed details in Chapter 5 which presented a structured approach to transforming raw network traffic data into meaningful features for spyware classification. The research emphasized the importance of preprocessing PCAP files by converting them into CSV format, ensuring that essential attributes such as IP addresses, ports, protocol, packet length, and flow duration were extracted. This transformation enabled efficient data handling and model training while storing network traffic characteristics.

A key aspect of feature engineering in this study was the differentiation between static and dynamic features. Static features, derived directly from packet headers, provided foundational information about network communication. In contrast, dynamic features were computed based on directional properties and traffic behaviour, which enhanced the classification capability. The study successfully derived 17 dynamic features, including flow duration, total packet count, packet lengths, and bandwidth-related metrics, which contributed to a detailed representation of spyware activities.

The development of two different datasets, MAS-2024-A and MAS-2024-B, showed the strengths of this research methodology. MAS-2024-A was based on single-directional packet flow, whereas MAS-2024-B was based on bidirectional packet flows, capturing a more complete interaction between communicating entities. The comparative analysis of these datasets showed the differences in feature distributions and sample counts, demonstrating the impact of directional properties on dataset structure.

Additionally, the chapter introduced rule-based sample labelling techniques to ensure accurate classification of spyware and normal traffic. Rules based on flow status, unique destination IPs, and uplink-downlink traffic ratios were applied to samples labelled in the datasets.

EDA further validated the dataset's integrity, uncovering correlations among key features and providing insights into traffic patterns. The study revealed strong correlations between total packet counts and packet lengths. These findings supported the effectiveness of the engineered features in capturing different spyware behaviours.

Overall, the feature engineering and dataset development process provided a strong foundation for spyware classification. By applying preprocessing techniques, dynamic feature extraction, and robust dataset structuring, the research ensured reliable data input for learning models. The insights gained from this chapter contributed to the broader goal of improving spyware classification through network traffic analysis, showing the way for further experimentation with classification models in next Chapters.

CHAPTER 6

BATCH PROCESSING

6.1 Introduction

After collecting the data and developing MAS-2024-A and MAS-2024-B, it is essential to identify whether both developed datasets are suitable for analysis. For this analysis various ML, DL and ensemble models are applied to find out whether it can classify those 14 types of spyware with Normal Traffic or not. During classification several ML, DL and ensemble models are applied in different scenario and after model classification and evaluation, the best models are saved using Pickle for future applications.

Batch processing, which involves processing large volumes of data in groups or batches, play an important role in classification (Acun, Murphy, Wang, Nie, Wu, & Hazelwood, 2021). This process allows for efficient handling of large datasets, helps to reduce computational load, and ensures consistent results (Bekkerman, Bilenko, & Langford, 2011). According to (Kharole, 2019) batch processing is beneficial in scenarios where data is collected over time and requires periodic analysis. By applying batch processing, the developed models can efficiently classify spyware types with Normal Traffic without real-time constraints, ultimately improving the overall performance and accuracy of the models (Devarakonda, Naumov, & Garland, 2017). Figure 6.1 illustrates the workflow for batch processing.

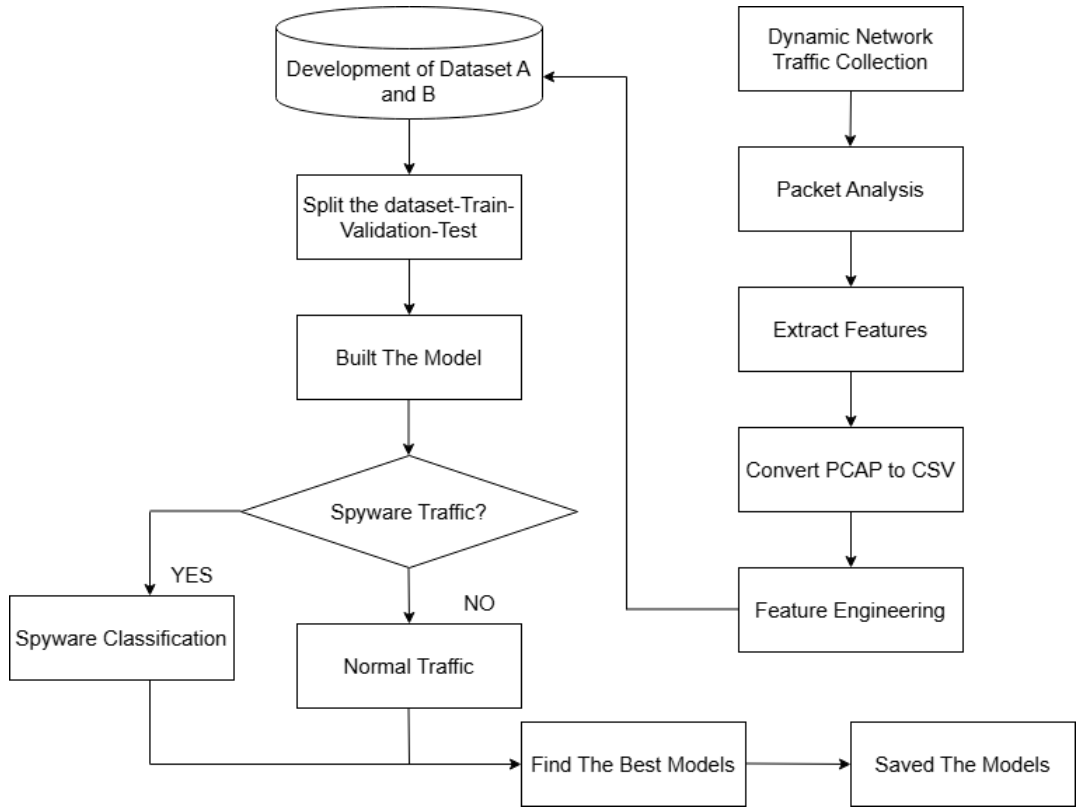


Figure 6.1 Steps for batch processing.

6.2 Metrics for Evaluation

This Section demonstrates the metrics measures used for various ML, DL, and ensemble models across different scenarios. XGB, CatBoost, LGB and RF are applied for ML model; CNN, LSTM, TabNet, and FFNN are applied for DL model; and CNN-LSTM, TabNet-XGB, FFNN-XGB and LSTM-XGB are applied for ensemble models. Also, several balancing methods are applied, such as SMOTE for oversampling, Random Under Sampling (RUS), and 150,000 samples per class for MAS-2024-A and 70,000 samples per class for MAS-2024-B. Tables 6.1 and 6.2 represent the accuracy, precision, recall, and F1-score for these different scenarios. The evaluation metrics for the classification algorithms are based on four main values: True Positive (TP_i), False Negative (TN_i), False Positive (FP_i), and True Negative (TN_i) for multi-class classification. Where N represents total number of samples, i represents total number of classes and w_i represents the number of true samples for each class. To calculate precision, recall, and F1-score, this research applies the weighted averaging to account

for class imbalance (Machine Learning Mastery & Medium 2024). Equations 6.1 to 6.4 represent the computation method for evaluation metrics.

- i. TP_i = The number of samples correctly predicted as class i .
- ii. TN_i = The number of samples that neither belong to nor were predicted as class i
- iii. FP_i = The number of samples predicted as class i but belonging to a different class.
- iv. FN_i = The number of instances that belong to class i but were predicted as a different class.

$$Accuracy = \frac{\sum TP_i + \sum TN_i}{N} \quad (6.1)$$

$$Precision_{weighted} = \frac{\sum_{i=1}^N w_i * Precision_i}{\sum_{i=1}^N w_i} \quad (6.2)$$

$$Recall_{weighted} = \frac{\sum_{i=1}^N w_i * Recall_i}{\sum_{i=1}^N w_i} \quad (6.3)$$

$$F1 - Score_{weighted} = \frac{\sum_{i=1}^N w_i * F1 - Score_i}{\sum_{i=1}^N w_i} \quad (6.4)$$

6.3 Results

The research evaluates classification performance using various metrics across different dataset distributions. First, 20% of each dataset is reserved as a test set to evaluate the final model's performance. From the remaining samples, several train-validation split ratios are allocated: 80-20, 70-30, and 60-40 for both datasets (Tang et al., 2017; Xu et al., 2018).

During batch processing, various balancing techniques are applied, including SMOTE, RUS. Also, specific sample number for each dataset are used to ensure consistency and to avoid errors during processing: 150,000 samples per class for MAS-2024-A and 70,000 samples per class for MAS-2024-B. The selection of these specific sample sizes is due to the characteristics of the datasets. In MAS-2024-A, the class with the lowest sample count is "Normal-Traffic," with 158,626 samples. To ensure a

balanced dataset for avoiding oversampling errors, a sample size of 150,000 is chosen—this ensured all classes are equalized while remaining within the data constraints. Selecting a number greater than 158,626 would have led to processing errors, making 150,000 the optimal choice. Similarly, in MAS-2024-B, the class with the lowest sample count is "Normal-Traffic," with 77,317 samples. A sample size of 70,000 is selected to balance the dataset for staying within the lower boundary of available samples. Exceeding this number have resulted in errors during processing. These show a careful balance between data sufficiency, computational feasibility, and the limitations imposed by the dataset itself, ensuring the integrity and reliability of the research outcomes. These balancing scenarios are combined with the different split ratios and tests across all learning models. After completing the experiments, the analysis reveals that the 80-20 split applies to MAS-2024-A and MAS-2024-B provides the best results compared to other dataset distributions and the scenarios with balancing techniques. Furthermore, the time takes for each type of experiment, considering both balancing and non-balancing scenarios, are also recorded.

This Section presents key results as follows: Table 6.1 illustrates the performance metrics (accuracy, precision, recall, and F1-score) for the 80-20 split on MAS-2024-A and MAS-2024-B across all models. Table 6.2 compares the accuracy of both datasets under various balancing techniques. The execution times for each balancing scenario on both datasets are summarized in Table 6.3, with the results are expressed in hours and minutes (hh:mm). Additional details results, including all metrics for different splits and balancing techniques, are provided in Appendix A, which contains a total of 24 Tables. Figures 6.2 and 6.3 illustrate the accuracy for different split ratios and Figures 6.4 and 6.5 illustrate the comparison of accuracy for different balancing and unbalancing scenarios.

**Table 6.1 Evaluation metrics results (accuracy, precision, recall, and f1-score)
for 80-20 split for both datasets.**

Models	MAS-2024-A				MAS-2024-B			
	Test (%)				Test (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.63	98.63	98.63	98.63	98.66	98.66	98.66	98.66
LGB	98.51	98.51	98.50	98.50	98.57	98.57	98.57	98.56
CatBoost	98.22	98.23	98.22	98.22	98. 21	98. 21	98. 21	98. 20
RF	93.33	93.36	93.33	93.33	87.82	87.82	87.82	87.78
CNN	85.23	85.61	85.24	85.29	87.72	87.89	87.73	87.67
LSTM	92.35	92.41	92.35	92.32	90.16	90.16	90.17	90.09
FFNN	77.00	77.67	77.00	76.85	72.09	73.13	72.10	72.26
TabNet	77.11	77.29	77.11	76.74	71.13	71.04	71.13	70.91
CNN-LSTM	96.07	96.09	96.07	96.07	95.86	95.88	95.86	95.86
FFNN-XGB	98.49	98.50	98.50	98.49	98.04	98.05	98.04	98.05
LSTM-XGB	99.03	99.03	99.03	99.03	99.04	99.04	99.04	99.04
TabNet-XGB	98.49	98.49	98.49	98.49	96.58	96.58	96.58	96.58

**Table 6.2 Comparison accuracy for 80-20 split for different balancing scenario
with the Original scenario.**

Models	MAS-2024-A				MAS-2024-B			
	Accuracy (%)				Accuracy (%)			
	Original	SMOTE	RUS	150K	Original	SMOTE	RUS	70K
XGB	98.63	98.15	98.22	98.08	98.66	97.97	98.38	98.36
LGB	98.51	98.18	98.33	98.43	98.57	97.82	98.56	98.24
CatBoost	98.22	97.59	98.27	98.08	98. 21	96.71	97.92	98.05
RF	93.33	92.37	90.66	89.79	87.82	85.25	82.55	81.94
CNN	85.23	83.13	81.74	80.39	87.72	86.19	86.25	83.17
LSTM	92.35	91.39	90.76	88.06	90.16	88.06	86.81	85.91
FFNN	77.00	75.97	72.52	69.13	72.09	71.73	64.20	66.47
TabNet	77.11	71.54	64.79	65.28	71.13	72.04	63.77	55.16
CNN-LSTM	96.07	95.64	95.12	94.49	95.86	95.20	94.76	95.37

Table 6.2 Comparison accuracy for 80-20 split for different balancing scenario with the Original scenario, continued.

Models	MAS-2024-A				MAS-2024-B			
	Accuracy (%)				Accuracy (%)			
	Original	SMOTE	RUS	150K	Original	SMOTE	RUS	70K
FFNN-XGB	98.49	97.89	98.30	98.25	98.04	96.38	97.39	97.60
LSTM-XGB	99.03	98.88	98.78	98.78	99.04	98.26	98.86	98.81
TabNet-XGB	98.49	98.45	98.20	98.36	96.58	95.39	96.38	96.04

Table 6.3 Run time for every model in different scenario.

	MAS-2024-A				MAS-2024-B			
Models	Original	SMOTE	RUS	150k-70k	Original	SMOTE	RUS	150k-70k
XGB	00:20	00:30	00:17	00:14	00:17	00:27	00:14	00:11
LGB	00:22	00:32	00:19	00:16	00:19	00:29	00:16	00:14
CatBoost	00:55	01:20	00:45	00:38	00:50	01:10	00:39	00:33
RF	01:15	01:30	01:00	00:55	01:05	01:20	00:50	00:40
CNN	00:28	00:45	00:20	00:18	00:25	00:40	00:17	00:14
LSTM	00:56	01:23	00:42	00:38	00:43	01:15	00:38	00:32
FFNN	00:35	00:55	00:28	00:24	00:30	00:45	00:25	00:21
TabNet	01:25	01:10	00:46	00:42	00:43	00:58	00:23	00:14
CNN-LSTM	01:28	01:40	01:10	00:58	01:20	01:30	01:03	00:49
FFNN-XGB	00:40	00:52	00:30	00:25	00:35	00:47	00:25	00:19
LSTM-XGB	01:10	01:25	01:00	00:55	01:00	01:15	00:53	00:45
TabNet-XGB	01:17	01:28	00:59	00:48	01:02	01:10	00:51	00:43

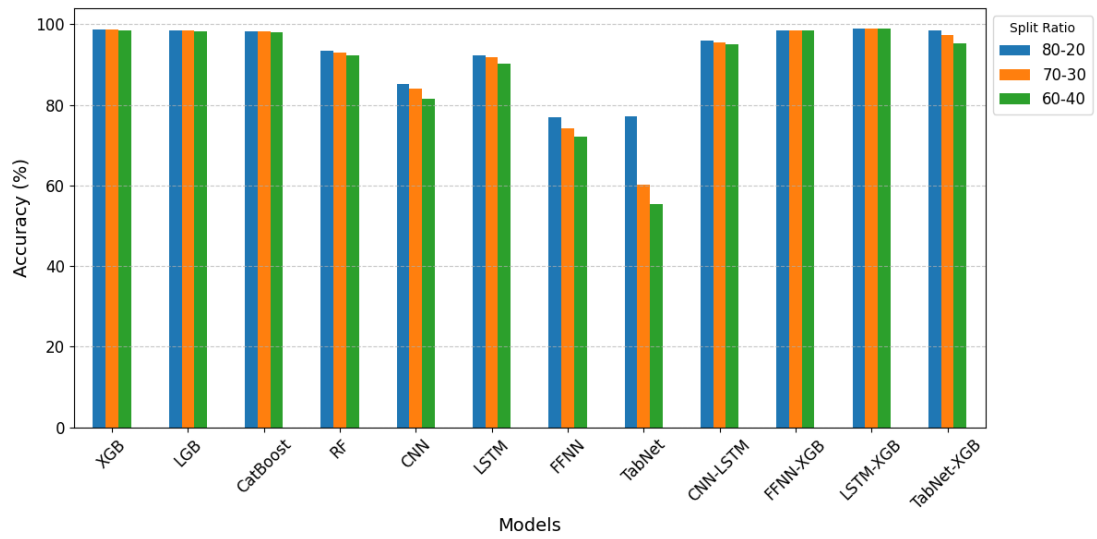


Figure 6.2 Models accuracy comparison across different split ratios for MAS-2024-A.

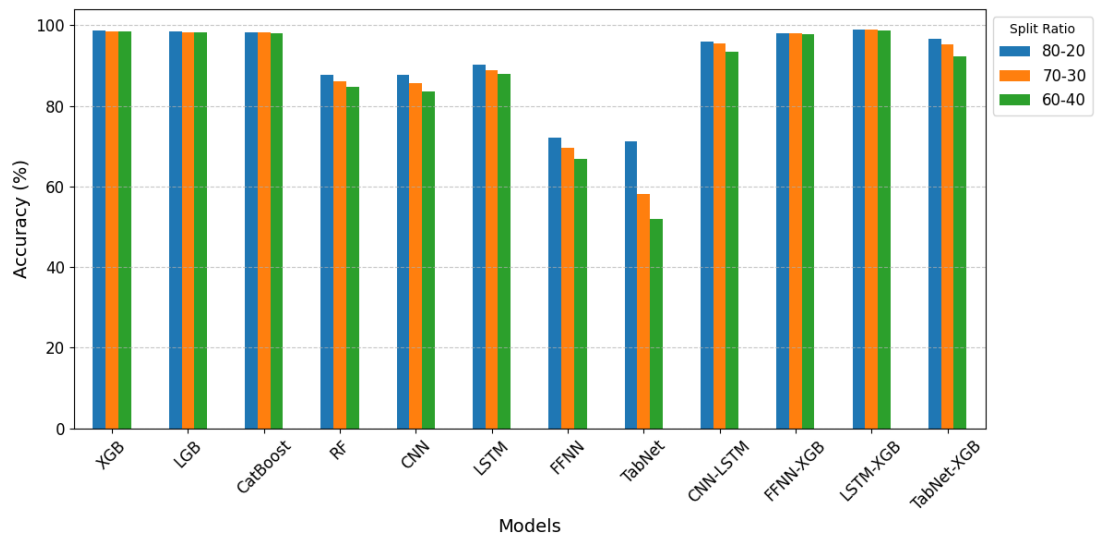


Figure 6.3 Models accuracy comparison across different split ratios for MAS-2024-B.

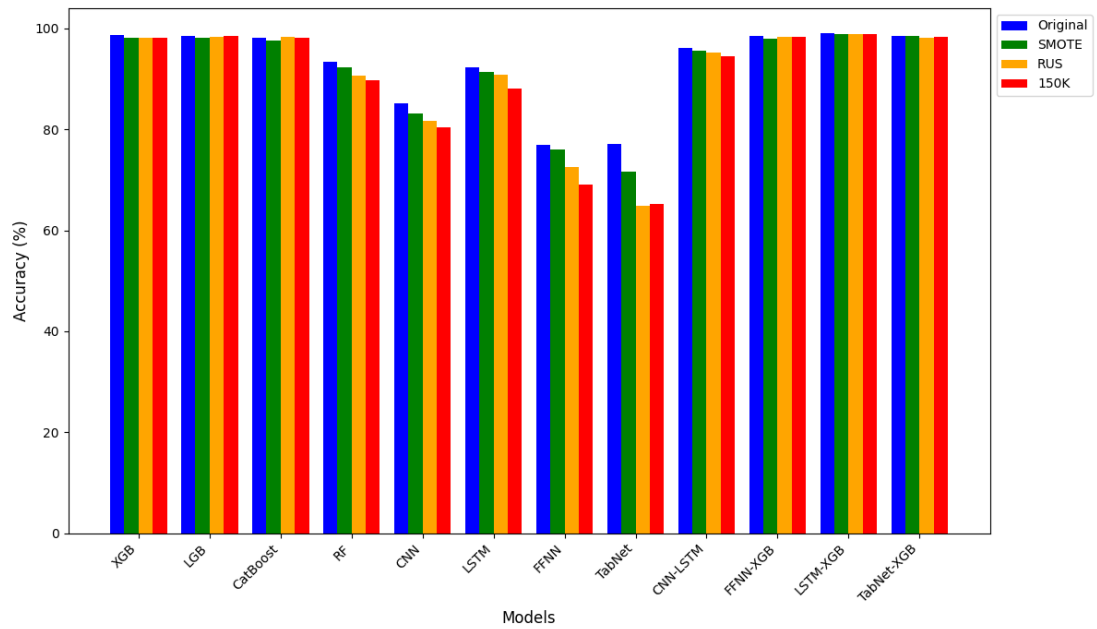


Figure 6.4 Comparison of accuracy for balancing (SMOTE, RUS, 150K) and original dataset (MAS-2024-A) for 80-20 split ratio.

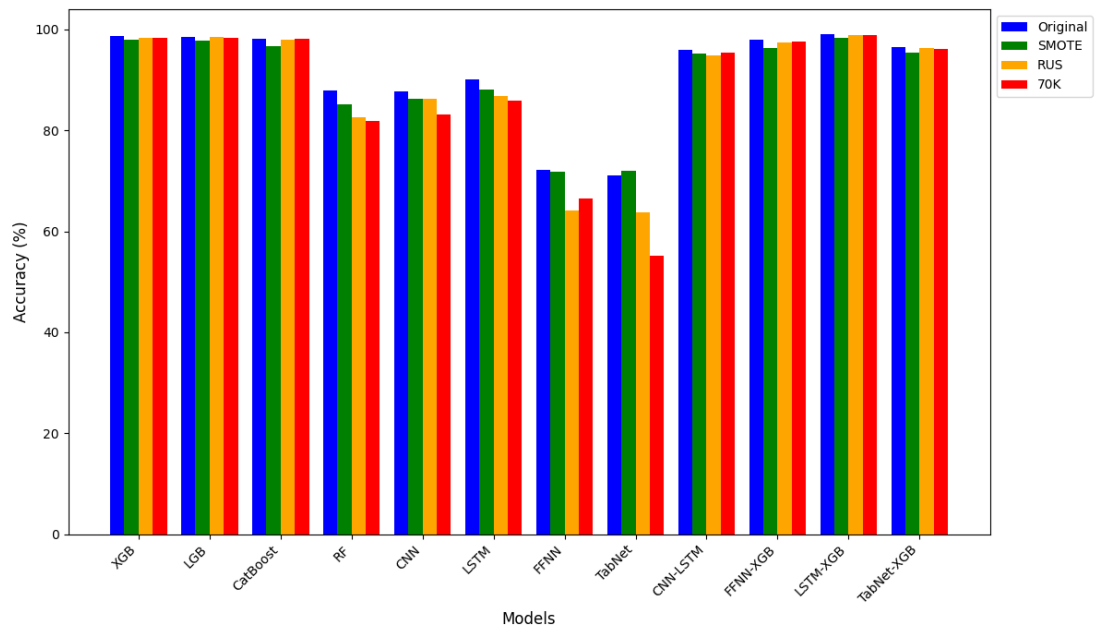


Figure 6.5 Comparison of accuracy for balancing (SMOTE, RUS, 70K) and original dataset (MAS-2024-B) for 80-20 split ratio.

6.4 Detailed Analysis of the Results

The evaluation results provide a strong insight into the performance and efficiency among ML, DL and ensemble approaches, as evaluated on the MAS-2024-A and MAS-2024-B datasets under various conditions. The performance metrics (Table 6.1) indicates that ensemble models, particularly LSTM-XGB, achieves the highest overall accuracy, precision, recall, and F1-score, outperforming all other models on both datasets. LSTM-XGB's superior performance, reaching 99.03% accuracy for MAS-2024-A and 99.04% for MAS-2024-B, illustrates its robustness and suitability for complex classification tasks. Traditional ML models demonstrate better performances, with XGB leading the category by achieving 98.63% accuracy on MAS-2024-A and 98.66% on MAS-2024-B. Among DL models, LSTM achieves stronger results than other DL models, with accuracies of 92.35% and 90.16% on MAS-2024-A and MAS-2024-B, respectively, confirming its ability to capture and process sequential dependencies in the data.

The impact of balancing techniques is illustrated from Table 6.2, where the Original datasets consistently provide slightly better accuracy than those balanced using SMOTE or RUS. For instance, XGB's accuracy for MAS-2024-A decreases from 98.63% (Original dataset) to 98.15% (SMOTE) and 98.22% (RUS). A similar pattern is observed across other models, highlighting that the Original datasets may already exhibit a suitable distribution for effective training without further balancing (Velarde et al., 2023). Additionally, while balancing methods help reduce biases in unbalanced data, their application here led to reduced performance, suggesting that SMOTE and RUS may have introduced noise or overfitting in the process (Alkhawaldeh, Albalkhi, & Naswhan, 2023). This pattern aligns across both datasets, validating the conclusions that balancing methods are not always advantageous, particularly when datasets are inherently well-structured (Elor & Averbuch, 2022).

The analysis of execution times presents in Table 6.3, provides critical insights into performance and computational efficiency across various models and balancing scenarios. XGB shows as the fastest model in all scenarios, completing training on the Original datasets in just 20 minutes (MAS-2024-A) and 17 minutes (MAS-2024-B). This efficiency is attributed to XGB's optimized implementation, which shows GB

techniques and parallelization to accelerate computation, making it particularly suitable for real-time applications (Bentéjac, Csörgő, & Martínez-Muñoz, 2019).

Alternatively, DL and ensemble models requires significantly longer training times. For instance, LSTM and CNN took 56 minutes and 28 minutes respectively for MAS-2024-A, while LSTM-XGB, a hybrid ensemble model, extends training to over an hour. This increase in execution time results from the complex architectures of DL models and ensemble strategies, which requires additional computational resources to handle large-scale datasets and multi-step operations (Thompson, Kristjan, Keeheon, & Manso, 2020).

Balancing techniques such as SMOTE and RUS adds further computational overhead, increasing training times by approximately 15% to 30% compared to Original datasets. SMOTE involves generating synthetic samples, thereby expanding the dataset size and increasing computational complexity (Chawla, Bowyer, Hall, & Kegelmeyer, 2002), while RUS reduces the dataset, demanding extra processing to maintain class balance during training (Brownlee, 2021).

The 150K-70K balancing scenario consistently shows the fastest execution times, as it involves a selective reduction of samples per class, resulting in smaller, more manageable datasets. However, this approach leads to poorer model performance because it discards a significant number of samples with diverse feature values, introducing information loss and reducing the classification models' ability to generalize effectively (Wolf, Payer, Lisson, Lisson, Beer, Götz, & Ropinski, 2024).

Additionally, MAS-2024-A, with a larger dataset size of 617.84 MB, takes longer to process across all models compares to MAS-2024-B, which has a dataset size of 331.44 KB. This difference is due to MAS-2024-A's increases the volume of data and higher feature dimensionality, which extends computational time during model training. These observations highlight the balance between computational efficiency and model accuracy, demonstrating that faster execution times often comes at the cost of poor performance, especially when balancing methods are applied or dataset sizes are reduced excessively (Brownlee, 2021).

Several observations come out from these findings. For real-time or near real-time scenarios where computational resources are limited, XGB is the most practical choice due to its high accuracy and low execution time. However, for tasks requiring maximum accuracy and where computational resources allow ensemble models such as LSTM-XGB are the ideal choice, as they deliver better classification performance. Dataset characteristics also plays an important role in model performance. MAS-2024-B consistently requires shorter execution times compared to MAS-2024-A, likely due to differences in dataset size as MAS-2024-A size is 617.84 KB and MAS-2024-B is 331.44 KB. Additionally, the Original datasets with an 80-20 split achieves the best balance between accuracy and execution time, making them the preferred choice for this study. So, for micro-batch processing and near real-time detection those best models are saved using Pickle (Ahmed et al., 2021).

Key findings include:

- i. The 80-20 split ratio provides the better results than other dataset distributions.
- ii. The Original datasets performed a little better than SMOTE, RUS, and the 150K-70K.
- iii. For MAS-2024-A, CatBoost, RF, LSTM, FFNN, TabNet, CNN-LSTM, and FFNN-XGB perform better.
- iv. For MAS-2024-B, XGB, LGB, CNN, and LSTM-XGB perform better.
- v. XGB performs better than other ML models based on evaluation metrics, LSTM leads DL models, and LSTM-XGB is better performance for ensemble model.
- vi. XGB shows the shortest execution time within 20 minutes.
- vii. MAS-2024-B is the faster execution time than MAS-2024-A
- viii. Though the 150K-70K scenario is faster, it provides poorer performance compared to SMOTE, RUS, and the Original datasets.

In conclusions, the analysis emphasizes the importance of aligning model selection with specific objectives. The findings suggest that while ensemble models like LSTM-XGB offers the best classification accuracy, practical considerations such as execution time and resource constraints make models like XGB more suitable for near real-time scenarios. The experiments also underscore the need to evaluate the

impact of balancing techniques carefully, as these approaches may not always enhance performance and can increase computational overhead. These insights provide a roadmap for optimizing model selection and dataset preparation for further micro-batch processing and near real-time detection.

6.5 Discussions

6.5.1 Learning Models

The accuracy range for RF was around 80%-90% which was a little lower than other ML models. In RF, highly correlated features (Total Fwd Pkt with Total Fwd Len, Total Bwd Pkt with Total Bwd Len, Max-Mean-Std Fwd Len, and Max-Mean-Std Bwd Len) increased redundancy, reducing model performance. The RF algorithm typically depends on Gini impurity by default (Analytics Vidhya, 2024; Medium, 2024), which measures the quality of splits but become less effective with high correlation, as redundant information causes lower split effectiveness. With redundant features it might reduce the classification ability of RF, impacting precision and recall, as indicated in the reported F1-scores ranging (80%-90%) (Gregorutti, Michel, & Saint-Pierre, 2017).

On the other hand, XGB, LGB, and CatBoost models provided high accuracy (up to 98%) because they handle feature correlations through feature importance ranking, iteratively focusing on errors. XGB performed well by penalizing misclassification through its loss function (Mushava & Murray, 2022). This approach addresses overfitting on noisy samples, especially where SMOTE has balanced the classes (Brownlee, 2020 & ML Journey, 2025).

CNNs captured spatial features (Total Fwd Pkt, Total Bwd Pkt, Total Fwd and Bwd Len, Mean, Max, Min, and Std Fwd and Bwd Len). The convolution operations apply a filter matrix over input data, reducing dimensionality and allowing CNN to capture local feature patterns effectively (Krizhevsky, Sutskever, & Hinton, 2012). Meanwhile, LSTM's temporal learning captured time dependencies for Flow Duration, Flow Bytes/s, Flow Pkt/s, Packet Rate, and Bandwidth. LSTM units work through a gating mechanism (forget gate, input gate and output gate), maintaining

long-term dependencies (Hochreiter & Schmidhuber, 1997). By combining CNN-LSTM; made it an effective model which provided better performance evaluation (95%-96%) rather than separately applied CNN and LSTM (Sudhamathy & Valliammal, 2023). As LSTM learns from temporal dependencies, the output of LSTM feed into XGB and XGB boosting the performance while LSTM-XGB this model provided better performance (Semmelmann, Henni, & Weinhardt, 2022). Similar approaches have been validated in cybersecurity research (Kim et al., 2020, Li et al, 2021, & Xu et al, 2023).

FFNN performed better with independent features (Bandwidth, Packet Rate, Flow Duration, Protocol, Source Port, Destination Port), and nonlinear features (Total Fwd Pkt with Total Bwd Pkt, Total Fwd Len with Total Bwd Len, Max-Mean-Min-Std for Fwd Len with Max-Mean-Min-Std for Bwd Len) (Hornik, Stinchcombe, & White, 1989). After getting the output from FFNN then feed the output into XGB. XGB boosting the performance and provides better performance (98 %) rather than separately FFNN performance (Khade et al., 2023).

As DL models focused on distinct types of features based on their specific mechanisms, individual DL models specialize in certain feature types (LeCun et al., 2015). However, when DL models were combined with ML or other DL models to create ensemble models, these ensemble models often achieved better performance (Zhou, 2012). This improvement was due to their ability to leverage multiple feature dependencies, capturing more complex patterns in the data.

The TabNet-XGB model achieved better performance too. TabNet-XGB found beneficial due to Tabnet's attention mechanism and XGB's boosting capabilities (Kanász, Drotár, Gnip, & Zoričák, 2024). TabNet, with its attention mechanism, selected informative features dynamically. The attention mechanism dynamically selected the features those focused on during different parts of the decision-making process (Arik et al., 2019). After training TabNet, the predicted probabilities from TabNet were feed into an XGB classifier for further classification which provided better performance.

The 150K/70K sample-based balancing slightly weak performance for all models compared to RUS and SMOTE. The static sampling approach, while ensuring a fixed class distribution, does not account for the diversity of features within each class. As a result, it might fail to capture the different characteristics of the data within each class. This static sample selection leads to a reduction in classification, reflects in the evaluation performance for both datasets (Wolf et al., 2024).

6.5.2 Network Traffic Analysis for Feature Engineering

The findings indicates that network traffic analysis can effectively classify spyware. By closely observing spyware traffic alongside Normal Traffic, it is possible to distinguish between them. The developed datasets, based on directional properties (SIP, DIP, SP, DP, and Protocol), facilitates a deeper understanding of network behaviour and spyware classification (Shabtai, Hasadya, & Elovici, 2012). The directional properties enhance spyware classification and network-performance optimization by mapping expect sequences, standard frequency, volume of data transmissions, and common paths taken by data packets (Stallings & Case, 2017). Although both datasets have the same features, the values of each feature differed between the two datasets due to the directional properties. The Bandwidth and Packet Rate are calculated based on the total packets and total packet length for each PCAP file, with the duration that each PCAP file are captured, which is why both datasets have the same Bandwidth and Packet rate value. Engineered features includes directional properties within the network data, offering insight into behavioural baselines, protocol analysis, traffic volume, and time-based patterns (Ahmed et al., 2016). This approach is important for accurately modelling spyware behaviour by understanding data flow direction, allowing for precise differentiation between normal and spyware activities (Al-Hamadi & Chen, 2018).

On the base of directional properties, the developed features like Total Fwd Pkt, Total Bwd Pkt, Total Fwd Len, Total Bwd Len, Min-Max-Mean-Std Fwd and Bwd Len, Flow Bytes/s, Flow Pkt/s are analysed to classify anomalies, such as unusual packet sizes, irregular transmission rates, and unexpected flow durations (Sharma et al., 2019). An unusually large packet size indicates data aggregation before transmission, which is normal for certain applications (e.g., video streaming, file

transfers, cloud storage, and backups) but suspicious for other (Medium, 2024, Mushava et al., 2022). Average packet size helps to understand typical network load, where a sudden increase suggests bulk data transfers (Xu et al., 2016). Standard deviation measures variability in packet sizes; high variability may indicate suspicious mixed traffic, while low variability points to uniform Normal Traffic (Mukherjee, Heberlein, & Levitt, 1994). A higher flow bytes/s rate suggests high-volume data transfer, which can be legitimate (e.g., video streaming) or suspicious (e.g., data exfiltration) (Iorliam, Tirunagari, Ho, Li, Waller, & Poh, 2016). Packet-length statistics are effective for classifying anomalies by establishing baselines and identifying deviations (Kim et al., 2004). These analyses help identify unusual patterns critical for maintaining network security. Spyware, DDoS attacks, or data exfiltration activities often involve unusually high bandwidth and packet rates due to the volume or frequency of data (Mirkovic & Reiher, 2004). Anomalous traffic shows higher bandwidth and packet rates, along with abnormal patterns like sudden spikes or sustained high rates (Zhou et al., 2020). Comparing observed bandwidth and packet rate to Normal Traffic baselines can identify significant differences, signalling potential anomalies (Lakhina, Crovella, & Diot, 2005).

6.6 Conclusions

The batch processing methodology applied in this research effectively optimized the classification of spyware and Normal Traffic. The experiments demonstrated that various learning models which could successfully distinguish between 14 types of spyware and Normal Traffic under different dataset distributions and balancing scenarios. The models' performances were systematically evaluated using accuracy, precision, recall, and F1-score, ensuring a details evaluation of classification capabilities.

The study initially allocated 20% of the dataset for testing, with the remaining data divided into training and validation sets at different ratios. Several balancing techniques, including SMOTE and RUS, were applied to address class imbalance. The results indicated that an 80:20 for the Original datasets achieved the best performance across multiple models. Among ML models XGB achieved the highest accuracy, reaching 98.63% for MAS-2024-A and 98.66% for MAS-2024-B. Similarly, LSTM

model demonstrated superior performance among DL models, with accuracy scores of 92.41% for MAS-2024-A and 90.16% for MAS-2024-B. In ensemble models, LSTM-XGB showed as the most effective, achieving an accuracy of 99.03% for MAS-2024-A and 99.04% for MAS-2024-B. These findings validated the robustness of the selected models and their capability to classify spyware traffic with high reliability.

The evaluation of different balancing techniques further validated the effectiveness of the classification models. When applied to the datasets, balancing techniques influences classification performance in various ways. While SMOTE and RUS decreased the results a little bit than the Original datasets, which might have introduced noise or overfitting.

Additionally, the execution time for each experiment was recorded to assess computational efficiency. The analysis revealed that MAS-2024-A required approximately 20 seconds for XGB, whereas MAS-2024-B took around 17 seconds for XGB. Also, it showed that while applying balancing techniques it took more execution time which created computational overhead. On the other hand, for MAS-2024-B took less execution time than MAS-2024-A because of the size of datasets. These differences showed the structural difference between single-directional and bi-directional packet flows.

The research also explored the effectiveness of different model architectures in handling large-scale batch processing. While XGB, LGB, and CatBoost showed strong classification capabilities, the incorporation of neural network-based models such as LSTM provided additional advantages in learning temporal dependencies within network traffic data. Ensemble models combining DL and ML approached, such as LSTM-XGB, further improved classification accuracy, demonstrating the potential of ensemble methodologies in spyware classification.

The success of batch processing in this study illustrated its significance in spyware classification. By processing large volumes of data in structured batches, this approach ensured consistent model performance while reducing computational overhead. The findings highlighted the advantages of batch processing as it allowed for systematic data evaluation. Moreover, the research established a foundation for

future advancements in spyware classification, enabling the way for integrating micro-batch processing and near real-time detection strategies.

In summary, the implementation of batch processing demonstrated to be an effective method for spyware classification, providing high accuracy and computational efficiency. The integration of learning models enabled a robust classification framework, while dataset balancing techniques further enhanced model performance. The insights gained from this study contributed to the broader field of network traffic analysis, providing the importance of systematic data processing for improving cybersecurity solutions. Next Chapter will show the further research for micro-batch processing and near real-time detection using those batch process models. After batch processing, the best models were saved using Pickle; and using those saved models to incorporate near real-time detection methods and exploring adaptive learning techniques to address spyware threats.

CHAPTER 7

MICRO-BATCH PROCESSING

7.1 Introduction

After batch processing and saved the models, this research applies micro-batch processing before near real-time detection. During micro-batch processing several micro-batch sizes are applies which are staring from 50, 100, 150, 200, and 250 packets, to determine the most appropriate size for near real-time detection (Gandhi et al, 2022 & Li et al., 2021). Algorithm 7.1 illustrates the procedure for micro-batch processing. For micro-batch processing, this research collects network traffic for each type of spyware alongside Normal Traffic in the format of PCAP files. During network traffic collection, each PCAP file containing specific spyware network traffic along with Normal Traffic is uploaded. Micro-batch sizes ranging from 50 to 250 are applied to every saved model. The accuracy for each of the 12 models (from XGB to TabNet-XGB) are averaged to determine the best option for micro-batch size during near real-time detection. Averaging the results allowed to identify a micro-batch size that performs consistently well across all models, ensuring unbiased evaluation and generalizability. This approach shows the potential impact of individual model performance outliers and provides a robust framework for determining the optimal micro-batch size. Tables 7.1 to 7.4 present the average results for each spyware class combine with Normal Traffic, showing the performance of 12 models across micro-batch sizes ranging from 50 to 250 packets. These tables summarize the findings of detailed experiments conducted for each spyware class and Normal Traffic using the 12 models. The individual results of these experiments, spanning 60 tables, have been placed in Appendix B for reference. The average results in Tables 7.1 to 7.4 are calculated to determine the most optimal micro-batch size.

7.2 Labelling Ground Truth During Micro-Batch and Near Real-Time Detection

During micro-batch process, this research captures network packets and organizes them into flows based on directional properties. The flows are then passed through a set of rules to determine whether the traffic is labelled as "Spyware" or "Normal Traffic," as detailed in Section 5.5 (Sample Labelling). This step is crucial since the captured packets lacks inherent labels, making the generation of accurate ground truth labels vital for the evaluation of learning models and their associated metrics. While the three rules effectively distinguish between "Spyware" and "Normal Traffic," this research addresses a more complex challenge: multi-class classification. The saved models aim to classify 15 different classes, comprising 14 different types of spyware and 1 "Normal Traffic" class. Too (Cover, 1967; Gupta & Zafra, 2023).

Algorithm 7.1 Micro-batch processing workflow.

```
1. pcap = read (open ( pcap file))
2. Require: SIP, DIP, FPL,BPL,TFP,TBP, Min-Max-Mean-Std , features values (fea_val),
   batch size
3. for batch size in batch size do
4.   for aggregate flow number in aggregate flow do
5.     if (Rule 1 || Rule 2 || Rule 3)
6.       flag = True
7.       Label= Spyware
8.       if (nearest_neighbor_label = knn.predict(fea_val)
9.         Label = Specific Spyware
10.      else
11.        Label = Unknown-Spyware
12.      else
13.        flag = False
14.        Label = Normal Traffic
15.      end if
16.    end for
17.    called saved model
18.    predict sample
```

19. calculate evaluation metrics
 20. **end for**
-

After identifying a flow as "Spyware," it becomes essential to precisely label the specific type of spyware. This step is critical because mislabelling significantly degrades the evaluation metrics and overall performance of the learning models. To address this, the features extracted during flow creation are compared against the developed dataset containing the feature values for each spyware class. However, a direct comparison is insufficient due to the similarity in features and signatures between different spyware types. In some cases, the features of two or more spyware classes overlap, making it unreliable to determine the correct label solely based on feature matches (Ahmed et al., 2021 & Fix et al., 1985).

To overcome this limitation, this research applied KNN into the algorithm. KNN is applied to find the nearest neighbour from the saved dataset based on extracted features. By getting the distance-based approach of KNN, the algorithm ensures that the most appropriate specific spyware label is assigned to the flagged flow. This methodology is particularly effective for multi-class problems where class boundaries are not well-defined, as KNN accounts for differences in feature distributions. Moreover, KNN is well-suited for this task because it does not rely on thresholds for classification, thereby reducing the risk of misclassification in overlapping feature spaces (Altman, 1992).

During micro-batch and near real-time processing, flow aggregation did not retain the original ground truth labels required for multi-class classification and evaluation. To address this, the research implemented a KNN approach to assign ground truth labels based on feature similarity with the previously developed and validated MAS-2024 datasets. This step was essential because, without labels, learning models can make predictions but cannot be evaluated using standard metrics. The KNN model was not used for training or evaluation of the saved models; it was solely used to assign labels where none existed. To ensure the reliability of this approach,

this research conducted validation tests using PCAP files containing only normal traffic and others containing specific spyware types. In each case, KNN correctly identified the traffic type—assigning "Normal Traffic" to benign flows and accurately labelling specific spyware types. This demonstrated that KNN could reliably assign labels based on known feature patterns. Furthermore, the saved models used for prediction and evaluation were selected during batch processing based on their performance and were not influenced by the KNN labelling process. This separation ensured that the evaluation of model performance remained unbiased and avoided methodological circularity.

KNN was employed during micro-batch processing to assist in assigning spyware labels to network flows. To mitigate concerns of circularity—where model performance might be evaluated against labels derived from another model—KNN was not used as part of the final predictive models. Instead, it served as an auxiliary tool to refine ground truth labels based on distance similarity within the training dataset. Each PCAP file used in micro-batch processing was captured in a controlled environment where only one specific spyware sample was installed during a defined time window, alongside normal traffic. This ensured that the captured traffic contained only the targeted spyware and benign activity, allowing KNN to differentiate between them without ambiguity. The final models (e.g., LSTM-XGB, RF, LSTM) were trained and evaluated independently using these fixed labels. Additionally, manual validation was conducted on a subset of flows to confirm that KNN label assignments aligned with known spyware signatures. This approach reduced the risk of circular evaluation bias and enhanced label reliability. Nonetheless, the potential for bias introduced by model-assisted labelling was acknowledged, and future work will explore external ground truth verification to further strengthen the methodology.

7.3 Results

Tables 7.1 to 7.4 illustrate the average micro-batch results for various batch sizes (50–250 packets) across 14 types of spyware with Normal Traffic. These tables analysed micro-batch processing across different batch sizes to determine the most suitable size for near real-time detection.

Table 7.1 Micro-batch processing-average accuracy for all the saved models.

Class	MAS-2024-A (Accuracy-%)					MAS-2024-B (Accuracy-%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
Air-Droid	74.73	75.17	77.38	75.06	74.11	73.12	73.67	76.18	73.89	73.54
AT-B-D-M	74.73	75.75	80.03	78.04	78.11	74.91	75.56	79.41	77.22	77.34
AT-C-M	78.99	79.61	81.89	79.64	79.36	75.57	76.26	80.39	76.13	77.53
AT-P-C	77.59	77.99	81.82	79.07	78.23	76.84	78.07	80.25	78.57	76.72
Eyezy	78.46	80.51	83.29	81.52	80.78	77.88	78.21	81.22	77.03	77.11
FamiGuardPro	76.09	77.79	82.44	80.49	79.02	76.21	77.06	80.72	78.54	78.26
FamiSafe	76.14	78.38	82.59	80.21	80.51	76.12	78.07	81.75	78.83	79.90
kidsGuardPro	79.23	79.80	83.58	80.62	80.67	79.71	77.95	82.15	79.53	79.62
Kids-360	79.49	80.39	84.36	81.58	81.58	78.12	79.98	83.15	80.11	79.28
mLite	78.89	80.37	83.17	79.77	79.87	78.14	79.62	81.77	78.21	79.37
MoniMasterPro	80.35	82.03	84.95	83.17	82.18	79.86	81.22	83.23	82.15	82.00
mSpy	77.92	79.14	82.13	79.47	79.05	77.85	78.00	80.74	79.64	78.44
Normal Traffic	80.77	83.00	84.56	82.09	82.12	79.54	80.94	82.94	81.51	80.08
Qustodio	79.89	80.69	82.91	82.02	80.99	79.01	78.85	82.80	80.95	80.01
TiSpy	71.66	72.81	76.29	74.68	74.80	71.53	70.53	74.18	71.75	71.09

Table 7.2 Micro-batch processing-average precision for all the saved models.

Class	MAS-2024-A (Precision-%)					MAS-2024-B (Precision-%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
Air-Droid	74.78	75.49	77.78	75.31	74.56	73.39	74.05	76.48	74.20	73.92
AT-B-D-M	75.01	76.20	80.45	78.35	78.33	75.35	76.17	79.49	77.51	77.89
AT-C-M	77.91	78.44	82.19	79.44	78.83	76.75	76.38	80.47	78.74	77.12
AT-P-C	79.27	79.83	82.28	79.91	79.70	75.87	76.57	80.68	76.97	77.95
Eyezy	80.12	82.09	84.75	81.76	81.31	78.12	78.51	81.47	77.34	77.47
FamiGuardPro	76.51	78.25	82.99	80.89	79.49	76.23	77.86	81.39	79.29	78.75
FamiSafe	76.55	78.83	83.03	80.64	81.04	76.35	78.50	82.17	79.39	80.23
kidsGuardPro	80.26	81.23	84.56	81.98	81.82	78.49	80.79	83.05	81.17	80.45
Kids-360	79.59	79.88	83.98	80.66	81.17	80.07	78.31	82.57	79.91	80.12
mLite	79.25	80.79	83.65	80.14	80.39	79.19	80.01	82.33	78.59	79.94
MoniMasterPro	80.79	82.39	85.38	83.57	82.68	80.06	81.59	83.49	82.53	82.51
mSpy	78.20	79.59	82.49	79.86	79.65	77.96	78.59	80.83	80.09	78.99
Normal Traffic	81.20	83.28	84.91	82.51	82.42	79.78	81.75	83.19	82.18	80.54
Qustodio	80.32	80.83	84.02	82.24	81.39	79.34	79.28	82.84	81.24	80.36
TiSpy	72.12	73.26	76.75	75.18	75.26	71.86	70.88	74.30	72.05	71.49

Table 7.3 Micro-batch processing-average recall for all the saved models.

Class	MAS-2024-A (Recall-%)					MAS-2024-B (Recall-%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
Air-Droid	74.43	75.17	77.36	75.06	74.11	73.14	73.71	76.21	73.93	73.59
AT-B-D-M	74.73	75.75	80.03	78.04	78.11	74.95	75.77	79.44	77.25	77.35
AT-C-M	77.59	77.99	81.82	79.07	78.23	76.65	78.09	80.51	78.62	76.79
AT-P-C	79.01	79.60	81.87	79.65	79.36	75.57	76.26	80.39	76.71	77.53
Eyezy	78.45	80.51	83.29	81.52	80.78	77.72	78.21	81.07	77.03	77.11
FamiGuardPro	76.09	77.83	82.51	80.49	79.03	76.22	77.07	80.72	78.55	78.29
FamiSafe	76.14	78.37	82.57	80.23	80.51	76.14	78.07	81.74	78.83	79.89
kidsGuardPro	79.49	80.38	84.36	81.58	81.59	78.12	79.98	83.17	80.12	79.28
Kids-360	79.22	79.77	83.57	80.60	80.63	79.71	77.96	82.17	79.54	79.62
mLite	78.91	80.39	83.17	79.78	79.90	79.14	79.62	81.77	78.21	79.37
MoniMasterPro	80.37	82.07	84.95	83.17	82.18	79.87	81.23	83.25	82.15	82.01
mSpy	77.94	79.17	82.14	79.47	79.08	77.84	77.99	80.75	79.64	78.44

**Table 7.3 Micro-batch processing-average recall for all the saved models,
continued.**

Class	MAS-2024-A (Recall-%)					MAS-2024-B (Recall-%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
Normal Traffic	80.75	82.99	84.57	82.09	82.12	79.53	80.94	82.92	81.52	80.07
Qustodio	79.87	80.69	82.89	82.01	81.01	79.01	78.85	82.80	80.95	80.01
TiSpy	71.68	72.83	76.30	74.68	74.81	71.53	70.53	74.18	71.75	71.09

Table 7.4 Micro-batch processing-average f1-score for all the saved models.

Class	MAS-2024-A (F1-Score-%)					MAS-2024-B (F1-Score-%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
Air-Droid	75.46	75.33	77.52	75.18	74.34	73.25	73.86	76.30	74.06	73.73
AT-B-D-M	74.78	75.99	80.23	78.19	78.28	75.12	75.96	79.35	77.01	77.68
AT-C-M	77.72	78.22	82.05	79.26	78.84	76.54	76.17	80.33	78.59	76.95
AT-P-C	79.12	79.68	82.09	79.77	79.53	75.73	76.39	80.54	76.82	77.74
Eyezy	79.93	81.88	84.56	81.67	81.14	77.99	78.39	81.28	77.17	77.31
FamiGuardPro	76.39	78.06	82.26	80.99	79.40	76.34	77.61	81.18	79.08	78.52
FamiSafe	76.33	78.60	82.81	80.43	80.77	76.15	78.29	81.96	79.18	79.99
kidsGuardPro	80.06	81.03	76.81	81.78	81.57	78.31	80.58	82.89	80.97	80.18
Kids-360	79.41	79.75	83.81	81.46	80.85	79.91	78.14	82.35	79.72	79.87
mLite	79.07	80.58	83.42	79.95	80.12	78.15	79.78	82.09	78.37	79.66
MoniMasterPro	80.64	82.21	85.18	83.36	82.39	79.86	81.41	83.45	82.33	83.00
mSpy	78.08	79.44	82.32	79.66	79.35	77.79	78.41	80.64	79.86	78.65
Normal Traffic	80.90	83.17	84.69	82.24	82.28	81.58	83.05	82.00	80.33	80.33
Qustodio	80.11	80.58	83.87	82.12	81.18	78.87	79.69	82.69	81.14	80.11
TiSpy	71.89	73.03	76.52	74.95	75.03	71.68	70.71	74.14	71.89	71.34

7.4 Detailed Analysis of the Results

Figures 7.1 and 7.2 visualize the results from Table 7.1. Figure 7.1 displays the average accuracy for different micro-batch sizes for MAS-2025-A, while Figure 7.2 shows the same for MAS-2024-B. These Figures indicate that a micro-batch size of 150 provides the better results. Based on the analysis conducted in Tables 1 to 64 (Attached

in Appendix B), this research averaged accuracy, precision, recall, and F1-score, as summarized in Tables 7.1 to 7.4. The findings reveals that a micro-batch size of 150 packets provides consistently better results across all classes with various matrix.

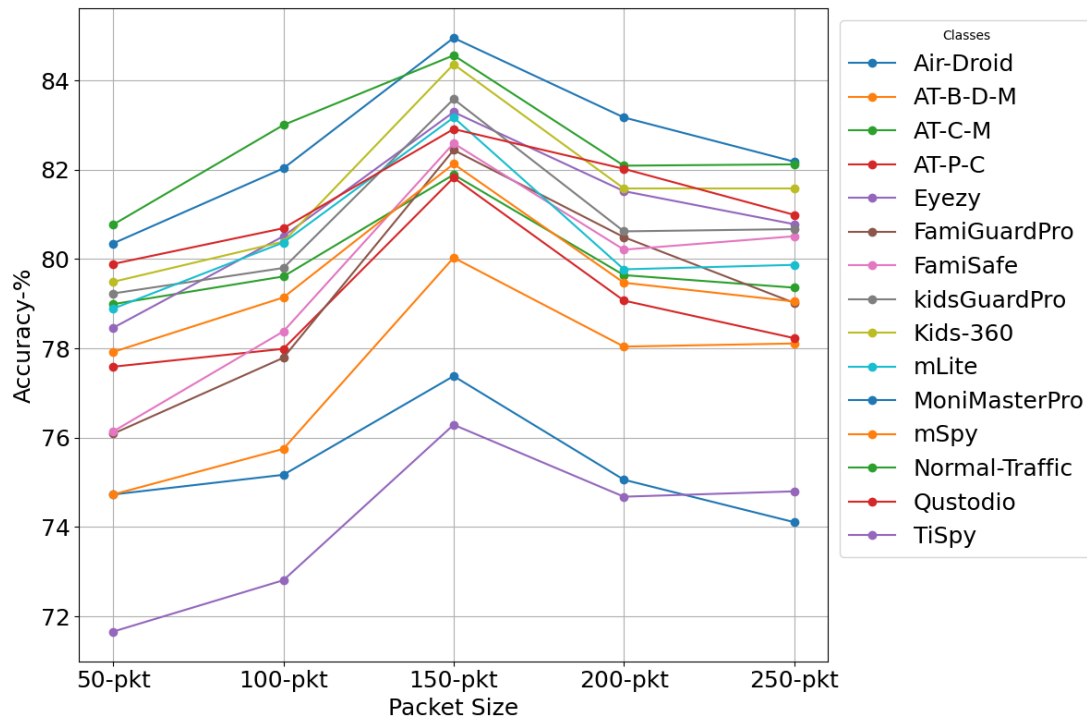


Figure 7.1 Average accuracy for 15 classes (MAS-2024-A).

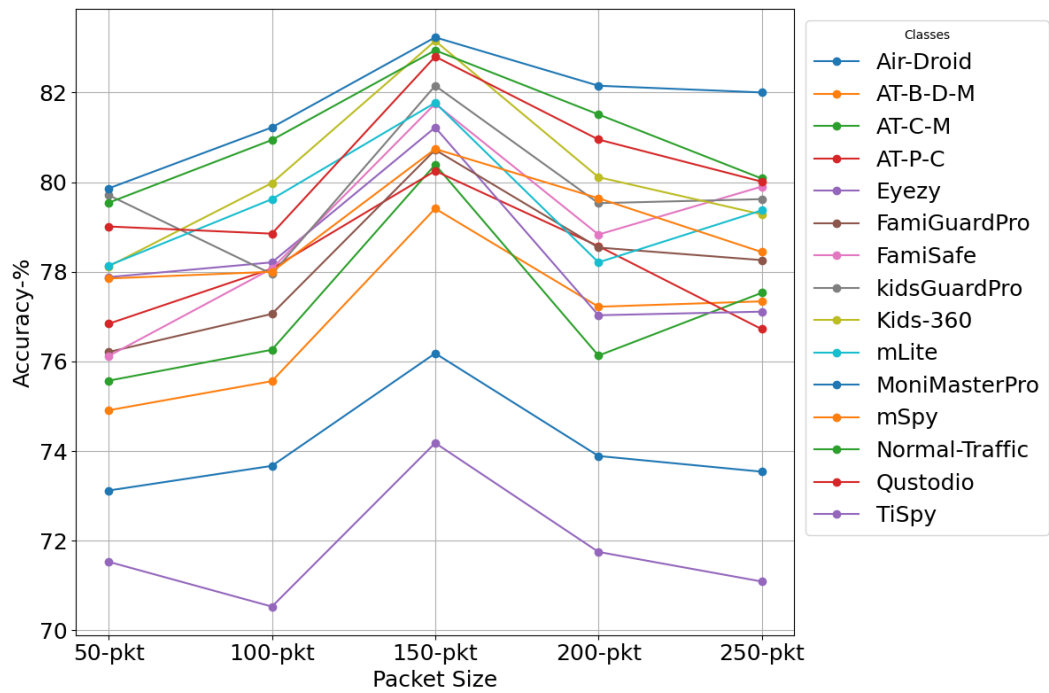


Figure 7.2 Average accuracy for 15 classes (MAS-2024-B).

The results from Chapter 7, including Tables 1–60, reveal valuable insights into the performance dynamics of learning models across various micro-batch processing sizes. Tables 1-60 provide detailed observations of model-specific performances across different spyware classes, while Tables 7.1–7.4 in the main chapter aggregate these findings to identify optimal micro-batch sizes. The analysis of these results highlights key factors affecting model behaviour and demonstrates relationships between the models' architectures, dataset characteristics, and their overall effectiveness.

Examining the results from Tables 1-60, it is evident that ML and ensemble models generally outperform then DL models for most spyware classes. Models like XGB, LGB, CatBoost, LSTM-XGB, and FFNN-XGB consistently achieve superior accuracy, precision, recall, and F1-scores, demonstrating the strength of combining the feature learning capabilities of DL with the decision-making abilities of ML (CodeProject, 2024 & Kumar et al., 2020). For example, LSTM-XGB achieves better accuracy for several spyware classes, such as 93.85% for FamiSafe and 92.62% for Normal Traffic in MAS-2024--B. This performance can be attributed to the ensemble models' ability to handle non-linear relationships and contextual dependencies, which are crucial in diverse and complex data environments (Zhou et al., 2020). On the other hand, ML models like XGB and CatBoost achieve better performance with much lower computational overhead, as seen in cases like AT-P-C (93.82% accuracy for XGB in MAS-2024-A). DL models such as LSTM and CNN, while delivering relatively a little lower accuracy scores, demonstrates capability in learning sequential patterns (e.g., Eyezy with LSTM achieving 85.99% accuracy) (Mortezapour, Perumal, Mustapha, & Mohamed, 2024).

The variation in model performance across methods and spyware classes demonstrates the correlation between dataset characteristics and the specific architectural strengths of the models (Talaie Khoei, Ismail, Shamaileh, Devabhaktuni, & Kaabouch, 2022). In MAS-202-A, ML models like XGB and LGB achieves notable success due to their ability to handle structured data efficiently. However, when applying MAS-2024-B, ensemble models like LSTM-XGB and TabNet-XGB provides to be more robust, handling data more effectively. For instance, TabNet-XGB

achieves higher precision (84.67%) and recall (84.56%) for KidsGuardPro in MAS-2024-B, showing its adaptability in such contexts. Notably, the DL models consistently underperformed in both datasets, indicating that their reliance on specific features such as LSTM reliance on temporal features (Bandwidth, Packet Rate etc) (Fang et al., 2022), CNN reliance on spatial features (Total Fwd and Bwd Pkt etc) in the dataset for optimal learning may have constraints their adaptability in this study (Wang et al., 2021).

Tables 7.1–7.4 offer a detailed view of the results, averaging the performances of all 12 models across micro-batch sizes ranging from 50 to 250 packets. These Tables show key insights into the relationship between micro-batch size and model performance. Across the spyware classes and both datasets (MAS-2024-A and MAS-2024-B), the 150-packet micro-batch consistently delivers the highest accuracy, precision, recall, and F1-score. For example, in MAS-2024-A, MoniMasterPro achieves an average accuracy of 84.95%, while Normal Traffic achieves 84.69%. Similarly, MAS-2024-B results follow this trend, with Kids-360 reaching 83.15% accuracy and Eyezy achieving 83.29%. These observations suggest that the 150-packet size shows an optimal balance between specificity and computational efficiency, allowing models to process meaningful context while avoiding excessive splitting (Lopez-Martin et al., 2018).

The performance decrease observed at larger micro-batch sizes (200 and 250 packets) highlights the reducing returns of aggregating too much data per batch (Mehmood & Bottaci, 2021). At higher sizes, contextual differentiation between data points may blur, reducing the effectiveness of model generalization (Hoffer, Ben-Nun, Hubara, Giladi, Hoefler, & Soudry, 2020). On the other end, smaller micro-batch sizes (50 and 100 packets) also result in suboptimal performance due to insufficient contextual information in each batch, leading to an incomplete understanding of the data (Keskar, Mudigere, Nocedal, Smelyanskiy, & Tang, 2017). This finding emphasizes the importance of selecting an appropriate micro-batch size that is adapted to the data characteristics and model architecture (Goodfellow, Bengio, & Courville, 2018).

The selection of a 150-packet micro-batch size was not only empirically justified but also aligned with known patterns of spyware communication. Spyware tools often exfiltrate data in short, burst-like transmissions to evade detection, might be ranging between 100 and 200 packets per session (Zhao et al., 2021; Chen et al., 2020). These bursts might design to mimic benign traffic and avoid triggering anomaly-based intrusion detection systems. A micro-batch size of 150 packets might capture the full scope of such exfiltration events without fragmenting them across batches, preserving temporal and contextual integrity (Zhao et al., 2021). This size might offer a balance: it might large enough to encompass complete malicious flows, yet small enough to maintain specificity for classification. In contrast, smaller batch sizes might fragment these bursts, losing critical behavioural features, while larger sizes might risk conflating malicious and benign traffic, decreased accuracy. Thus, the 150-packet configuration was optimal not only for model performance but also for faithfully representing the network behaviour of spyware during data exfiltration.

Normal Traffic consistently provides better average results across all metrics compared to other spyware classes, especially for precision and recall. This indicates that Normal Traffic data likely demonstrates more distinct and learnable patterns compared to the noisier and more complex spyware classes (Ring, Wunderlich, Scheuring, Landes, & Hotho, 2019). However, the spyware-specific results vary significantly, with some classes like Qustodio and KidsGuardPro performing well, while others like TiSpy and AT-P-C show slightly lower scores across all the metrics. These differences can be attributed to the built-in variability in data patterns and imbalances within individual spyware classes, presenting unique challenges to the models (Xu et al., 2022).

Overall, the logical pattern in the results emphasizes the importance of utilizing ensemble models for robust classification tasks, especially in complex and data scenarios (Zhang et al., 2012). The consistent performance of the 150-packet micro-batch size suggests that this configuration effectively balances the exchange between processing efficiency and classification accuracy (Lyzer AI, 2024). The findings also highlight that selecting the appropriate model and micro-batch size combination for a specific class or dataset can significantly influence the overall

performance, showing the way for more efficient real-time classification strategies (Sze, Chen, Yang, & Emer, 2017).

7.5 Conclusions

The research executed in Chapter 7 focused on the implementation and evaluation of micro-batch processing for spyware classification. The study applied micro-batch sizes ranging from 50 to 250 packets to determine the most effective size for near real-time detection. The models used in this analysis were saved during batch-processing data, ensuring that their performance was evaluated under realistic conditions. The experiments demonstrated that a micro-batch size of 150 packets provided the best balance between accuracy, computational efficiency, and classification effectiveness.

The findings indicated that certain ensemble and ML models, particularly LSTM-XGB and FFNN-XGB, XGB, LGB, and CatBoost better perform than other ML, DL and ensemble models in most spyware classification. These models showed the advantages of both DL's sequential pattern recognition and ML's structured decision-making capabilities. The results showed that LSTM-XGB achieved better accuracy for several spyware classes, confirming its robustness in handling diverse network traffic patterns. While ML models such as XGB also demonstrated strong classification performance, it showed lower computational overhead compared to DL models, making them efficient for near real-time applications.

Analysis of micro-batch sizes revealed that small batch sizes (50 and 100 packets) results in fragmented data, leading to reduced classification accuracy due to insufficient contextual information. Conversely, larger batch sizes (200 and 250 packets) obscured key patterns within the data, causing reduced model performance. The optimal micro-batch size of 150 packets provided a balance between maintaining meaningful packet sequences and minimizing noise, leading to improved accuracy and reliability in classification. The study also highlighted the challenges presents by data imbalance among spyware classes. Certain spyware types, such as Qustodio and KidsGuardPro, showed high classification performance, while others, including TiSpy and AT-P-C, faced difficulties due to less distinguishable traffic patterns.

Furthermore, the study validated that micro-batch processing offered a practical solution for adapting from batch classification to near real-time detection. The ability to classify network traffic in micro-batches allowed for more responsive and adaptive spyware classification without requiring model retraining. These insights provided a basis for aligning micro-batch processing with near real-time detection strategies, ensuring enhanced adaptability in dynamic network environments.

In conclusions, the research in Chapter 7 established that micro-batch processing was a feasible intermediate step between batch processing and near real-time detection. The findings emphasized the importance of selecting an optimal micro-batch size to enhance classification accuracy while maintaining computational efficiency. The results contributed to the field of network traffic analysis by showing the effectiveness of ensemble models in micro-batch scenarios and facilitating the way for further advancements in near real-time spyware detection.

CHAPTER 8

NEAR REAL-TIME DETECTION

8.1 Introduction

After micro-batch processing, near real-time detection is applied. Capturing packets directly from the Android device and predicting using saved models is a challenge. The models run on Jupyter Notebook, which is not feasible on the Android device, so they run on a Desktop. To enable near real-time detection, network packets must be captured directly from the Android device. For this, Bluetooth tethering is activated on the Android device. The desktop computer, where the saved models are running, connects to the Android device via Bluetooth tethering. This setup allows the network traffic transferred to the Android device to be simultaneously captured by the desktop (Pixel Phone Help, 2025; Unit 3, 2025 & Madhava, 2024).

The process begins by establishing the Bluetooth tethering connection between the Android device and the desktop computer. Once the connection is verified using the “ipconfig” command, confirming that the desktop is utilizing the Android devices internet connection, Scapy is employed. It sniffs network traffic directly from the Android device, and the saved models running on the desktop predict the network behaviour base on this captured traffic (Pixel Phone Help, 2025; Unit 3, 2025 & Madhava, 2024). Figure 8.1 illustrates the steps for near real-time detection.

To maintain efficiency in near real-time detection, micro-batch processing is utilized, with an optimal batch size of 150 packets. Each batch of packets is grouped into network flows based on IP addresses, ports, and protocols, ensuring structured feature extraction. Then dynamic features such as packet rate, bandwidth, flow duration, total packet count are computed.

For detection, extracted features are fed into the saved model, stores in a pickle format. Additionally, the system applied rules-based analysis to detect anomalies, such as abnormal uplink-to-downlink ratios, unique destination IP thresholds, and packet flow status (Huabing et al., 2021). Traffic detection occurs in controlled micro-batches

preventing unnecessary retraining of the models while maintaining timely detection (Gandhi et al., 2022; Kim et al., 2020 & Kumar et al., 2023).

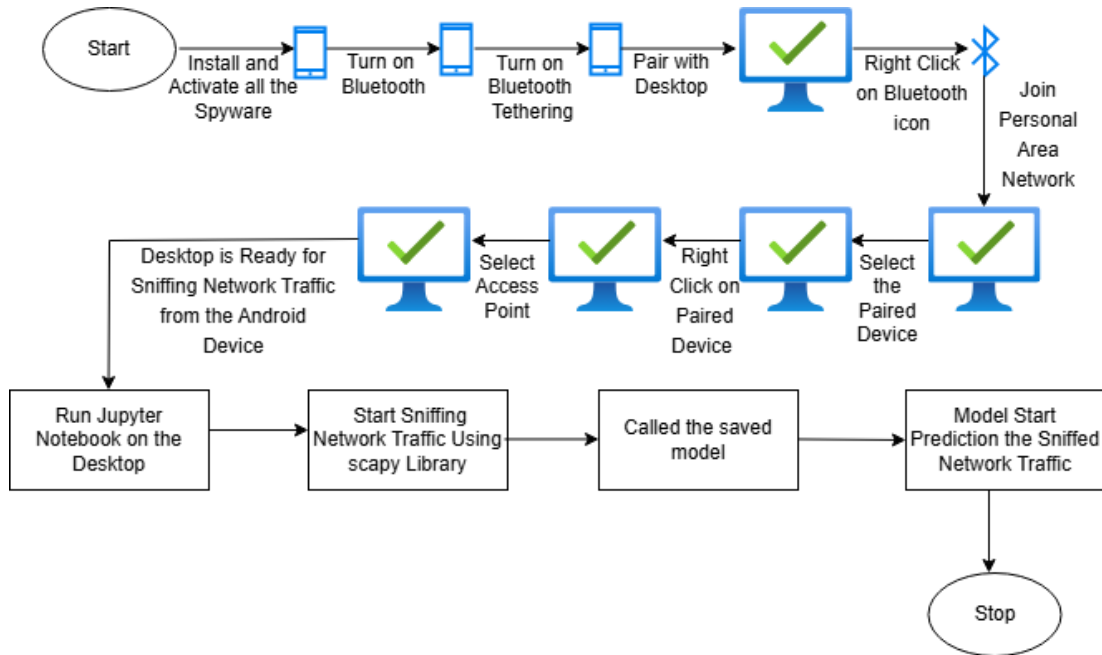


Figure 8.1 Flow chart-shows the steps for the set-up for near real-time spyware detection.

The Bluetooth tethering configuration was chosen over Wi-Fi, hotspot or network tap methods due to its controlled and isolated connectivity, which minimized interference from external networks. This setup ensured that all packet flows originated solely from the designated Android device, enhancing experimental precision and data integrity (Pixel Phone Help, 2025). While Bluetooth offers lower bandwidth compared to Wi-Fi, the limited throughput was sufficient for spyware traffic analysis since most spyware transmits lightweight command-and-control or metadata packets (Shirsat, Shirsat, & Yadav, 2018). An additional advantage was the ability to avoid IP conflicts or background internet traffic from other devices sharing the same Wi-Fi network (Cäsar, Pawelke, Steffan, & Terhorst, 2022).

8.2 Prototype

This Section presents the prototype of the spyware classification system, which is designed to classify spyware types alongside normal traffic by analysing

single-directional or bidirectional flow properties. The system is developed with a user-friendly interface, allowing users to select the preferred learning model and specify whether to use single-directional or bidirectional flow for near real-time detection of spyware and Normal Traffic. Additionally, the system provides an evaluation mechanism to assess the performance of the selected model (Ahmed et al, 2021).

The prototype integrates multiple learning models, which is saved during batch processing and each supporting both single-directional and bidirectional flow analysis. The system is implemented using Flask, a lightweight web framework, to facilitate near real-time execution of classification models. The interface enables users to capture network packets, execute the selected model, and visualize the classification results dynamically (Ahmed et al, 2021).

To ensure efficient processing, the system suppresses unnecessary warnings and optimizes execution by handling notebook-based model execution. The output is cleaned to remove irrelevant information, ensuring that only meaningful classification results are displayed. The interface also incorporates a background image for improved readability.

By providing the selection of multiple models, near real-time classification capabilities, and an intuitive interface, this prototype serves as a robust spyware detection system. The design ensured flexibility, usability, and accuracy, making it suitable for practical deployment in cybersecurity applications (Ahmed et al, 2021).

The primary function of the system is to allow users to choose the learning model and decide whether it goes for single directional or bidirectional flow for classifying spyware and Normal Traffic. From the drop-down button the user will choose the options as the user want, click on the “Capture Packets” button and displays the output from the selected learning model as shown in Figures 8.2 to 8.4.

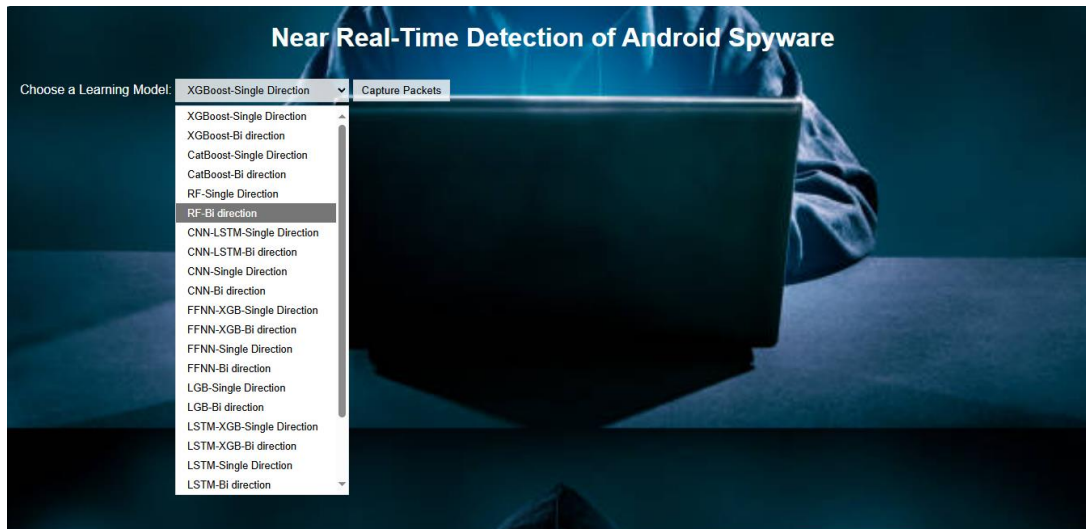


Figure 8.2 User interface for model and flow type selection.

After choosing the preferred learning model with the flow direction (single or bidirectional), then the user needs to click on the button “Capture Packets”.

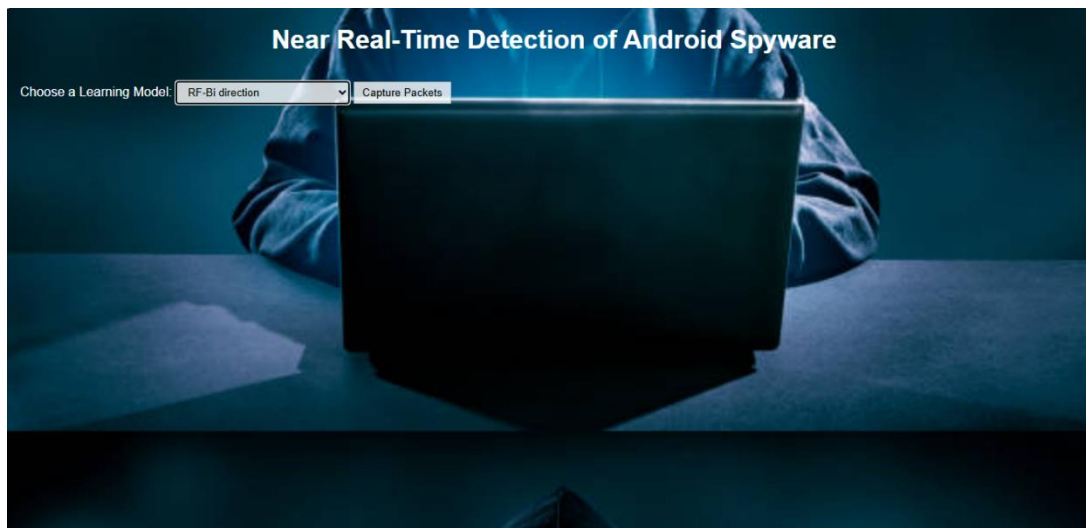


Figure 8.3 Interface showing packet capture after model selection.

After click on the “Capture Packets” button, within 40 seconds it provides the prediction with evaluation results (accuracy, precision, recall and f1-score) for that specific model. This time limit is chosen based on the performance analysis of different models. As shown in Table 8.2, the average maximum execution time for single-directional processing was 36.86 seconds, while for bi-directional processing, it was

39.89 seconds. To ensure the system accommodates all models efficiently, a 40-second limit is set for the user interface.



Figure 8.4 Classification result output of the selected model.

The Flask-based prototype was implemented as a lightweight interface within the Jupyter Notebook, connecting the network sniffer, feature extractor, and classification model (Ahmed et al, 2021). Figure 8.1 illustrated the complete data flow architecture. Packets were first captured from the Android device via Bluetooth tethering and received by the Scapy sniffer on the host machine. The sniffer then extracted key directional features (packet count, flow duration) and transferred these to the Flask application. Flask handled the data routing and passed formatted feature vectors to the saved model, which performed real-time spyware classification and displayed results through the web dashboard (Ahmed et al, 2021). This design allowed modular testing, rapid model replacement, and near real-time display of classification outcomes. On the other hand, Flask's single-threaded processing might limit concurrent flow handling, and Bluetooth throughput might constrain scalability beyond one active device (Ahmed et al, 2021).

8.3 Results

After micro-batch processing, this research finds out that the micro-batch size of 150 packet provides the better results so during near real-time detection this research applies 150 packets which shows in Tables 8.1 and 8.2.

Table 8.1 Evaluation matrix results for near real-time detection.

Models	Single direction				Bi direction			
	Acc (%)	Pre (%)	Rec (%)	F1 (%)	Acc (%)	Pre (%)	Rec (%)	F1 (%)
XGB	80.01	80.46	79.58	80.38	76.31	76.58	76.42	76.12
LGB	79.02	79.09	79.01	79.03	74.44	74.35	74.19	74.25
CatBoost	79.15	78.56	79.15	78.85	76.36	76.82	76.24	76.53
RF	73.27	72.63	72.54	72.58	71.01	70.14	69.91	70.02
CNN	73.02	72.68	73.50	73.09	70.88	69.93	71.41	70.66
LSTM	78.69	78.56	79.66	79.11	75.39	75.71	76.17	75.94
FFNN	66.69	66.89	64.77	65.81	61.12	61.53	62.66	62.09
TabNet	59.85	59.92	60.60	60.26	58.27	59.27	57.05	58.14
CNN-LSTM	76.48	76.22	77.95	77.08	77.72	78.07	77.00	77.53
FFNN-XGB	79.15	79.14	78.02	78.58	76.09	75.84	75.67	75.75
LSTM-XGB	76.26	75.98	76.32	76.15	77.86	78.65	78.19	78.42
TabNet-XGB	78.36	77.64	79.17	78.39	77.93	77.30	78.12	77.71
Average	74.99	74.81	75.02	74.94	72.66	72.85	72.75	72.76

Table 8.2 Execution time for near real-time detection.

Models	Single direction				Bi direction			
	Time (Second)		Number of Flow		Time (Second)		Number of Flow	
	Min	Max	Min	Max	Min	Max	Min	Max
XGB	10.33	36.74	8	19	11.31	48.50	8	18
LGB	7.62	59.11	10	21	14.82	75.33	12	18
CatBoost	2.33	26.82	4	14	32.87	46.80	8	13
RF	1.35	31.89	2	20	0.56	39.84	2	24
CNN	0.02	33.07	2	28	2.76	37.99	10	44
LSTM	0.02	30.88	2	35	0.02	37.23	2	22
FFNN	10.27	39.22	10	18	13.21	33.77	9	20
TabNet	0.87	34.99	6	26	14.16	33.81	10	16
CNN-LSTM	10.06	31.47	12	21	0.24	35.46	4	22
FFNN-XGB	14.47	34.14	12	18	28.29	32.31	13	20
LSTM-XGB	1.65	33.48	2	23	2.33	38.68	2	20
TabNet-XGB	14.71	38.79	13	25	1.41	30.83	2	25
Average	6.56	36.86	6	22	10.17	39.89	6	21

8.4 Detailed Analysis of the Results

The results in the evaluation metrics demonstrates that XGB consistently achieves the highest performance across all metrics under single directional properties, which is likely due to its ability to handle complex relationships in the data effectively. However, its performance slightly decreases under bidirectional properties, possibly due to increases data complexity. While single directional has less complex structure then bidirectional flow which involves communication moving in only one direction, whereas bidirectional flow requires synchronization and interaction in both directions, adding complexity to the structure and process (Atlassian, 2024 & This vs. That, 2024). LGB, while comparable to XGB, shows a little lower result, which can have been connected to its emphasis on speed over complex pattern classification (Restack,

2024). CatBoost maintains stable performance due to its handling of categorical features, though minor declines in MAS-2024-B suggests adaptation challenges. RF and CNN shows moderate performance, with RF's ensemble nature limiting its ability to classify intricate patterns, and CNN which is designed to identify and process such spatial relationships effectively through convolution operations (Krizhevsky et al., 2012). LSTM demonstrates strong recall and F1-scores, reflecting its strength in identifying sequential patterns, its performance also moderates for both datasets, indicating the dataset's temporal dependencies (Hochreiter et al., 1997). TabNet delivers the lowest results, which is expected given its design focus on transparency rather than absolute performance (Arik et al., 2021). Ensemble models, such as CNN-LSTM and LSTM-XGB, performs better due to their complementary strengths, although some fluctuation between methods suggested differences in adaptability to near real-time conditions (Zhao et al., 2020). Execution times varied, with simpler models like XGB completing tasks faster, while hybrid and deep-learning models requires more time due to their higher computational demands. Overall, the performance differences stem from the unique strengths and limitations of each model, as well as their adaptability to near real-time tabular data analysis.

Table 8.2. represents the execution time during near real-time detection, where the minimum and maximum execution time are influenced by the number of flows. In near real-time detection, a micro-batch of 150 packets is processed at a time. Instead of making prediction for each individual packet, the system generates flows based on directional properties within these packets. When the number of flows is lower, the execution time is less because fewer flows require less computation for feature extraction and prediction. Conversely, when the number of flows, which involves more computation. This variation reflects the direct correlation between the computational load and the volume of flows generated from the packets (Saydam et al & Maggio 2020). After near real-time detection, it found that the XGB model provides better accuracy for single directional feature engineering. Also, single directional feature engineering generally has a faster execution time than bidirectional feature engineering.

8.5 Discussions

8.5.1 Near Real-Time Detection

When the micro-batch size was too small (50-100 packets), the amount of information available for classification is limited. This restriction hinders the classification algorithm's ability to extract meaningful features, leading to lower accuracy (Xu et al., 2023). On the other hand, if the micro-batch size was too large (200-250 packets), the latter part of the payload often contains irrelevant transmission data, which increased computational overhead and introduces noise, making it more challenging to identify anomalies (Xu et al., 2023). This research found that an optimal micro-batch size is 150 packets makes a balance between having sufficient informative data and minimizing noise, resulting in better classification accuracy.

Near real-time and micro-batch processing must adapt to dynamic network conditions, such as varying traffic loads and packet loss. These conditions introduce variability in the data being analysed, making it more challenging to maintain consistent accuracy which provides batch processing better results than micro-batch processing and near real-time detection (Mohammad et al, 2009; Gandhi et al, 2022; Warriar et al, 2022).

In near real-time analysis, the models do not retrain with new samples; they are applied to predict and provided evaluation metrics. This approach reduces computation time and resource usage significantly. The results showed that single directional features achieve an average accuracy of 74.99%, and bidirectional features achieves 72.66%, showing the efficiency of near real-time packet sniffing and prediction without retraining. This efficiency is critical for practical deployment, where low computational overhead and quick response are important.

The efficiency and effectiveness align with findings from other studies such as (Niu et al, 2022) highlighted the benefits of dynamically updating models with streaming data, although this added computational overhead. Kim et al., (2020) discussed optimizing models for near real-time traffic, emphasizing continuous training's computational demands. This research overcomes these issues by using

saved models, ensuring faster processing and less resource allocation, which is important for near real-time spyware detection.

Near real-time and micro-batch processing must adapt to dynamic network conditions, such as varying traffic loads and packet loss. These conditions introduce variability in the data being analysed, making it more challenging to maintain consistent accuracy which provided batch processing better results than micro-batch processing and near real-time detection (Gandhi et al., 2022; Mohammed et al., 2009 & Warriar et al., 2022).

In near real-time analysis, the models were not retrained with new samples; they were applied to predict and provided evaluation metrics. This approach reduced computation time and resource usage significantly. The results showed that Method A achieved an average accuracy of 74.99%, and Method B achieved 72.66%, showing the efficiency of real-time packet sniffing and prediction without retraining. This efficiency is critical for practical deployment, where low computational overhead and quick response are important.

To address variability in network conditions such as packet loss, latency variations, and heavy traffic loads, this study incorporated adaptive mechanisms to maintain consistent classification performance. Specifically, the proposed system utilized micro- batch processing, which balanced the adjustment between having sufficient informative data and minimizing noise caused by excessive payload. This study ensured that even under high traffic loads or packet loss scenarios, the system could process data efficiently without significant degradation in accuracy. Additionally, the use of saved models for near real-time analysis reduced the computational overhead and latency, enabling faster predictions and adaptability to dynamic network conditions. Similar approaches had been validated in prior research, such as the application of artificial intelligence to mitigate network delay and packet loss through predictive analytics and real-time adjustments (Mohammed et al., 2024). Furthermore, ML-based anomaly detection systems have demonstrated effectiveness in identifying congestion and packet loss as key factors in network variability, ensuring scalability in real-world deployments (Schummer et al., 2024). These strategies

enhanced the robustness of the classification system, making it suitable for deployment in less controlled, real-world environments.

While confidence intervals or statistical significance tests were critical in many studies to provide a range of plausible values for performance metrics and ensure the reliability of results, this study focused on providing comparative performance analysis directly through evaluation metrics such as accuracy, precision, recall, and F1- score. Similar practices were adopted in ML research to emphasize model performance and computational trade-offs (Caruana & Niculescu-Mizil, 2006 & Kotsiantis, Zaharakis, & Pintelas, 2007). Confidence intervals, while valuable, might not always align with studies that prioritize near real-time processing and performance evaluation under dynamic constraints. As highlighted by (Mohammed et al.; 2024), practical considerations in dynamic environments often emphasized direct comparative metrics rather than statistical confidence intervals due to the adaptive nature of the models. Future work will be explored the integration of confidence intervals or statistical significance tests to enhance the statistical rigor of comparisons.

The efficiency and effectiveness align with findings from other studies such as (Bendiab, Shiaeles, Alruban, & Kolokotronis, 2020) highlighted the benefits of dynamically updating models with streaming data, although this added computational overhead. (Niu et al., 2021) discussed optimizing models for real- time traffic, emphasizing continuous training's computational demands. This research overcomes these issues by using saved models, ensuring faster processing and less resource allocation, which is important for near real-time spyware detection.

8.5.2 Advancements in ML/DL Methods for Near Real-Time Analysis

This research contributed to enhancing near real-time spyware detection by applying directional properties that extend beyond simply applying existing models. By integrating CNN-LSTM and LSTM-XGB, this study showed the strengths of spatial and temporal feature learning while improving classification accuracy. The LSTM output was fed into XGB, which optimally boosts performance by penalizing mis- classification and addressing overfitting in noisy samples. Additionally, micro-batch processing was adapted to near real-time constraints, optimizing batch sizes to

reduce latency and computation time while maintaining high accuracy. The use of Bluetooth tethering and Scapy for real-time packet capture introduces a novel framework that bridges the gap between mobile device limitations and desktop processing capabilities (Pixel, 2025; Unit 3, 2025 & Madhava, 2024). These innovations addressed practical challenges in near real-time analysis while ensuring scalability and efficiency. Collectively, these advancements contribute to refining ML and DL methods for near real-time spyware detection, offering a robust and practical solution for dynamic cybersecurity environments.

This research advanced the state-of-the-art in network traffic-based near real-time spyware detection by addressing key challenges faced in existing methods. Unlike prior studies, which relied on continuously updating models or computationally expensive real-time training (Bendiab et al., 2024 & Niu et al, 2022), this research applied saved models that significantly reduced computational overhead and latency, enabling efficient near real-time analysis (Kim et al., 2020; Kumar et al., 2023 & Yu et al., 2024). Additionally, while many studies focused on either ML or DL models, this research integrated several learning models, including CNN-LSTM and LSTM-XGB, to leverage spatial and temporal feature learning, which enhanced classification accuracy by addressing the challenges of noisy samples and misclassification. Moreover, the used of adaptive micro-batch processing strike a balance between processing efficiency and maintaining good accuracy, better methods that prioritize either speed or accuracy alone. The real-time packet capture framework using Bluetooth tethering and Scapy further set this approach apart by introducing a novel solution to bridge mobile device constrained with desktop-based processing (Pixel, 2025; Unit 3, 2025 & Madhava, 2024). Collectively, these innovations offered a robust, scalable, and efficient solution, showing clear advancements over existing state-of-the-art methods in real-world, dynamic cybersecurity environments.

8.5.3 Performance Validation on Alternative Dataset

To evaluate whether other datasets can be effectively used with the proposed near real-time detection pipeline, this study utilized the publicly available dataset from Qabalin et al. (2022). After downloading the dataset, the same pipeline was applied,

including feature engineering, rule-based labelling, and the creation of two directional datasets: single-directional and bi-directional network traffic flow features.

Multiple ML models were tested across different train-test split ratios, where the 70:30 split demonstrated the optimal performance. For single-directional data, SVM achieved the highest accuracy at 99.88%, while RF and XGB performed best on bi-directional data with 99.24% accuracy. These best performing models were saved for further evaluation.

Next, micro-batch processing was conducted using micro-batch sizes ranging from 50 to 180 packets. The experiments revealed that a batch size of 120 packets delivered the best performance across all saved models. Subsequently, near real-time analysis was performed, where XGB achieved the highest accuracy of 75.00% on bi-directional data.

This confirms that the near real-time detection pipeline developed in this research is applicable to other datasets. While the optimal micro-batch size for the original research dataset was 150 packets, the downloaded dataset performed best at 120 packets. In both cases during near real time detection, XGB consistently better performed than other models, achieving 80.01% accuracy on the research dataset and 75.00% on the downloaded dataset. Tables 8.3 through 8.8 present the experimental results obtained from the downloaded dataset, along with a comparative analysis of performance between the research-developed and downloaded datasets.

Table 8.3 Results of batch processing with a 70:30 train-test validation on the downloaded dataset.

Single Direction	Models	Accuracy	Precision	Recall	F1-Score
	SVM	99.88%	99.88%	99.88%	99.88%
Bi-Direction	RF	99.24%	99.25%	99.24%	99.24%
	XGB	99.24%	99.25%	99.24%	99.24%

Table 8.4 Micro-batch processing accuracy (%) of RF across different classes in the downloaded dataset.

Classes	50	60	70	80	90	100	110	120	130	140	150	160	170	180
FlexiSpy	68.98	67.14	71.37	73.12	75.49	76.15	81.17	81.59	77.83	75.18	71.47	69.63	66.36	66.95
MobileSpy	68.84	67.72	69.08	74.14	76.15	77.72	79.87	81.00	76.06	74.20	71.35	70.44	66.28	64.79
mSPY	64.61	65.94	67.72	68.28	71.14	75.48	76.49	77.72	72.09	70.74	67.19	66.23	63.91	60.61
TheWISPY	63.87	64.28	68.94	68.87	72.44	74.52	76.65	78.94	76.33	73.76	72.55	70.40	67.00	66.98
uMobix	62.66	63.43	65.50	66.66	68.04	68.26	71.72	73.12	69.88	67.49	64.79	63.49	63.20	61.31

Table 8.5 Micro-batch processing accuracy (%) of XGB across different classes in the downloaded dataset.

Classes	50	60	70	80	90	100	110	120	130	140	150	160	170	180
FlexiSpy	64.50	65.92	66.35	67.33	68.29	71.62	73.16	76.62	72.19	72.70	67.50	66.49	64.04	62.35
MobileSpy	65.00	67.84	68.20	71.99	72.59	75.70	76.62	78.68	76.64	74.42	72.71	68.04	64.81	62.88
mSPY	64.17	66.24	69.11	70.77	74.17	76.83	76.88	79.54	76.52	73.66	72.19	69.42	66.31	63.22
TheWISPY	65.98	68.37	68.59	70.14	71.12	75.76	75.96	77.86	75.62	73.54	68.58	66.96	63.90	61.43
uMobix	63.96	66.17	68.32	72.47	75.55	78.86	75.84	78.82	74.51	73.32	68.86	65.51	62.12	60.50

Table 8.6 Micro-batch processing accuracy (%) of SVM across different classes in the downloaded dataset.

Classes	50	60	70	80	90	100	110	120	130	140	150	160	170	180
FlexiSpy	60.39	62.61	65.72	66.47	69.84	72.21	74.00	74.43	72.75	68.25	66.81	64.48	62.58	60.35
MobileSpy	61.52	63.36	66.52	68.14	70.28	72.31	74.80	77.73	74.59	71.32	69.03	66.84	63.58	60.62
mSPY	61.59	63.99	65.76	67.82	68.75	70.39	74.06	77.62	73.05	70.99	68.96	66.52	62.71	61.18
TheWISPY	60.72	62.31	65.65	67.42	68.44	72.05	73.57	76.45	74.06	70.24	68.59	64.48	62.29	61.19
uMobix	62.19	65.28	67.94	68.89	71.08	73.07	74.72	78.27	76.08	71.34	68.81	65.49	63.76	60.29

Table 8.7 Near real-time detection results with the downloaded dataset.

Single Direction	Models	Accuracy	Precision	Recall	F1-Score
	SVM	73.07%	73.75%	73.07%	73.41%
Bi Direction	XGB	75.00%	75.67%	75.00%	75.33%
	RF	71.15%	70.89%	69.21%	70.04%

Table 8.8 Performance comparison between developed and downloaded datasets.

Performance		Developed Datasets		Downloaded Datasets	
Best train-test split ratio		80-20		70-30	
Best batch processing model	Single Direction	LSTM-XGB	99.03% accuracy	SVM	99.88% accuracy
	Bi direction	LSTM-XGB	99.04% accuracy	RF, XGB	99.24% accuracy
Optimal micro batch size		150 pkt		120 pkt	
Near real time performance		XGB	75.00% accuracy	XGB	80.01% accuracy

8.5.4 Limitations of Real-Time Detection: Computational and Practical Constraints

The detection method applied in this research is near real-time detection, rather than real-time system, due to the feature engineering process and batch classification strategy. In real-time detection system, each individual packet will require instant processing and detection, meaning prediction will occur on a per-packet basis. However, the packet flow-based feature engineering methodology applies in this research relies on grouping packets based on directional properties, rather than analysing single packets independently.

This distinction is critical because certain key features such as Total Fwd and Bwd Pkt, min-max-std-mean calculations for Fwd and Bwd Pkt remain unchanged when considering individual packets. However, when packets are grouped into flows, these feature values vary dynamically, providing meaningful insights into spyware traffic behaviour (Conti et al., 2023 & Qabalin et al., 2022).

Additionally, a micro-batch processing framework is applied, where packets are structured into flows before detection. Micro-batch sizes of 150 packets are selected to optimize execution speed while maintaining detection accuracy. This approach enables efficient network traffic detection while ensuring the faster execution time, even though it does not process each packet in isolation (Gandhi et al., 2022; Kim et al., 2020 & Kumar et al., 2023).

Implementing real-time detection system presents several practical and computational challenges. One of the primary limitations lies in feature engineering design, as spyware traffic analysis relies on flow-based feature extraction rather than per-packet detection. Additionally, computational constraints further complicate real-time implementation, as instantaneous feature extraction and prediction for every packet would significantly increase processing overhead (Pandit, Bian, Prasad, Kwatra, Lu, Riess, Xu., 2018 & Clark, Jacobson, Romkey, Salwen 1989).

The observed reduction in accuracy from 98–99% in batch processing to approximately 80% in near real-time detection primarily stemmed from the reduced context available within smaller micro-batches (150 packets per window). Smaller windows limit temporal flow continuity, preventing the models from fully capturing sequential spyware communication patterns (Gandhi et al., 2022). Additionally, real-time traffic might introduce minor noise and irregular packet delays that differ from the stable offline dataset conditions (Pandit et al., 2018). The use of static saved models without online adaptation also might contribute to reduced generalization under dynamic network behaviour (Pandit et al., 2018). These combined factors explained the gap between offline and live detection accuracy.

8.5.5 Comparative Analysis with Existing Studies

To evaluate the performance of the proposed system, this study compared its results with several key existing works discussed in Chapter 2. Qabalin et al. (2022), achieved an accuracy of 67% using DT on five types of spyware based on network traffic. Conti et al. (2020) achieved 93.8% accuracy using RF for spyware classification on static datasets. Similarly, Li et al. (2021) achieved 99% accuracy on CICIDS2017 dataset for anomaly traffic classification using CNN-LSTM, while Kim

et al. (2020) achieved 91% accuracy for real-time HTTP traffic. Unlike previous studies, this research achieved 99.03% accuracy for MAS-2024-A and 99.04% accuracy for MAS-2024-B in batch processing using the LSTM-XGB ensemble model (Table 6.1) and maintained 80-82% accuracy during micro-batch and near real-time detection (Tables 7.1 and 8.1). Unlike previous studies that relied on static APK analysis or computationally heavy training, this study introduced directional network traffic features, rule-based labelling, and saved models for near real time detection, reducing latency while maintaining high accuracy. These comparative results indicated that the proposed study provided a balanced improvement in both accuracy and processing efficiency, positioning it as a practical advancement over previous methods.

8.6 Conclusions

The study successfully implemented near real-time spyware detection using saved learning models. The research evaluated various micro-batch sizes, with 150 packets proving to be the most effective in accuracy and computational efficiency. Near real-time classification was conducted by capturing network traffic from an Android device and processing it using saved models on a desktop. The system enabled users to choose different models and flow types, allowing flexibility in classification.

Among the saved models, XGB consistently better performed others, achieving the highest accuracy in classification spyware. LightGBM, CatBoost, and LSTM also demonstrated good results, but XGB remained the most reliable for near real-time detection. The evaluation of execution time indicated that the complexity of the model and the number of flows directly influenced the performance. Single-directional feature showed faster processing times compared to bi-directional features, confirming its suitability for real-time scenarios.

The findings demonstrated that near real-time detection using saved models was feasible and effective in identifying spyware. The system eliminated the need for continuous model retraining, reducing computational overhead while maintaining good classification accuracy. This research contributed to the advancement of spyware classification by providing a practical and efficient framework for near real-time

detection. Future work could focus on optimizing model efficiency and expanding the dataset to improve classification accuracy further.

While specific false positive and false negative rates were not computed due to the multi-class classification setup across 14 spyware types along with Normal Traffic, the reduced accuracy suggested that some misclassifications were expected. These were likely to occur between spyware classes with overlapping behavioural features, especially while each class did not contain same sample number count. For example, a flow from one spyware type might be incorrectly predicted as another, though the system still correctly identified it as spyware. Normal traffic, validated through rule-based labelling and manual inspection of clean PCAP files, remained reliably classified. In practice, this meant that while the system can flag spyware presence effectively, analysts might need to verify the exact type before acting. This supported the system's potential for real-world use, with future work aimed at improving per-class precision and reducing misclassification noise.

CHAPTER 9

CONCLUSIONS

9.1 Conclusions

This research focused on spyware classification, applying both offline and near real-time detection. 14 types of spyware network traffic were collected alongside Normal Traffic, including Air-Droid, AT-B-D-M, AT-P-C-M, AT-C-M, Eyezy, FamiSafe, FamiGuardPro, mLite, kids-360, kidsGuardPro, MoniMasterPro, mSpy, Qustodio, and TiSpy. A total of 22 features was extracted from the network traffic, and two datasets were developed based on packet directionality: MAS-2024-A (single-directional) and MAS-2024-B (bi-directional) and applied rule-base labelling for distinguish between spyware and Normal-Traffic.

For offline classification, batch processing was applied using various scenarios, including balancing techniques like SMOTE and RUS, as well as Original datasets. Different train-validation-test splits were applied, and the analysis utilized various learning models. ML models included XGB, LGB, CatBoost, and RF. DL models included CNN, LSTM, FFNN, and TabNet. Ensemble models combined DL and ML, such as CNN-LSTM, TabNet-XGB, LSTM-XGB, and FFNN-XGB. It observed that the 80-20 split ratio provided the best performance for the Original datasets. Among ML models, XGB achieved the highest accuracy, at 98.63% for MAS-2024-A and 98.66% for MAS-2024-B. For DL models, LSTM achieved better results with 92.41% accuracy for MAS-2024-A and 90.16% for MAS-2024-B. Among ensemble models, LSTM-XGB achieved 99.03% accuracy for MAS-2024-A and 99.04% for MAS-2024-B. All the models with 80-20 split ratio for the Original datasets were saved as .pkl files for further classification.

For micro-batch processing, packet sizes ranging from 50 to 250 were analysed, with 150-packet micro-batches demonstrating optimal results. The average accuracies achieve were 76.29% to 84.95% across different classes for MAS-2024-A and 74.18% to 83.23% for MAS-2024-B. During near real-time analysis, utilizing the

saved models and 150-packet micro-batch size, single directional feature achieved 74.99% average accuracy, while bidirectional feature achieved 72.66%. XGB showed as the better performing model in near real-time analysis, with 80.01% accuracy for single directional features. Additionally, single directional feature engineering was more efficient for near real-time detection, with a faster execution time of 36.36 seconds compared to bidirectional feature engineering's 39.89 seconds.

This research successfully applied network traffic-based spyware classification techniques using various learning models for both offline and near real-time detection, providing valuable insights and practical solutions for spyware classification.

9.2 Recommendations for Future Work

Future research will focus on enhancing real-time spyware classification by integrating adaptive learning mechanisms. One direction will be the implementation of federated learning models, where classification models will continuously be updated based on incoming traffic data without compromising user privacy (Xu et al., 2023). Building on this, low-latency classification models for packet level analysis may be explored to reduce reliance on batch processing (Xu et al., 2023).

To extend coverage under known threats, integrating anomaly-based techniques will support the identification of zero-day spyware attacks. Combining supervised and unsupervised models may boost robustness and adaptability (Cloud Security Web, Groenewald & Tinybird, 2024).

Finally, the use of synthetic spyware traffic through GANs may enhance model training and maintain effectiveness against evolving threats (Wiles et al., 2024).

REFERENCES

- Aboaoja, F. A., Zainal, A., Ghaleb, F. A., Al-Rimy, B. A. S., Eisa, T. A. E., & Elnour, A. A. H. (2022). Malware detection issues, challenges, and future directions: A survey. *Applied Sciences*, 12(17), 8482.
- Abri, F., Siami-Namini, S., Khanghah, M. A., Soltani, F. M., & Namin, A. S. (2019). The performance of machine and deep learning classifiers in detecting zero-day vulnerabilities. *arXiv preprint arXiv:1911.09586*.
- Acun, B., Murphy, M., Wang, X., Nie, J., Wu, C. J., & Hazelwood, K. (2021, February). Understanding training efficiency of deep learning recommendation models at scale. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)* (pp. 802-814). IEEE.
- Adair, S., Varol, C., Liang, F., & Pham, V. (2024, April). Malware API Call-Based Multiclass-Classification Using Machine Learning and Deep Learning. In *2024 12th International Symposium on Digital Forensics and Security (ISDFS)* (pp. 1-7). IEEE.
- Afuwape, A. A., Xu, Y., Anajemba, J. H., & Srivastava, G. (2021). Performance evaluation of secured network traffic classification using a machine learning approach. *Computer Standards & Interfaces*, 78, 103545.
- Ahmed, A. A., & Agunsoye, G. (2021). A real-time network traffic classifier for online applications using machine learning. *Algorithms*, 14(8), 250.
- Ahmed, M., Mahmood, A. N., & Hu, J. (2016). A survey of network anomaly detection techniques. *Journal of Network and Computer Applications*, 60, 19-31. <https://doi.org/10.1016/j.jnca.2015.11.016>
- Akhtar, M. S., & Feng, T. (2022). Malware analysis and detection using machine learning algorithms. *Symmetry*, 14(11), 2304.
- Al-Hamadi, H. A., & Chen, Y. (2018). Botnet detection based on network behavior. *Applied Soft Computing*, 80, 579–589. <https://doi.org/10.1016/j.asoc.2019.03.025>
- Al-Hashedi, M., Soon, L. K., Goh, H. N., Lim, A. H. L., & Siew, E. G. (2023). Cyberbullying detection based on emotion. *IEEE Access*, 11, 53907-53918.

Alkhawaldeh, I. M., Albalkhi, I., & Naswhan, A. J. (2023). Challenges and limitations of synthetic minority oversampling techniques in machine learning. *World journal of methodology*, 13(5), 373.

Allix, K., Bissyandé, T. F., Klein, J., & Le Traon, Y. (2016, May). Androzoo: Collecting millions of android apps for the research community. In *Proceedings of the 13th international conference on mining software repositories* (pp. 468-471).

Ali, S., Elgharabawy, M., Duchaussoy, Q., Mannan, M., & Youssef, A. (2020, December). Betrayed by the guardian: Security and privacy risks of parental control solutions. In *Proceedings of the 36th Annual Computer Security Applications Conference* (pp. 69-83).

Almomani, I., Almashat, T., & El-Shafai, W. (2024). Maloid-DS: Labeled Dataset for Android Malware Forensics. *IEEE Access*.

Al-Musawi, H. S. H. (2022). Hybrid Malware Detection and Classification in Real-Time by Deep Learning Techniques (Doctoral dissertation).

Alsumaidae, Y. A. M., Yahya, M. M., & Yaseen, A. H. (2025). Optimizing malware detection and classification in real-time using hybrid deep learning approaches. *International Journal of Safety and Security Engineering*, 15(1).

Altman, N. S. (1992). An introduction to kernel and nearest-neighbor nonparametric regression. *American Statistician*, 46(3), 175-185. <https://doi.org/10.1080/00031305.1992.10475879>

Amro, B. (2018). Malware detection techniques for mobile devices. *arXiv*. Cornell University.

Analytics Vidhya, "Gini Impurity," Nov. 1, 2024. [Online]. Available: <https://www.analyticsvidhya.com/articles/gini-impurity/>

Anderson, H. S., & Roth, P. (2018). Ember: an open dataset for training static pe malware machine learning models. *arXiv preprint arXiv:1804.04637*.

Apktool. (2024, December 1). Apktool. [Online]. Available: <https://apktool.org/>

Arik, S. Ö., & Pfister, T. (2021, May). Tabnet: Attentive interpretable tabular learning. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 35, No. 8, pp. 6679-6687).

- Arjunan, T. (2024). Real-time detection of network traffic anomalies in big data environments using deep learning models. *International Journal for Research in Applied Science and Engineering Technology*, 12(9). <https://doi.org/10.22214/ijraset.2024.58946>
- Aslan, Ö., & Yilmaz, A. A. (2021). A new malware classification framework based on deep learning algorithms. *Ieee Access*, 9, 87936-87951.
- Aswathy, M. C., & Rajkumar, T. (2024). Real-time anomaly detection in network traffic: A comparative analysis of machine learning algorithms. *International Research Journal on Advanced Engineering Hub (IRJAEH)*, 2(7), 1968–1977.
- Atlassian. Unidirectional vs Bidirectional Integration: Choosing the Right Approach. Retrieved March 22, 2024, from <https://community.atlassian.com/forums/App-Central-articles/Unidirectional-vs-Bidirectional-Integration-Choosing-the-Right/ba-p/2920403>
- Attobrah, M. (2024). *Essential Data Analytics, Data Science, and AI*.
- Azab, A., Khasawneh, M., Alrabaee, S., Choo, K. K. R., & Sarsour, M. (2024). Network traffic classification: Techniques, datasets, and challenges. *Digital Communications and Networks*, 10(3), 676-692.
- Bdair, A. H., Abdullah, R., Manickam, S., & Al-Ani, A. K. (2020). Brief of intrusion detection systems in detecting ICMPv6 attacks. In *Computational Science and Technology: 6th ICCST 2019*, Kota Kinabalu, Malaysia, 29-30 August 2019 (pp. 199-213). Springer Singapore.
- Bekkerman, R., Bilenko, M., & Langford, J. (Eds.). (2011). *Scaling up machine learning: Parallel and distributed approaches*. Cambridge University Press.
- Bendiab, G., Shiaeles, S., Alruban, A., & Kolokotronis, N. (2020, June). IoT malware network traffic classification using visual representation and deep learning. In *2020 6th IEEE Conference on Network Softwarization (NetSoft)* (pp. 444-449). IEEE.
- Bentéjac, C., Csörgő, A., & Martínez-Muñoz, G. (2019). A Comparative Analysis of XGBoost. *arXiv*. <https://arxiv.org/abs/1911.01914>
- Botacin, M., Ceschin, F., Sun, R., Oliveira, D., & Grégio, A. (2021). Challenges and pitfalls in malware research. *Computers & Security*, 106, 102287.

Bouchaib, P., & Bouhorma, M. (2021, April). Transfer learning and smote algorithm for image-based malware classification. In Proceedings of the 4th International Conference on Networking, Information Systems & Security (pp. 1-6).

Brown, A., Gupta, M., & Abdelsalam, M. (2024). Automated machine learning for deep learning based malware detection. *Computers & Security*, 137, 103582.

Brownlee, J. (2021). Random Oversampling and Undersampling for Imbalanced Classification. *Machine Learning Mastery*. Retrieved from Machine Learning Mastery

Brownlee, J. (2020). A Gentle Introduction to XGBoost Loss Functions. *Machine Learning Mastery*. Retrieved from <https://machinelearningmastery.com/xgboost-loss-functions/>

Buxton, O. (2024). 15 types of malware: Real-world examples and protection tips. Norton. Retrieved April 11, 2025, from <https://us.norton.com/blog/malware/types-of-malware>

Caruana, R., & Niculescu-Mizil, A. (2006, June). An empirical comparison of supervised learning algorithms. In Proceedings of the 23rd international conference on Machine learning (pp. 161-168).

Cäsar, M., Pawelke, T., Steffan, J., & Terhorst, G. (2022). A survey on Bluetooth Low Energy security and privacy. *Computer Networks*, 205, 108712.

Chanesh, D. (2017). Using SNORT Network Intrusion Detection for Real-Time Packet Inspection (Doctoral dissertation). University of Technology, Mauritius.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>

Chowdhury, P., Barua, P., & Uddin, M. N. (2024, September). Diabetes Prediction Using Machine Learning and Hybrid Deep Learning Ensemble Technique. In 2024 IEEE International Conference on Computing, Applications and Systems (COMPAS) (pp. 1-7). IEEE.

Clark, D. D., Jacobson, V., Romkey, J., & Salwen, H. (1989). An analysis of TCP processing overhead. *IEEE Communications magazine*, 27(6), 23-29.

Clarry, M., Andersen, R., Papadopoulos, M., Grigoriev, T., & Hofmann, G. (2024). Dynamic pattern recognition for enhanced ransomware detection via adaptive signature analysis.

Cloud Security Web: Detecting anomalies in real-time: The power of AI-driven solutions. Retrieved March 27, 2024, from <https://cloudsecurityweb.com/articles/2025/03/27/detecting-anomalies-in-real-time-the-power-of-ai-driven-solutions/>

CodeProject: Faster XGBoost, LightGBM, and CatBoost inference on GPUs. Retrieved March 25, 2024, from <https://www.codeproject.com/Articles/5372413/Faster-XGBoost-LightGBM-and-CatBoost-Inference-on>

Conti, M., Rigoni, G., & Toffalini, F. (2020, August). ASAIN: A spy App identification system based on network traffic. In Proceedings of the 15th International Conference on Availability, Reliability and Security (pp. 1-8).

Corlatescu, D. G., Dinu, A., Gaman, M. P., & Sumedrea, P. (2023). EMBERSim: a large-scale databank for boosting similarity search in malware analysis. *Advances in Neural Information Processing Systems*, 36, 26722-26743.

Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE transactions on information theory*, 13(1), 21-27.

D'Angelo, G., Ficco, M., & Palmieri, F. (2021). Association rule-based malware classification using common subsequences of API calls. *Applied Soft Computing*, 105, 107234.

Deepthi, B. G., Rani, K. S., Krishna, P. V., & Saritha, V. (2024). An efficient architecture for processing real-time traffic data streams using apache flink. *Multimedia Tools and Applications*, 83(13), 37369-37385.

Devarakonda, A., Naumov, M., & Garland, M. (2017). Adabatch: Adaptive batch sizes for training deep neural networks. *arXiv preprint arXiv:1712.02029*.

Dias, K. L., Pongelupe, M. A., Caminhas, W. M., & Errico, L. D. (2019). An innovative approach for real-time network traffic classification. *Compute Networks*, 158, 143-157.

Duan, G., Liu, H., Cai, M., Sun, J., & Chen, H. (2024). MaDroid: A maliciousness-aware multifeatured dataset for detecting android malware. *Computers & Security*, 144, 103969.

Elngar, A., & Burlea-Schiopoiu, A. (2023). Feature selection and dynamic network traffic congestion classification based on machine learning for Internet of Things. *Wasit Journal of Computer and Mathematics Science*, 2(2), 72-86.

Elor, Y., & Averbuch-Elor, H. (2022). To SMOTE, or not to SMOTE?. arXiv preprint arXiv:2201.08528.

Ericsson. (2024, April 15). Uplink traffic dominated by communication and cloud-storage services. Retrieved from <https://www.ericsson.com/en/reports-and-papers/mobility-report/dataforecasts/analysis-traffic-profiles>.

Faizal, M. A., Zaki, M. M., Shahrin, S., Robiah, Y., Rahayu, S. S., & Nazrulazhar, B. (2009). Threshold verification technique for network intrusion detection system. arXiv preprint arXiv:0906.3843.

FamiGuard: Parental control app guide. Retrieved March 19, 2024, from <https://www.famiguard.com/parental-control-app/guide/>

Fang, Y., & Cheng, C. (2022). Analyzing the performance of LSTM models on diverse feature types in network traffic classification. IEEE Access, 10, 20485–20495. <https://doi.org/10.1109/ACCESS.2022.3154021>

Ferreira, G. O., Ravazzi, C., Dabbene, F., Calafiore, G. C., & Fiore, M. (2023). Forecasting network traffic: A survey and tutorial with open-source comparative evaluation. IEEE Access, 11, 6018-6044.

Fidelis Security: Network traffic pattern analysis. Retrieved March 20, 2024, from <https://fidelissecurity.com/threatgeek/network-security/network-traffic-pattern-analysis/>

Fix, E. (1985). Discriminatory analysis: nonparametric discrimination, consistency properties (Vol. 1). USAF school of Aviation Medicine.

Fortinet. (2024, April 15). Network traffic. Retrieved from <https://www.fortinet.com/resources/cyberglossary/network-traffic>.

Gandhi, H., & Ribeiro, V. (2022, January). Packet batching for reducing system resource consumption for botnet detection using network traffic analysis. In 2022 14th International Conference on COMmunication Systems & NETworkS (COMSNETS) (pp. 1-6). IEEE.

Garcia, A. M., Griebler, D., Schepke, C., & Fernandes, L. G. L. (2022, March). Evaluating micro-batch and data frequency for stream processing applications on multi-cores. In 2022 30th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP) (pp. 10-17). IEEE.

Gavrilov, A. D., Jordache, A., Vasdani, M., & Deng, J. (2018). Preventing model overfitting and underfitting in convolutional neural networks. *International Journal of Software Science and Computational Intelligence (IJSSCI)*, 10(4), 19-28.

GeeksforGeeks. (2024). Impact of dataset size on deep learning model. Retrieved from <https://www.geeksforgeeks.org/impact-of-dataset-size-on-deep-learning-model/>

Gregorutti, B., Michel, B., & Saint-Pierre, P. (2017). Correlation and variable importance in random forests. *Statistics and Computing*, 27(3), 659–678. <https://doi.org/10.1007/s11222-016-9646-1>

Groenewald, E. S., Dodda, S. B., Soni, M., Groenewald, C. A., Dhumane, A., & Tiwari, V. (2024, March). Real-Time Anomaly Detection in Industrial Systems Using Stream Processing and Online Machine Learning. In *International Conference on Deep Learning and Visual Artificial Intelligence* (pp. 513-529). Singapore: Springer Nature Singapore.

Gibert, D., Planes, J., Mateu, C., & Le, Q. (2022). Fusing feature engineering and deep learning: A case study for malware classification. *Expert Systems with Applications*, 207, 117957.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. Retrieved from <https://www.deeplearningbook.org>

Gupta, A. K., Chakroborty, S., Ghosh, S. K., & Ganguly, S. (2023). A machine learning model for multi-class classification of quenched and partitioned steel microstructure type by the k-nearest neighbor algorithm. *Computational Materials Science*, 228, 112321.

Guardian Digital. (2024). Spyware: What it is and how to protect against it. Retrieved April 19, 2024, from <https://guardiandigital.com/resources/blog/spyware>.

Harang, R., & Rudd, E. M. (2020). SOREL-20M: A large scale benchmark dataset for malicious PE detection. *arXiv preprint arXiv:2012.07634*.

Heinrich, A., Bittner, N., & Hollick, M. (2022, May). AirGuard-protecting android users from stalking attacks by apple find my devices. In *Proceedings of the 15th ACM Conference on Security and Privacy in Wireless and Mobile Networks* (pp. 26-38).

Hewlett Packard Enterprise. (2024, April 14). HPE 5500 HI Switch Series Layer 2—LAN Switching Configuration Guide. Retrieved from <https://support.hpe.com/techhub/eginfolib/networking/docs/switches/5500hi/5998->

5328_l2lan_cg/content/377991906.htm#:~:text=Uplink%20traffic%E2%80%94Traffic%20transmitted%20from,to%20the%20service%20provider%20network.

Hindarto, D., & Djajadi, A. (2023). Android-manifest extraction and labeling method for malware compilation and dataset creation. *International Journal of Electrical & Computer Engineering* (2088-8708), 13(6).

Hoffer, E., Ben-Nun, T., Hubara, I., Giladi, N., Hoefler, T., & Soudry, D. (2020). Augment your batch: Improving generalization through instance repetition. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, 1247–1256.

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366. [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8)

Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>

Holdaway, K. R. (2009, October). Exploratory data analysis in reservoir characterization projects. In *SPE/EAGE Reservoir Characterization & Simulation Conference* (pp. cp-170). European Association of Geoscientists & Engineers.

Howarth, J. (2024, June 30). How many people own smartphones? (2024-2029). Exploding Topics. Retrieved from <https://explodingtopics.com/blog/smartphone-stats>.

Huabing, Z., Sisi, Y., Xiaoming, C., & Zhida, L. (2021, April). Real-time detection method for mobile network traffic anomalies considering user behavior security monitoring. In *2021 International Conference on Computer, Blockchain and Financial Development (CBFD)* (pp. 11-16). IEEE.

Humayun, M., Jhanjhi, N. Z., & Alamri, M. Z. (2020). Smart secure and energy efficient scheme for e-health applications using IoT: a review. *International Journal of Computer Science and Network Security*, 20(4), 55-74.

Hu, Q., & Dinev, T. (2005). Is spyware an internet nuisance or public menace?. *Communications of the ACM*, 48(8), 61-66.

IBM. (n.d.). (2024, January 16). What is ground truth in machine learning? IBM. Retrieved from <https://www.ibm.com/cloud/blog/what-is-ground-truth>.

Internet Engineering Task Force (IETF). (2008). The Transport Layer Security (TLS) Protocol Version 1.2 (RFC 5246). Retrieved March 19, 2025, from <https://datatracker.ietf.org/doc/html/rfc5246>

Iorliam, A., Tirunagari, S., Ho, A. T., Li, S., Waller, A., & Poh, N. (2016). "Flow Size Difference" Can Make a Difference: Detecting Malicious TCP Network Flows Based on Benford's Law. arXiv preprint arXiv:1609.04214.

Joyce, R. J., Amlani, D., Nicholas, C., & Raff, E. (2023). Motif: A malware reference dataset with ground truth family labels. *Computers & Security*, 124, 102921.

Joyce, R. J., Raff, E., Nicholas, C., & Holt, J. (2023). MalDICT: Benchmark Datasets on Malware Behaviors, Platforms, Exploitation, and Packers. arXiv preprint arXiv:2310.11706.

Jung, J., Park, J., Cho, S. J., Han, S., Park, M., & Cho, H. H. (2021). Feature engineering and evaluation for android malware detection scheme. *Journal of Internet Technology*, 22(2), 423-440.

Kanász, R., Drotár, P., Gnip, P., & Zoričák, M. (2024, June). Clash of titans on imbalanced data: TabNet vs XGBoost. In *2024 IEEE Conference on Artificial Intelligence (CAI)* (pp. 320-325). IEEE.

Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., & Tang, P. T. P. (2017). On large-batch training for deep learning: Generalization gap and sharp minima. *Proceedings of the International Conference on Learning Representations (ICLR)*, Retrieved from <https://arxiv.org/abs/1609.04836>

Khade, A., Vidhate, A. V., & Vidhate, D. (2023). FFN-XGB-design of a hybrid feed forward neural network and extreme gradient boosting model for early prediction of chronic kidney disease. *International Journal of System Assurance Engineering and Management*, 1-10.

Khandait, P., Hubballi, N., & Mazumdar, B. (2020). Efficient keyword matching for deep packet inspection based network traffic classification. In *2020 International Conference on Communication Systems & Networks (COMSNETS)* (pp. 567-570). IEEE.

Kharole, S. (2019). Big IoT data analytics by combining batch and stream processing technologies. *International Journal for Research in Applied Science and Engineering Technology*, 7(6), 645-652.

Kim, A., Park, M., & Lee, D. H. (2020). AI-IDS: Application of deep learning to real-time Web intrusion detection. *IEEE Access*, 8, 70245-70261.

Kim, H., & Kim, J. (2004). Detection of malicious traffic using packet size distribution. *International Journal of Computer Science and Network Security*, 4(8), 15–18.

- Kotsiantis, S. B., Zaharakis, I., & Pintelas, P. (2007). Supervised machine learning: A review of classification techniques. *Emerging artificial intelligence applications in computer engineering*, 160(1), 3-24.
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25.
- Kumar, B., & Kumar, T. (2020). Comparative analysis of ML-based gradient boosting algorithms: XGBoost, CatBoost, and LightGBM. *Journal of Scientific and Engineering Research*, 7(8), 235–239.
- Kumar, M., & Dash, C. S. (2024). Detecting and Preventing ARP Spoofing Attacks Using Real-Time Data Analysis and Machine Learning. *International Journal of Innovative Research in Computer Science and Technology*, 12(5).
- Kumar, N. (2024, December 31). Demandsage: Android usage statistics 2025: Devices & market share. Retrieved January 23, 2025, from <https://www.demandsage.com/android-statistics/>
- Kumar, P., Pandey, D., Srivastav, R. K., & Pandey, P. K. (2023). Network traffic analysis and prediction using machine learning. *International Journal of Research Publication and Reviews*, 4(8), 2071–2075.
- Kunasaikaran, J., Ismail, R., & Ahmad, A. R (2019). Challenges and solutions of real-time clustering for network anomaly detection.
- Lakhina, A., Crovella, M., & Diot, C. (2005). Diagnosing network-wide traffic anomalies. *Proceedings of the 2004 conference on Applications, technologies, architectures, and protocols for computer communications*, 219–230. <https://doi.org/10.1145/1015467.1015492>
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>
- Li, M., Han, D., Yin, X., Liu, H., & Li, D. (2021). Design and implementation of an anomaly network traffic detection model integrating temporal and spatial features. *Security and Communication Networks*, 2021(1), 7045823.
- Liu, L., Wang, P., Lin, J., & Liu, L. (2020). Intrusion detection of imbalanced network traffic based on machine learning and deep learning. *IEEE access*, 9, 7550-7563.

- Lockett, A. (2021). Assessing the effectiveness of yara rules for signature-based malware detection and classification. arXiv preprint arXiv:2111.13910.
- Loi, N., Borile, C., & Ucci, D. (2021). Towards an automated pipeline for detecting and classifying malware through machine learning. arXiv preprint arXiv:2106.05625.
- Lopez-Martin, M., Carro, B., & Sanchez-Esguevillas, A. (2018). Application of deep reinforcement learning to intrusion detection for network traffic. *Neurocomputing*, 312, 18–28.
<https://doi.org/10.1016/j.neucom.2018.05.082>
- Low, M. X., Yap, T. T. V., Soo, W. K., Ng, H., Goh, V. T., Chin, J. J., & Kuek, T. Y. (2022). Comparison of label encoding and evidence counting for malware classification. *Journal of System and Management Sciences*, 12(6), 17-30.
- Lyzer AI: Batch size. Retrieved March 25, 2024, from <https://www.lyzr.ai/glossaries/batch-size/#:~:text=Batch%20size%20refers%20to%20the%20number%20of%20training%20examples%20utilized,more%20effective%20for%20noisy%20data>
- Machine Learning Mastery. (2024, November 01). Precision, recall, and F-measure for imbalanced classification. Retrieved from <https://machinelearningmastery.com/precision-recall-and-f-measure-for-imbalanced-classification/>
- Madhava, R. (2024). Network packet analyzer.
- Maggio, M. (2020, February 25). Scheduling theory: Real-time systems (Lecture 12). Retrieved March 8, 2025, from <http://www.control.lth.se>
- Mahesh, V., & KA, S. D. (2020, July). Detection and Prediction of Spyware for user Applications by interdisciplinary approach. In 2020 International Conference on Computational Intelligence for Smart Power System and Sustainable Energy (CISPSSE) (pp. 1-6). IEEE.
- Malik, J., & Kaushal, R. (2016, July). CREDROID: Android malware detection by network traffic analysis. In Proceedings of the 1st acm workshop on privacy-aware mobile computing (pp. 28-36).
- Mangeard, P., Tejaswi, B., Mannan, M., & Youssef, A. (2024). WARNE: A stalkerware evidence collection tool. *Forensic Science International: Digital Investigation*, 48, 301677.
- Mannan, M., Youssef, A., Mangeard, P., Yu, X., Tejaswi, B., & Pagey, R. (2023). Privacy Analysis of Technologies Used in Intimate Partner Abuse. Technical Report. Concordia University, Montreal, CA.

Retrieved 2024-02-13 from https://www.priv.gc.ca/en/opc-actions-and-decisions/research/fundingfor-privacy-research-and-knowledge-translation/completed-contributionsprogram-projects/2022-2023/p_202223_11.

Masabo, E., Kaawaase, K. S., Sansa-Otim, J., Ngubiri, J., & Hanyurwimfura, D. (2020). Improvement of malware classification using hybrid feature engineering. *SN Computer Science*, 1, 1-14.

Medium. (2024, November 01). Classification report explained: Precision, recall, accuracy, macro average, and weighted average. Retrieved from <https://medium.com/@chanakapinfo/classification-report-explained-precision-recall-accuracy-macro-average-and-weighted-average-8cd358ee2f8a>

Mehmood, R., & Bottaci, L. (2021). Micro-batch-based traffic classification: Impact on model performance. *Computer Networks*, 181, 107586. <https://doi.org/10.1016/j.comnet.2020.107586>

Mekruksavanich, S., Jantawong, P., & Jitpattanakul, A. (2022, January). LSTM-XGB: a new deep learning model for human activity recognition based on LSTM and XGBoost. In 2022 Joint International Conference on Digital Arts, Media and Technology with ECTI Northern Section Conference on Electrical, Electronics, Computer and Telecommunications Engineering (ECTI DAMT & NCON) (pp. 342-345). IEEE.

Miha, K., Kawasaki, A., Makuda, G., Takana, T., & Barakata, A. (2024). A novel adaptive signature extraction framework for ransomware detection.

Mirkovic, J., & Reiher, P. (2004). A taxonomy of DDoS attack and DDoS defense mechanisms. *ACM SIGCOMM Computer Communication Review*, 34(2), 39–53. <https://doi.org/10.1145/997150.997156>

ML Journey. (2025). LightGBM vs XGBoost vs CatBoost: A Comprehensive Comparison. Retrieved from <https://mljourney.com/lightgbm-vs-xgboost-vs-catboost-a-comprehensive-comparison/>

ML Journey. (2024, January 16). What is a label in machine learning? ML Journey. Retrieved from <https://ml-journey.com/what-is-a-label-in-machine-learning>.

Mohammed, A. B., & Nor, S. M. (2009). Near real time online flow-based internet traffic classification using machine learning (C4. 5). *International Journal of Engineering*, 3(4), 370-379.

Mortezapour Shiri, F., Perumal, T., Mustapha, N., & Mohamed, R. (2024). A comprehensive overview and comparative analysis on deep learning models: CNN, RNN, LSTM, GRU. *Journal on Artificial Intelligence*, 6(1), 301–360. <https://doi.org/10.32604/jai.2024.054314>

MSpy Team: How-to guides. Monimaster. Retrieved March 19, 2024, from

<https://www.monimaster.com/how-to/>

Mukherjee, B., Heberlein, L. T., & Levitt, K. N. (1994). Network intrusion detection. *IEEE Network*, 8(3), 26–41. <https://doi.org/10.1109/65.283931>

Munusamy, T., & Khodadi, T. (2023). Building Cyber Resilience: Key Factors for Enhancing Organizational Cyber Security. *Journal of Informatics and Web Engineering*, 2(2), 59-71.

Mushava, J., & Murray, M. (2022). A novel XGBoost extension for credit scoring class-imbalanced data combining a generalized extreme value link and a modified focal loss function. *Expert Systems with Applications*, 202, 117233.

Naser, M., & Abu Al-Haija, Q. (2023). Spyware Identification for Android Systems Using Fine Trees. *Information*, 14(2), 102.

Nawaz, A. (2021). Feature engineering based on hybrid features for malware detection over Android framework. *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*, 12(10), 2856-2864.

Najm, I. A., Saeed, A. H., Ahmad, B. A., Ahmed, S. R., Sekhar, R., Shah, P., & Veena, B. S. (2024, May). Enhanced Network Traffic Classification with Machine Learning Algorithms. In *Proceedings of the Cognitive Models and Artificial Intelligence Conference* (pp. 322-327).

Niu, Z., Xue, J., Qu, D., Wang, Y., Zheng, J., & Zhu, H. (2022). A novel approach based on adaptive online analysis of encrypted traffic for identifying malware in IIoT. *Information Sciences*, 601, 162–174.

Noetzold, D., Rossetto, A. G. D. M., & Leithardt, V. R. Q. (2023). Spyware Integrated with Prediction Models for Monitoring Corporate Computers.

Ong, J. (2013, November 28). mSpy: A terrifying app for spying on another smartphone or tablet user. *Thenextweb.com*. Retrieved June 24, 2024.

Ozcan, Y., & Rosenberg, C. (2021, August). Performance Evaluation of Full Duplex Communications in 5G Networks: The Impact of Interference and Traffic Asymmetry. In *2021 33th International Teletraffic Congress (ITC-33)* (pp. 1-9). IEEE.

Packetlabs. (2024, October 22). A guide to replay attacks and how to defend against them. Retrieved from <https://www.packetlabs.net/posts/a-guide-to-replay-attacks-and-how-to-defend-againstthem/#:~:text=Kerberos:%20Kerberos%20is%20vulnerable%20to,as%20a%20legitimate%20authentication%20attempt>

Pandit, K., Bian, B., Prasad, V. M., Kwatra, A., Lu, P., Riess, M., & Xu, G. (2018). Modeling the impact of CPU properties to optimize and predict packet-processing performance. Intel case study.

Patel, S., Gupta, A., Kumari, S., Singh, M., & Sharma, V. (2018, October). Network traffic classification analysis using machine learning algorithms. In 2018 International Conference on Advances in Computing, Communication Control and Networking (ICACCCN) (pp. 1182-1187). IEEE.

Peng, J., Guo, C., Ping, Y., Cui, Y., Chen, Y., & Shen, G. (2024). SNDMI: Spyware network traffic detection method based on inducement operations. *Computers & Security*, 140, 103806.

Petrini, L. (2023). Breaking the Curse of Dimensionality in Deep Neural Networks by Learning Invariant Representations. arXiv preprint arXiv:2310.16154.

Pierazzi, F., Mezzour, G., Han, Q., Colajanni, M., & Subrahmanian, V. S. (2020). A data-driven characterization of modern Android spyware. *ACM Transactions on Management Information Systems (TMIS)*, 11(1), 1-38.

Pixel Phone Help. (2025, February 5). Pixel phone help. Retrieved from <https://support.google.com/pixelphone/answer/2812516?hl=en>

Qabalin, M. K., Naser, M., & Alkasassbeh, M. (2022). Android spyware detection using machine learning: a novel dataset. *sensors*, 22(15), 5765.

Rafrastara, F. A., Supriyanto, C., Paramita, C., Astuti, Y. P., & Ahmed, F. (2023). Performance Improvement of Random Forest Algorithm for Malware Detection on Imbalanced Dataset using Random Under-Sampling Method. *Jurnal Informatika: Jurnal Pengembangan IT*, 8(2), 113-118.

Rawat, A., Kumar, A., Singh, A. K., & Arora, K. (2024). Exploring Android Security Landscape: Threats, Vulnerabilities, and Best Practices. *International Research Journal on Advanced Engineering and Management (IRJAEM)*, 2(06), 1831-1839.

Restack: LightGBM vs. CatBoost vs. XGBoost: Answer by Cat AI. Retrieved March 25, 2024, from <https://www.restack.io/p/lightgbm-vs-catboost-vs-xgboost-answer-cat-ai>

Rettig, L., Khayati, M., Cudré-Mauroux, P., & Piórkowski, M. (2019). Online anomaly detection over big data streams. *Applied Data Science: Lessons Learned for the Data-Driven Business*, 289-312

RF Wireless World. (2024, April 14). Uplink vs Downlink. Retrieved from <https://www.rfwireless-world.com/Terminology/Uplink-vs-Downlink.html>

Ring, M., Wunderlich, S., Scheuring, D., Landes, D., & Hotho, A. (2019). A survey of network-based intrusion detection data sets. *Computers & Security*, 86, 147–167.
<https://doi.org/10.1016/j.cose.2019.06.005>

Sangher, K. S., Singh, A., & Pandey, H. M. (2024). Signature based ransomware detection based on optimizations approaches using RandomClassifier and CNN algorithms. *International Journal of System Assurance Engineering and Management*, 15(5), 1687-1703.

Sarhan, M., Layeghy, S., Moustafa, N., Gallagher, M., & Portmann, M. (2024). Feature extraction for machine learning-based intrusion detection in IoT networks. *Digital Communications and Networks*, 10(1), 205-216.

Saydam, B., Orhan, C., Toker, N., & Turasan, M. (2020). Optimisation of scheduled tasks by real-time measurement and correlation. *New Trends and Issues Proceedings on Advances in Pure and Applied Sciences*, 12, 36–43.

Sbai, H., Happa, J., Goldsmith, M., & Meftali, S. (2020, December). Dataset construction and analysis of screenshot malware. In *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)* (pp. 646-655). IEEE.

Scapy Documentation: Scapy: Packet crafting for Python. Retrieved August 25, 2024, from <https://scapy.readthedocs.io/en/latest>

Schummer, P., del Rio, A., Serrano, J., Jimenez, D., Sánchez, G., & Llorente, Á. (2024). Machine Learning-Based Network Anomaly Detection: Design, Implementation, and Evaluation. *AI*, 5(4), 2967-2983.

Semmelmann, L., Henni, S., & Weinhardt, C. (2022). Load forecasting for energy communities: A novel LSTM-XGBoost hybrid model based on smart meter data. *Energy Informatics*, 5(1), Article 24.
<https://doi.org/10.1186/s42162-022-00212-9>

Shabtai, A., Hasadya, B., & Elovici, Y. (2012). A survey of data leakage detection and prevention solutions. SpringerBriefs in Computer Science.

Shahzad, R. M. K. (2024). Automated Malware Detection and Classification Using Supervised Learning (Doctoral dissertation, Blekinge Tekniska Högskola).

Sharma, S., & Jain, M. (2019). Anomaly detection in network traffic using machine learning. *International Journal of Advanced Research in Computer Science*, 10(6), 23–28.
<https://doi.org/10.26483/ijarcs.v10i6.6477>

Shezan, F. H., Afroze, S. F., & Iqbal, A. (2017, January). Vulnerability detection in recent Android apps: An empirical study. In *2017 International Conference on Networking, Systems and Security (NSysS)* (pp. 55-63). IEEE.

Shirsat, A. S., Shirsat, S. A., & Yadav, D. M. (2018). Performance of Bluetooth in Homogeneous Environment. *International Journal of Engineering & Technology*, 7(4.36), 506-510.

Sipior, J. C., Ward, B. T., & Roselli, G. R. (2005, August). A united states perspective on the ethical and legal issues of spyware. In *Proceedings of the 7th international conference on Electronic commerce* (pp. 738-743).

Stafford, T. F., & Urbaczewski, A. (2004). Spyware: The ghost in the machine. *The Communications of the Association for Information Systems*, 14(1), 49.

Stallings, W., & Case, L. (2017). *Network Security Essentials: Applications and Standards* (6th ed.). Pearson. Retrieved from Pearson Network Security Resources.

Sudhamathy, G., & Valliammal, N. (2023). The Bayesian CNN-LSTM classification model to predict and evaluate learner's performance.

Sze, V., Chen, Y. H., Yang, T. J., & Emer, J. S. (2017). Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, 105(12), 2295–2329.
<https://doi.org/10.1109/JPROC.2017.2761740>

Talabani, H. S., & Abdulhadi, H. M. T. (2022). Bitcoin ransomware detection employing rule-based algorithms. *Science Journal of University of Zakho*, 10(1), 5-10.

Talaei Khoei, T., Ismail, S., Shamaileh, K. A., Devabhaktuni, V. K., & Kaabouch, N. (2022). Impact of dataset and model parameters on machine learning performance for the detection of GPS spoofing attacks on unmanned aerial vehicles. *Applied Sciences*, 13(1), 383.

Tang, C., Wang, D., Tan, A. H., & Miao, C. (2017). EEG-based emotion recognition via fast and robust feature smoothing. In *Brain Informatics: International Conference, BI 2017, Beijing, China, November 16-18, 2017, Proceedings* (pp. 83-92). Springer International Publishing

Techtarget|Security. (2024, October 9). [Online]. Available :
<https://www.techtarget.com/searchsecurity/definition/promiscuous-mode#:~:text=Promiscuous%20mode%20is%20often%20used,Ethernet%20address%20display%20in%20tcpdump>

This vs. That:. Bidirectional vs. Unidirectional - What's the Difference? Retrieved March 22, 2024, from <https://thisvsthat.io/bidirectional-vs-unidirectional>

Thompson Neil, C., Kristjan, G., Keeheon, L., & Manso Gabriel, F. (2020). The computational limits of deep learning. *ArXiv*, Cornell University, juillet.

Tinybird:Real-time anomaly detection: Use cases and code examples. Retrieved March 27, 2024, from <https://www.tinybird.co/blog-posts/real-time-anomaly-detection>

Tirumala, S. S., Valluri, M. R., & Nanadigam, D. (2020, January). Evaluation of feature and signature based training approaches for malware classification using autoencoders. In *2020 International Conference on COMMunication Systems & NETworkS (COMSNETS)* (pp. 1-5). IEEE.

Unit 3: Packet Sniffing with Python. (2025, February 5). Packet sniffing with Python. Retrieved from <https://egyankosh.ac.in/bitstream/123456789/93209/1/Unit-3.pdf>

UCI Machine Learning Repository. (2023, October 1). [Online]. Available:
<https://archive.ics.uci.edu/about>.

Velarde, G., Sudhir, A., Deshmane, S., Deshmunkh, A., Sharma, K., & Joshi, V. (2023). Evaluating XGBoost for balanced and imbalanced data: application to fraud detection. *arXiv preprint arXiv:2303.15218*.

Wang, Y., Han, T., & Li, W. (2021). Comparative performance of deep learning models in network traffic classification. *IEEE Transactions on Neural Networks and Learning Systems*, 32(10), 4513–4525. <https://doi.org/10.1109/TNNLS.2021.3075593>

Warrier, H., & Gupta, Y. (2022). Network state estimation using raw video analysis: vQoS-GAN based non-intrusive deep learning approach. arXiv preprint, arXiv:2204.07062.

Wei, G., & Wang, Z. (2021). Adoption and realization of deep learning in network traffic anomaly detection device design. *Soft Computing*, 25(2), 1147-1158.

Wikipedia contributors: KidGuard. In Wikipedia. Retrieved March 19, 2024, from https://en.wikipedia.org/wiki/KidGuard#cite_note-8

Wiles, A., Colombo, F., & Mascorro, R. (2024). Ransomware detection using network traffic analysis and generative adversarial networks.

Wireshark. Wireshark documentation - network traffic analysis. Retrieved March 19, 2024, from <https://www.wireshark.org/docs/>

Wolf, D., Payer, T., Lisson, C. S., Lisson, C. G., Beer, M., Götz, M., & Ropinski, T. (2024). Less is More: Selective Reduction of CT Data for Self-Supervised Pre-Training of Deep Learning Models with Contrastive Learning Improves Downstream Classification Performance. *Computers in Biology and Medicine*, 183, 109242. <https://doi.org/10.1016/j.combiomed.2024.109242>

Xu, H., Xie, Q., Zhang, Y., & Zhang, T. (2022). Class imbalance and feature variability in cybersecurity data: Challenges and solutions. *Journal of Cybersecurity Research*, 8(2), 101–115. <https://doi.org/10.1016/j.cysec.2022.101115>

Xu, H., Han, S., Li, X., & Han, Z. (2023). Anomaly traffic detection based on communication-efficient federated learning in space-air-ground integration network. *IEEE Transactions on Wireless Communications*, 22(12), 9346-9360.

Xu, Y., Wu, Q., Zhou, H., & Zhang, Y. (2016). Traffic modeling and performance evaluation in cloud computing data center networks. *IEEE Transactions on Cloud Computing*, 4(2), 276–287. <https://doi.org/10.1109/TCC.2015.243033>

Xu, Y., & Goodacre, R. (2018). On splitting training and validation set: a comparative study of cross-validation, bootstrap and systematic sampling for estimating the generalization performance of supervised learning. *Journal of analysis and testing*, 2(3), 249-262.(

- Yang, L., Ciptadi, A., Laziuk, I., Ahmadzadeh, A., & Wang, G. (2021, May). BODMAS: An open dataset for learning based temporal analysis of PE malware. In 2021 IEEE Security and Privacy Workshops (SPW) (pp. 78-84). IEEE.
- Yu, H., Yang, W., Cui, B., Sui, R., & Wu, X. (2024). Renyi entropy-driven network traffic anomaly detection with dynamic threshold. *Cybersecurity*, 7(1), 1-13.
- Zafra, A., & Gibaja, E. (2023). Nearest neighbor-based approaches for multi-instance multi-label classification. *Expert Systems with Applications*, 232, 120876.
- Zahan, N., Burckhardt, P., Lysenko, M., Aboukhadijeh, F., & Williams, L. (2024, April). MalwareBench: Malware samples are not enough. In 2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR) (pp. 728-732). IEEE.
- Zhang, C., & Ma, Y. (2012). *Ensemble machine learning: Methods and applications*. Springer.
<https://doi.org/10.1007/978-1-4419-9326-7>
- Zhao, Z., Chen, W., Wu, X., Chen, P., & Liu, J. (2020). LSTM-CNN: A hybrid model for traffic flow prediction. *IEEE Transactions on Intelligent Transportation Systems*, 21(9), 3518–3528.
<https://doi.org/10.1109/TITS.2020.2964858>
- Zhong, G., Li, X., Song, J., Qu, B., Wang, F., Wang, Y., & Duan, L. (2022). Reconstruction of global surface ocean p CO₂ using region-specific predictors based on a stepwise FFNN regression algorithm. *Biogeosciences*, 19(3), 845-859.
- Zhou, Z.-H. (2012). *Ensemble Methods: Foundations and Algorithms*. Chapman and Hall/CRC.
<https://doi.org/10.1201/b12207>
- Zhou, Z., & He, Z. (2020). Anomaly detection in network traffic using statistical and machine learning approaches. *IEEE Transactions on Network Science and Engineering*, 7(1), 38–47.
<https://doi.org/10.1109/TNSE.2019.2941846>
- Zhou, Z., & Huang, F. (2020). Machine learning-based hybrid models for sequential anomaly detection. *IEEE Transactions on Neural Networks and Learning Systems*, 31(4), 1206–1216.
<https://doi.org/10.1109/TNNLS.2019.2916066>
- Zou, F., Zhang, S., Wan, T., & Pan, L. (2014, September). A survey of android mobile platform security. In 10th International Conference on Wireless Communications, Networking and Mobile Computing (WiCOM 2014) (pp. 520-527). IET.

APPENDICES

APPENDIX A

Experimental Results for Batch Processing

Table 1 Evaluation metrics results for MAS-2024-A (80-20).

MAS-2024-A (80-20)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.60	98.60	98.60	98.60	98.63	98.63	98.63	98.63
LGB	98.53	98.52	98.52	98.51	98.51	98.51	98.50	98.50
CatBoost	98.24	98.25	98.24	98.24	98.22	98.23	98.22	98.22
RF	93.31	93.34	93.31	93.31	93.33	93.36	93.33	93.33
CNN	85.25	85.61	85.25	85.30	85.23	85.61	85.24	85.29
LSTM	92.46	92.53	92.46	92.44	92.35	92.41	92.35	92.32
FFNN	77.09	77.73	77.09	76.94	77.00	77.67	77.00	76.85
TabNet	77.20	77.37	77.20	76.84	77.11	77.29	77.11	76.74
CNN-LSTM	96.05	96.07	96.05	96.05	96.07	96.09	96.07	96.07
FFNN-XGB	98.51	98.52	98.52	98.52	98.49	98.50	98.50	98.49
LSTM-XGB	99.04	99.04	99.04	99.04	99.03	99.03	99.03	99.03
TabNet-XGB	98.50	98.50	98.50	98.50	98.49	98.49	98.49	98.49

Table 2 Evaluation metrics results for MAS-2024-A (70-30).

MAS-2024-A (70-30)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.61	98.61	98.61	98.61	98.61	98.61	98.61	98.61
LGB	98.49	98.48	98.48	98.48	98.49	98.48	98.47	98.49
CatBoost	98.22	98. 23	98.22	98.22	98. 21	98. 21	98.21	98.20
RF	93.09	93.13	93.09	93.09	93.08	93.12	93.08	93.08
CNN	84.04	83.95	84.04	83.90	84.08	84.00	84.09	83.95
LSTM	91.80	91.76	91.80	91.73	91.75	91.71	91.75	91.68
FFNN	74.14	75.14	74.14	73.71	74.19	75.15	74.19	73.77
TabNet	60.07	60.21	60.07	59.21	60.11	60.24	60.11	59.25
CNN-LSTM	95.59	95.64	95.59	95.60	95.57	95.63	95.58	95.59
FFNN-XGB	98.49	98.50	98.50	98.50	98.48	98.49	98.48	98.48

Table 2 Evaluation metrics results for MAS-2024-A (70-30), continued.

LSTM-XGB	99.02	99.02	99.02	99.02	99.00	99.00	99.00	99.00
TabNet-XGB	95.30	95.31	95.30	95.30	95.37	95.37	95.37	95.36

Table 3 Evaluation metrics results for MAS-2024-A (60-40).

MAS-2024-A (60-40)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.52	98.51	98.52	98.51	98.50	98.49	98.50	98.49
LGB	98.35	98.35	98.35	98.35	98.33	98.32	98.33	98.32
CatBoost	98.02	98.03	98.02	98.02	98.03	98.03	98.03	98.03
RF	92.30	92.35	92.30	92.30	92.30	92.34	92.30	92.30
CNN	81.54	81.89	81.54	81.52	81.55	81.88	81.55	81.53
LSTM	90.11	90.24	90.12	90.12	90.13	90.24	90.13	90.13
FFNN	72.18	73.70	72.18	72.10	72.12	73.63	72.12	72.03
TabNet	55.48	56.01	55.48	55.33	55.45	55.98	55.45	55.29
CNN-LSTM	95.18	95.19	95.18	95.18	95.15	95.16	95.15	95.15
FFNN-XGB	98.49	98.49	98.49	98.49	98.44	98.44	98.44	98.44
LSTM-XGB	98.99	99.00	99.00	99.00	98.99	98.99	98.99	98.99
TabNet-XGB	97.40	97.40	97.40	97.40	97.39	97.38	97.39	97.38

Table 4 Evaluation metrics results for MAS-2024-B (80-20).

MAS-2024-B (80-20)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.70	98.70	98.70	98.70	98.66	98.66	98.66	98.66
LGB	98.62	98.63	98.62	98.62	98.57	98.57	98.57	98.56
CatBoost	98.26	98. 26	98.26	98.25	98. 21	98. 21	98. 21	98. 20
RF	87.72	87.71	87.72	87.68	87.82	87.82	87.82	87.78
CNN	87.74	87.91	87.74	87.69	87.72	87.89	87.73	87.67
LSTM	90.25	90.26	90.25	90.18	90.16	90.16	90.17	90.09
FFNN	72.12	73.17	72.13	72.28	72.09	73.13	72.10	72.26
TabNet	71.10	70.99	71.10	70.87	71.13	71.04	71.13	70.91

Table 4 Evaluation metrics results for MAS-2024-B (80-20), continued.

CNN-LSTM	95.83	95.86	95.83	95.84	95.86	95.88	95.86	95.86
FFNN-XGB	98.01	98.01	98.01	98.01	98.04	98.05	98.04	98.05
LSTM-XGB	99.06	99.06	99.06	99.06	99.04	99.04	99.04	99.04
TabNet-XGB	96.52	96.52	96.52	96.52	96.58	96.58	96.58	96.58

Table 5 Evaluation metrics results for MAS-2024-B (70-30).

MAS-2024-B (70-30)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.63	98.63	98.63	98.62	98.58	98.59	98.58	98.58
LGB	98.40	98.40	98.40	98.39	98.34	98.34	98.34	98.34
CatBoost	98.23	98.23	98.23	98.23	98.20	98.20	98.20	98.20
RF	86.00	86.10	86.00	85.98	86.04	86.14	86.04	86.02
CNN	85.58	85.97	85.58	85.46	85.59	85.97	85.59	85.47
LSTM	88.83	88.90	88.83	88.78	88.88	88.94	88.88	88.82
FFNN	69.91	71.07	69.90	69.63	69.64	70.86	69.64	69.40
TabNet	58.12	59.14	58.12	57.60	58.20	59.26	58.20	57.70
CNN-LSTM	95.58	95.60	95.58	95.58	95.55	95.57	95.55	95.56
FFNN-XGB	98.01	98.01	98.01	98.01	97.99	98.00	98.00	98.00
LSTM-XGB	98.95	98.95	98.95	98.95	98.95	98.95	98.95	98.95
TabNet-XGB	95.30	95.31	95.30	95.30	95.37	95.37	95.37	95.36

Table 6 Evaluation metrics results for MAS-2024-B (60-40).

MAS-2024-B (60-40)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.61	98.61	98.61	98.61	98.56	98.56	98.56	98.56
LGB	98.30	98.30	98.30	98.29	98.24	98.25	98.24	98.24
CatBoost	98.18	98.18	98.18	98.18	98.10	98.10	98.10	98.10
RF	84.64	84.74	84.64	84.61	84.65	84.75	84.65	84.62
CNN	83.67	84.36	83.67	83.70	83.69	84.40	83.70	83.73
LSTM	87.95	88.05	87.95	87.89	87.91	88.00	87.91	87.84

Table 6 Evaluation metrics results for MAS-2024-B (60-40), continued.

FFNN	66.96	69.05	66.96	67.02	66.79	68.89	66.79	66.86
TabNet	52.03	52.72	52.03	51.35	52.06	52.68	52.06	51.36
CNN-LSTM	93.41	93.46	93.41	93.42	93.38	93.44	93.39	93.40
FFNN-XGB	97.74	97.74	97.74	97.74	97.73	97.73	97.73	97.73
LSTM-XGB	98.72	98.72	98.72	98.72	98.72	98.72	98.72	98.72
TabNet-XGB	92.17	92.18	92.17	92.16	92.25	92.26	92.25	92.24

Table 7 Evaluation metrics results for SMOTE-MAS-2024-A (80-20).

SMOTE-MAS-2024-A (80-20)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.16	98.17	98.16	98.16	98.15	98.16	98.15	98.16
LGB	98.19	98.20	98.19	98.19	98.18	98.19	98.18	98.18
CatBoost	97.55	97.57	97.55	97.56	97.59	97.62	97.59	97.60
RF	92.35	92.50	92.35	92.41	92.37	92.52	92.37	92.42
CNN	82.99	84.09	82.99	83.27	83.13	84.22	83.13	83.40
LSTM	91.38	91.68	91.38	91.46	91.39	91.69	91.39	91.46
FFNN	76.01	78.01	76.01	76.28	75.97	77.97	75.98	76.25
TabNet	71.55	72.90	71.55	71.95	71.54	72.89	71.54	71.95
CNN-LSTM	95.61	95.75	95.65	95.68	95.64	95.74	95.64	95.67
FFNN-XGB	97.94	97.99	97.94	97.96	97.89	97.95	97.89	97.91
LSTM-XGB	98.91	98.92	98.91	98.92	98.88	98.89	98.88	98.88
TabNet-XGB	98.54	98.56	98.54	98.54	98.54	98.55	98.54	98.54

Table 8 Evaluation metrics results for SMOTE-MAS-2024-A (70-30).

SMOTE-MAS-2024-A (70-30)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.11	98.13	98.11	98.11	98.09	98.11	98.09	98.09
LGB	98.19	98.20	98.19	98.19	98.18	98.19	98.18	98.18
CatBoost	97.56	97.58	97.56	97.56	97.57	97.59	97.57	97.57
RF	92.11	92.26	92.11	92.17	92.12	92.27	92.12	92.18
CNN	81.78	83.26	81.78	82.12	81.79	83.27	81.80	82.14
LSTM	90.87	91.37	90.87	91.03	90.82	91.33	90.82	90.99
FFNN	73.67	76.26	73.67	74.02	73.71	76.30	73.71	74.06
TabNet	68.42	70.44	68.42	68.84	68.43	70.43	68.43	68.84
CNN-LSTM	95.52	95.60	95.52	95.54	95.49	95.58	95.50	95.52
FFNN-XGB	97.48	97.55	97.48	97.50	97.46	97.53	97.46	97.48
LSTM-XGB	98.89	98.90	98.90	98.90	98.86	98.86	98.86	98.86
TabNet-XGB	98.47	98.49	98.47	98.48	98.45	98.47	98.45	98.46

Table 9 Evaluation metrics results for SMOTE-MAS-2024-A (60-40).

SMOTE-MAS-2024-A (60-40)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.03	98.05	98.03	98.03	98.01	98.03	98.01	98.01
LGB	97.95	97.96	97.95	97.95	97.96	97.98	97.96	97.97
CatBoost	97.56	97.58	97.56	97.56	97.56	97.58	97.56	97.56
RF	91.49	91.65	91.49	91.55	91.51	91.68	91.51	91.57
CNN	79.08	81.15	79.08	79.66	79.07	81.16	79.07	79.65
LSTM	89.87	90.30	89.87	90.01	89.87	90.31	89.87	90.01
FFNN	73.66	77.64	73.67	74.29	73.64	77.66	73.65	74.29
TabNet	53.67	55.70	53.67	54.28	53.67	55.69	53.67	54.27
CNN-LSTM	95.44	95.54	95.44	95.47	95.41	95.52	95.41	95.44
FFNN-XGB	97.35	97.40	97.35	97.37	97.32	97.37	97.32	97.34
LSTM-XGB	98.86	98.87	98.86	98.86	98.83	98.84	98.83	98.83
TabNet-XGB	97.27	97.30	97.27	97.28	97.25	97.28	97.25	97.26

Table 10 Evaluation metrics results for SMOTE-MAS-2024-B (80-20).

SMOTE-MAS-2024-B (80-20)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	97.99	98.01	97.99	97.99	97.97	97.99	97.97	97.97
LGB	97.86	97.87	97.86	97.86	97.82	97.84	97.82	97.83
CatBoost	96.63	96.68	96.63	96.64	96.71	96.75	96.71	96.72
RF	85.19	85.41	85.19	85.27	85.25	85.46	85.25	85.32
CNN	86.15	87.47	86.15	86.41	86.19	87.50	86.20	86.45
LSTM	87.94	88.30	87.94	88.07	88.06	88.41	88.06	88.18
FFNN	71.76	74.21	71.76	72.47	71.73	74.20	71.73	72.44
TabNet	72.12	71.98	72.12	71.85	72.04	71.92	72.04	71.78
CNN-LSTM	95.14	95.23	95.14	95.17	95.20	95.29	95.20	95.23
FFNN-XGB	96.41	96.48	96.41	96.43	96.38	96.45	96.38	96.40
LSTM-XGB	98.24	98.25	98.24	98.24	98.26	98.27	98.26	98.27
TabNet-XGB	95.39	95.49	95.39	95.42	95.39	95.49	95.39	95.42

Table 11 Evaluation metrics results for SMOTE-MAS-2024-B (70-30).

SMOTE-MAS-2024-B (70-30)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	97.79	97.81	97.79	97.80	97.75	97.77	97.75	97.75
LGB	97.81	97.83	97.81	97.81	97.75	97.77	97.75	97.75
CatBoost	96.64	96.69	96.64	96.65	96.69	96.73	96.69	96.70
RF	84.00	84.25	84.00	84.09	84.04	84.27	84.04	84.12
CNN	85.21	85.87	85.21	85.29	85.24	85.92	85.24	85.32
LSTM	86.52	87.09	86.52	86.71	86.64	87.18	86.64	86.81
FFNN	69.32	72.17	69.32	70.10	69.18	72.03	69.18	69.96
TabNet	62.93	64.37	62.90	63.24	62.93	64.38	62.93	63.26
CNN-LSTM	94.65	94.73	94.65	94.68	94.64	94.71	94.64	94.66
FFNN-XGB	95.53	95.61	95.53	95.55	95.57	95.64	95.57	95.59
LSTM-XGB	98.06	98.07	98.05	98.06	98.07	98.09	98.08	98.08
TabNet-XGB	92.91	93.06	92.91	92.96	93.02	93.16	93.02	93.06

Table 12 Evaluation metrics results for SMOTE-MAS-2024-B (60-40).

SMOTE-MAS-2024-B (60-40)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	97.78	97.80	97.78	97.78	97.74	97.76	97.74	97.75
LGB	97.73	97.74	97.73	97.73	97.69	97.71	97.69	97.69
CatBoost	96.65	96.70	96.65	96.66	96.66	96.71	96.66	96.67
RF	83.35	83.61	83.35	83.44	83.31	83.57	83.31	83.40
CNN	83.63	84.73	83.63	83.89	83.56	84.66	83.56	83.82
LSTM	86.01	86.72	86.01	86.21	86.09	86.77	86.09	86.28
FFNN	69.12	71.52	69.12	69.42	68.86	71.31	68.86	69.18
TabNet	57.03	60.04	57.03	57.84	56.99	59.96	56.99	57.78
CNN-LSTM	94.05	94.17	94.05	94.09	94.02	94.13	94.02	94.05
FFNN-XGB	95.12	95.21	95.12	95.15	95.13	95.22	95.13	95.16
LSTM-XGB	97.69	97.71	97.70	97.70	97.71	97.73	97.71	97.71
TabNet-XGB	91.39	91.61	91.39	91.46	91.53	91.73	91.53	91.59

Table 13 Evaluation metrics results for RUS-MAS-2024-A (80-20).

RUS-MAS-2024-A (80-20)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.22	98.23	98.22	98.22	98.22	97.87	98.00	97.93
LGB	98.35	98.37	98.35	98.35	98.33	98.33	98.33	98.33
CatBoost	98.23	98.26	98.23	98.24	98.27	98.30	98.27	98.28
RF	90.69	91.08	90.69	90.81	90.66	91.06	90.66	90.78
CNN	81.74	82.32	81.74	81.87	81.74	82.33	81.74	81.87
LSTM	90.75	91.02	90.76	90.85	90.76	91.02	90.76	90.85
FFNN	72.48	73.95	72.48	72.88	72.52	73.99	72.52	72.92
TabNet	64.83	66.54	64.83	65.18	64.79	66.49	64.79	65.13
CNN-LSTM	95.11	95.19	95.11	95.14	95.12	95.20	95.12	95.15
FFNN-XGB	98.31	98.33	98.31	98.32	98.30	98.33	98.30	98.31
LSTM-XGB	98.78	98.78	98.78	98.78	98.78	98.78	98.78	98.78
TabNet-XGB	98.83	98.84	98.83	98.84	98.83	98.84	98.83	98.84

Table 14 Evaluation metrics results for RUS-MAS-2024-A (70-30).

RUS-MAS-2024-A (70-30)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.18	98.19	98.18	98.18	98.18	97.74	98.03	97.88
LGB	98.30	98.32	98.30	98.31	98.27	98.28	98.27	98.27
CatBoost	98.24	98.27	98.24	98.24	98.24	98.27	98.24	98.25
RF	89.85	90.29	89.85	89.99	89.86	90.29	89.86	89.99
CNN	81.73	83.38	81.73	82.12	81.72	83.36	81.72	82.11
LSTM	88.81	89.05	88.81	88.87	88.78	89.02	88.78	88.85
FFNN	69.54	71.40	69.54	69.67	69.53	71.38	69.53	69.65
TabNet	59.77	60.44	59.77	59.65	59.67	60.34	59.67	59.55
CNN-LSTM	94.48	94.55	94.48	94.49	94.44	94.52	94.45	94.46
FFNN-XGB	98.32	98.34	98.32	98.33	98.28	98.32	98.30	98.30
LSTM-XGB	98.61	98.62	98.61	98.62	98.59	98.60	98.59	98.60
TabNet-XGB	98.19	98.19	98.19	98.19	98.20	98.20	98.20	98.20

Table 15 Evaluation metrics results for RUS-MAS-2024-A (60-40).

RUS-MAS-2024-A (60-40)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.07	98.08	98.07	98.07	98.05	97.68	97.85	97.76
LGB	98.21	98.23	98.21	98.22	98.19	98.21	98.19	98.19
CatBoost	98.23	98.25	98.23	98.24	98.25	98.28	98.25	98.26
RF	89.11	89.59	89.11	89.25	89.11	89.59	89.11	89.25
CNN	79.30	80.88	79.30	79.74	79.39	80.95	79.39	79.82
LSTM	87.59	88.05	87.60	87.71	87.55	88.01	87.55	87.66
FFNN	68.85	71.99	68.85	69.53	68.86	72.01	68.86	69.54
TabNet	53.95	56.86	53.95	54.58	54.02	56.93	54.02	54.63
CNN-LSTM	93.93	93.99	93.93	93.95	93.89	93.97	93.90	93.92
FFNN-XGB	97.94	97.99	97.94	97.96	97.89	97.95	97.89	97.91
LSTM-XGB	98.05	98.07	98.05	98.06	98.08	98.09	98.08	98.08
TabNet-XGB	93.48	93.52	93.48	93.49	93.48	93.52	93.48	93.49

Table 16 Evaluation metrics results for RUS-MAS-2024-B (80-20).

RUS-MAS-2024-B (80-20)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.42	98.42	98.42	98.42	98.38	98.06	98.26	98.16
LGB	98.59	98.60	98.59	98.59	98.56	98.57	98.56	98.56
CatBoost	97.91	97.94	97.91	97.92	97.92	97.95	97.92	97.92
RF	82.54	83.33	82.54	82.78	82.55	83.31	82.55	82.78
CNN	86.25	86.76	86.25	86.35	86.25	86.77	86.25	86.36
LSTM	86.75	86.99	86.75	86.81	86.81	87.06	86.81	86.87
FFNN	64.23	67.83	64.24	64.94	64.20	67.74	64.20	64.90
TabNet	63.72	65.82	63.72	63.90	63.77	65.89	63.77	63.95
CNN-LSTM	94.79	94.90	94.79	94.82	94.76	94.88	94.76	94.80
FFNN-XGB	97.41	97.43	97.41	97.42	97.39	97.41	97.39	97.40
LSTM-XGB	98.87	98.87	98.87	98.87	98.86	98.86	98.86	98.86
TabNet-XGB	98.22	98.23	98.22	98.22	98.18	98.20	98.18	98.19

Table 17 Evaluation metrics results for RUS-MAS-2024-B (70-30).

RUS-MAS-2024-B (70-30)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.31	98.31	98.31	98.31	98.29	98.06	98.16	98.10
LGB	98.45	98.47	98.45	98.45	98.40	98.41	98.40	98.40
CatBoost	97.93	97.96	97.93	97.94	97.89	97.92	97.89	97.89
RF	81.25	82.07	81.25	81.50	81.25	82.05	81.25	81.49
CNN	84.24	84.64	84.24	84.28	84.10	84.51	84.10	84.15
LSTM	86.34	86.66	86.34	86.44	86.40	86.70	86.40	86.49
FFNN	64.23	67.83	64.24	64.94	64.20	67.74	64.20	64.90
TabNet	53.95	56.86	53.95	54.58	54.02	56.93	54.02	54.63
CNN-LSTM	94.37	94.47	94.38	94.40	94.38	94.47	94.38	94.41
FFNN-XGB	97.19	97.23	97.20	97.21	97.22	97.25	97.22	97.23
LSTM-XGB	98.61	98.62	98.61	98.62	98.59	98.60	98.59	98.60
TabNet-XGB	97.04	97.08	97.04	97.05	97.08	97.13	97.08	97.10

Table 18 Evaluation metrics results for RUS-MAS-2024-B (60-40).

RUS-MAS-2024-B (60-40)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.22	98.23	98.22	98.22	98.22	97.87	98.00	97.93
LGB	98.20	98.21	98.20	98.20	98.17	98.19	98.17	98.17
CatBoost	97.92	97.95	97.92	97.93	97.88	97.91	97.88	97.89
RF	80.32	81.16	80.32	80.57	80.25	81.10	80.25	80.50
CNN	79.51	80.78	79.51	79.83	79.37	80.65	79.37	79.69
LSTM	84.08	84.56	84.09	84.20	84.16	84.62	84.16	84.27
FFNN	62.64	65.85	62.64	63.06	62.58	65.80	62.57	62.99
TabNet	46.03	50.51	46.03	47.02	46.14	50.60	46.14	47.09
CNN-LSTM	92.49	92.63	92.49	92.53	92.48	92.63	92.48	92.52
FFNN-XGB	96.37	96.42	96.37	96.39	96.41	96.45	96.41	96.42
LSTM-XGB	98.42	98.43	98.42	98.42	98.43	98.44	98.43	98.43
TabNet-XGB	96.38	96.42	96.38	96.39	96.38	96.43	96.38	96.40

Table 19 Evaluation metrics results for 150K-MAS-2024-A (80-20).

150K-MAS-2024-A (80-20)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.07	98.08	98.07	98.06	98.08	98.08	98.08	98.07
LGB	98.44	98.43	98.43	98.43	98.43	98.43	98.43	98.44
CatBoost	98.07	98.08	98.07	98.06	98.08	98.08	98.08	98.07
RF	89.77	89.96	89.77	89.83	89.79	89.98	89.79	89.85
CNN	80.51	81.50	80.51	80.63	80.39	81.43	80.40	80.53
LSTM	88.19	88.36	88.19	88.24	88.06	88.24	88.06	88.11
FFNN	69.07	71.79	69.07	69.45	69.13	71.81	69.13	69.50
TabNet	65.40	66.11	65.40	65.21	65.28	65.98	65.28	65.12
CNN-LSTM	94.48	94.50	94.48	94.48	94.49	94.51	94.49	94.50
FFNN-XGB	98.27	98.27	98.27	98.27	98.25	98.26	98.25	98.26
LSTM-XGB	98.78	98.78	98.78	98.78	98.78	98.78	98.78	98.78
TabNet-XGB	92.17	92.18	92.17	92.16	92.25	92.26	92.25	92.24

Table 20 Evaluation metrics results for 150K-MAS-2024-A (70-30).

150K-MAS-2024-A (70-30)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.18	98.19	98.18	98.18	98.20	98.20	98.20	98.19
LGB	98.14	98.14	98.14	98.13	98.13	98.13	98.12	98.14
CatBoost	98.07	98.08	98.07	98.07	98.06	98.06	98.06	98.06
RF	89.13	89.32	89.13	89.19	89.11	89.30	89.11	89.18
CNN	77.97	78.25	77.97	78.01	77.91	78.19	77.91	77.95
LSTM	88.17	88.36	88.17	88.21	88.03	88.22	88.03	88.07
FFNN	67.35	68.44	67.35	67.26	67.43	68.50	67.43	67.33
TabNet	59.77	60.44	59.77	59.65	59.67	60.34	59.67	59.55
CNN-LSTM	94.09	94.14	94.09	94.10	94.10	94.15	94.10	94.11
FFNN-XGB	98.08	98.09	98.08	98.08	98.06	98.07	98.06	98.06
LSTM-XGB	98.06	98.07	98.05	98.06	98.08	98.09	98.08	98.08
TabNet-XGB	98.19	98.19	98.19	98.19	98.20	98.20	98.20	98.20

Table 21 Evaluation metrics results for 150K-MAS-2024-A (60-40).

150K-MAS-2024-A (60-40)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	97.97	97.99	97.97	97.97	97.97	97.98	97.97	97.97
LGB	98.01	98.02	98.01	98.01	97.99	98.00	97.99	97.99
CatBoost	97.97	97.99	97.97	97.97	97.97	97.98	97.97	97.97
RF	88.70	88.91	88.70	88.76	88.66	88.87	88.66	88.73
CNN	77.85	78.75	77.86	77.95	77.92	78.84	77.92	78.02
LSTM	85.78	86.03	85.78	85.85	85.73	85.99	85.73	85.80
FFNN	66.87	68.25	66.87	67.08	66.95	68.30	66.95	67.15
TabNet	53.70	54.80	53.70	53.69	53.66	54.78	53.66	53.67
CNN-LSTM	94.05	94.08	94.05	94.06	94.01	94.04	94.01	94.01
FFNN-XGB	98.03	98.04	98.03	98.03	97.97	97.97	97.97	97.97
LSTM-XGB	98.91	98.91	98.91	98.91	98.88	98.88	98.88	98.88
TabNet-XGB	98.34	98.34	98.34	98.34	98.36	98.36	98.36	98.36

Table 22 Evaluation metrics results for 150K-MAS-2024-B (80-20).

70K-MAS-2024-B (80-20)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.34	98.35	98.34	98.34	98.36	98.37	98.36	98.36
LGB	98.30	98.30	98.30	98.30	98.24	98.24	98.24	98.24
CatBoost	98.11	98.12	98.11	98.11	98.05	98.06	98.05	98.05
RF	81.74	82.03	81.74	81.83	81.94	82.23	81.94	82.04
CNN	83.17	83.40	83.17	83.05	83.17	83.36	83.17	83.03
LSTM	85.93	86.17	85.93	86.00	85.91	86.13	85.91	85.96
FFNN	66.27	67.98	66.27	66.50	66.47	68.17	66.47	66.70
TabNet	55.27	57.94	55.27	55.78	55.16	57.94	55.16	55.71
CNN-LSTM	95.43	95.45	95.43	95.43	95.37	95.39	95.37	95.38
FFNN-XGB	97.74	97.75	97.74	97.74	97.60	97.61	97.60	97.61
LSTM-XGB	98.84	98.84	98.84	98.84	98.81	98.81	98.81	98.81
TabNet-XGB	95.94	95.96	95.94	95.95	96.04	96.07	96.04	96.05

Table 23 Evaluation metrics results for 150K-MAS-2024-B (70-30).

70K-MAS-2024-B (70-30)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.30	98.30	98.30	98.29	98.32	98.33	98.32	98.32
LGB	98.27	98.27	98.27	98.26	98.24	98.23	98.24	98.23
CatBoost	98.04	98.06	98.04	98.05	98.01	98.03	98.01	98.02
RF	80.28	80.61	80.28	80.39	80.38	80.69	80.38	80.49
CNN	83.10	83.97	83.10	83.27	83.01	83.87	83.01	83.18
LSTM	84.58	84.69	84.58	84.56	84.48	84.59	84.48	84.46
FFNN	62.46	66.02	62.46	62.66	62.51	66.04	62.51	62.70
TabNet	52.14	53.41	52.14	51.36	51.93	53.22	51.93	51.16
CNN-LSTM	94.32	94.36	94.32	94.32	94.32	94.37	94.32	94.33
FFNN-XGB	96.52	96.53	96.52	96.53	96.39	96.41	96.40	96.40
LSTM-XGB	98.78	98.78	98.78	98.78	98.78	98.78	98.78	98.78
TabNet-XGB	95.71	95.73	95.71	95.72	95.72	95.74	95.72	95.73

Table 24 Evaluation metrics results for 150K-MAS-2024-B (60-40).

70K-MAS-2024-B (60-40)								
Models	Validation (%)				Testing (%)			
	Accu	Pre	Rec	F1	Accu	Pre	Rec	F1
XGB	98.12	98.13	98.12	98.12	98.19	98.19	98.19	98.18
LGB	98.25	98.25	98.25	98.23	98.23	98.23	98.23	98.25
CatBoost	97.99	98.00	97.99	97.99	97.93	97.93	97.93	97.93
RF	78.87	79.23	78.87	78.99	78.96	79.28	78.96	79.08
CNN	82.22	83.14	82.22	82.32	82.06	82.96	82.06	82.15
LSTM	81.42	81.81	81.43	81.46	81.31	81.68	81.31	81.33
FFNN	62.18	64.62	62.18	62.53	62.31	64.76	62.31	62.65
TabNet	42.93	44.31	42.93	42.79	43.13	44.54	43.13	42.99
CNN-LSTM	93.58	93.61	93.58	93.58	93.52	93.55	93.52	93.53
FFNN-XGB	95.49	95.51	95.49	95.49	95.35	95.37	95.35	95.36
LSTM-XGB	98.15	98.15	98.15	98.15	98.14	98.14	98.14	98.14
TabNet-XGB	95.05	95.07	95.05	95.05	95.05	95.07	95.05	95.06

APPENDIX B

Experimental Results for Micro-Batch Processing

Table 1 Accuracy and precision for all the models for Air-Droid (MAS-2024-A).

Class- Air-Droid-MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	61.17	64.93	67.65	67.41	67.34	61.29	65.15	67.78	67.53	67.69
FFNN-XGB	65.10	65.52	66.62	65.82	63.56	65.44	65.64	67.18	65.94	63.68
LSTM	64.45	67.60	69.66	64.16	65.76	64.98	67.72	70.12	64.28	66.44
LSTM-XGB	77.82	74.48	79.79	75.85	74.69	77.94	74.71	79.92	75.98	75.25
CNN-LSTM	85.60	86.68	86.98	85.07	84.27	85.72	86.97	87.11	85.19	84.84
CNN	68.88	68.61	69.47	69.25	67.89	69.43	68.73	70.03	69.37	68.01
FFNN	62.81	63.64	65.28	63.31	61.68	62.87	63.77	65.84	63.43	61.80
TabNet	53.42	52.96	57.42	51.14	50.67	53.55	53.48	57.55	51.27	51.34
RF	90.06	90.17	91.32	89.79	88.67	90.19	90.28	91.45	89.92	88.80
CatBooat	88.78	89.45	91.73	90.01	88.82	89.46	90.09	92.41	90.69	89.47
LGB	87.34	89.21	91.58	90.64	88.27	88.07	89.89	92.26	91.22	88.95
XGB	87.69	88.74	91.07	88.28	87.73	88.37	89.42	91.75	88.94	88.41

Table 2 Accuracy and precision for all the models for AT-B-D-M (MAS-2024-A).

Class- AT-B-D-M- MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet- XGB	65.07	65.90	70.89	70.29	68.89	63.12	66.12	71.02	70.41	69.24
FFNN- XGB	83.29	83.51	84.00	81.83	83.08	83.65	83.63	84.56	81.95	83.21
LSTM	64.61	64.45	65.75	64.13	65.32	65.36	64.98	65.87	64.25	65.99
LSTM- XGB	77.51	76.45	82.71	81.11	81.83	78.04	77.07	82.53	81.19	81.70
CNN- LSTM	86.55	86.57	88.01	86.16	86.56	86.67	86.70	88.23	86.26	87.12
CNN	66.02	65.63	69.54	69.26	68.46	66.58	66.19	69.66	69.38	68.59
FFNN	61.81	64.57	67.61	65.54	63.78	61.94	64.69	68.17	65.67	63.91
TabNet	63.91	66.28	71.01	70.88	69.21	64.04	66.81	71.69	71.01	69.34
RF	85.58	84.48	87.26	87.60	87.56	86.26	85.16	87.94	88.28	87.56
CatBooat	80.85	84.52	91.17	86.55	88.43	81.53	85.09	91.85	87.23	88.43
LGB	78.79	83.14	92.29	86.55	88.17	79.47	83.82	92.97	87.23	88.24
XGB	82.75	83.51	90.17	86.62	85.98	83.43	84.19	90.85	87.31	86.67

Table 3 Accuracy and precision for all the models for AT-P-C (MAS-2024-A).

Class- AT-P-C- MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet- XGB	65.38	70.67	74.81	72.96	71.56	65.49	70.89	74.93	73.09	71.91
FFNN- XGB	85.25	87.17	88.91	85.84	86.45	85.58	87.29	89.03	86.39	86.57
LSTM	71.91	68.85	72.25	69.67	67.87	72.67	69.38	73.25	69.79	68.12
LSTM- XGB	80.49	84.47	87.21	85.06	83.55	81.02	84.69	87.76	85.19	83.67
CNN- LSTM	81.46	83.05	87.14	85.01	85.55	81.58	83.87	86.68	85.54	86.07
CNN	70.23	66.47	73.46	71.87	64.72	70.79	66.59	73.58	72.42	68.84

Table 3 Accuracy and precision for all the models for AT-P-C (MAS-2024-A), continued

FFNN	67.13	67.18	70.00	69.08	68.54	67.26	67.73	70.12	69.33	68.66
TabNet	49.58	52.64	57.16	52.91	55.24	49.71	53.16	57.28	53.04	55.91
RF	85.66	86.11	91.30	88.39	89.74	84.79	86.24	91.98	88.52	89.27
CatBooat	91.83	88.23	92.33	90.72	88.25	92.52	88.91	93.01	91.35	88.91
LGB	91.49	90.57	93.49	88.02	86.52	92.17	91.25	94.17	88.70	87.18
XGB	90.67	90.57	93.82	89.26	90.78	91.35	91.25	94.49	89.93	90.79

Table 4 Accuracy and precision for all the models for AT-C-M (MAS-2024-A).

Class- AT-C-M- MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	85.29	86.56	87.04	86.61	86.17	85.40	86.69	87.16	86.74	86.29
XGB										
FFNN-	81.16	83.43	87.25	84.21	84.01	81.49	83.55	87.81	84.34	84.12
XGB										
LSTM	63.27	68.54	72.50	68.08	69.15	64.03	69.06	73.50	68.20	69.27
LSTM-	81.94	84.34	86.62	84.25	82.68	82.46	84.56	87.17	84.38	82.79
XGB										
CNN-	83.12	86.78	86.93	86.12	84.75	83.21	87.60	87.46	86.64	85.30
LSTM										
CNN	70.66	66.66	73.44	67.02	68.96	71.24	66.78	73.56	67.15	69.52
FFNN	62.06	57.67	62.29	62.19	62.01	62.18	57.37	62.42	62.26	62.14
TabNet	66.92	68.37	69.33	68.00	66.49	67.04	68.89	69.99	68.11	66.62
RF	89.59	89.27	89.75	89.34	86.69	89.72	89.39	90.43	90.02	87.36
CatBooat	87.22	86.78	88.25	87.49	87.38	87.38	86.93	88.40	88.01	87.91
LGB	87.74	87.60	89.46	86.07	87.15	87.89	87.76	89.16	86.59	87.68
XGB	89.00	89.27	89.77	86.27	86.92	89.16	89.43	90.29	86.43	87.45

Table 5 Accuracy and precision for all the models for Eyezy (MAS-2024-A).

Class- Eyezy- MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	64.29	67.45	70.01	69.22	69.02	64.52	67.68	70.24	69.46	69.01
FFNN-XGB	85.69	87.53	88.12	87.71	87.58	85.84	87.67	88.26	87.90	87.76
LSTM	79.65	83.04	85.99	85.64	85.45	79.66	83.05	86.00	85.64	85.45
LSTM-XGB	85.22	87.76	89.93	86.10	85.71	85.37	87.89	90.08	86.26	85.85
CNN-LSTM	85.64	86.03	87.78	87.58	85.03	85.64	86.21	87.95	87.76	85.20
CNN	77.29	79.53	81.96	81.46	80.56	77.48	79.65	82.11	81.65	80.75
FFNN	67.77	69.60	71.86	70.41	69.84	67.87	69.76	71.98	70.40	69.83
TabNet	62.83	69.91	70.90	70.02	69.02	63.61	70.69	71.68	70.80	69.80
RF	82.45	82.23	89.65	85.05	85.43	83.14	82.91	90.33	85.73	86.11
CatBooat	92.25	92.26	92.41	88.55	87.29	93.18	92.91	93.09	88.54	87.76
LGB	74.77	76.80	78.46	78.15	78.16	87.71	87.67	92.13	78.23	78.65
XGB	83.64	84.00	92.48	88.34	86.30	87.41	89.05	93.20	88.73	89.49

Table 6 Accuracy and precision for all the models for FGP (MAS-2024-A).

Class-FGP-MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	67.71	73.31	78.77	78.49	72.61	67.84	73.53	79.28	78.61	72.95
FFNN-XGB	84.37	86.79	93.05	89.26	87.67	84.36	86.92	93.17	89.82	87.79
LSTM	66.42	65.40	69.54	66.34	68.56	67.20	65.92	69.66	66.45	69.56
LSTM-XGB	84.51	86.47	92.39	89.75	87.04	85.03	86.69	92.93	90.27	87.16
CNN-LSTM	84.79	86.53	89.31	88.36	88.61	84.91	87.34	89.86	88.36	89.14
CNN	72.24	74.11	76.46	76.36	75.05	72.79	74.23	76.55	76.91	75.17

Table 6 Accuracy and precision for all the models for FGP (MAS-2024-A), continued.

FFNN	65.67	64.53	66.88	66.87	65.69	65.79	64.65	67.44	66.99	66.21
TabNet	50.41	54.15	55.32	55.16	54.34	50.53	54.67	55.96	55.28	54.46
RF	79.52	81.01	90.51	85.05	85.50	80.19	81.75	91.19	85.72	86.18
CatBooat	86.41	88.42	92.77	90.65	87.54	87.08	89.09	93.45	91.32	88.21
LGB	85.17	88.31	92.12	90.00	88.87	85.84	88.99	92.75	90.68	89.54
XGB	85.82	84.52	93.04	89.57	86.78	86.51	85.19	93.72	90.30	87.46

Table 7 Accuracy and precision for all the models for FamiSafe (MAS-2024-A).

Class-FamiSafe- MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	69.78	73.15	76.36	75.25	75.04	69.90	73.36	76.88	75.38	75.39
XGB										
FFNN-	83.07	86.99	93.98	88.48	88.15	83.41	87.11	94.10	89.04	88.27
XGB										
LSTM	64.07	66.23	70.00	67.87	67.19	64.83	66.75	70.12	67.99	68.18
LSTM-	84.67	87.09	92.93	90.24	86.18	85.18	87.31	93.48	90.75	86.30
XGB										
CNN-	85.01	86.33	89.94	87.74	88.54	85.14	87.15	90.47	88.26	89.11
LSTM										
CNN	76.74	76.89	80.23	79.82	79.53	77.29	77.01	80.35	80.38	79.65
FFNN	62.28	65.51	68.37	65.17	68.09	62.41	65.63	68.93	65.29	68.62
TabNet	52.28	54.31	57.24	54.59	56.64	52.41	54.82	57.37	54.58	57.31
RF	86.06	86.02	90.52	88.56	88.23	86.73	86.69	91.19	89.24	88.98
CatBooat	82.85	87.13	89.94	88.66	89.40	83.12	87.81	90.56	89.33	90.18
LGB	82.73	84.48	90.55	88.75	89.54	83.41	85.16	91.22	89.43	90.21
XGB	84.18	86.45	91.05	87.33	89.54	84.84	87.12	91.72	88.01	90.22

Table 8 Accuracy and precision for all the models for kids-360 (MAS-2024-A).

Class-kids-360- MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	81.20	86.09	89.90	85.32	87.60	82.73	86.31	90.25	85.43	88.12
FFNN-XGB	86.92	86.61	91.87	90.87	86.67	87.25	86.73	91.99	91.43	86.78
LSTM	64.14	66.61	71.63	71.24	67.07	64.89	67.14	71.75	71.36	68.06
LSTM-XGB	86.38	88.00	91.42	87.50	90.24	86.89	88.22	91.55	88.02	90.79
CNN-LSTM	91.16	91.69	92.87	91.67	91.87	91.28	92.20	93.69	92.18	92.42
CNN	76.95	75.62	80.36	74.00	74.89	77.51	75.77	80.92	74.14	75.01
FFNN	62.28	63.83	68.29	66.67	66.78	63.95	67.33	68.81	66.78	63.93
TabNet	54.49	54.95	58.13	56.55	55.81	54.62	55.47	58.79	56.66	55.93
RF	89.14	86.96	90.58	89.09	88.41	90.70	89.26	87.63	89.63	89.09
CatBooat	86.38	87.49	91.47	90.66	88.62	87.05	88.15	92.15	91.35	89.29
LGB	87.54	88.69	92.76	89.09	89.02	88.21	89.37	93.44	89.76	89.70
XGB	87.39	88.17	93.09	86.31	92.00	88.07	88.85	93.76	86.98	92.67

Table 9 Accuracy and precision for all the models for KGP (MAS-2024-A).

Class-KGP- MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	74.28	77.95	84.67	80.87	81.27	74.28	78.16	85.18	80.99	81.61
FFNN-XGB	85.37	84.73	92.06	90.80	86.55	85.71	84.85	92.17	91.35	86.72
LSTM	79.87	76.53	81.59	75.95	75.82	80.62	76.65	82.12	76.06	76.29
LSTM-XGB	81.99	77.98	84.50	82.88	83.53	82.11	78.21	85.02	83.62	84.06
CNN-LSTM	86.26	88.54	88.78	87.29	86.49	86.38	89.06	89.59	83.39	87.05
CNN	73.17	73.96	74.65	73.36	72.39	73.69	74.08	75.20	73.48	72.51

Table 9 Accuracy and precision for all the models for KGP (MAS-2024-A), continued.

FFNN	65.37	67.09	70.57	66.14	70.73	65.49	67.65	70.68	66.26	71.25
TabNet	56.68	54.12	59.33	56.44	54.35	56.85	54.64	59.46	56.56	55.01
RF	85.69	85.37	88.91	84.43	82.52	85.88	85.42	89.04	84.98	83.21
CatBooat	86.68	88.59	92.68	90.00	91.60	87.35	89.27	93.02	90.68	92.27
LGB	86.83	89.08	93.49	88.44	91.95	87.51	89.41	94.17	89.04	92.62
XGB	88.61	90.44	91.84	90.84	90.82	89.29	91.21	92.15	91.51	91.49

Table 10 Accuracy and precision for all the models for mLite (MAS-2024-A).

Class-mLite- MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	84.49	83.47	85.80	82.83	79.27	84.62	83.82	86.03	82.95	79.79
XGB										
FFNN-	84.71	86.23	89.86	89.67	88.13	85.04	86.35	89.97	90.24	88.25
XGB										
LSTM	80.35	82.20	82.43	77.50	76.08	80.47	82.77	83.19	77.62	77.08
LSTM-	81.19	86.22	87.59	87.38	86.32	81.72	86.44	88.11	87.93	86.44
XGB										
CNN-	84.50	83.41	86.26	74.33	82.16	84.60	83.93	87.08	74.82	82.71
LSTM										
CNN	73.08	73.16	76.27	73.72	71.17	73.74	73.34	76.82	73.84	71.29
FFNN	62.88	67.19	68.94	63.36	67.21	63.00	67.75	69.06	63.48	67.68
TabNet	55.26	54.23	57.04	54.67	54.07	55.37	54.75	57.15	54.79	54.73
RF	85.52	84.64	87.89	84.66	85.47	85.64	84.76	88.45	84.77	86.14
CatBooat	85.64	87.96	90.99	89.68	86.66	86.32	88.65	91.65	90.35	87.26
LGB	84.56	88.29	91.87	90.42	91.18	85.23	88.96	92.54	91.11	91.86
XGB	84.57	87.41	93.12	89.02	90.75	85.23	88.04	93.80	89.70	91.43

Table 11 Accuracy and precision for all the models for MMP (MAS-2024-A).

Class-MoniMasterPro- MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	92.64	90.94	93.42	92.64	91.57	92.65	91.17	93.42	92.76	91.91
FFNN-XGB	85.16	88.71	91.84	89.95	87.54	85.49	88.83	91.96	90.51	87.66
LSTM	86.15	87.08	87.78	87.74	85.45	87.50	87.21	88.29	87.85	86.44
LSTM-XGB	83.04	85.33	91.35	87.44	86.35	83.56	85.55	91.47	87.96	86.90
CNN-LSTM	85.55	87.20	90.73	88.03	86.98	85.57	88.02	91.25	88.54	87.53
CNN	74.74	73.25	74.86	73.56	72.19	75.28	73.37	75.41	73.68	72.31
FFNN	59.22	62.87	70.12	68.97	67.76	59.77	62.98	70.23	69.50	67.86
TabNet	53.56	55.18	56.74	56.09	55.17	53.68	55.60	56.83	56.21	55.81
RF	84.48	86.24	88.67	86.87	81.56	84.60	86.36	89.22	86.99	82.23
CatBooat	87.09	87.89	90.87	90.49	90.28	87.77	88.56	91.55	91.18	90.95
LGB	86.95	90.68	91.02	86.39	89.24	87.62	91.35	91.70	87.07	89.92
XGB	85.66	89.03	92.06	89.88	92.04	86.00	89.71	93.17	90.55	92.72

Table 12 Accuracy and precision for all the models for mSpy (MAS-2024-A).

Class-mSpy- MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	71.30	74.43	76.25	74.24	75.23	71.42	74.65	76.36	74.37	75.35
FFNN-XGB	83.52	87.98	90.42	88.41	86.48	83.68	88.09	90.48	88.96	86.60
LSTM	82.21	88.00	88.12	87.76	86.26	82.96	88.26	88.24	87.88	87.25
LSTM-XGB	63.77	63.42	69.48	65.01	64.90	63.89	63.59	70.00	65.55	65.45
CNN-LSTM	86.18	85.21	91.01	86.59	86.35	86.31	86.04	91.53	87.11	86.90
CNN	81.48	80.53	81.81	80.07	76.97	82.03	82.03	82.37	80.19	77.09

Table 12 Accuracy and precision for all the models for mSpy (MAS-2024-A), continued.

FFNN	64.38	66.13	69.35	65.03	68.80	64.45	66.68	69.47	65.15	69.33
TabNet	55.99	54.60	59.16	56.57	52.74	56.11	55.12	59.28	56.69	53.41
RF	84.43	86.13	88.12	83.35	81.52	84.54	86.25	88.67	83.47	82.20
CatBooat	87.31	89.56	90.78	87.50	89.29	87.74	89.98	91.20	88.45	90.25
LGB	87.40	85.57	91.06	89.70	90.76	87.82	85.99	92.01	90.12	91.71
XGB	87.05	88.08	89.94	89.39	89.29	87.49	88.51	90.36	90.35	90.24

Table 13 Accuracy and precision for all the models for Normal Traffic (MAS-2024-A).

Class-Normal-Traffic- MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet- XGB	80.93	83.24	89.08	87.82	84.42	81.45	83.67	89.45	89.12	84.62
FFNN- XGB	85.49	89.24	89.34	89.02	88.21	85.89	89.54	89.74	89.02	88.51
LSTM	84.99	86.68	87.65	87.28	85.64	85.29	86.98	87.95	87.58	85.94
LSTM- XGB	89.59	89.58	91.49	89.55	91.46	89.78	89.88	91.99	89.85	91.96
CNN- LSTM	85.23	85.73	88.83	87.68	88.67	85.56	85.73	88.83	87.99	89.02
CNN	72.33	74. 21	74.74	72.63	69.80	72.73	74. 51	75.22	72.98	70.12
FFNN	64.01	67.76	72.76	65.73	69.19	64.56	67.90	72.98	65.98	69.56
TabNet	68.26	68.96	70.55	67.77	66.94	68.86	69.34	70.97	67.98	67.34
RF	87.56	88.90	90.10	85.00	88.08	87.98	89.23	90.18	85.76	88.45
CatBooat	84.87	83.24	85.02	84.20	84.01	85.05	83.67	85.76	84.45	84.34
LGB	82.96	84.87	86.31	85.68	83.88	83.61	85.12	86.52	86.10	84.01
XGB	82.99	84.82	88.86	82.73	85.15	83.66	85.06	89.29	83.36	85.20

Table 14 Accuracy and precision for all the models for Qustodio (MAS-2024-A).

Class-Qustodio- MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	89.62	85.62	92.62	92.16	92.60	89.72	85.83	92.74	92.28	92.71
FFNN-XGB	84.95	89.20	90.27	88.49	86.76	85.28	89.32	90.82	88.61	86.87
LSTM	84.98	86.20	87.32	87.07	86.74	85.73	86.72	87.44	87.18	87.74
LSTM-XGB	66.22	68.87	73.13	67.78	67.01	66.34	65.09	73.68	68.30	67.55
CNN-LSTM	87.56	86.86	91.39	89.35	87.04	87.67	87.68	91.92	89.87	87.59
CNN	74.39	74.22	75.02	73.72	72.60	74.94	74.34	75.57	73.84	72.72
FFNN	69.37	69.52	70.83	70.56	70.49	69.49	69.66	71.38	70.68	71.01
TabNet	55.79	55.01	59.08	56.16	54.41	55.91	55.50	59.20	56.25	55.11
RF	89.78	87.09	90.10	87.87	87.88	89.89	87.19	90.23	88.41	88.56
CatBooat	85.41	88.87	90.57	89.66	88.17	86.37	89.82	90.72	89.82	88.32
LGB	84.48	88.61	92.97	90.41	89.08	85.43	89.56	93.14	90.56	89.19
XGB	86.17	88.25	91.29	90.99	89.12	87.12	89.20	91.44	91.06	89.26

Table 15 Accuracy and precision for all the models for TiSpy (MAS-2024-A).

Class-TiSpy- MAS-2024-A										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	71.51	76.99	79.85	78.11	79.65	71.52	77.01	79.87	78.14	79.67
FFNN-XGB	56.87	59.15	64.74	62.37	60.00	57.03	59.30	64.90	62.53	60.15
LSTM	67.05	71.25	78.16	76.77	77.93	67.24	71.44	78.36	76.97	78.14
LSTM-XGB	73.54	76.64	76.96	73.96	73.52	73.69	76.79	77.11	74.11	73.68
CNN-LSTM	84.21	85.40	86.75	86.37	85.31	84.87	86.06	87.41	87.03	85.97
CNN	68.78	65.42	70.82	68.62	67.49	69.77	66.40	71.81	69.60	68.48

Table 15 Accuracy and precision for all the models for TiSpy (MAS-2024-A), continued.

FFNN	59.81	60.44	62.11	60.96	61.76	59.96	60.60	62.26	61.11	61.91
TabNet	54.53	54.78	55.64	54.74	54.95	55.42	55.62	56.53	55.63	55.84
RF	88.88	89.12	89.86	89.54	88.89	89.10	89.35	90.08	89.76	89.18
CatBooat	76.38	73.78	79.72	78.17	83.64	77.05	74.45	80.40	78.81	84.28
LGB	74.77	76.80	78.46	78.15	78.16	75.44	77.47	79.14	79.39	78.84
XGB	83.64	84.00	92.48	88.34	86.30	84.32	84.65	93.10	89.03	86.98

Table 16 Recall and f1-score for all the models for Air-Droid (MAS-2024-A).

Class- Air-Droid- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	61.17	64.93	67.35	67.41	67.35	61.17	65.04	67.72	67.47	67.52
XGB										
FFNN-	65.10	65.52	66.62	65.82	63.56	65.27	65.58	66.89	65.88	63.62
XGB										
LSTM	64.45	67.60	69.66	64.16	65.76	64.72	67.66	69.83	64.22	66.10
LSTM-	77.82	74.48	79.79	75.85	74.69	77.87	74.59	79.86	75.92	74.97
XGB										
CNN-	85.60	86.68	86.98	85.07	84.27	85.66	86.91	87.05	85.14	84.56
LSTM										
CNN	68.88	68.61	69.47	69.25	67.89	69.15	68.67	69.75	69.31	67.95
FFNN	62.81	63.64	65.28	63.31	61.68	62.94	63.71	65.56	63.37	61.74
TabNet	53.42	52.96	57.42	51.14	50.67	53.49	53.22	57.49	51.21	51.01
RF	90.06	90.17	91.32	89.79	88.67	90.14	90.22	91.38	89.86	88.74
CatBooat	88.78	89.45	91.73	90.01	88.82	89.12	89.76	92.07	90.36	89.13
LGB	87.34	89.21	91.58	90.64	88.27	87.73	89.56	91.19	90.88	88.61
XGB	87.69	88.74	91.07	88.28	87.73	88.03	89.08	91.41	88.59	88.07

Table 17 Recall and f1-score for all the models for AT-B-D-M (MAS-2024-A).

Class- AT-B-D-M- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	65.07	65.90	70.89	70.29	68.89	63.07	66.01	70.96	70.35	69.07
FFNN-XGB	83.29	83.51	84.00	81.83	83.08	83.46	83.57	84.28	81.89	83.15
LSTM	64.61	64.45	65.75	64.13	65.32	64.98	64.72	65.81	64.19	65.65
LSTM-XGB	77.51	76.45	82.71	81.11	81.83	77.77	76.96	82.46	81.14	81.42
CNN-LSTM	86.55	86.57	88.01	86.16	86.56	86.61	86.64	88.12	86.19	86.85
CNN	66.02	65.63	69.54	69.26	68.46	66.29	65.91	69.60	69.33	68.53
FFNN	61.81	64.57	67.61	65.54	63.78	61.87	64.57	67.89	65.61	63.85
TabNet	63.91	66.28	71.01	70.88	69.21	63.97	66.54	71.35	70.95	69.28
RF	85.58	84.48	87.26	87.60	87.56	85.92	84.82	87.59	87.94	87.90
CatBooat	80.85	84.52	91.17	86.55	88.43	81.19	84.75	91.51	86.89	88.77
LGB	78.79	83.14	92.29	86.55	88.17	79.13	83.48	92.63	86.89	88.51
XGB	82.75	83.51	90.17	86.62	85.98	83.09	83.85	90.51	86.97	86.33

Table 18 Recall and f1-score for all the models for AT-P-C (MAS-2024-A).

Class- AT-P-C- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	65.38	70.67	74.81	72.96	71.56	65.44	70.79	74.87	73.02	71.73
FFNN-XGB	85.25	87.17	88.91	85.84	86.45	85.42	87.23	88.97	86.12	86.51
LSTM	71.91	68.85	72.25	69.67	67.87	72.28	69.12	72.75	69.74	69.79
LSTM-XGB	80.49	84.47	87.21	85.06	83.55	80.75	84.58	87.49	85.12	83.61
CNN-LSTM	81.46	83.05	87.14	85.01	85.55	81.52	83.46	87.39	85.27	85.81
CNN	70.23	66.47	73.46	71.87	64.72	70.51	66.53	73.52	72.15	68.78

Table 18 Recall and f1-score for all the models for AT-P-C (MAS-2024-A), continued.

FFNN	67.13	67.18	70.00	69.08	68.54	67.19	67.45	70.06	69.26	68.60
TabNet	49.58	52.64	57.16	52.91	55.24	49.70	52.90	57.22	52.97	55.57
RF	85.66	86.11	91.30	88.39	89.74	84.72	86.18	91.64	88.46	89.80
CatBooat	91.83	88.23	92.33	90.72	88.25	92.18	88.57	92.67	91.01	88.57
LGB	91.49	90.57	93.49	88.02	86.52	91.83	90.91	93.82	88.36	86.88
XGB	90.67	90.57	93.82	89.26	90.78	91.04	90.91	94.16	89.59	90.45

Table 19 Recall and f1-score for all the models for AT-C-M (MAS-2024-A).

Class- AT-C-M- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	85.29	86.55	87.04	86.61	86.17	85.34	86.63	87.09	86.68	86.23
XGB										
FFNN-	81.16	83.43	87.25	84.21	84.01	81.32	83.49	87.53	84.28	84.07
XGB										
LSTM	63.26	68.53	72.50	68.05	69.15	63.65	68.79	72.99	68.14	69.21
LSTM-	81.92	84.35	86.62	84.29	82.70	82.19	84.44	86.89	84.31	82.74
XGB										
CNN-	83.10	86.74	86.93	86.17	84.75	83.15	87.19	87.19	86.38	85.03
LSTM										
CNN	70.86	66.66	73.44	67.02	68.92	70.96	66.72	73.50	67.08	69.23
FFNN	62.06	57.66	62.29	62.19	62.01	62.12	57.09	62.36	62.20	62.07
TabNet	66.92	68.37	69.34	68.00	66.49	66.98	68.63	69.66	68.07	66.55
RF	89.54	89.27	89.75	89.34	86.69	89.66	89.33	90.09	89.67	87.02
CatBooat	87.22	86.78	88.25	87.52	87.38	87.29	86.86	88.32	87.75	87.65
LGB	87.74	87.60	89.46	86.07	87.15	87.82	87.68	89.54	86.33	87.42
XGB	89.05	89.26	89.77	86.27	86.92	89.08	89.35	90.03	86.35	87.14

Table 20 Recall and f1-score for all the models for Eyezy (MAS-2024-A).

Class-Eyezy- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	64.29	67.45	70.01	69.22	69.02	64.44	67.56	70.14	69.34	69.15
FFNN-XGB	85.69	87.53	88.14	87.71	87.58	85.77	87.59	88.19	87.80	87.66
LSTM	79.65	83.04	85.99	85.64	85.45	79.66	83.05	86.00	85.65	85.45
LSTM-XGB	85.18	87.76	89.93	86.10	85.71	85.29	87.81	90.00	86.18	85.69
CNN-LSTM	85.64	86.03	87.78	87.58	85.03	85.56	86.12	87.86	87.63	85.11
CNN	77.29	79.53	81.96	81.46	80.56	77.39	79.68	82.03	81.55	80.66
FFNN	67.77	69.60	71.86	70.41	69.84	67.80	69.68	71.92	70.48	69.91
TabNet	62.83	69.91	70.90	70.02	69.02	63.21	70.29	71.29	70.41	69.41
RF	82.45	82.23	89.65	85.05	85.43	82.79	82.57	89.99	85.39	85.76
CatBooat	92.25	92.26	92.41	88.55	87.29	92.84	92.57	92.75	88.88	87.42
LGB	74.77	76.80	78.46	78.10	78.16	87.37	87.33	91.79	78.46	78.25
XGB	83.64	84.00	92.48	88.34	86.30	87.07	88.38	92.85	88.38	89.15

Table 21 Recall and f1-score for all the models for FGP (MAS-2024-A).

Class-FGP- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	67.73	73.38	78.69	78.47	72.62	67.78	73.42	79.02	78.55	72.78
FFNN-XGB	84.37	86.79	93.05	89.26	87.67	84.53	86.85	86.85	93.11	89.54
LSTM	66.42	65.40	69.54	66.34	68.56	66.82	65.66	69.59	66.39	69.06
LSTM-XGB	84.56	86.53	92.59	89.86	87.00	84.77	86.55	92.65	90.01	87.10
CNN-LSTM	84.82	86.55	89.36	88.38	88.65	84.85	86.93	89.58	88.62	88.87
CNN	72.27	74.16	76.48	76.39	75.07	72.52	74.17	76.49	76.63	75.11

Table 21 Recall and f1-score for all the models for FGP (MAS-2024-A), continued.

FFNN	65.67	64.53	66.88	66.87	65.69	65.73	64.59	67.16	66.93	65.95
TabNet	50.41	54.15	55.32	55.16	54.34	50.47	54.41	55.62	55.22	54.40
RF	79.62	81.07	90.34	85.01	85.48	79.85	81.42	90.85	85.11	85.84
CatBooat	86.39	88.52	92.67	90.55	87.64	86.74	88.75	93.11	90.64	87.87
LGB	85.17	88.31	92.12	90.00	88.87	85.51	88.65	92.80	90.34	89.21
XGB	85.72	84.62	93.02	89.59	86.82	86.16	85.40	93.38	90.32	87.12

Table 22 Recall and f1-score for all the models for FamiSafe (MAS-2024-A).

Class-FamiSafe- MAS-2024-A										
Models	Recall (%0					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	69.68	73.17	76.38	75.23	75.00	69.84	73.25	76.62	75.32	75.21
XGB										
FFNN-	83.02	86.90	93.92	88.49	88.19	83.24	87.05	94.04	88.76	88.21
XGB										
LSTM	64.02	66.25	70.05	67.89	67.24	64.45	66.49	70.06	67.93	67.68
LSTM-	84.69	87.12	92.91	90.29	86.16	84.92	87.20	93.21	90.49	86.23
XGB										
CNN-	85.05	86.23	89.84	87.76	88.58	85.07	86.73	90.21	88.00	88.82
LSTM										
CNN	76.78	76.84	80.25	79.85	79.56	77.01	76.95	80.29	80.10	79.59
FFNN	62.25	65.55	68.39	65.19	68.04	62.35	65.56	68.64	65.23	68.36
TabNet	52.28	54.41	57.32	54.55	56.62	52.34	54.56	57.31	54.65	56.97
RF	86.06	86.07	90.26	88.56	88.23	86.39	86.36	90.85	88.90	88.56
CatBooat	82.89	87.16	89.87	88.67	89.40	82.78	87.46	90.23	88.99	89.84
LGB	82.73	84.42	90.59	88.85	89.54	83.06	84.82	90.88	89.09	89.87
XGB	84.18	86.41	91.09	87.43	89.54	84.49	86.78	91.38	87.67	89.87

Table 23 Recall and f1-score for all the models for kids-360 (MAS-2024-A).

Class-kids-360- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	81.25	86.09	89.91	85.35	87.63	82.67	86.19	90.07	85.37	87.86
FFNN-XGB	86.82	86.51	91.89	90.89	86.69	87.08	86.69	91.93	91.15	86.72
LSTM	64.19	66.65	71.67	71.29	67.09	64.51	66.86	71.68	71.29	67.56
LSTM-XGB	86.39	88.07	91.45	87.55	90.29	86.63	88.11	91.49	87.76	90.52
CNN-LSTM	91.19	91.67	92.86	91.66	91.86	91.22	91.95	93.27	91.92	92.14
CNN	76.98	75.66	80.39	74.05	74.84	77.23	75.71	80.64	74.06	74.95
FFNN	62.23	63.81	68.21	66.60	66.70	63.89	67.06	68.55	66.72	63.87
TabNet	54.46	54.92	58.10	56.50	55.80	54.55	55.21	58.46	56.60	55.87
RF	89.19	86.95	90.59	89.12	88.46	90.64	89.20	87.29	89.36	88.75
CatBooat	86.32	87.42	91.41	90.62	88.65	86.71	87.82	91.80	91.00	88.95
LGB	87.50	88.60	92.71	89.02	89.08	87.87	89.03	93.10	89.43	89.36
XGB	87.37	88.19	93.04	86.34	92.09	87.72	88.51	93.42	86.64	92.33

Table 24 Recall and f1-score for all the models for KGP (MAS-2024-A).

Class-KGP- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	74.25	77.90	84.69	80.82	81.21	74.22	78.04	84.92	80.93	81.44
FFNN-XGB	85.37	84.75	92.05	90.70	86.51	85.54	84.79	92.12	91.07	86.65
LSTM	79.87	76.50	81.58	75.90	75.79	80.24	76.58	81.85	76.00	75.79
LSTM-XGB	81.89	77.90	84.51	82.89	83.50	82.05	78.09	84.76	83.35	83.78
CNN-LSTM	86.29	88.50	88.72	87.22	86.45	86.32	88.80	89.18	83.14	86.77
CNN	73.15	73.93	74.61	73.33	72.30	73.41	74.02	74.92	73.42	72.45

Table 24 Recall and f1-score for all the models for KGP (MAS-2024-A), continued.

FFNN	65.39	67.03	70.59	66.15	70.74	65.42	67.37	70.62	66.20	70.99
TabNet	56.69	54.17	59.38	56.40	54.30	56.79	54.38	59.39	56.50	54.67
RF	85.69	85.37	88.81	84.41	82.40	85.82	85.37	88.97	84.71	82.86
CatBooat	86.61	88.52	92.61	90.05	91.61	87.01	88.93	93.36	90.34	91.93
LGB	86.80	89.03	93.39	88.48	91.90	87.17	89.75	93.83	88.69	92.28
XGB	88.65	90.42	91.84	90.87	90.89	88.95	90.86	91.84	91.17	91.14

Table 25 Recall and f1-score for all the models for mLite (MAS-2024-A).

Class-mLite- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	84.47	83.49	85.85	82.86	79.29	84.55	83.65	85.91	82.89	79.53
XGB										
FFNN-	84.71	86.27	89.89	89.69	88.16	84.87	86.28	89.91	89.95	88.19
XGB										
LSTM	80.35	82.25	82.46	77.57	76.12	80.41	82.46	82.81	77.56	76.58
LSTM-	81.25	86.27	87.49	87.32	86.37	81.45	86.32	87.85	87.66	86.38
XGB										
CNN-	84.57	83.42	86.26	74.37	82.16	84.54	83.66	86.66	74.56	82.43
LSTM										
CNN	73.15	73.16	76.29	73.78	71.19	73.45	73.28	76.54	73.78	71.23
FFNN	62.82	67.19	68.99	63.32	67.25	62.94	67.47	69.08	63.42	67.42
TabNet	55.29	54.27	57.07	54.63	54.05	55.31	54.48	57.09	54.73	54.39
RF	85.52	84.69	87.82	84.69	85.42	85.57	84.70	88.17	84.71	85.81
CatBooat	85.60	87.90	90.89	89.58	86.69	85.97	88.31	91.31	90.01	86.92
LGB	84.51	88.32	91.84	90.49	91.29	84.89	88.62	92.21	90.77	91.52
XGB	84.67	87.48	93.19	89.09	90.84	84.89	87.69	93.45	89.36	91.09

Table 26 Recall and f1-score for all the models for MMP (MAS-2024-A).

Class-MMP- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	92.66	90.99	93.32	92.60	91.50	92.71	91.05	93.47	92.64	91.74
FFNN-XGB	85.18	88.73	91.84	89.96	87.52	85.32	88.76	91.90	90.23	87.60
LSTM	86.17	87.07	87.79	87.77	85.49	87.12	87.14	88.03	87.79	85.94
LSTM-XGB	83.07	85.35	91.38	87.48	86.39	83.30	85.44	91.41	87.69	86.63
CNN-LSTM	85.56	87.20	90.79	88.06	86.92	85.51	87.61	90.99	88.28	87.25
CNN	74.74	73.25	74.86	73.56	72.19	75.00	73.31	75.12	73.62	72.25
FFNN	59.26	62.86	70.11	68.98	67.78	59.49	62.92	70.17	69.23	67.80
TabNet	53.58	55.12	56.77	56.07	55.19	53.62	55.41	56.77	56.16	55.47
RF	84.52	86.27	88.69	86.89	81.58	84.54	86.29	88.94	86.93	81.88
CatBooat	87.14	87.89	90.85	90.44	90.29	87.42	88.23	91.21	90.84	90.28
LGB	86.85	90.98	91.00	86.39	89.24	87.28	91.01	91.36	86.72	89.57
XGB	85.69	89.07	92.03	89.85	92.06	86.34	89.36	92.82	90.22	92.37

Table 27 Recall and f1-score for all the models for mSpy (MAS-2024-A).

Class-mSpy- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	71.32	74.45	76.23	74.28	75.29	71.36	74.54	76.30	74.31	75.29
FFNN-XGB	83.58	87.92	90.49	88.43	86.47	83.85	88.04	90.54	88.68	86.54
LSTM	82.23	88.09	88.17	87.72	86.28	82.58	88.52	88.18	87.81	86.75
LSTM-XGB	63.79	63.47	69.49	65.08	64.93	63.83	63.48	69.73	65.26	65.17
CNN-LSTM	86.19	85.27	91.05	86.49	86.39	86.23	85.62	91.27	86.85	86.62
CNN	81.49	80.56	81.78	80.02	76.87	81.75	81.75	82.09	80.14	77.03

Table 27 Recall and f1-score for all the models for mSpy (MAS-2024-A), continued.

FFNN	64.36	66.12	69.39	65.07	68.87	64.39	66.41	69.41	65.09	69.07
TabNet	55.89	54.63	59.14	56.52	52.78	56.05	54.86	59.22	56.64	53.07
RF	84.45	86.17	88.17	83.39	81.59	84.48	86.19	88.39	83.41	81.86
CatBooat	87.35	89.58	90.75	87.55	89.39	87.52	89.77	90.99	87.97	89.76
LGB	87.49	85.59	91.08	89.70	90.76	87.61	85.78	91.53	89.91	91.24
XGB	87.09	88.15	89.92	89.33	89.35	87.26	88.29	90.15	89.87	89.76

Table 28 Recall and f1-score for all the models for Normal Traffic (MAS-2024-A).

Class-Normal Traffic- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	80.93	83.27	89.06	87.89	84.46	80.99	83.45	89.15	87.99	84.62
XGB										
FFNN-	85.43	89.27	89.37	89.07	88.25	85.69	89.74	89.56	89.02	88.35
XGB										
LSTM	84.94	86.63	87.61	87.22	85.60	85.04	86.78	87.75	87.38	85.74
LSTM-	89.52	89.50	91.41	89.50	91.44	89.69	89.68	91.59	89.65	91.56
XGB										
CNN-	85.26	85.79	88.87	87.64	88.63	85.33	85.83	88.93	87.78	88.77
LSTM										
CNN	72.30	74.20	74.79	72.65	69.82	72.33	74.21	74.74	72.73	70.23
FFNN	64.08	67.76	72.74	65.72	69.14	64.12	67.86	72.86	65.83	69.29
TabNet	68.20	68.90	70.50	67.74	66.90	68.36	69.12	70.65	67.87	67.16
RF	87.52	88.92	90.18	85.07	88.12	87.66	88.97	90.22	85.12	88.18
CatBooat	84.89	83.27	85.09	84.25	84.09	84.97	83.34	85.09	84.28	84.09
LGB	82.99	84.82	86.36	85.65	83.82	83.24	85.23	86.63	85.98	83.99
XGB	82.89	84.80	88.86	82.73	85.11	83.42	84.92	89.19	83.26	85.32

Table 29 Recall and f1-score for all the models for Qustodio (MAS-2024-A).

Class-Qustodio- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	89.66	85.65	92.65	92.19	92.65	89.66	85.72	92.68	92.22	92.65
FFNN-XGB	84.92	89.23	90.25	88.45	86.78	85.11	89.26	90.55	88.55	86.81
LSTM	84.92	86.23	87.35	87.05	86.79	85.35	86.46	87.38	87.12	87.23
LSTM-XGB	66.25	68.84	73.19	67.73	67.08	66.28	64.98	73.42	68.04	67.28
CNN-LSTM	87.58	86.8	91.32	89.36	87.09	87.61	87.27	91.65	89.61	87.31
CNN	74.33	74.24	75.05	73.73	72.62	74.66	74.28	75.29	73.77	72.66
FFNN	69.39	69.57	70.84	70.52	70.45	69.43	69.58	71.10	70.62	70.75
TabNet	55.73	55.07	59.07	56.18	54.42	55.85	55.23	59.14	56.19	54.77
RF	89.72	87.03	90.12	87.89	87.89	89.83	87.08	90.16	88.14	88.22
CatBooat	85.43	88.84	90.52	89.61	88.12	85.89	89.33	90.64	89.74	88.24
LGB	84.42	88.62	92.92	90.42	89.03	84.95	89.08	93.05	90.48	89.11
XGB	86.12	88.21	91.26	90.93	89.15	86.64	88.72	91.36	90.98	89.18

Table 30 Recall and f1-score for all the models for TiSpy (MAS-2024-A).

Class-TiSpy- MAS-2024-A										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	71.55	76.93	79.81	78.14	79.62	71.51	77.00	79.86	78.12	79.66
FFNN-XGB	56.84	59.19	64.79	62.39	60.05	56.95	59.22	64.82	62.44	60.07
LSTM	67.09	71.29	78.19	76.79	77.99	67.14	71.34	78.26	76.87	78.03
LSTM-XGB	73.51	76.61	76.92	73.92	73.59	73.61	76.71	77.04	74.04	73.61
CNN-LSTM	84.24	85.45	86.72	86.33	85.34	84.54	85.73	87.08	86.70	85.64
CNN	68.73	65.47	70.85	68.64	67.44	69.27	65.91	71.32	69.11	67.98

Table 30 Recall and f1-score for all the models for TiSpy (MAS-2024-A), continued.

FFNN	59.89	60.45	62.15	60.94	61.72	59.89	60.52	62.18	61.04	61.83
TabNet	54.56	54.72	55.62	54.72	54.93	54.98	55.17	56.08	55.18	55.39
RF	88.89	89.17	89.89	89.56	88.88	89.06	89.24	89.97	89.65	89.07
CatBooat	76.32	73.73	79.78	78.12	83.64	76.71	74.11	80.05	78.46	83.94
LGB	74.79	76.89	78.42	78.19	78.19	75.11	77.14	78.80	79.05	78.50
XGB	83.69	84.09	92.49	88.39	86.38	83.97	84.31	92.76	88.69	86.64

Table 4 Accuracy and precision for all the models for Air-Droid (MAS-2024-B).

Class-Air-Droid- MAS-2024-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	61.73	64.53	65.34	61.59	63.11	61.85	64.75	65.68	61.71	63.23
XGB										
FFNN-	61.93	57.10	65.56	64.31	63.75	62.26	57.22	65.68	64.86	63.86
XGB										
LSTM	63.49	66.38	67.30	63.86	64.69	63.61	66.90	67.85	63.98	65.35
LSTM-	76.34	78.00	82.62	78.25	77.03	76.43	78.22	82.73	78.37	77.59
XGB										
CNN-	81.53	82.33	84.40	81.73	80.33	81.65	82.45	84.95	81.85	80.45
LSTM										
CNN	71.75	69.96	72.14	69.95	69.81	72.30	70.08	72.26	70.50	69.93
FFNN	61.47	62.34	64.41	60.97	61.05	61.59	62.90	64.53	61.09	61.17
TabNet	54.42	52.13	54.66	52.81	53.77	54.54	52.65	54.78	52.92	54.41
RF	84.31	83.93	85.43	84.66	84.09	84.43	84.05	85.55	85.01	84.21
CatBooat	83.80	89.42	90.02	89.83	88.82	84.47	90.09	90.69	90.00	89.47
LGB	88.04	89.22	90.64	90.54	88.27	88.20	89.89	91.31	91.21	88.95
XGB	88.68	88.74	91.58	88.28	87.74	89.35	89.42	91.75	88.93	88.41

Table 5 Accuracy and precision for all the models for AT-B-D-M (MAS-2024-B).

Class-AT-B-D-M- MAS-2024-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	66.90	65.90	70.42	70.29	68.89	67.42	66.12	70.54	70.41	69.24
FFNN-XGB	81.24	81.83	83.51	81.29	83.08	81.57	81.95	83.62	81.85	83.20
LSTM	62.95	64.46	65.37	63.60	64.13	63.70	64.97	65.99	63.72	64.25
LSTM-XGB	77.51	81.83	85.81	81.11	81.15	78.03	82.05	85.92	81.19	81.70
CNN-LSTM	86.55	84.98	86.83	86.16	86.57	86.67	85.18	86.94	86.26	87.12
CNN	66.02	68.79	71.55	69.27	68.51	66.57	69.34	71.67	69.38	68.59
FFNN	61.81	61.48	65.55	64.59	63.79	61.93	62.03	65.67	64.71	63.91
TabNet	63.86	66.28	70.88	69.21	69.72	64.03	66.80	70.99	69.33	70.39
RF	83.01	82.38	83.83	81.40	81.09	83.14	82.49	83.95	81.52	81.77
CatBooat	81.80	84.52	88.43	86.55	86.96	82.44	85.09	87.04	87.22	89.11
LGB	87.23	83.14	89.43	86.55	88.17	87.88	83.82	89.60	87.23	88.85
XGB	80.09	83.51	91.25	86.63	85.99	80.77	84.19	91.92	87.30	86.66

Table 6 Accuracy and precision for all the models for AT-P-C (MAS-2024-B).

Class-AT-P-C- MAS-2024-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	67.12	64.73	71.99	70.98	66.81	67.25	64.94	72.11	71.09	67.15
FFNN-XGB	79.17	81.13	90.07	87.28	83.09	79.50	81.25	90.19	87.76	83.21
LSTM	66.16	62.96	69.62	68.99	66.62	66.91	63.48	69.74	69.12	67.61
LSTM-XGB	81.59	82.73	90.06	87.28	85.55	82.11	82.94	86.55	85.24	86.10
CNN-LSTM	79.81	83.05	85.13	85.01	84.39	80.01	83.87	85.65	85.53	84.95
CNN	72.37	66.47	75.57	71.87	68.71	72.92	66.59	75.69	72.42	68.83

Table 33 Accuracy and precision for all the models for AT-P-C (MAS-2024-B), continued.

FFNN	67.13	67.18	69.08	65.42	68.67	67.25	67.73	69.32	65.54	68.79
TabNet	51.13	52.64	55.24	54.81	52.91	51.48	53.16	55.91	54.94	53.03
RF	81.24	80.00	84.66	83.21	79.03	81.35	80.12	84.78	83.33	79.70
CatBooat	89.42	89.33	91.83	90.72	88.24	90.06	90.00	92.51	91.35	88.19
LGB	90.71	90.57	91.49	88.02	86.52	90.86	91.24	88.73	88.70	87.17
XGB	93.82	90.57	90.68	89.26	90.13	91.35	91.24	94.49	89.91	90.78

Table 34 Accuracy and precision for all the models for AT-C-M (MAS-2024-B).

Class-AT-C-M- MAS-2024-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet- XGB	65.00	70.88	73.21	71.79	72.54	65.89	71.10	73.33	72.14	72.66
FFNN- XGB	80.29	82.63	87.32	83.26	83.16	80.63	82.75	87.44	83.81	83.26
LSTM	64.36	64.15	72.07	67.09	70.25	64.88	64.66	72.82	67.21	71.24
LSTM- XGB	83.00	79.95	89.77	85.05	84.91	83.52	80.14	90.44	85.17	85.02
CNN- LSTM	80.29	82.41	86.42	82.56	85.25	80.41	83.23	86.94	83.08	85.86
CNN	70.28	73.03	75.17	68.89	72.51	70.79	73.19	75.29	69.44	72.62
FFNN	63.46	65.92	69.57	61.61	67.39	63.54	66.42	69.69	61.73	67.51
TabNet	52.02	54.40	56.99	52.93	50.26	52.14	54.92	57.11	52.71	50.92
RF	84.15	83.12	85.32	82.35	82.61	84.27	83.23	85.44	82.46	83.28
CatBooat	87.22	84.73	88.31	87.49	87.38	87.37	84.88	88.44	88.01	87.91
LGB	87.74	87.60	90.79	88.27	87.15	87.89	87.85	90.93	88.49	87.68
XGB	89.00	86.27	89.77	89.27	86.92	89.15	86.42	90.29	89.42	87.44

Table 35 Accuracy and precision for all the models for Eyezy (MAS-2024-B).

Class-Eyezy- MAS-2024-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	66.79	65.90	68.89	66.85	67.13	66.91	66.12	69.24	66.97	67.25
FFNN-XGB	83.29	83.51	83.03	75.81	81.83	83.62	83.63	83.20	76.23	81.95
LSTM	64.61	64.46	70.70	71.76	65.75	65.36	64.97	70.82	72.43	65.87
LSTM-XGB	88.88	90.00	86.22	82.40	81.15	89.41	90.22	86.34	82.52	81.70
CNN-LSTM	86.55	86.58	86.57	86.16	84.21	86.67	86.70	87.12	86.19	84.43
CNN	75.64	69.54	80.02	69.27	74.35	76.19	69.66	80.57	69.38	74.14
FFNN	61.81	64.57	67.61	65.55	63.79	61.93	64.69	68.10	65.67	63.91
TabNet	63.86	66.28	69.21	61.09	64.46	64.03	66.80	69.33	61.21	65.07
RF	77.51	77.43	82.33	79.39	80.42	77.61	77.46	82.60	79.51	81.09
CatBooat	89.88	92.26	92.41	88.55	84.06	90.55	92.90	93.09	89.22	84.74
LGB	87.03	89.60	93.35	89.53	89.39	87.71	90.23	94.03	89.99	90.03
XGB	86.73	88.38	92.51	88.05	88.81	87.41	88.70	93.20	88.72	89.49

Table 36 Accuracy and precision for all the models for FGP (MAS-2024-B).

Class-FGP- MAS-2024-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	72.77	73.31	78.49	72.61	77.41	72.89	73.53	78.61	72.95	77.93
FFNN-XGB	86.54	86.79	89.26	87.67	88.24	86.66	86.92	89.82	87.79	88.34
LSTM	67.08	65.40	68.57	64.75	66.34	67.82	65.92	69.56	64.87	66.45
LSTM-XGB	85.66	86.47	92.48	89.75	87.04	86.18	86.69	93.03	90.27	87.16
CNN-LSTM	85.77	86.54	88.61	88.37	86.62	85.89	87.34	89.14	88.88	87.16
CNN	72.24	74.12	78.96	77.26	75.05	72.79	74.23	79.08	77.81	75.17

Table 36 Accuracy and precision for all the models for FGP (MAS-2024-B), continued.

FFNN	65.68	63.52	66.87	64.71	65.69	65.79	64.07	66.99	64.83	66.21
TabNet	53.55	54.15	55.16	54.34	55.05	53.65	54.67	55.28	54.46	55.71
RF	70.43	77.06	77.95	76.84	75.04	70.55	77.18	78.02	77.33	75.67
CatBooat	81.80	84.52	88.43	86.55	86.96	81.97	89.09	90.96	91.31	88.21
LGB	86.81	88.31	91.89	90.00	88.87	87.63	88.99	92.56	90.68	89.54
XGB	86.21	84.52	91.92	89.67	86.78	86.50	85.74	93.72	90.30	87.46

Table 37 Accuracy and precision for all the models for FamiSafe (MAS-2024-B).

Class-FamiSafe- MAS-2024-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	70.61	73.15	75.26	75.05	74.98	70.73	73.36	75.38	75.39	75.49
XGB										
FFNN-	83.73	86.99	89.64	88.85	88.15	84.06	87.11	89.76	89.04	88.27
XGB										
LSTM	63.38	66.23	67.87	62.54	67.19	64.14	66.75	67.99	62.66	68.18
LSTM-	84.68	87.09	93.85	90.24	86.87	85.18	87.31	94.38	90.75	86.99
XGB										
CNN-	85.29	86.33	89.38	87.75	88.55	85.42	87.15	89.91	88.26	89.10
LSTM										
CNN	76.73	76.89	81.98	79.24	79.53	77.72	77.01	82.53	79.36	79.65
FFNN	65.51	67.83	68.27	65.17	68.09	65.62	68.38	68.87	65.29	68.62
TabNet	50.05	54.31	56.64	53.88	54.59	50.17	54.82	57.30	53.99	54.70
RF	78.15	79.95	84.78	78.45	82.39	78.26	80.07	84.90	79.00	83.07
CatBooat	81.95	87.13	89.84	88.66	89.40	82.23	87.80	90.18	89.33	90.51
LGB	87.68	84.48	92.43	88.75	89.53	87.81	85.16	93.07	89.43	90.21
XGB	85.70	86.45	91.04	87.33	89.54	84.84	87.12	91.72	90.22	88.01

Table 38 Accuracy and precision for all the models for kids-360 (MAS-2024-B).

Class-kids-360- MAS-2024-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	82.79	86.09	89.90	85.32	83.04	82.88	86.31	90.24	85.43	83.92
FFNN-XGB	80.80	86.61	91.87	83.43	86.67	81.14	86.73	91.99	83.98	86.78
LSTM	56.86	66.61	72.48	63.69	67.07	57.61	67.14	72.69	71.74	68.06
LSTM-XGB	85.56	88.00	92.31	87.50	89.52	85.68	88.22	92.45	88.02	91.55
CNN-LSTM	92.52	91.69	93.15	91.67	91.87	92.64	92.20	93.97	92.18	92.42
CNN	76.95	75.66	74.81	80.36	74.89	77.50	75.77	74.73	80.92	75.01
FFNN	63.83	63.81	68.92	66.67	54.25	63.95	69.26	63.93	66.78	54.78
TabNet	53.29	54.96	55.81	55.63	54.67	53.56	55.47	55.93	55.74	58.79
RF	81.65	81.98	84.29	82.35	80.67	81.80	82.03	84.36	82.57	81.29
CatBooat	85.22	87.48	90.67	89.29	88.62	85.89	88.15	91.34	89.96	89.29
LGB	90.58	88.69	91.62	89.09	89.02	91.25	89.37	92.29	89.76	89.70
XGB	87.39	88.17	92.00	86.31	91.06	88.07	88.82	92.67	86.98	93.76

Table 39 Accuracy and precision for all the models for KGP (MAS-2024-B).

Class-KGP-Method-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	77.02	77.95	83.99	80.89	81.27	77.14	78.04	84.49	80.87	81.61
FFNN-XGB	86.61	84.73	91.39	90.80	86.55	86.88	84.88	91.50	91.35	86.71
LSTM	79.38	76.53	79.82	75.95	75.29	79.99	76.65	80.34	76.06	76.29
LSTM-XGB	81.99	77.84	85.31	83.09	83.53	82.11	78.06	85.53	83.62	84.06
CNN-LSTM	86.27	87.59	87.10	82.88	86.49	86.38	88.41	87.62	83.39	87.05
CNN	76.47	73.96	78.03	73.36	72.39	77.03	74.08	78.58	73.48	72.51

Table 39 Accuracy and precision for all the models for KGP (MAS-2024-B), continued.

FFNN	65.37	60.06	67.09	63.21	64.36	65.49	60.12	67.65	63.39	64.88
TabNet	56.69	51.63	57.47	56.44	54.35	56.85	52.15	57.54	56.56	55.01
RF	82.91	77.01	81.21	78.48	77.44	83.03	77.11	81.26	79.02	77.61
CatBooat	85.24	88.59	91.60	90.01	90.96	85.56	89.27	92.27	90.68	91.64
LGB	90.47	89.08	91.95	88.44	91.03	91.14	89.75	92.62	89.00	91.69
XGB	88.09	90.44	90.82	90.84	91.83	89.29	91.20	91.48	91.51	92.49

Table 40 Accuracy and precision for all the models for mLite (MAS-2024-B).

Class-mLite-Method-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	83.82	83.48	84.56	82.83	79.27	84.18	83.82	84.93	82.95	79.79
XGB										
FFNN-	84.71	86.23	88.21	86.78	88.13	85.04	86.35	88.33	87.33	88.25
XGB										
LSTM	75.22	81.74	80.35	77.50	76.09	85.97	82.26	80.47	77.62	77.08
LSTM-	86.32	86.22	88.79	84.84	87.38	86.47	86.44	89.32	85.34	87.93
XGB										
CNN-	84.50	83.41	85.85	73.53	82.16	84.60	83.93	86.63	73.96	82.71
LSTM										
CNN	73.18	73.22	78.57	73.72	77.33	73.73	73.34	78.89	73.85	77.80
FFNN	63.85	67.19	67.21	64.68	63.36	63.97	67.75	67.68	64.86	63.48
TabNet	54.67	54.23	55.28	51.35	52.08	55.37	54.75	54.79	51.38	52.88
RF	77.28	75.68	79.82	77.17	74.21	77.39	75.80	79.94	77.77	74.82
CatBooat	83.59	87.97	89.68	86.67	89.84	84.27	88.64	90.35	87.26	90.52
LGB	83.39	88.29	92.05	90.43	91.87	84.04	88.96	92.77	91.11	92.54
XGB	87.12	87.81	90.92	89.02	90.75	85.23	88.03	93.80	89.70	91.43

Table 41 Accuracy and precision for all the models for MMP (MAS-2024-B).

Class MMP-Method-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	92.65	87.93	92.89	92.64	91.57	92.77	88.12	93.40	92.76	91.91
FFNN-XGB	85.16	88.71	88.35	83.56	87.54	85.49	88.82	88.46	84.03	87.66
LSTM	84.72	84.56	87.08	87.74	85.45	85.46	84.77	85.20	87.85	86.44
LSTM-XGB	83.05	85.33	91.84	86.33	86.35	83.56	85.55	92.35	86.45	86.90
CNN-LSTM	82.32	87.20	89.64	88.02	86.98	82.94	88.02	89.76	88.54	87.53
CNN	74.74	73.26	74.88	73.48	74.10	75.28	73.37	75.44	73.68	74.22
FFNN	62.87	67.76	69.42	68.98	65.03	62.98	67.86	69.97	69.50	65.15
TabNet	55.50	55.18	51.81	56.16	55.17	55.62	55.68	51.93	56.22	55.81
RF	75.85	77.12	81.53	78.38	81.57	75.95	77.37	81.97	78.82	82.23
CatBooat	86.02	87.89	90.40	90.49	90.28	86.69	88.56	91.08	91.17	90.95
LGB	86.95	90.68	88.91	90.11	89.24	87.62	91.35	89.58	90.78	89.91
XGB	88.49	89.03	92.03	89.88	90.74	86.34	89.70	92.71	90.55	91.41

Table 42 Accuracy and precision for all the models for mSpy (MAS-2024-B).

Class-mSpy-Method-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	73.72	74.43	75.23	74.41	74.25	73.84	74.65	75.35	74.48	74.37
FFNN-XGB	84.90	87.98	87.50	88.41	86.48	85.23	88.09	87.62	88.96	86.60
LSTM	85.75	85.22	87.76	86.53	86.26	86.46	88.52	87.88	86.65	87.25
LSTM-XGB	63.77	64.36	71.64	65.01	64.91	63.89	64.57	72.16	65.55	65.45
CNN-LSTM	86.18	80.85	86.35	86.59	84.67	86.30	81.67	86.90	87.11	85.19
CNN	82.12	80.53	81.48	82.39	80.08	82.68	80.62	82.03	82.50	80.19

Table 42 Accuracy and precision for all the models for mSpy (MAS-2024-B), continued.

FFNN	61.39	66.13	68.81	65.31	65.03	61.91	66.68	69.33	65.43	65.15
TabNet	55.99	54.60	55.26	56.58	50.47	56.11	55.12	55.54	56.69	51.14
RF	76.35	75.42	81.52	80.20	81.53	76.47	75.50	81.64	80.75	82.20
CatBooat	86.87	89.56	88.74	89.29	87.50	87.30	89.98	89.17	90.24	88.44
LGB	87.40	88.86	91.06	91.55	90.76	87.82	89.28	92.01	92.48	91.71
XGB	89.76	88.08	93.53	89.39	89.29	87.47	88.51	90.36	90.35	90.24

Table 43 Accuracy and precision for all the models for Normal Traffic (MAS-2024-B).

Class-Normal-Traffic- MAS-2024-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	82.88	83.24	87.82	85.82	84.82	82.97	81.05	87.94	86.12	84.94
XGB										
FFNN-	85.49	80.88	89.02	86.04	88.21	85.83	89.36	89.57	89.46	88.33
XGB										
LSTM	82.65	86.68	87.28	82.18	85.64	83.40	87.20	87.38	82.70	86.63
LSTM-	89.59	89.58	92.62	89.56	89.08	89.71	89.80	93.14	90.07	89.63
XGB										
CNN-	80.15	85.73	88.67	87.68	84.39	80.48	86.55	89.22	88.20	85.36
LSTM										
CNN	72.33	75.50	74.28	72.63	72.62	72.89	75.49	74.76	72.75	72.74
FFNN	64.01	67.54	69.24	65.73	62.28	64.14	68.05	69.31	65.85	62.79
TabNet	68.26	68.97	67.77	68.47	66.35	68.38	69.49	67.88	69.06	66.40
RF	76.26	80.16	81.08	81.22	78.18	76.38	80.22	81.20	81.90	78.78
CatBooat	84.87	83.35	84.26	84.20	84.01	85.54	83.63	84.39	84.63	84.43
LGB	83.88	84.87	88.05	85.68	82.69	84.01	85.12	88.15	86.10	83.08
XGB	84.06	84.82	85.15	88.86	82.73	83.66	85.06	85.28	89.28	83.36

Table 44 Accuracy and precision for all the models for Qustodio (MAS-2024-B).

Class-Qustodio-Method-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	89.62	83.37	92.16	88.76	91.81	89.72	83.55	92.28	88.88	91.91
FFNN-XGB	83.07	89.20	88.49	87.06	86.76	83.40	89.32	88.61	87.61	86.87
LSTM	80.85	86.20	87.07	86.68	86.74	81.60	86.72	87.18	86.80	87.74
LSTM-XGB	66.22	63.25	75.29	67.78	67.01	66.34	63.74	75.80	68.30	67.55
CNN-LSTM	87.56	81.74	89.35	85.19	87.04	87.67	82.56	89.87	85.70	87.59
CNN	74.39	74.22	79.52	73.72	74.44	74.94	73.34	80.07	73.84	74.49
FFNN	69.37	69.19	69.52	70.49	68.53	69.49	69.74	69.64	71.01	68.64
TabNet	55.79	55.01	53.48	56.16	51.58	55.91	55.50	53.60	56.25	52.24
RF	79.56	78.23	83.13	82.89	79.85	79.65	78.35	83.35	83.44	80.50
CatBooat	86.93	88.87	89.66	91.34	88.17	87.88	89.81	89.81	91.48	88.32
LGB	87.55	88.61	92.33	90.41	89.08	88.59	89.56	92.48	90.56	89.19
XGB	87.18	88.25	93.62	90.91	89.11	87.12	89.20	91.44	91.06	89.26

Table 45 Accuracy and precision for all the models for TiSpy (MAS-2024-B).

Class-TiSpy-Method-B										
Models	Accuracy (%)					Precision (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	74.70	64.93	77.69	67.41	67.65	74.82	65.12	78.03	67.53	67.77
FFNN-XGB	55.88	62.50	63.56	60.56	62.03	55.99	62.62	63.68	61.11	62.15
LSTM	71.29	72.66	67.60	64.16	65.76	72.07	73.18	67.77	64.28	66.43
LSTM-XGB	77.82	74.49	79.79	75.86	74.69	77.93	74.71	79.91	75.97	75.24
CNN-LSTM	81.76	80.82	85.60	85.08	84.28	81.87	80.94	85.72	85.19	84.83
CNN	69.47	68.61	71.73	69.25	67.89	70.02	68.73	72.28	69.37	68.01

Table 45 Accuracy and precision for all the models for TiSpy (MAS-2024-B), continued.

FFNN	60.51	57.46	61.68	60.55	56.68	60.63	58.16	61.80	60.67	56.81
TabNet	51.15	52.96	53.43	50.85	50.67	51.27	53.48	51.27	51.06	51.34
RF	76.55	78.18	80.28	78.00	79.87	77.03	78.30	80.37	78.07	79.95
CatBooat	76.38	72.92	79.72	78.17	79.17	77.05	73.20	80.40	78.81	79.84
LGB	78.69	76.80	78.46	82.80	78.16	79.35	77.47	79.14	83.47	78.84
XGB	84.14	84.00	90.57	88.35	86.30	84.31	84.65	91.23	89.02	86.63

Table 46 Recall and f1-score for all the models for Air-Droid (MAS-2024-B).

Class-Air-Droid- MAS-2024-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	61.70	64.55	65.39	61.58	63.19	61.79	64.64	65.51	61.65	63.17
XGB										
FFNN-	61.99	57.19	65.57	64.39	63.79	62.10	57.16	65.62	64.58	63.80
XGB										
LSTM	63.48	66.39	67.35	63.89	64.68	63.55	66.63	67.57	63.92	65.02
LSTM-	76.30	78.17	82.69	78.29	77.09	76.36	78.07	82.67	78.31	77.31
XGB										
CNN-	81.50	82.30	84.42	81.77	80.39	81.59	82.39	84.68	81.79	80.39
LSTM										
CNN	71.79	69.99	72.19	69.99	69.89	72.02	70.02	72.20	70.22	69.87
FFNN	61.45	62.33	64.40	60.90	61.09	61.53	62.62	64.47	61.03	61.11
TabNet	54.49	52.19	54.69	52.89	53.79	54.48	52.39	54.72	52.86	54.10
RF	84.39	83.99	85.49	84.69	84.16	84.37	83.98	85.48	84.94	84.15
CatBooat	83.88	89.49	90.09	89.89	88.87	84.14	89.75	90.35	89.91	89.12
LGB	88.09	89.25	90.69	90.59	88.29	88.11	89.55	90.97	90.87	88.61
XGB	88.62	88.70	91.56	88.27	87.79	89.01	89.08	91.41	88.59	88.07

Table 47 Recall and f1-score for all the models for AT-B-D-M (MAS-2024-B).

Class-AT-B-D-M- MAS-2024-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	66.95	65.95	70.47	70.35	68.78	67.16	66.01	70.48	70.35	69.06
FFNN-XGB	81.29	81.80	83.59	81.35	83.09	81.40	81.89	83.56	81.57	83.14
LSTM	62.99	64.49	65.39	63.69	64.19	63.32	64.71	65.65	63.66	64.19
LSTM-XGB	77.55	81.89	85.89	81.19	81.19	77.77	81.93	85.86	81.14	81.42
CNN-LSTM	86.59	84.88	86.89	86.19	86.59	86.61	85.07	86.88	86.19	86.84
CNN	66.09	68.75	71.52	69.17	68.59	66.29	69.06	71.61	69.32	68.53
FFNN	61.83	61.46	65.59	64.58	63.77	61.87	61.76	65.60	64.65	63.84
TabNet	63.88	66.29	70.89	69.23	69.77	63.97	66.54	70.93	69.27	70.05
RF	83.05	82.39	83.85	81.42	81.07	83.07	82.43	83.89	81.46	81.43
CatBooat	81.86	84.51	88.41	86.59	86.99	82.10	84.75	86.70	86.88	88.77
LGB	87.27	83.18	89.49	86.59	88.19	87.54	83.47	89.51	86.88	88.51
XGB	80.07	83.59	91.29	86.65	85.98	80.43	83.84	91.58	86.96	86.32

Table 48 Recall and f1-score for all the models for AT-P-C (MAS-2024-B).

Class-AT-P-C- MAS-2024-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	67.15	64.75	71.97	70.99	66.89	67.18	64.83	72.05	71.03	66.98
FFNN-XGB	79.15	81.19	90.15	87.34	83.14	79.33	81.19	90.14	87.48	83.15
LSTM	66.19	62.89	69.69	68.97	66.69	66.53	63.22	69.68	69.05	67.11
LSTM-XGB	81.69	82.79	90.17	87.35	85.59	81.85	82.83	86.50	85.18	85.82
CNN-LSTM	79.89	83.09	85.19	85.09	84.45	79.95	83.45	85.39	85.27	84.67
CNN	72.39	66.44	75.59	71.89	68.79	72.64	66.53	75.63	72.14	68.77

Table 48 Recall and f1-score for all the models for AT-P-C (MAS-2024-B), continued.

FFNN	67.19	67.25	69.19	65.49	68.72	67.19	67.45	69.26	65.47	68.73
TabNet	51.12	52.65	55.29	54.88	52.99	51.42	52.98	55.57	54.87	52.97
RF	81.25	80.08	84.69	83.29	79.09	81.29	80.06	84.72	83.27	79.36
CatBooat	89.32	89.30	91.85	90.79	88.29	89.37	89.66	92.17	91.40	88.57
LGB	90.61	90.59	91.59	88.09	86.59	90.77	90.90	88.65	88.36	86.83
XGB	93.89	90.59	90.69	89.29	90.19	91.01	90.90	94.15	89.59	90.44

Table 49 Recall and f1-score for all the models for AT-C-M (MAS-2024-B).

Class-AT-C-M- MAS-2024-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	65.00	70.88	73.21	71.79	72.54	65.83	70.92	73.27	71.96	72.60
XGB										
FFNN-	80.29	82.63	87.32	83.26	83.16	80.46	82.69	87.37	83.50	83.20
XGB										
LSTM	64.36	64.15	72.07	67.09	70.25	64.42	64.40	72.44	67.14	70.74
LSTM-	83.00	79.95	89.77	85.05	84.91	83.26	80.03	90.16	85.11	84.96
XGB										
CNN-	80.29	82.41	86.42	82.56	85.25	80.34	82.82	86.68	82.82	85.58
LSTM										
CNN	70.28	73.03	75.17	68.89	72.51	70.51	73.09	75.23	69.16	72.56
FFNN	63.46	65.92	69.57	61.61	67.39	63.48	66.14	69.63	61.67	67.45
TabNet	52.02	54.40	56.99	52.93	50.26	52.08	54.66	57.05	52.65	50.58
RF	84.15	83.12	85.32	82.35	82.61	84.21	83.16	85.38	82.40	82.96
CatBooat	87.22	84.73	88.31	87.49	87.38	87.29	84.81	88.36	87.74	87.64
LGB	87.74	87.60	90.79	88.27	87.15	87.81	87.68	90.85	88.38	87.41
XGB	89.00	86.27	89.77	89.27	86.92	89.08	86.34	90.03	89.34	87.18

Table 50 Recall and f1-score for all the models for Eyezy (MAS-2024-B).

Class-Eyezy- MAS-2024-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	66.79	65.90	68.89	66.85	67.13	66.85	66.01	69.06	66.90	67.19
FFNN-XGB	83.29	83.51	83.03	75.81	81.83	83.41	83.51	83.14	75.95	81.89
LSTM	64.61	64.46	70.70	71.76	65.75	65.36	64.71	70.76	72.09	65.81
LSTM-XGB	88.88	90.00	86.22	82.40	81.15	89.14	90.11	86.27	82.46	81.42
CNN-LSTM	86.55	86.58	86.57	86.16	84.21	86.61	86.63	86.84	86.26	84.32
CNN	75.64	69.54	80.02	69.27	74.35	75.91	69.60	80.29	69.32	74.08
FFNN	61.81	64.57	67.61	65.55	63.79	61.87	64.63	67.89	65.60	63.84
TabNet	63.86	66.28	69.21	61.09	64.46	63.97	66.54	69.27	61.15	65.12
RF	77.51	77.43	82.33	79.39	80.42	77.55	77.40	82.53	79.45	80.75
CatBooat	89.88	92.26	92.41	88.55	84.06	90.21	92.56	92.75	88.88	84.40
LGB	87.03	89.60	93.35	89.53	89.39	87.36	89.89	93.69	89.64	89.73
XGB	86.73	88.38	92.51	88.05	88.81	87.69	89.04	92.85	88.38	89.14

Table 51 Recall and f1-score for all the models for FGP (MAS-2024-B).

Class-FamiGuardPro- MAS-2024-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	72.75	73.29	78.47	72.60	77.47	72.82	73.40	78.55	72.78	77.66
FFNN-XGB	86.58	86.78	89.28	87.69	88.29	86.71	86.85	89.54	87.73	88.28
LSTM	67.09	65.47	68.59	64.79	66.39	67.45	65.66	69.06	64.82	66.39
LSTM-XGB	85.69	86.49	92.49	89.79	87.09	85.92	86.58	92.75	90.01	87.10
CNN-LSTM	85.78	86.58	88.68	88.38	86.68	85.83	86.93	88.87	88.62	86.88
CNN	72.27	74.17	78.97	77.27	75.07	72.51	74.17	79.02	77.54	75.11

Table 51 Recall and f1-score for all the models for FGP (MAS-2024-B), continued.

FFNN	65.66	63.56	66.86	64.76	65.66	65.73	63.79	66.93	64.77	65.95
TabNet	53.55	54.15	55.15	54.35	55.05	53.59	54.41	55.22	54.40	55.37
RF	70.44	77.04	77.94	76.84	75.09	70.49	77.12	77.96	77.06	75.33
CatBooat	81.83	84.53	88.43	86.53	86.98	81.63	88.75	90.62	90.97	87.87
LGB	86.82	88.32	91.82	90.02	88.87	87.29	88.65	92.22	90.32	89.20
XGB	86.21	84.51	91.91	89.61	86.76	86.16	85.04	93.38	89.96	87.12

Table 52 Recall and f1-score for all the models for FamiSafe (MAS-2024-B).

Class-FamiSafe- MAS-2024-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	70.69	73.19	75.29	75.09	74.99	70.67	73.25	75.32	75.21	75.23
XGB										
FFNN-	83.78	86.98	89.68	88.88	88.18	83.89	87.05	89.69	88.76	88.21
XGB										
LSTM	63.37	66.27	67.87	62.57	67.17	63.75	66.49	67.93	62.60	67.68
LSTM-	84.66	87.06	93.86	90.26	86.86	84.92	87.20	94.10	90.49	86.93
XGB										
CNN-	85.25	86.35	89.35	87.74	88.55	85.36	86.73	89.65	88.00	88.82
LSTM										
CNN	76.74	76.84	81.94	79.25	79.54	77.01	76.95	82.25	79.30	79.59
FFNN	65.53	67.83	68.23	65.13	68.03	65.56	68.19	68.81	65.22	68.36
TabNet	50.02	54.32	56.62	53.82	54.52	50.11	54.65	56.97	53.93	54.64
RF	78.11	79.91	84.71	78.41	82.31	78.20	80.01	84.84	78.72	82.73
CatBooat	81.99	87.19	89.84	88.66	89.41	82.09	87.46	89.84	88.99	90.17
LGB	87.66	84.48	92.45	88.79	89.54	87.73	84.82	92.73	89.09	89.87
XGB	85.77	86.47	91.09	87.38	89.59	84.49	86.78	91.38	89.87	87.67

Table 53 Recall and f1-score for all the models for kids-360 (MAS-2024-B).

Class-kids-360- MAS-2024-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	82.78	86.09	89.99	85.39	83.09	82.82	86.19	90.07	85.37	83.66
FFNN-XGB	80.89	86.68	91.88	83.48	86.68	80.97	86.69	91.93	83.70	86.72
LSTM	56.87	66.67	72.47	63.67	67.07	57.23	66.86	72.58	71.68	67.57
LSTM-XGB	85.56	88.06	92.36	87.55	89.56	85.62	88.11	92.38	87.76	91.48
CNN-LSTM	92.55	91.65	93.15	91.66	91.85	92.58	91.94	93.56	91.92	92.14
CNN	76.94	75.64	74.84	80.34	74.84	77.22	75.71	74.67	80.64	74.95
FFNN	63.84	63.83	68.93	66.63	54.23	63.89	68.98	63.87	66.72	54.52
TabNet	53.23	54.92	55.82	55.62	54.62	53.50	55.21	55.87	55.68	58.46
RF	81.62	81.91	84.21	82.31	80.61	81.69	81.97	84.41	82.46	80.95
CatBooat	85.21	87.49	90.69	89.29	88.69	85.55	87.81	91.00	89.62	88.95
LGB	90.59	88.68	91.68	89.08	89.08	90.91	89.03	91.95	89.42	89.36
XGB	87.38	88.17	92.07	86.37	91.07	87.72	88.51	92.33	86.64	93.42

Table 54 Recall and f1-score for all the models for KGP (MAS-2024-B).

Class-KGP-Method-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	77.09	77.98	83.98	80.88	81.29	77.08	78.15	84.23	80.93	81.44
FFNN-XGB	86.68	84.79	91.38	90.89	86.58	86.71	84.78	91.44	91.07	86.65
LSTM	79.37	76.57	79.87	75.97	75.27	79.62	76.58	80.08	76.00	75.79
LSTM-XGB	81.96	77.85	85.36	83.06	83.56	82.05	77.95	85.41	83.35	83.78
CNN-LSTM	86.25	87.56	87.15	82.85	86.45	86.32	87.99	87.36	83.14	86.77
CNN	76.44	73.93	78.04	73.34	72.34	76.75	74.01	78.30	73.42	72.45

Table 54 Recall and f1-score for all the models for KGP (MAS-2024-B), continued.

FFNN	65.33	60.02	67.03	63.23	64.33	65.42	60.06	67.37	63.33	64.61
TabNet	56.62	51.61	57.42	56.42	54.32	56.79	51.88	57.48	56.50	54.67
RF	82.90	77.09	81.21	78.41	77.41	82.97	77.04	81.14	78.75	77.52
CatBooat	85.29	88.58	91.69	90.09	90.99	85.40	88.93	91.93	90.34	91.24
LGB	90.48	89.07	91.98	88.48	91.08	90.80	89.41	92.28	88.69	91.35
XGB	88.07	90.46	90.87	90.87	91.87	88.95	90.86	91.14	91.17	92.15

Table 55 Recall and f1-score for all the models for mLite (MAS-2024-B).

Class-mLite-Method-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	83.82	83.48	84.56	82.83	79.27	84.11	83.47	84.82	82.89	79.52
XGB										
FFNN-	84.71	86.23	88.21	86.78	88.13	84.87	86.28	88.27	87.05	88.19
XGB										
LSTM	75.22	81.74	80.35	77.50	76.09	75.59	81.99	80.41	77.56	76.58
LSTM-	86.32	86.22	88.79	84.84	87.38	86.38	86.32	89.05	85.04	87.66
XGB										
CNN-	84.50	83.41	85.85	73.53	82.16	84.52	83.66	86.21	73.70	82.43
LSTM										
CNN	73.18	73.22	78.57	73.72	77.33	73.45	73.28	78.73	73.78	77.60
FFNN	63.85	67.19	67.21	64.68	63.36	63.91	67.47	67.42	64.75	63.42
TabNet	54.67	54.23	55.28	51.35	52.08	55.31	54.48	54.73	51.24	52.55
RF	77.28	75.68	79.82	77.17	74.21	77.33	75.73	79.88	77.44	74.51
CatBooat	83.59	87.97	89.68	86.67	89.84	83.93	88.38	90.00	86.92	90.17
LGB	83.39	88.29	92.05	90.43	91.87	83.70	88.62	92.38	90.76	92.20
XGB	87.12	87.81	90.92	89.02	90.75	84.89	87.69	93.12	89.36	91.09

Table 56 Recall and f1-score for all the models for MMP (MAS-2024-B).

Class MMP-Method-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	92.69	87.99	92.89	92.69	91.59	92.71	88.01	93.14	92.70	91.74
FFNN-XGB	85.18	88.78	88.38	83.58	87.58	85.32	88.76	88.40	83.75	87.60
LSTM	84.77	84.57	87.07	87.77	85.47	85.09	84.66	87.14	87.79	85.94
LSTM-XGB	83.06	85.36	91.86	86.36	86.36	83.30	85.44	92.09	86.39	86.63
CNN-LSTM	82.35	87.25	89.65	88.05	86.95	82.88	87.61	89.70	88.28	87.25
CNN	74.74	73.24	74.84	73.44	74.14	75.00	73.31	75.16	73.62	74.16
FFNN	62.83	67.73	69.43	68.93	65.03	62.92	67.80	69.69	69.23	65.09
TabNet	55.52	55.12	51.82	56.12	55.12	55.56	55.41	51.87	56.16	55.47
RF	75.81	77.11	81.51	78.31	81.51	75.89	77.31	81.91	78.61	81.88
CatBooat	86.09	87.89	90.49	90.49	90.29	86.35	88.22	90.74	90.83	90.61
LGB	86.98	90.68	88.98	90.18	89.28	87.28	91.00	89.24	90.43	98.57
XGB	88.47	89.07	92.07	89.87	90.77	86.00	89.36	92.37	90.21	91.07

Table 57 Recall and f1-score for all the models for mSpy (MAS-2024-B).

Class-mSpy-Method-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	73.70	74.43	75.25	74.47	74.29	73.78	74.54	75.29	74.42	74.31
FFNN-XGB	84.91	87.92	87.54	88.46	86.48	85.06	88.03	87.56	88.68	86.54
LSTM	85.72	85.21	87.73	86.55	86.27	86.08	88.26	87.81	86.59	86.75
LSTM-XGB	63.73	64.30	71.62	65.04	64.96	63.83	64.46	71.90	65.26	65.17
CNN-LSTM	86.14	80.81	86.31	86.53	84.65	86.23	81.26	86.62	86.85	84.93
CNN	82.15	80.52	81.40	82.32	80.04	82.40	80.56	81.75	82.44	80.14

Table 57 Recall and f1-score for all the models for mSpy (MAS-2024-B), continued.

FFNN	61.36	66.14	68.89	65.31	65.03	61.65	66.40	69.06	65.37	65.09
TabNet	55.97	54.65	55.28	56.50	50.42	56.05	54.86	55.48	56.63	50.08
RF	76.38	75.46	81.57	80.29	81.51	76.40	75.44	81.57	80.47	81.86
CatBooat	86.89	89.57	88.76	89.28	87.50	87.09	89.77	88.95	89.76	87.97
LGB	87.41	88.88	91.05	91.57	90.79	87.61	89.07	91.53	92.00	91.23
XGB	89.76	88.09	93.54	89.36	89.28	87.26	88.29	90.15	89.87	89.76

Table 58 Recall and f1-score for all the models for Normal Traffic (MAS-2024-B).

Class-Normal Traffic- MAS-2024-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-	82.89	83.29	87.80	85.82	84.80	82.91	80.94	87.87	86.06	84.88
XGB										
FFNN-	85.48	80.88	89.01	86.03	88.21	85.66	89.30	89.30	89.40	88.27
XGB										
LSTM	82.67	86.67	87.21	82.14	85.62	83.02	86.94	87.32	82.64	86.14
LSTM-	89.56	89.56	92.63	89.55	89.03	89.65	89.69	92.87	89.81	89.35
XGB										
CNN-	80.15	85.75	88.64	87.66	84.34	80.42	86.14	88.94	87.94	85.09
LSTM										
CNN	72.34	75.54	74.25	72.67	72.65	72.61	75.43	74.48	72.69	72.68
FFNN	64.03	67.53	69.26	65.78	62.26	64.07	67.77	69.25	65.79	62.53
TabNet	68.22	68.92	67.77	68.49	66.37	68.32	69.22	67.82	68.73	66.34
RF	76.21	80.11	81.08	81.20	78.18	76.32	80.16	81.14	81.63	78.44
CatBooat	84.89	83.30	84.29	84.29	84.09	85.20	83.49	84.32	84.41	84.22
LGB	83.88	84.81	88.00	85.67	82.60	83.94	84.98	88.08	85.88	82.86
XGB	84.07	84.89	85.11	88.88	82.71	83.32	84.92	85.21	89.07	83.14

Table 59 Recall and f1-score for all the models for Qustodio (MAS-2024-B).

Class-Qustodio-Method-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	89.62	83.37	92.16	88.76	91.81	81.22	89.66	92.22	88.82	91.85
FFNN-XGB	83.07	89.20	88.49	87.06	86.76	83.23	89.26	88.55	87.33	86.81
LSTM	80.85	86.20	87.07	86.68	86.74	86.42	87.12	86.74	87.23	86.42
LSTM-XGB	66.22	63.25	75.29	67.78	67.01	66.28	63.63	75.54	68.04	67.28
CNN-LSTM	87.56	81.74	89.35	85.19	87.04	87.61	82.15	89.61	85.44	87.31
CNN	74.39	74.22	79.52	73.72	74.44	74.66	74.28	79.79	73.77	74.43
FFNN	69.37	69.19	69.52	70.49	68.53	69.43	69.46	69.58	70.75	68.58
TabNet	55.79	55.01	53.48	56.16	51.58	55.85	55.23	53.54	56.19	51.91
RF	79.56	78.23	83.13	82.89	79.85	79.59	78.28	83.29	83.16	80.16
CatBooat	86.93	88.87	89.66	91.34	88.17	87.40	89.35	89.74	91.41	88.24
LGB	87.55	88.61	92.33	90.41	89.08	88.11	89.08	92.40	90.48	89.11
XGB	87.18	88.25	93.62	90.91	89.11	86.64	88.72	91.36	90.98	89.18

Table 60 Recall and f1-score for all the models for TiSpy (MAS-2024-B).

Class-TiSpy-Method-B										
Models	Recall (%)					F1-Score (%)				
	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt	50-pkt	100-pkt	150-pkt	200-pkt	250-pkt
TabNet-XGB	74.70	64.93	77.69	67.41	67.65	74.76	65.04	77.86	67.47	67.71
FFNN-XGB	55.88	62.50	63.56	60.56	62.03	55.83	62.56	63.62	60.84	62.09
LSTM	71.29	72.66	67.60	64.16	65.76	71.93	72.92	67.66	64.22	66.10
LSTM-XGB	77.82	74.49	79.79	75.86	74.69	77.87	74.59	79.85	75.91	74.97
CNN-LSTM	81.76	80.82	85.60	85.08	84.28	81.81	80.88	85.66	85.14	84.55
CNN	69.47	68.61	71.73	69.25	67.89	69.74	68.67	72.00	69.31	67.95
FFNN	60.51	57.46	61.68	60.55	56.68	60.57	57.88	61.74	60.61	56.74
TabNet	51.15	52.96	53.43	50.85	50.67	51.20	53.22	51.20	51.00	51.00
RF	76.55	78.18	80.28	78.00	79.87	76.77	78.24	80.31	78.01	79.93
CatBooat	76.38	72.92	79.72	78.17	79.17	76.71	73.06	80.05	78.46	79.50
LGB	78.69	76.80	78.46	82.80	78.16	79.01	77.14	78.80	83.14	78.50
XGB	84.14	84.00	90.57	88.35	86.30	83.97	84.31	90.89	88.68	86.98

LIST OF PUBLICATIONS

Journals

- [1] Mimi, M. A., Ng, H., & Yap, T. T. V. (2025). Directional features and rule-based labeling for real-time network traffic-based Android spyware classification. *Journal of Supercomputing*, 81(932). <https://doi.org/10.1007/s11227-025-07391-8>
- [2] Mimi, M. A., Ng, H., & Yap, T. T. V. (2025). Curating datasets to enhance spyware classification. *Jordanian Journal of Computers and Information Technology (JJCIT)*, 11(1), March. <https://doi.org/10.5455/jjcit.71-1719026602>.
- [3] Mimi, M A., Yap, T. T. V., & Ng, H. (2024). Advancements in Machine Learning Techniques for Intrusion Detection Systems: An Overview of Perspectives and Datasets. *Journal of Logistics, Informatics and Service Science*, 11(3), 1–20.
<https://doi.org/10.33168/JLISS.2024.0301>